

TOTAL TYPESCRIPT

THE ESSENTIALS

MATT POCKOCK

with TAYLOR BELL



CONTENTS IN DETAIL

FOREWORD

INTRODUCTION

[The Birth of TypeScript](#)

[Why Now?](#)

[Who Is This Book For?](#)

[What We'll Cover](#)

[Online Resources](#)

[The Book's Exercises](#)

[@ts-expect-error](#)

[Vitest](#)

PART I: GETTING STARTED

1

KICKSTART YOUR TYPESCRIPT SETUP

[What's Different About TypeScript?](#)

[A High-Level View of How TypeScript Works](#)

[Tools for TypeScript Development](#)

[Installing Node.js](#)

[Installing the npm Package Manager](#)

[Installing TypeScript](#)

[Summary](#)

2

IDE SUPERPOWERS

[Autocomplete](#)

[Manual Autocomplete](#)

[Exercise 2-1: Autocomplete](#)

[TypeScript Error Checking](#)

[Runtime Errors](#)

[Non-Runtime Errors](#)

[Warning Locations](#)

[Multiline Errors](#)

[Introspecting Variables and Declarations](#)

[Exercise 2-2: Hovering Over a Function Call](#)

[JSDoc Comments](#)

[Exercise 2-3: Adding Documentation for Hovers](#)

[Navigating with Go to Definition and Go to References](#)

[Rename Symbol](#)

[Automatic Imports](#)

[Quick Fixes](#)

[Restarting the VS Code Server](#)

[Working in JavaScript](#)

[Exercise 2-4: Quick Fix Refactoring](#)

[Summary](#)

3

TYPESCRIPT IN THE DEVELOPMENT PIPELINE

[The Problem with TypeScript in the Browser](#)

[Transpiling TypeScript](#)

[Initializing a TypeScript Project](#)

[Running tsc](#)

[Does TypeScript Change Your JavaScript?](#)

[A Note on Version Control](#)

[Running TypeScript in Watch Mode](#)

[Errors in the TypeScript CLI](#)

[TypeScript with Modern Frameworks](#)

[TypeScript as a Linter](#)

[Summary](#)

PART II: FUNDAMENTALS

4

ESSENTIAL TYPES AND ANNOTATIONS

[Type Annotations](#)

[The Basic Types](#)

[Function Parameter Annotations](#)

[Variable Annotations](#)

[Type Inference](#)

[The any Type](#)

[Exercise 4-1: Basic Types with Function Parameters](#)

[Exercise 4-2: Annotating Empty Parameters](#)

[Exercise 4-3: The Basic Types](#)

[Exercise 4-4: The any Type](#)

[Object Literal Types](#)

[Optional Object Properties](#)

[Exercise 4-5: Object Literal Types](#)

[Exercise 4-6: Optional Property Types](#)

[Type Aliases](#)

[Sharing Types Across Modules](#)

[Exercise 4-7: The type Keyword](#)

[Arrays](#)

[Arrays of Objects](#)

[Tuples](#)

[Named Tuples](#)

[Exercise 4-8: Array Type](#)

[Exercise 4-9: Arrays of Objects](#)

[Exercise 4-10: Tuples](#)

[Exercise 4-11: Optional Members of Tuples](#)

[Passing Types to Functions](#)

[Passing Types to Set](#)

[Not All Functions Can Receive Types](#)

[Exercise 4-12: Passing Types to Map](#)

[Exercise 4-13: JSON.parse\(\) Can't Receive Type Arguments](#)

[Typing Functions](#)

[Optional Parameters](#)

[Default Parameters](#)

[Function Return Types](#)

[Rest Parameters](#)

[Function Types](#)

[The void Type](#)

[Async Functions](#)

[Exercise 4-14: Optional Function Parameters](#)

[Exercise 4-15: Default Function Parameters](#)

[Exercise 4-16: Rest Parameters](#)

[Exercise 4-17: Function Types](#)

[Exercise 4-18: Functions Returning void](#)

[Exercise 4-19: void vs. undefined](#)

[Exercise 4-20: Async Functions](#)

[Summary](#)

5

UNIONS, LITERALS, AND NARROWING

[Union Types](#)

[Declaring Union Types](#)

[Literal Types](#)

[Combining Unions with Unions](#)

[Exercise 5-1: string or null](#)

[Exercise 5-2: Restricting Function Parameters](#)

[Wide and Narrow Types](#)

[Unions Are Wider Than Their Members](#)

[The Process of Narrowing](#)

[Exercise 5-3: Narrowing with if Statements](#)

[Exercise 5-4: Throwing Errors to Narrow](#)

[Exercise 5-5: Using in to Narrow](#)

[The unknown and never Types](#)

[The Widest Type: unknown](#)

[The Difference Between unknown and any](#)

[The Narrowest Type: never](#)

[Exercise 5-6: Narrowing Errors with instanceof](#)

[Exercise 5-7: Narrowing unknown to a Value](#)

[Discriminated Unions](#)

[The Problem: The Bag of Optionals](#)

[The Solution: Discriminated Unions](#)

[Exercise 5-8: Deconstructing a Discriminated Union](#)

[Exercise 5-9: Narrowing a Discriminated Union with a switch Statement](#)

[Exercise 5-10: Discriminated Tuples](#)

[Exercise 5-11: Handling Defaults with a Discriminated Union](#)

[Summary](#)

[PART III: OBJECTS, CLASSES, AND MUTABILITY](#)

[6](#)

[OBJECTS](#)

[Extending Objects](#)

[Intersection Types](#)

[Interfaces](#)

[Intersections vs. interface extends](#)

[Types vs. Interfaces](#)

[Exercise 6-1: Creating an Intersection Type](#)

[Exercise 6-2: Extending Interfaces](#)

[Dynamic Object Keys](#)

[Index Signatures](#)

[The Record Type](#)

[A Combination of Known and Dynamic Keys](#)

[The PropertyKey Type](#)

[Exercise 6-3: Using an Index Signature for Dynamic Keys](#)

[Exercise 6-4: Default Properties with Dynamic Keys](#)

[Exercise 6-5: Restricting Object Keys with Records](#)

[Exercise 6-6: Dynamic Key Support](#)

[Reducing Duplication with Utility Types](#)

[The Partial Type](#)

[The Required Type](#)

[The Pick Type](#)

[The Omit Type](#)

[Union Types with Omit and Pick](#)

[Exercise 6-7: Expecting Certain Properties](#)

[Exercise 6-8: Updating a Product](#)

[Summary](#)

[7](#)

[MUTABILITY](#)

[Mutability and Inference](#)

[How TypeScript Infers let](#)

[How TypeScript Infers const](#)

[Object Property Inference](#)

[Readonly Object Properties](#)

[Exercise 7-1: Inference with an Array of Objects](#)

[Exercise 7-2: Avoiding Array Mutation](#)

[Exercise 7-3: An Unsafe Tuple](#)

[Deep Immutability with as const](#)

[as const vs. Variable Annotation](#)

[as const vs. Object.freeze](#)

[Exercise 7-4: Inferring Literal Values in Arrays](#)

[Summary](#)

8

CLASSES

[Creating a Class](#)

[Adding a Constructor](#)

[Adding Arguments to the Constructor](#)

[Using a Class as a Type](#)

[Properties in Classes](#)

[Class Property Initializers](#)

[readonly Class Properties](#)

[Optional Class Properties](#)

[public and private Properties](#)

[Class Methods](#)

[Class Inheritance](#)

[Extending a Class](#)

[protected Properties](#)

[Safe Overrides with override](#)

[The implements Keyword](#)

[Abstract Classes](#)

[Abstract Methods](#)

[Exercise 8-1: Creating a Class](#)

[Exercise 8-2: Implementing Class Methods](#)

[Exercise 8-3: Implementing a Getter](#)

[Exercise 8-4: Implementing a Setter](#)

[Exercise 8-5: Extending a Class](#)

[Summary](#)

9

TYPESCRIPT-ONLY FEATURES

[Class Parameter Properties](#)

[Enums](#)

[Numeric Enums](#)

[String Enums](#)

[Enums Are Strange](#)

[Should You Use Enums?](#)

[Namespaces](#)

[How Namespaces Compile](#)

[Merging Namespaces](#)
[Should You Use Namespaces?](#)
[Type-Only Namespaces](#)
[When to Use ECMAScript vs. TypeScript](#)
[The Future of TypeScript: Erasable Syntax Only](#)
[Summary](#)

PART IV: WORKING WITH THE COMPILER

10

DERIVING TYPES

[Deriving Types from Other Types](#)

[The keyof Operator](#)

[The typeof Operator](#)

[You Can't Create Values from Types](#)

[Indexed Access Types](#)

[Chaining Multiple Indexed Access Types](#)

[Passing a Union to an Indexed Access Type](#)

[Getting an Object's Values with keyof](#)

[Using as const for JavaScript-Style Enums](#)

[Enums Require You to Pass the Enum Value](#)

[Enums Are Nominal](#)

[Which Approach Should You Use?](#)

[Exercise 10-1: Reducing Key Repetition](#)

[Exercise 10-2: Deriving a Type from a Value](#)

[Exercise 10-3: Accessing Specific Values](#)

[Exercise 10-4: Unions with Indexed Access Types](#)

[Exercise 10-5: Extract a Union of All Values](#)

[Exercise 10-6: Creating a Union from an as const Array](#)

[Deriving Types from Functions](#)

[Parameters](#)

[ReturnType](#)

[Awaited](#)

[Why Derive Types from Functions?](#)

[Exercise 10-7: A Single Source of Truth](#)

[Exercise 10-8: Typing Based on a Return Value](#)

[Exercise 10-9: Unwrapping a Promise](#)

[Transforming Derived Types](#)

[Exclude](#)

[NonNullable](#)

[Extract](#)

[Deriving vs. Decoupling](#)

[When Decoupling Makes Sense](#)

[When Deriving Makes Sense](#)

[Summary](#)

11

ANNOTATIONS AND ASSERTIONS

[Annotating Variables vs. Values](#)

[Annotating Values with satisfies](#)

[Narrowing Values with satisfies](#)

[Assertions: Forcing the Type of Values](#)

[The as Assertion](#)

[The Non-Null Assertion](#)

[Error Suppression Directives](#)

[@ts-expect-error](#)

[@ts-ignore](#)

[@ts-nocheck](#)

[Suppressing Errors vs. as any](#)

[When to Suppress Errors](#)

[When You Know More Than TypeScript](#)

[When TypeScript Is Being “Dumb”](#)

[When You Don’t Understand the Error](#)

[**Exercise 11-1: Providing Additional Info to TypeScript**](#)

[**Exercise 11-2: Solving Issues with Assertions**](#)

[**Exercise 11-3: Enforcing a Valid Configuration**](#)

[**Exercise 11-4: Variable Annotation vs. as vs. satisfies**](#)

[**Exercise 11-5: Creating a Deeply Read-Only Object**](#)

[Summary](#)

12

THE WEIRD PARTS

[The Evolving any Type](#)

[Excess Property Warnings](#)

[No Excess Property Checks on Variables](#)

[No Excess Property Checks When Comparing Functions](#)

[Open vs. Closed Object Types](#)

[Fresh and Stale Objects](#)

[Object Keys Are Loosely Typed](#)

[The Empty Object Type](#)

[The Type and Value Worlds](#)

[Classes Can Cross Between Worlds](#)

[Enums Can Cross Between Worlds](#)

[The this Keyword Can Cross Between Worlds](#)

[Naming Types and Values the Same](#)

[Using this in Functions](#)

[Arrow Functions Don’t Support this](#)

[Function Assignability](#)

[Unions of Functions Intersect Parameters](#)

[**Exercise 12-1: Accepting Anything Except null and undefined**](#)

[**Exercise 12-2: Detecting Excess Properties in an Object**](#)

[**Exercise 12-3: Detecting Excess Properties in a Function**](#)

[**Exercise 12-4: Iterating over Objects**](#)

[Exercise 12-5: Function Parameter Comparisons](#)

[Exercise 12-6: Unions of Functions with Object Params](#)

[Exercise 12-7: Unions of Functions with Incompatible Parameters](#)

[Summary](#)

PART V: UNDERSTANDING THE ENVIRONMENT

13

MODULES, SCRIPTS, AND DECLARATION FILES

[Understanding Modules and Scripts](#)

[Modules Have Local Scope](#)

[Scripts Have Global Scope](#)

[TypeScript Guesses Which to Use](#)

[Forcing Modules with moduleDetection](#)

[Declaration Files](#)

[Declaration Files Describe JavaScript](#)

[Declaration Files Can Add to the Global Scope](#)

[Declaration Files Can't Contain Implementations](#)

[The declare Keyword](#)

[Typing Global Variables](#)

[Scoping Global Variables to One File](#)

[More Ways to declare](#)

[Module Augmentation vs. Module Overriding](#)

[Declaration Files You Don't Control](#)

[TypeScript's Types](#)

[DOM Types](#)

[Types That Ship with Libraries](#)

[DefinitelyTyped](#)

[The skipLibCheck Config Option](#)

[Authoring Declaration Files](#)

[Should You Store Your Types in Declaration Files?](#)

[Is Using Global Types a Good Idea?](#)

[**Exercise 13-1: Typing a JavaScript Module**](#)

[**Exercise 13-2: Ambient Context**](#)

[**Exercise 13-3: Modifying window**](#)

[**Exercise 13-4: Modifying process.env**](#)

[Summary](#)

14

CONFIGURING TYPESCRIPT

[Recommended Configuration](#)

[Additional Configuration Options](#)

[The Complete Base Configuration](#)

[Base Options](#)

[target](#)

- [esModuleInterop](#)
 - [isolatedModules](#)
- [Strictness Options](#)
 - [noUncheckedIndexedAccess](#)
 - [Other Strictness Options](#)
- [The Two Choices for module](#)
 - [NodeNext](#)
 - [Preserve](#)
- [noEmit](#)
- [Source Maps](#)
- [Transpiling Code for Library Use](#)
 - [outDir](#)
 - [Creating Declaration Files](#)
 - [Declaration Maps](#)
- [jsx](#)
- [A Note on Node's TypeScript Support](#)
- [Managing Multiple TypeScript Configurations](#)
 - [How TypeScript Finds tsconfig.json](#)
 - [Extending Configurations](#)
 - [--project](#)
 - [Project References](#)
- [Summary](#)

PART VI: ADVANCED APPLICATION DEVELOPMENT

15

DESIGNING YOUR TYPES

- [Generic Types](#)
 - [Multiple Type Parameters](#)
 - [All Type Arguments Must Be Provided](#)
 - [Default Type Parameters](#)
 - [Type Parameter Constraints](#)
- [Template Literal Types in TypeScript](#)
 - [Combining Template Literal Types with Union Types](#)
 - [Transforming String Types](#)
- [Conditional Types](#)
- [Mapped Types](#)
 - [Key Remapping with as](#)
 - [Using Mapped Types with Union Types](#)
 - [Exercise 15-1: Creating a DataShape Type Helper](#)**
 - [Exercise 15-2: Typing PromiseFunc](#)**
 - [Exercise 15-3: Working with the Result Type](#)**
 - [Exercise 15-4: Constraining the Result Type](#)**
 - [Exercise 15-5: A Stricter Omit Type](#)**

[Exercise 15-6: Route Matching](#)

[Exercise 15-7: Sandwich Permutations](#)

[Exercise 15-8: Attribute Getters](#)

[Exercise 15-9: Renaming Keys in a Mapped Type](#)

[Summary](#)

16

BUILDING POWERFUL SHARED UTILITIES

[Generic Functions](#)

[Generic Function Type Alias vs. Generic Type](#)

[Missing or Conflicting Type Arguments](#)

[There Is No Such Thing as a “Generic”](#)

[The Problem Generic Functions Solve](#)

[Debugging the Inferred Type of Generic Functions](#)

[Type Parameter Defaults](#)

[Constraining Type Parameters](#)

[Type Predicates](#)

[Assertion Functions](#)

[Function Overloads](#)

[The Implementation Signature](#)

[Function Overloads vs. Unions](#)

[Exercise 16-1: Making a Function Generic](#)

[Exercise 16-2: Default Type Arguments](#)

[Exercise 16-3: Inference in Generic Functions](#)

[Exercise 16-4: Type Parameter Constraints](#)

[Exercise 16-5: Combining Generic Types and Functions](#)

[Exercise 16-6: Multiple Type Arguments in a Generic Function](#)

[Exercise 16-7: Assertion Functions](#)

[Summary](#)

INDEX

TOTAL TYPESCRIPT

The Essentials

by Matt Pocock with Taylor Bell



**no starch
press®**

San Francisco

TOTAL TYPESCRIPT. Copyright © 2026 by Matt Pocock with Taylor Bell.

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner and the publisher.

First printing

30 29 28 27 26 1 2 3 4 5

ISBN-13: 978-1-7185-0416-5 (print)

ISBN-13: 978-1-7185-0417-2 (ebook)



® Published by No Starch Press®, Inc.
245 8th Street, San Francisco, CA 94103
phone: +1.415.863.9900
www.nostarch.com;
info@nostarch.com

Publisher: William Pollock

Managing Editor: Jill Franklin

Production Manager: Sabrina Plomitallo-González

Production Editor: Sydney Cromwell

Developmental Editor: Jill Franklin

Cover Illustrator: Rob Fiore

Interior Design: Octopod Studios

Technical Reviewer: Titian Cernicova-Dragomir

Copyeditor: Kim Wimpsett

Proofreader: Audrey Doyle

Indexer: BIM Creatives, LLC

Library of Congress Cataloging-in-Publication Data

Names: Pocock, Matt author | Bell, Taylor, (Computer science researcher) author

Title: Total Typescript / by Matt Pocock with Taylor Bell.

Description: San Francisco, CA : No Starch Press, [2026] | Includes index.

Identifiers: LCCN 2025048382 (print) | LCCN 2025048383 (ebook) | ISBN 9781718504165
print | ISBN 9781718504172 ebook

Subjects: LCSH: TypeScript (Computer program language)

Classification: LCC QA76.73.T97 P63 2026 (print) | LCC QA76.73.T97 (ebook)

LC record available at <https://lcn.loc.gov/2025048382>

LC ebook record available at <https://lcn.loc.gov/2025048383>

For customer service inquiries, please contact info@nostarch.com. For information on distribution, bulk sales, corporate sales, or translations: sales@nostarch.com. For permission to translate this work: rights@nostarch.com. To report counterfeit copies or piracy: counterfeit@nostarch.com.

The authorized representative in the EU for product safety and compliance is EU Compliance Partner, Pärnu mnt. 139b-14, 11317 Tallinn, Estonia, hello@eucompliancepartner.com, +3375690241.

No Starch Press and the No Starch Press iron logo are registered trademarks of No Starch Press, Inc. Other product and company names mentioned herein may be the trademarks of their respective owners. Rather than use a trademark symbol with every occurrence of a trademarked name, we are using the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of this work, neither the authors nor No Starch Press, Inc. shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in it.

For Louise and Isaac

About the Authors

Matt Pocock is a full-time TypeScript educator, having worked as a developer and educator for Vercel and Stately.ai. He teaches TypeScript workshops, builds online courses, and watches horses gallop by his office in Oxfordshire, England.

Taylor Bell has worked as a technical copywriter, researcher, developer, and associated roles in between. He studied communication theory and computer science at Boise State University in his hometown of Boise, Idaho.

About the Technical Reviewer

Titian Cernicova-Dragomir is a developer on the JavaScript infrastructure team at Bloomberg, where he works on JavaScript and TypeScript tooling for internal developers. He's passionate about TypeScript and is an active member of the TypeScript community on Stack Overflow. Cernicova-Dragomir is also a TypeScript compiler contributor and recently contributed the implementation of ES class private methods and private static members.

FOREWORD

In early 2012, I joined a small team of engineers at Microsoft that was tasked with improving the JavaScript development experience for projects across the company. The concept was simple: Create a language, based on JavaScript, that added gradual static typing, and use that static typing to empower developers to code more quickly, more confidently, and more accurately. At the time, static typing was out of fashion among most JavaScript developers. I figured that if we did a fantastic job, we might eventually gain mindshare among 20 percent of developers, maybe a little more in larger projects.

Our initial release was met with guarded skepticism. Was Microsoft *really* turning over a new leaf and embracing open source? Could TypeScript find a way to describe the behavior of all our favorite libraries? Wasn't the chaos of JavaScript half the fun of using it?

At the same time, we saw a tremendous enthusiasm start to form. TypeScript was the solution that so many people had been looking for. They saw great potential in the project and realized what it might eventually be able to deliver. A vibrant community of developers, content creators, authors, and commentators began to form.

Thirteen years later, TypeScript has become the de facto standard for writing modern JavaScript, with virtually 100 percent adoption in large projects and three quarters of JavaScript developers using TypeScript for most if not all of their code. It's been an incredible journey to see TypeScript grow from a small moonshot project to a truly dominant player in the language space.

The community has expanded alongside this growth. Much like I never believed we'd have a majority share of JavaScript developers, I've been

pleasantly surprised to see the incredible network of TypeScript educators, influencers, and creators grow around our project.

I've always been so energized by that community. TypeScript would not be where it is today without the intelligent, engaged, and thoughtful contributions from so many different people from around the world. We stand not just on the shoulders of giants but on a foundation of thousands of experts and enthusiasts who want to make JavaScript development a better experience.

Matt Pocock in particular stands out for his tireless work to explain TypeScript. His focus on approachable, comprehensive, and pragmatic TypeScript education has been an invaluable resource to countless developers. This book takes that work to the next level, providing a clear and concise overview of the language. What I've always appreciated about Matt's work as a technical writer is that he's unafraid to stake out an opinion and consistently displays good taste. (In other words, he tends to agree with me!)

TypeScript development offers a huge amount of flexibility, and using the language effectively is as much art as science. As an exercise for the reader, I offer you an additional challenge: As you read the book, think about the trade-offs we make every time we write a line of code. Do we want to be explicit or concise? Are error cases handled immediately or deferred? How do we balance the precision of types we write against the complexity of those types? The text of the book offers excellent guidance in these matters, often more implicitly than explicitly. Test yourself for mastery of these concepts by seeing if you can realize alternative solutions to the tasks here that vary on these axes.

My hope is that you take away from this book not just an understanding of TypeScript as a language but also a feel for its community. The way we balance safety, flexibility, structure, creativity, and so much more is one of the things that make the language such an expressive joy to work with.

Happy coding!

Ryan Cavanaugh
Development Lead for the TypeScript Team at Microsoft
Seattle, Washington

INTRODUCTION



If you were building JavaScript apps in the 2000s, you were probably having a bad time. Users were demanding more interactive experiences on the web. Developers were building large, complex web apps. And JavaScript, the language of the web, began to creak under the strain.

Back then, the experience of writing JavaScript was not particularly fun. Editor features that are now commonplace, like autocomplete and error highlighting, often weren't available. As applications grew more and more complex, working with JavaScript became a nightmare.

This came to a head at Microsoft, where our story starts.

The Birth of TypeScript

Around 2010, Microsoft noticed something strange. Some of its teams were using a community project called Script# (ScriptSharp) to build JavaScript apps. The Script# library allowed for developers to write code in C#, which would then be transformed into JavaScript.

There are a lot of benefits to writing code in a strongly typed language like C#. When you make silly errors like misspelling a variable name, you'll get a warning right away instead of finding out when the build doesn't work. More importantly, refactoring code in a strongly typed language is much simpler. If you needed to change how a function was called, this could be done in a few clicks instead of a few hours. By comparison, writing in JavaScript felt like carving your code in stone tablets. Once written, it was hard to change.

Microsoft tasked Anders Hejlsberg, the creator of C#, with investigating Script#. He was surprised to find that people were so annoyed with JavaScript that they were willing to code in a completely different language to avoid it. However, he found the combination of C# and the Script# library bizarre. Wouldn't it make more sense to create a new language that was closer to JavaScript and enabled developers to use all the IDE features that JavaScript was missing?

And so, TypeScript was born.

In the decade or so since its introduction, TypeScript has become a staple of modern development. It's been adopted by huge companies like Google, Netflix, and Airbnb. Teams that use TypeScript ship better software at greater speed—and are happier as a result.

Why Now?

We are in a moment of enormous change, both for developers and for TypeScript. For developers, artificial intelligence is revolutionizing the way we work. AI is finding a home in our IDEs, helping us write code faster than ever. But one thing is clear: Without good feedback loops, AI can quickly lose its way. This makes TypeScript, which provides a fast feedback loop *inside* the IDE, a natural fit for AI-driven development. Most IDEs are already utilizing TypeScript's error checking to improve their AI agents, and I feel this space is still underexplored.

But TypeScript itself is in flux. In 2025, the TypeScript team announced it was porting TypeScript's type checker to Go, promising a 10× speedup for autocomplete, in-IDE errors, and project type checking. A faster TypeScript means faster feedback loops, especially in large projects. And it has TypeScript developers extremely excited.

Learning TypeScript has rarely been more appealing. You'll be coding in a language that AI loves to use. You'll ship better code, faster than before.

Learning TypeScript *totally* changed my career. I hope it will do the same for you.

Who Is This Book For?

The aim of this book is to build your confidence with TypeScript and to teach all the essential things you'll need to know to build a TypeScript application. You'll need to have some experience with JavaScript, but that's it. No TypeScript experience is necessary.

TypeScript is used both on the frontend (on the web, and on desktop and mobile apps) *and* on the backend (in APIs, scripts, and much more). We'll be looking at the core language features that help you build *any* project, instead of focusing on one over the other.

We'll be starting from the basics, but we won't stop there. By the end of the book, you'll feel like a TypeScript wizard. You'll be able to work *with* TypeScript instead of against it, quickly diagnosing errors and moving faster than you ever did with JavaScript.

What We'll Cover

We'll be covering everything you need to build a TypeScript product. Beyond learning the basic syntax, we'll be building mental models for how it works at a deeper level. You'll get useful patterns you can immediately apply in your projects, and you'll learn helpful workarounds for TypeScript's most common pitfalls.

We believe the best time to learn something is the moment you need to apply it practically. So, this book contains dozens of exercises that you can code as you follow along. These exercises are battle-tested in Matt Pocock's live workshops at <https://www.totaltypescript.com>. Every TypeScript language feature solves a specific need, and the exercises help link the knowledge with the instant you need it.

We've been very intentional with the ordering of what gets taught when. We'll start from a solid foundation and move through to the level of the TypeScript wizard. Here's what each chapter will cover:

Chapter 1: Kickstart Your TypeScript Setup We'll start by showing you how to set up a TypeScript development environment on your machine.

Chapter 2: IDE Superpowers TypeScript enables dozens of amazing features in your IDE. We'll go through them one by one and

look at our first pieces of TypeScript code.

[Chapter 3: TypeScript in the Development Pipeline](#) TypeScript is not just a language; it's also a set of tools that give you confidence in the code you're writing. We'll examine TypeScript's CLI and how TypeScript typically works on production applications.

[Chapter 4: Essential Types and Annotations](#) We'll explain how to add types to functions, arrays, and objects, and how to make reusable types with type aliases.

[Chapter 5: Unions, Literals, and Narrowing](#) We'll discuss union types and literal types, which let you express OR logic in your functions. We'll also look at narrowing: how TypeScript can understand your code to give you better warnings.

[Chapter 6: Objects](#) We'll teach you how to express object types, including index signatures, Record, and interfaces.

[Chapter 7: Mutability](#) We'll look at `let`, `const`, and `as const`, and at how mutability (whether a value can be changed later) affects how TypeScript understands your code.

[Chapter 8: Classes](#) We'll teach you how understanding classes helps you harness classic object-oriented patterns in your code.

[Chapter 9: TypeScript-Only Features](#) We'll take a peek at enums and namespaces and look at why you may not want to use them in your code.

[Chapter 10: Deriving Types](#) We'll use `keyof`, `typeof`, and more to create types from other types (or even from values). We'll see how these tools can massively cut down on duplicated code.

[Chapter 11: Annotations and Assertions](#) We'll examine `satisfies`, `as`, and error directives, and how you can leverage them for better type safety (or even to lie to TypeScript).

[Chapter 12: The Weird Parts](#) In some situations, TypeScript can behave quite strangely. We'll examine the mental models you need to understand these oddities.

[Chapter 13: Modules, Scripts, and Declaration Files](#) We'll cover how to add types to the global scope, as well as `.d.ts` files and the DOM

typings.

Chapter 14: Configuring TypeScript We'll dive into *tsconfig.json* project references and describe how to configure TypeScript for any situation.

Chapter 15: Designing Your Types We'll look at TypeScript's powerful features for designing your own types: generic types, mapped types, template literals, and conditional types.

Chapter 16: Building Powerful Shared Utilities We'll provide information on all the tools you need to build powerful shared utilities in your code: generic functions, function overloads, and more.

Online Resources

- The book repository: <https://github.com/total-typescript/total-typescript-book>
- Total TypeScript free tutorials: <https://www.totaltypescript.com/tutorials>
- Total TypeScript articles: <https://www.totaltypescript.com/articles>
- TypeScript Official Handbook: <https://www.typescriptlang.org/docs/handbook/intro.html>

The Book's Exercises

Most chapters in the book have practical exercises designed to test your understanding of the material. You can run the exercises by visiting the GitHub repository for the book at <https://totalts.link/e-repo>. There, you'll find instructions for cloning the repository to your local machine, installing the dependencies, and running the exercises.

Each exercise in the book will have a link to an online version on our site, <https://www.totaltypescript.com>. There will also be a handy button to open the exercise in your local editor.

Most of the exercises will involve this process:

1. Read the instructions.
2. Open the code in your local editor.

3. Start the exercise command line interface (CLI), which checks your work as you go.
4. Fix the TypeScript errors and runtime errors until the exercise is complete.
5. Check your work against my solution.

Let's quickly discuss a couple of conventions and tools used in the exercises.

@ts-expect-error

TypeScript checks your code in your editor for things it thinks might fail when the code is run. In your IDE, it shows these issues as red squiggly lines under the code.

For instance, here we are calling a method on `null`:

```
const x = null;  
x.toUpperCase(); Error: Property 'toUpperCase' does not exist on type 'null'.
```

TypeScript also provides a directive to tell it to *expect* an error on the following line:

```
const x = null;  
  
// @ts-expect-error  
x.toUpperCase();
```

Now the error no longer appears, because TypeScript is expecting it.

This is useful as a teaching tool, since it allows me to write exercises that *expect* errors. So, if you see an `@ts-expect-error` comment, we are *expecting* a TypeScript error on the next line.

Vitest

TypeScript checks your code only in your editor. To make the exercises as realistic as possible, I've also added a few runtime tests. These tests are

written with Vitest, a popular testing framework for TypeScript. We're not using many complex Vitest features, only a few simple ones.

For instance, this test checks whether $1 + 1$ equals 2:

```
import {expect, it} from "vitest";

it("1 + 1 = 2", () => {
  expect(1 + 1).toBe(2);
});
```

In the preceding code, `it` is a function used to define a test, and `expect` is a function used to make assertions.

The exercise CLI will automatically run the tests as you work through the exercises.

PART I

GETTING STARTED

1

KICKSTART YOUR TYPESCRIPT SETUP



In this chapter, we'll provide a whirlwind tour of TypeScript essentials to get you up and running. We'll first briefly discuss what makes TypeScript different from JavaScript. We'll then give a broad overview of how the language works. We'll also cover the tools you'll need to start working with TypeScript.

What's Different About TypeScript?

What makes TypeScript different from JavaScript can be summed up in a single word: *types*. But there's a common misconception here. People think TypeScript's core mission is to make JavaScript a strongly typed language, like C# or Rust, which is not quite accurate. TypeScript wasn't invented to make JavaScript strongly typed. It was built to allow amazing *tooling* for JavaScript.

Imagine you're building an integrated development environment (IDE) and you want to warn people when they mistype a function name or object property. If you don't know the shapes of the variables, parameters, and objects in your code, you'll have to resort to guesswork. But if you *do* know the types of everything in your app, you can begin implementing powerful IDE features like autocomplete, inline errors, and automatic refactors.

TypeScript aims to provide *just enough* strong typing to make working with JavaScript more pleasant and productive.

A High-Level View of How TypeScript Works

With a JavaScript-only project, you would typically write your code in files with `.js` file extensions. Those files are then able to be directly executed in the browser or a runtime environment like Node.js (which is used to run JavaScript on servers or on your laptop). The JavaScript you write is the JavaScript that gets executed, as illustrated in [Figure 1-1](#).

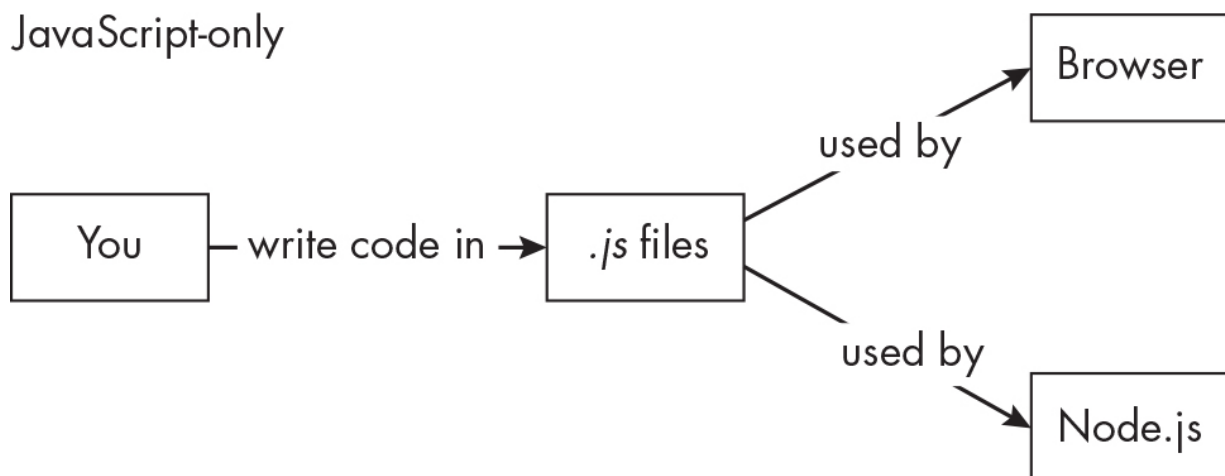


Figure 1-1: An overview of a JavaScript-only workflow

If you're testing whether your code works, you need to test it inside the runtime, which is the browser or Node.js.

For a TypeScript project, your code is primarily in `.ts` or `.tsx` files. In your IDE, those files are monitored by TypeScript's *language server*. This server watches as you type and powers IDE features like autocompletion and error checking, among others.

Unlike `.js` files, `.ts` files can't usually be executed directly by the browser or a runtime. Instead, they require an initial build process. This is where TypeScript's `tsc` CLI comes in. It transforms your `.ts` files into `.js` files, which means you are able to take advantage of TypeScript's features while writing your code, but the output is still plain JavaScript, as shown in [Figure 1-2](#).

TypeScript

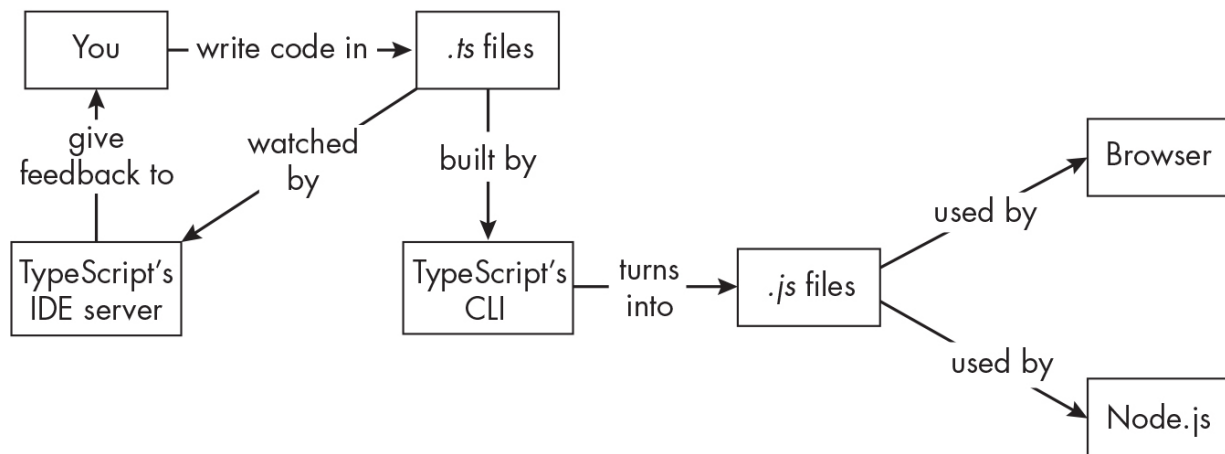


Figure 1-2: An overview of how a TypeScript project works

The great benefit of this system is that you get into a *feedback loop* with TypeScript: You write code. The in-IDE server gives you feedback. You adjust based on the feedback. In addition, all of this happens *before* your code goes into the browser. In JavaScript, you need to go all the way to the runtime before you can detect any defects. TypeScript's loop happens entirely within your IDE, so it can help you create higher-quality code more quickly.

NOTE

Automated testing can also provide a high-quality feedback loop. While we won't cover automated tests in this book, they're a great companion to TypeScript for creating extremely high-quality code.

Tools for TypeScript Development

Let's break down the tools you need to work with TypeScript:

IDE To write code, you need an editor or IDE. While you can use any IDE, the assumption in this book is that you are using Microsoft's Visual Studio Code (VS Code). The TypeScript integration with VS Code is excellent, as you will see shortly. Install it from <https://code.visualstudio.com/download> if you haven't already. It'll work on Windows, macOS, or Linux.

Execution environment You need a place to run your emitted JavaScript, such as Node.js or a web browser like Chrome.

TypeScript CLI You need Node.js to run the TypeScript CLI. This tool converts your TypeScript to JavaScript and warns you of any issues in your project.

Installing Node.js

Download the Node.js installer from <https://nodejs.org>. When you visit the site, you'll see two options: LTS and Current. *LTS* is short for *Long Term Support* and is the recommended version for production use. It's also the most stable version and the one we'll be using in this book. The Current version contains the latest features, but it's not recommended for production use.

Click the **LTS** button to download the installer, and follow the installation instructions.

After running the installer, you can verify it installed correctly by opening a terminal and running the following command:

```
node -v
```

If Node.js is installed correctly, this command should display the version number. If you see an error message like `node command not found`, it means the installation wasn't successful, and you should try again.

Installing the pnpm Package Manager

To run the exercises, you'll need a tool that helps install some third-party packages. Instead of using the traditional `npm` package manager, we'll be using a library called `pnpm`.

Node.js includes the `npm` package manager by default. If you've worked with JavaScript repositories, you're likely familiar with `npm` and the `package.json` file. The `package.json` file represents all the packages you need to install to run the code in the repository. For example, in the repository for this material, we have a special CLI for running exercises along with helper packages and various other dependencies like `cross-fetch` and `nodemon`:

```
// Inside package.json
{
  "devDependencies": {
    "@total-typescript/exercise-cli": "0.4.0",
    "@total-typescript/helpers": "~0.0.1",
    "cross-fetch": "~3.1.5",
    "nodemon": "~3.0.1",
    "npm-run-all": "~4.1.5",
    "prettier": "~2.8.7",
    "typescript": "~5.2.2",
    "vite-tsconfig-paths": "~4.0.7",
    "vitest": "0.34.4"
  }
}
```

To install these packages, you would typically run the `npm install` command, which downloads them from the npm registry into your `node_modules` folder. The `node_modules` folder contains JavaScript files that the exercises in the `src` directory need to run.

However, for the book's repository, we will be using the `pnpm` package manager instead. It works the same way as `npm`, but it's more efficient. Instead of having individual `node_modules` folders for each project, `pnpm` uses a single location on your computer and hard-links the dependencies from there, which makes it run faster and use less disk space. We use `pnpm` in all our projects. To install `pnpm`, follow the instructions provided in the official documentation at <https://pnpm.io/installation>.

Once it's installed, you can check it's working by running `pnpm --version`.

Installing TypeScript

TypeScript and its dependencies are contained within a single package, called `typescript`. You can install it on your system with `npm`:

```
npm install --global typescript
```

By adding `--global`, you're adding the `tsc` CLI to your system globally. This means you'll be able to run `tsc` on your command line to perform various tasks, like creating TypeScript config files and checking your code.

TypeScript is usually also installed locally in your *projects* folder to make sure that all developers using the project are using the same version. For the purposes of this book, a global installation will do just fine.

Summary

This chapter introduced TypeScript as a tool designed primarily to enable amazing JavaScript tooling rather than to make JavaScript strongly typed. TypeScript provides just enough typing to make development more pleasant and productive through IDE features like autocomplete and inline errors.

The development workflow differs from JavaScript in that you write code in `.ts` files, which are monitored by TypeScript's language server for real-time feedback. These files must be compiled to `.js` files using the `tsc` CLI before execution, creating a faster feedback loop entirely within your IDE.

To get started with TypeScript development, you need an IDE like VS Code, an execution environment like Node.js or a browser, and the TypeScript CLI. The chapter walked through installing Node.js (preferring the LTS version), the `pnpm` package manager for better efficiency than `npm`, and TypeScript globally using `npm`.

The key advantage of this setup is getting immediate feedback while coding, allowing you to catch and fix issues before your code reaches the runtime environment and ultimately leading to higher-quality code development.

2

IDE SUPERPOWERS



TypeScript works the same no matter what IDE you use, but for this book, we'll assume you're using VS Code. When you open VS Code, the TypeScript server starts in the background. It will be active as long as you have a TypeScript file open. In this chapter, we'll introduce some of the awesome VS Code features that are powered by the TypeScript server.

Autocomplete

If we were to name the single TypeScript feature we couldn't live without, it would be autocomplete. TypeScript knows what type everything is inside your app. Because of that, it can give you suggestions when you're typing, speeding up your work enormously.

In the following example, just entering `p` after `audioElement` brings up all the properties that start with *p*:

```
const audioElement = document.createElement("audio");  
  
audioElement.p; // Suggests play, pause, part, and so on
```

This feature is really powerful for exploring APIs you might not be familiar with, like the `HTMLAudioElement` API in this case.

Manual Autocomplete

You'll often want to trigger autocomplete manually. In VS Code, the `CTRL-spacebar` keyboard shortcut will list suggestions for what you're typing. For example, if you were adding an event listener to an element, you would see a list of available events:

```
document.addEventListener(  
    "", // Autocomplete here  
);
```

Pressing `CTRL-spacebar` with the cursor inside the quotation marks will show a list of events that can be listened for:

```
DOMContentLoaded  
abort  
animationcancel  
. . .
```

If you want to narrow down the list to the events you were interested in, enter **drag** before pressing `CTRL-spacebar`; only the appropriate events will display:

```
drag  
dragend  
dragenter  
dragleave  
. . .
```

Autocomplete is an essential tool for writing TypeScript code, and it's available right out of the box in VS Code.

Exercise 2-1: Autocomplete

Here's an example of some code where autocomplete can be triggered:

```
const acceptsObj = (obj: {foo: string; bar: number; baz: boolean}) => {};  
  
acceptsObj({  
  // Autocomplete in here!  
});
```

Practice using the autocomplete shortcut to fill in the object when calling `acceptsObj`.

See <https://totalts.link/essentials-2-1/>.

Solution

When you press CTRL-spacebar inside the object, you'll see a list of the possible properties based on the `{foo: string; bar: number; baz: boolean}` type:

```
bar;  
baz;  
foo;
```

As you select each property, the autocomplete list will update to show you the remaining properties.

TypeScript Error Checking

TypeScript is most famous for its errors. These are a set of rules that TypeScript uses to make sure your code is doing what you think it's doing. With every change you make to a file, the TypeScript server checks your code.

If the server finds any errors, it tells VS Code to draw a red squiggly line under the part of the code that has a problem. Hovering over the underlined code shows you the error message. Once you make a change, the TypeScript server checks the code again and removes the red squiggly line if the error is fixed. You can think of it like a teacher hovering over your shoulder, underlining your work in red pen while you type.

Let's look at these errors a bit more deeply.

Runtime Errors

Sometimes TypeScript warns you about things that will definitely fail at runtime. For example, consider the following code:

```
const a = null;  
  
a.toString(); // red squiggly line under a
```

TypeScript tells you that there is a problem with `a`. Hovering over it shows the following error message:

```
'a' is possibly 'null'.
```

This tells you *where* the problem is, but it doesn't necessarily tell you *what* the problem is. In this case, you need to stop and think about why you can't call `toString()` on `null`. If you do, it will throw an error at runtime:

```
Uncaught TypeError: Cannot read properties of null (reading  
  'toString').
```

Here, TypeScript tells you that an error might happen even without you needing to run the code—very handy.

Non-Runtime Errors

Not everything TypeScript warns you about will actually fail at runtime. Take a look at the following example where we assign a property to an empty object:

```
const obj = {};  
  
const result = obj.foo; // red squiggly line under foo
```

TypeScript draws a red squiggly line below `foo`. But if you think about it, this code won't actually cause an error at runtime. We're trying to access a property that doesn't exist in this object: `foo`. This won't error; it will just mean that the result is undefined.

It may seem strange that TypeScript would warn you about something that won't cause a runtime error, but it's actually a good thing. If TypeScript didn't warn you about this, it would be saying that you can access any property on any object at any time. Over the course of an entire application, this could result in quite a few bugs.

It's best to think of TypeScript's rules as opinionated. They are a collection of helpful tips that will make your application safer as a whole.

Warning Locations

TypeScript will try to provide warnings as close to the source of the problem as possible. Let's take a look at an example:

```
type Album = {  
  artist: string;  
  title: string;  
  year: number;  
};  
  
const album: Album = {  
  artsist: "Television", // red squiggly line under artsist  
  title: "Marquee Moon",  
  year: 1977,  
};
```

We define an Album type, which is an object with three properties. Then, we say that `const album` needs to be of that type via `const album: Album`. Don't worry if you don't understand all the syntax yet; we'll cover it all later.

Can you see the problem? There's a typo on the artist property when creating an album. Hovering over `artsist` shows the following error message:

```
Type '{artsist: string; title: string; year: number;}' is no  
t assignable to type 'Album'.
```

That's because we've said that the album variable needs to be of type Album, but we've misspelled artist as artsist. TypeScript is saying that

we've made a mistake and even suggests the correct spelling.

Multiline Errors

Sometimes TypeScript will give you an error in a less helpful spot. In the following example, we have a function called `logUserJobTitle` that logs the job title of a user:

```
const logUserJobTitle = (user: {  
  job: {  
    title: string;  
  };  
}) => {  
  console.log(user.job.title);  
};
```

The important thing is that `logUserJobTitle` takes a user object with a `job` property that has a `title` property.

Now let's call `logUserJobTitle` with a user object where the `job.title` is a number, not a string:

```
const exampleUser = {  
  job: {  
    title: 123,  
  },  
};  
  
logUserJobTitle(exampleUser); // red squiggly line under exampleUser
```

It might seem like TypeScript should give an error on `title` in the example `User` object. But instead, it gives an error on the `exampleUser` variable itself.

Hovering over `exampleUser` shows the following error message:

```
Argument of type '{job: {title: number;}}' is not assignable to  
parameter of type '{job: {title: string;}}'.
```

The types of 'job.title' are incompatible between these types.

Type 'number' is not assignable to type 'string'.

It's multiple lines long, which can feel pretty scary. A good rule of thumb with multiline errors is to start at the bottom:

Type 'number' is not assignable to type 'string'.

This tells you that a number is being passed into a slot where a string is expected, which is the root of the problem.

Let's go to the next line:

The types of 'job.title' are incompatible between these types.

This says that the title property in the job object is the problem, and we already understand the issue without needing to see the top line, which is usually a long summary of the problem. Reading errors from bottom to top can be a helpful strategy when dealing with multiline TypeScript errors.

Introspecting Variables and Declarations

You can hover over more than just error messages. Any time you hover over a variable or declaration, VS Code will show you information about it.

In this example, you could hover over `thing` and see that it's of type `number`:

```
let thing = 123;
```

```
// hovering over thing shows:
```

```
let thing: number;
```

Hovering works for more involved examples as well. Here, `otherObject` spreads in the properties of `otherThing` as well as adding `thing`:

```
let otherThing = {  
  name: "Alice",  
};  
  
const otherObject = {  
  . . .otherThing,  
  thing,  
};  
otherObject.thing;
```

Hovering over `otherObject` gives us a computed readout of all of its properties:

```
// hovering over otherObject shows:  
  
const otherObject: {  
  thing: number;  
  name: string;  
};
```

Depending on what you hover over, VS Code shows different information. For example, hovering over `.thing` in `otherObject.thing` shows the type of `thing`:

```
(property) thing: number
```

Get used to the ability to float around your codebase introspecting variables and declarations because it's a great way to understand what the code is doing.

Exercise 2-2: Hovering Over a Function Call

In the following code snippet, we're trying to grab an element using `document.getElementById` with an ID of 12. However, TypeScript is complaining:

```
let element = document.getElementById(12); // red squiggly line under 12
```

How can hovering help determine what argument `document.getElementById` actually requires? And for a bonus point, what type is `element`?

See <https://totalts.link/essentials-2-2/>.

Solution

First, you can hover over the red squiggly error itself. In this case, hovering over 12 gives you the following error message:

```
Argument of type 'number' is not assignable to parameter of type 'string'.
```

You can also get a readout of the `getElementById` function by hovering there:

```
(method) Document.getElementById(elementId: string): HTMLElement | null
```

In the case of `getElementById`, you can see that it looks something like a function call. You can see that the function takes in an `elementId` of type `string`, and it returns an `HTMLElement | null`. We'll look at this syntax later, but it basically means either an `HTMLElement` or a `null`.

Note that this readout appears only when you hover over `getElementById`. If you hover over the document itself, you won't see the information for `getElementById`, so it's important to be precise with your hovering.

This tells you that you can fix the error by changing the argument to a string:

```
let element = document.getElementById("12");
```

You also know that `element`'s type will be what `document.getElementById` returns, which you can confirm by hovering over `element`:

```
// hovering over element shows:  
  
const element: HTMLElement | null;
```

Hovering in different places reveals different information, so when we're working in TypeScript, we hover around constantly to get a better sense of what our code is doing.

JSDoc Comments

JSDoc is a syntax for adding documentation to the types and functions in your code. It allows VS Code to show additional information in the pop-up that appears when hovering. This feature is extremely useful when working with a team.

Here's an example of how a `logValues` function could be documented:

```
/**  
 * Logs the values of an object to the console.  
 *  
 * @param obj - The object to log.  
 *  
 * @example  
 * ```ts  
 * logValues({a: 1, b: 2});  
 * // Output:  
 * // a: 1  
 * // b: 2  
 * ```  
 */  
  
const logValues = (obj: any) => {  
  for (const key in obj) {  
    console.log(`${key}: ${obj[key]}`);  
  }  
}
```

```
}  
};
```

The `@param` tag is used to describe the parameters of the function. The `@example` tag is used to provide an example of how the function can be used, using markdown syntax.

Many tags are available for use in JSDoc comments. You can find a full list of them in the JSDoc documentation (<https://jsdoc.app>).

Adding JSDoc comments is a useful way to communicate the purpose and usage of your code, whether you're working on a library, a team, or your own personal projects.

Exercise 2-3: Adding Documentation for Hovers

Here's a simple function that adds two numbers together:

```
const myFunction = (a: number, b: number) => {  
  return a + b;  
};
```

To understand what this function does, you'd have to read the code. Add some documentation to the function so that when you hover over it, you can read a description of what it does.

See <https://totalts.link/essentials-2-3/>.

Solution

In this case, you'll specify that the function "adds two numbers together." You can also use an `@example` tag to show an example of how the function is used:

```
/**  
 * Adds two numbers together.  
 * @example  
 * myFunction(1, 2);  
 * // Will return 3  
 */
```

```
const myFunction = (a: number, b: number) => {  
    return a + b;  
};
```

Now whenever you hover over this function, you'll see the signature of the function along with the comment and full syntax highlighting for anything after the `@example` tag:

```
// hovering over myFunction shows:  
  
const myFunction: (a: number, b: number) => number  
  
Adds two numbers together.  
  
@example  
  
myFunction(1, 2);  
  
// Will return 3
```

While this example is trivial (you could, of course, just give the function a better name), JSDoc can be an extremely important tool for documenting your code.

Navigating with Go to Definition and Go to References

The TypeScript server also provides the ability to navigate to the definition of a variable or declaration. In VS Code, this Go to Definition shortcut is invoked by `COMMAND`-clicking on macOS, `CTRL`-right-clicking on Windows, or pressing `F12` on Windows for the current cursor position. You can also right-click and select Go to Definition from the context menu on either platform. For the sake of brevity, we'll use the macOS shortcut.

Once you've arrived at the definition location, repeating the `COMMAND`-click shortcut shows you every location where the variable or declaration is used. This is called Go to References, and it is especially useful for navigating around large codebases.

You also can use the shortcut to find the type definitions for built-in types and libraries. For example, if you COMMAND-click `document` when using the `getElementById` method, you'll be taken to the type definitions for `document`. It's a great feature that will help you understand how built-in types and libraries work.

Rename Symbol

In some situations, you might want to rename a variable across your entire codebase. Imagine that a database column changes from `id` to `entityId`. Doing a simple find and replace won't work because `id` is used in many places for different purposes. A TypeScript-enabled feature called *rename symbol* allows you to do that with a single action. Let's look at an example:

```
const filterUsersById = (id: string) => {  
    return users.filter((user) => user.id === id);  
};
```

Right-click the `id` parameter of the `filterUsersById` function and select **Rename Symbol**. (You can also press F2.) A panel should appear that prompts for the new name. Enter **userIdToFilterBy** and press ENTER. VS Code is smart enough to realize that you want to rename only the `id` parameter for the function and not the `user.id` property:

```
const filterUsersById = (userIdToFilterBy: string) => {  
    return users.filter((user) => user.id === userIdToFilterB  
y);  
};
```

The rename symbol feature is a great tool for refactoring code, and it works across multiple files as well.

Automatic Imports

Large JavaScript applications can be composed of many, many modules, and manually importing from other files can be tedious. Fortunately, TypeScript supports automatic imports.

When you start typing the name of a variable you want to import, TypeScript shows a list of suggestions when you use the CTRL-spacebar shortcut. Select a variable from the list, and TypeScript automatically adds an import statement to the top of the file.

You need to be a little bit careful when using autocompletion in the middle of a name since the rest of the line could be unintentionally altered. To avoid this issue, make sure your cursor is at the end of the name before pressing CTRL-spacebar.

Quick Fixes

VS Code also offers a “quick fix” feature that can be used to run quick refactor scripts. For now, let’s use it to import multiple missing imports at the same time.

To open the Quick Fixes menu, press COMMAND-. (dot) for macOS or CTRL-. (dot) for Windows. If you do this on a line of code that references a value that hasn’t been imported yet, a pop-up will show:

```
const triangle = new Triangle(); // red squiggly line under
Triangle
```

One of the options in the Quick Fixes menu is Add All Missing Imports. Selecting this option adds all the missing imports to the top of the file:

```
import {Triangle} from "../shapes";

const triangle = new Triangle();
```

The Quick Fixes menu provides a lot of options for refactoring your code, and it’s a great way to learn about TypeScript’s capabilities. (We’ll return to this menu again in Exercise 2-4.)

Restarting the VS Code Server

You’ve seen several examples of the cool things that TypeScript can do for you in VS Code. However, running a language server is not a simple task.

The TypeScript server can sometimes get into a bad state and stop working properly. This might happen if configuration files are changed or when working with a particularly large codebase. Errors can occasionally stop appearing as expected, or types can occasionally get mixed up.

If you're experiencing strange behavior, it's a good idea to restart the TypeScript server. To do this, open the VS Code Command Palette with **COMMAND-SHIFT-P** in macOS or **CTRL-SHIFT-P** in Windows. Then, search for **Restart TS Server**. After a couple of seconds, the server should kick back into gear and ensure that errors are being reported properly.

Working in JavaScript

If you're a JavaScript user, you might have noticed that lots of these features are already available without using TypeScript. Autocomplete, organizing imports, auto imports, and hovering all work in JavaScript. Why is that?

It's because of TypeScript. TypeScript's IDE server is running not just on TypeScript files but on JavaScript files too. That means some of TypeScript's advanced IDE capabilities are also available in JavaScript.

Some features, however, aren't available in JavaScript out of the box. The most prominent are in-IDE errors. Without type annotations, TypeScript isn't confident enough about the shape of your code to give you accurate warnings.

NOTE

There is a system for adding types to .js files using JSDoc comments that TypeScript supports, but it isn't fully enabled by default. You'll learn how to configure it later.

The reason TypeScript does this is to support a better experience when working in JavaScript for VS Code users. A subset of TypeScript's features is better than nothing at all. The upshot is that moving to TypeScript should feel extremely familiar for JavaScript users. It's the same IDE experience, only better.

Exercise 2-4: Quick Fix Refactoring

Let's revisit the VS Code Quick Fixes menu you saw earlier. In this example, you have a function that contains a `randomPercentage` variable, which is created by calling `Math.random()` and converting the result to a fixed number:

```
const func = () => {  
  // Refactor this to be its own function.  
  const randomPercentage = `${(Math.random() * 100).toFixed(2)}%`;  
  
  console.log(randomPercentage);  
};
```

The goal here is to refactor the logic that generates the random percentage into its own separate function.

Highlight a variable, line, or entire code block and then press `COMMAND-.` to open the Quick Fixes menu. You'll find several options for modifying the code, depending on where your cursor is when you open the menu. Experiment with different options to see how they affect the example function.

See <https://totalts.link/essentials-2-4/>.

Solution

The Quick Fixes menu will show different refactoring options depending on where your cursor is when you open it:

Inlining variables

If you target `randomPercentage`, you can select an `Inline Variable` option, which would remove the variable and inline its value into the `console.log`:

```
const func = () => {  
  console.log(`${(Math.random() * 100).toFixed(2)}%`);  
};
```

Extracting constants

When selecting a smaller portion of code, like `Math.random() * 100`, the option **Extract Constant in Enclosing Scope** will appear. Selecting this option creates a new local variable that you are prompted to name and assigns the selected value to it. After saving and running a code formatter, everything is cleaned up nicely:

```
const func = () => {  
  const randomTimes100 = Math.random() * 100;  
  const randomPercentage = `${randomTimes100.toFixed(2)}%`;   
  console.log(randomPercentage);  
};
```

Similarly, the **Extract to Constant in Module Scope** option will create a new constant in the module scope:

```
const randomTimes100 = Math.random() * 100;  
const func = () => {  
  const randomPercentage = `${randomTimes100.toFixed(2)}%`;   
  console.log(randomPercentage);  
};
```

Extracting functions

Selecting the entire random percentage logic enables some other extraction options. The **Extract to Function in Module Scope** option will act similarly to the constant option but create a function instead:

```
const func = () => {  
  const randomPercentage = getRandomPercentage();  
  console.log(randomPercentage);  
};  
function getRandomPercentage() {  
  return `${(Math.random() * 100).toFixed(2)}%`;   
};
```

These are just a few of the options the Quick Fixes menu provides. There's so much you can achieve with them, and we're only scratching the surface. Keep exploring and experimenting to discover their full potential!

Summary

This chapter introduced the powerful IDE features that make TypeScript development so productive in VS Code. The TypeScript server running in the background provides real-time feedback and assistance as you code.

Autocomplete stands out as perhaps the most valuable feature, offering intelligent suggestions based on TypeScript's understanding of our code's types. You can trigger it manually with CTRL-spacebar to explore APIs and discover available properties and methods.

TypeScript's error checking catches both runtime errors (like calling methods on `null`) and potential issues that won't crash your app but could lead to bugs. The red squiggly lines appear instantly as you type, with hover tooltips explaining the problems.

The hover feature lets you inspect any variable or declaration to see its inferred type, helping you understand what your code is doing. When you combine this feature with JSDoc comments, you can provide rich documentation that appears in these hover tooltips.

Navigation features like Go to Definition and Go to References help you move around large codebases efficiently. The rename symbol feature safely renames variables across your entire project without breaking unrelated code.

Automatic imports and quick fixes streamline your workflow by handling tedious tasks like adding import statements or refactoring code blocks into separate functions. When things go wrong, restarting the TypeScript server usually resolves any issues.

Even JavaScript files benefit from many of these features, since the TypeScript server analyzes them too, making the transition to full TypeScript feel natural and familiar.

3

TYPESCRIPT IN THE DEVELOPMENT PIPELINE



You’ve explored the relationship between JavaScript and TypeScript and how TypeScript improves your life as a developer. But let’s go a bit deeper. In this chapter, you’ll get the TypeScript CLI up and running and see how it fits into the development pipeline. We’ll cover the challenges of running TypeScript, the process of how TypeScript becomes JavaScript, and how to integrate TypeScript with modern frontend frameworks.

As an example, we’ll use TypeScript to show you how to build a web application. Note that you can use TypeScript anywhere JavaScript can be used—in a Node, Electron, React Native, or any other app.

The Problem with TypeScript in the Browser

Consider this TypeScript file, *example.ts*, containing a run function that logs a message to the console:

```
const run = (message: string) => {  
  console.log(message);  
}
```

```
};  
  
run("Hello!");
```

Alongside the *example.ts* file, you have a basic *index.html* file that references the *example.ts* file in a script tag:

```
<!DOCTYPE html>  
  
<html lang="en">  
  <head>  
    <meta charset="UTF-8" />  
  
    <title>My App</title>  
  </head>  
  
  <body>  
    <script src="example.ts"></script>  
  </body>  
</html>
```

However, when you open the *index.html* file in a browser, you see an error in the console:

```
Unexpected token ':'
```

No red lines appear in the TypeScript file, so why does this happen? This happens because *browsers can't run TypeScript*. Browsers (and other runtimes like Node.js) can't understand TypeScript on their own; they understand only JavaScript. In the case of the `run` function, the `: string` after `message` in the function declaration is not valid JavaScript:

```
const run = (message: string) => {  
  // : string is not valid JavaScript!  
  
  console.log(message);  
};
```

Removing `: string` gives you something that looks a bit more like JavaScript, but now TypeScript displays an error underneath message:

```
const run = (message) => {}; // red squiggly line under message
```

Hovering over the red squiggly line in VS Code, you see the error message that tells you `message` implicitly has an any type. We'll discuss what that error means later, but for now, the point is that the *example.ts* file contains syntax that the browser can't understand, and the TypeScript CLI isn't happy when you remove it. To get the browser to understand your TypeScript code, you need to turn it into JavaScript.

Transpiling TypeScript

The TypeScript CLI, `tsc`, is what handles the process of turning TypeScript to JavaScript. The CLI is installed automatically when you install TypeScript, but before you can use it, you need to set up your TypeScript project.

To do this, open a terminal and navigate to the parent directory of *example.ts* and *index.html*. To double-check that you have TypeScript installed properly, run `tsc --version` in your terminal. If you see a version number, you're good to go. Otherwise, run the following to install TypeScript globally:

```
npm add -g typescript
```

With the terminal open to the correct directory and TypeScript installed, you can now initialize the TypeScript project.

Initializing a TypeScript Project

For TypeScript to know how to transpile your code, you need to create a *tsconfig.json* file in the root of your project. Run the following command to generate the *tsconfig.json* file:

```
tsc --init
```

In the *tsconfig.json* file, you'll find several useful starter configuration options, as well as many other commented-out options.

For now, let's stick with the defaults:

```
// Excerpt from tsconfig.json
{
  "compilerOptions": {
    "target": "es2016",
    "module": "commonjs",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  }
}
```

With the *tsconfig.json* file in place, you can begin transpilation.

Running tsc

Running the **tsc** command in the terminal without any arguments will take advantage of the defaults in *tsconfig.json* and transpile all TypeScript files in the project to JavaScript:

tsc

In this case, the TypeScript code in the *example.ts* file will become JavaScript code in *example.js*. In *example.js*, the TypeScript syntax has been transpiled into JavaScript:

```
// In the example.js file

"use strict";

const run = (message) => {
  console.log(message);
};

run("Hello!");
```

Now that you have a JavaScript file, you can update the *index.html* file to reference *example.js* instead of *example.ts*:

```
// In index.html

<script src="example.js"></script>
```

Opening the *index.html* file in the browser will now show the expected "Hello!" output in the console without any errors!

Does TypeScript Change Your JavaScript?

In your JavaScript file, little has changed from the TypeScript code:

```
"use strict";

const run = (message) => {
  console.log(message);
};

run("Hello!");
```

It has removed the `: string` from the `run` function and added `"use strict";` to the top of the file, but other than that, the code is identical.

A key guiding principle for TypeScript is that it gives you a thin layer of syntax on top of JavaScript, but it doesn't change the way your code works. It doesn't add runtime validation. It doesn't attempt to optimize your code's performance. It just adds types to give you a better developer experience and removes them when it's time to turn your code into JavaScript.

NOTE

There are some exceptions to this guiding principle, such as enums, namespaces, and class parameter properties, but we'll cover those later.

A Note on Version Control

You've transpiled the TypeScript code to JavaScript successfully, but you've also added a new file to the project. Adding a *.gitignore* file to the root of the project and including **.js* will prevent the JavaScript files from being added to version control. Importantly, this communicates to other developers using the repo that the **.ts* files are the source of truth for the JavaScript.

Running TypeScript in Watch Mode

You might have noticed that if you make changes to your TypeScript file, you'll need to run `tsc` again to see the changes in the browser, which will get old fast. You might forget to do it and wonder why your changes aren't in the browser.

Fortunately, the TypeScript CLI uses a `--watch` flag that recompiles your TypeScript files automatically on save:

```
tsc --watch
```

To see it in action, open VS Code with the *example.ts* and *example.js* files side by side. If you change the message being passed to the `run` function in *example.ts* to something else, you'll see the *example.js* file update automatically.

Errors in the TypeScript CLI

If `tsc` encounters an error, it displays the error in the terminal, and the file with the error is marked with a red squiggly line in VS Code. For example, try changing the message: `string` to **`message: number`** in the `run` function in *example.ts*:

```
const run = (message: number) => {  
  console.log(message);  
};  
  
run("Hello world!"); // red squiggly line under "Hello world!"
```

In the terminal, tsc should display an error message:

```
// In the terminal
```

```
Argument of type 'string' is not assignable to parameter of  
type 'number'.
```

```
run("Hello world!");
```

```
Found 1 error. Watching for file changes.
```

Reverting the change to message: string will remove the error, and the *example.js* file will again update automatically.

Running tsc in watch mode is useful for compiling TypeScript files automatically and catching errors as you write code. It can be especially useful on large projects because it checks the entire project. This differs from your IDE, which shows only the errors of the files that are currently open.

TypeScript with Modern Frameworks

So far, the setup is simple: a TypeScript file, a `tsc --watch` command, and a JavaScript file. But to build a frontend app, you'll need to do a lot more: handle CSS, minification, bundling, and so forth. TypeScript can't help you with all of that.

Fortunately, many frontend frameworks can help. Vite is one example of a frontend tooling suite that not only transpiles *.ts* files to *.js* files but also provides a dev server with hot module replacement (HMR). Working with an HMR setup allows you to see changes in your code reflected in the browser without reloading the page manually.

But there's a drawback: While Vite and other tools handle the transpilation of TypeScript to JavaScript, they don't provide type checking out of the box. This means you could introduce errors into your code and Vite would continue running the dev server without telling you. It would even allow you to push errors into production because it doesn't know any

better, so you still need the TypeScript CLI to catch errors. But if Vite transpiles your code, you don't need TypeScript to do it too.

NOTE

Several JavaScript server runtimes, like Node and Bun, now support TypeScript out of the box. But their support is similar to Vite's: They provide transpilation without type checking. You'll still need to use `tsc --watch` to get type checking.

TypeScript as a Linter

Fortunately, you can configure TypeScript's CLI to allow for type checking without interfering with other tools. In the `tsconfig.json` file, there's an option called `noEmit` that tells `tsc` whether to emit JavaScript files.

Setting `noEmit` to `true` ensures that no JavaScript files will be created when you run `tsc`. This makes TypeScript act more like a linter than a transpiler. This makes the `tsc` step a great addition to a CI/CD system, since it prevents merging a pull request with TypeScript errors. Later in the book, we'll take a closer look at more advanced TypeScript configurations for application development.

Summary

This chapter explored how TypeScript fits into the development pipeline and the challenges of running TypeScript code in browsers. The core problem is that browsers can't understand TypeScript syntax; they only understand JavaScript.

When you try to run TypeScript directly in a browser, you'll encounter errors because features like type annotations (`: string`) aren't valid JavaScript. This is where the TypeScript CLI comes in to transpile your code.

The `tsc` command converts TypeScript files into JavaScript by removing type annotations and adding necessary JavaScript syntax. You initialize a TypeScript project with `tsc --init`, which creates a `tsconfig.json` file with configuration options.

Running `tsc` transpiles all TypeScript files in your project. The resultant JavaScript is nearly identical to your TypeScript code, just with types removed. TypeScript follows a guiding principle of being a thin layer over JavaScript without changing how your code works.

The `--watch` flag automatically recompiles files when you make changes, making development more efficient. The CLI also catches type errors and displays them in the terminal, helping you fix issues as you code.

Modern frontend frameworks like Vite can handle TypeScript transpilation but don't provide type checking. You can configure TypeScript as a linter by setting `noEmit: true` in *tsconfig.json*, allowing other tools to handle transpilation while TypeScript focuses on type checking.

This setup makes TypeScript an excellent addition to CI/CD pipelines, preventing code with type errors from being merged into production.

PART II

FUNDAMENTALS

4

ESSENTIAL TYPES AND ANNOTATIONS



Now that we've covered most of the *why* of TypeScript, it's time to start with the *how*. In this chapter, we'll cover key concepts such as type annotations and type inference, as well as how to start writing type-safe functions.

It's important to build a solid foundation, as everything you'll learn later builds upon what you'll learn in this chapter.

Type Annotations

One of the most common tasks you'll need to perform as a TypeScript developer is to annotate your code. Type annotations tell TypeScript what type something is supposed to be. Annotations will often use a colon (:), which is used to tell TypeScript that a variable or function parameter is of a certain type.

The Basic Types

TypeScript has a number of basic types you can use to annotate your code. Here are a few of the most common ones:

```
let example1: string = "Hello World!";  
let example2: number = 42;  
let example3: boolean = true;
```

```
let example4: symbol = Symbol();
let example5: bigint = 123n;
let example6: null = null;
let example7: undefined = undefined;
```

Each of these types is used to tell TypeScript what type a variable or function parameter is supposed to be. The basic types like `string`, `boolean`, and `number` are covered. But TypeScript also has types for JavaScript features you may not have heard of, such as `bigint` and `symbol`. It also has separate types for `null` and `undefined`. You can express much more complex types in TypeScript, such as arrays, objects, functions, and more (we'll cover these in later chapters).

Function Parameter Annotations

One of the most important annotations you'll use is for function parameters. For example, here is a `logAlbumInfo` function that takes in a `title` `string`, a `trackCount` `number`, and an `isReleased` `boolean`:

```
const logAlbumInfo = (
  title: string,

  trackCount: number,

  isReleased: boolean,
) => {
  // implementation
};
```

Each parameter's type annotation enables TypeScript to check that the arguments passed to the function are of the correct type. If the type doesn't match up, TypeScript will show a red squiggly line under any offending arguments:

```
logAlbumInfo("Black Gold", false, 15); // red squiggly line
first under false, then under 15
```

In the previous example, you would first get an error under `false` because a boolean can't be passed in place of a number. TypeScript's errors describe this as not being "assignable" to boolean. If you fix that like so

```
logAlbumInfo("Black Gold", 20, 15); // red squiggly line under 15
```

you would still have an error under `15` because a number isn't assignable to a boolean.

Variable Annotations

Just as with function parameters, you can also annotate variables. You use variable annotations to explicitly tell TypeScript what you expect the types of your variables to be. Here's an example of some variables with their associated types:

```
let albumTitle: string = "Midnights";  
let isReleased: boolean = true;  
let trackCount: number = 13;
```

Notice how each variable name is followed by a `:` and its primitive type before its value is set.

Once a variable has been declared with a specific type annotation, TypeScript ensures that the variable remains compatible with the type you specified. For example, this reassignment would work:

```
let albumTitle: string = "Midnights";  
  
albumTitle = "1989"; // albumTitle has been reassigned without error
```

But this one would show an error:

```
let isReleased: boolean = true;  
  
isReleased = "yes"; // red squiggly line under isReleased
```

TypeScript's static type checking is able to spot errors at compile time, which is happening behind the scenes as you write your code.

In the case of the previous `isReleased` example, the error message reads:

```
Type 'string' is not assignable to type 'boolean'.
```

In other words, TypeScript says that it expected `isReleased` to be a `boolean` but instead received a `string`. It's nice to be warned about these kinds of errors before you even run your code!

Type Inference

TypeScript gives you the ability to annotate almost any value, variable, or function in your code. You might be thinking, "Wait, do I need to annotate everything? That's a lot of extra code." Don't worry. In many cases, TypeScript is smart enough to infer a type where none exists. This is called *type inference*.

Variables Don't Always Need Annotations

Let's look again at the variable annotation example but drop the annotations:

```
let albumTitle = "Midnights";  
let isReleased = true;  
let trackCount = 13;
```

We didn't add the annotations, but TypeScript isn't complaining. What's going on?

Try hovering your cursor over each variable, and you'll see the following:

```
// hovering over each variable name shows:  
  
let albumTitle: string;  
let isReleased: boolean;  
let trackCount: number;
```

Even though they aren't annotated, TypeScript is able to *infer* the type of each.

TypeScript still behaves as if variables have been annotated, warning you if you try to assign it a different type from what it was assigned originally. For example, if you assign `isReleased` the boolean `true` and then try to reassign it to a string, TypeScript will give you an error message:

```
let isReleased = true;
isReleased = "yes"; // red squiggly line under isReleased
// hovering over isReleased shows:
Type 'string' is not assignable to type 'boolean'.
```

When TypeScript infers the type of a variable, it also provides autocomplete on its associated methods. In this case, since `albumTitle` is inferred as a string, typing `albumTitle.toUpperCase` will allow autocompletion for `toUpperCase`.

This is an extremely powerful part of TypeScript. It means you can mostly *not* annotate variables and your IDE still will know what type things are.

Function Parameters Always Need Annotations

Type inference can't work everywhere, however. Let's see what happens if you remove the type annotations from the `logAlbumInfo` function's parameters:

```
const logAlbumInfo = (
  title, // error: Parameter 'title' implicitly has an 'any'
  type.
  trackCount, // error: Parameter 'trackCount' implicitly ha
  s an 'any' type.
  isReleased, // error: Parameter 'isReleased' implicitly ha
  s an 'any' type.
) => {
  // rest of function body
};
```

Without type annotations for the function parameters, TypeScript now displays errors. This is because functions are very different from variables. TypeScript can see what value is assigned to which variable, so it can make a good guess about the type, but TypeScript can't tell from a function parameter alone what type it's supposed to be.

It also can't detect the type from usage. If you had an add function that took two parameters, TypeScript wouldn't be able to tell that they were supposed to be numbers:

```
function add(a, b) {  
    return a + b;  
}
```

The `a` and `b` could be strings, Booleans, or anything else. TypeScript can't know from the function body what type they're supposed to be. So, while inference works in most situations, function parameters are an exception. You'll mostly need to annotate them in TypeScript.

NOTE

There are a couple of exceptions to this rule. Let's say you're mapping over an array of strings. You pass a function to `.map`, which receives the string you're currently mapping over as an argument:

```
const strings = ['a', 'b'];  
strings.map(str => str.toUpperCase());
```

In this context, the parameter `str` does not need to be annotated because TypeScript can infer it's a string. Very clever!

The any Type

The error you just encountered with the `logAlbumInfo` function's parameters was pretty scary:

```
Parameter 'title' implicitly has an 'any' type.
```

When TypeScript doesn't know what type something is, it assigns it the any type.

The any type disables TypeScript's type system. It turns off type safety on the thing it's assigned to, which means that anything can be assigned to it, any property on it can be accessed or assigned to, and it can be called like a function:

```
let anyVariable: any = "This can be anything!";

anyVariable(); // no error

anyVariable.deep.property.access; // no error
```

This code will error at runtime, but TypeScript isn't giving you a warning!

You can use the any type to turn off error messages in TypeScript, which can be a useful escape hatch for when a type is too complex to describe. It's also useful for migrating legacy JavaScript codebases to TypeScript. Overusing any, however, defeats the purpose of using TypeScript, so it's best to avoid it whenever possible.

Exercise 4-1: Basic Types with Function Parameters

Let's start with an add function that takes two Boolean parameters, a and b, and returns a + b:

```
export const add = (a: boolean, b: boolean) => {
  return a + b; // red squiggly line under a + b
};
```

Calling the add function creates the result variable, which is then checked to see if it is equal to a number:

```
const result = add(1, 2); // red squiggly line under 1

type test = Expect<Equal<typeof result,
  number>>; // red squiggly line under Equal through number
```

You're using a couple of type annotations, `Expect` and `Equal`, from the library `@totaltypescript/helpers` to act as tests for the exercises. In this case, you expect the result to be a number.

Currently, a few errors in the code are marked by red squiggly lines. The first is on the return line of the `add` function, where you have `a + b`:

```
// hovering over a + b shows:
```

```
Operator '+' cannot be applied to types 'boolean' and 'boolean'
```

There's also an error after the 1 argument in the `add` function call:

```
Argument of type 'number' is not assignable to parameter of type 'boolean'
```

Finally, you can see that the test result has an error because the result is currently typed as `any`, which is not equal to `number`.

Your challenge is to consider how you can change the types to make the errors go away and to ensure that `result` is a number. Hover over `result` to check it.

See <https://totalts.link/essentials-4-1/>.

Solution

Common sense tells you that the Booleans in the `add` function should be replaced with some sort of number type. If you are coming from another language, you might be tempted to try using `int` or `float`, but TypeScript has only the `number` type:

```
function add(a: number, b: number) {  
    return a + b;  
}
```

Making this change resolves the errors and also gives you some other bonuses.

If you try calling the add function with a string instead of a number, you'll get an error that type string is not assignable to type number:

```
add("something", 2); // red squiggly line under something
```

Not only that, but the result of the function is now inferred for you:

```
const result = add(1, 2); // result is a number!
```

TypeScript can infer not only variables but also the return types of functions.

Exercise 4-2: Annotating Empty Parameters

In this exercise, you have a concatTwoStrings function that is similar in shape to the add function. It takes two parameters, a and b, and returns a string:

```
const concatTwoStrings = (a, b) => {  
  // red squiggly line under a and b  
  return [a, b].join(" ");  
};
```

The a and b parameters have errors, which have not been annotated with types:

Parameter 'a' implicitly has an 'any' type.

Parameter 'b' implicitly has an 'any' type.

The result of calling concatTwoStrings with "Hello" and "World" and checking if it is a string does not show any errors:

```
const result = concatTwoStrings("Hello", "World");  
  
type test = Expect<Equal<typeof result, string>>;
```

Your job is to add some function parameter annotations to the `concatTwoStrings` function to make the errors go away.

See <https://totalts.link/essentials-4-2/>.

Solution

As we've mentioned, function parameters always need annotations in TypeScript. With this in mind, let's update the function declaration parameters so that `a` and `b` are both specified as `string`:

```
const concatTwoStrings = (a: string, b: string) => {  
  return [a, b].join(" ");  
};
```

This change fixes the errors.

For a bonus point, what type will the return type be inferred as?

```
const result = concatTwoStrings("Hello", "World"); // result  
is a string!
```

Exercise 4-3: The Basic Types

As you've seen, TypeScript shows errors when types don't match. The following set of examples shows the basic types that TypeScript gives you to describe JavaScript:

```
export let example1: string = "Hello World!";  
export let example2: string = 42; // red squiggly line under  
example2  
export let example3: string = true; // red squiggly line und  
er example3  
export let example4: string = Symbol(); // red squiggly line  
under example4  
export let example5: string = 123n; // red squiggly line und  
er example5
```

You'll also notice there are several errors. Hovering over each of the underlined variables displays any associated error messages. For example,

hovering over example2 shows this:

```
Type 'number' is not assignable to type 'string'.
```

The type error for example3 tells you this:

```
Type 'boolean' is not assignable to type 'string'.
```

Change the types of the annotations on each variable to make the errors go away.

See <https://totalts.link/essentials-4-3/>.

Solution

Each of the following examples represents one of TypeScript's basic types and would be annotated as follows:

```
let example1: string = "Hello World!";  
let example2: number = 42;  
let example3: boolean = true;  
let example4: symbol = Symbol();  
let example5: bigint = 123n;
```

You've already seen string, number, and boolean. The symbol type is used for symbols, which are used to ensure that property keys are unique. The bigint type is used for JavaScript bigints.

NOTE

JavaScript has some limitations on how large numbers can be. Numbers larger than `Number.MAX_SAFE_INTEGER` can't reliably be added together. Bigints solve this problem by letting us represent large numbers in JavaScript.

However, in practice, you typically won't annotate variables like this. If you remove the explicit type annotations, there won't be any errors at all:

```
let example1 = "Hello World!";
let example2 = 42;
let example3 = true;
let example4 = Symbol();
let example5 = 123n;
```

These basic types are useful to know, even if you don't always need them for your variable declarations.

Exercise 4-4: The any Type

The following function, called `handleFormData`, accepts an `e` typed as `any`. The function prevents the default form submission behavior and then creates an object from the form data and returns it:

```
const handleFormData = (e: any) => {
  e.preventDefault();

  const data = new FormData(e.target);

  const value = Object.fromEntries(data.entries());

  return value;
};
```

Here is a test for the function that creates a form, sets the `innerHTML` to add an input, and then manually submits the form. When it submits, you expect the value to equal the value that was in your form that you grafted in there:

```
it("Should handle a form submit", () => {
  const form = document.createElement("form");

  form.innerHTML = `

  `;

  form.onsubmit = (e) => {
```



```
    const value = handleFormData(e);

    expect(value).toEqual({name: "John Doe"});
  };

  form.requestSubmit();

  expect.assertions(1);
});
```

This isn't the normal way you would test a form, but it provides a way to test the example `handleFormData` function more extensively.

In the code's current state, no red squiggly lines are present. However, when running the test with Vitest, you get an error similar to the following:

```
This error originated in "any.problem.ts" test file.
It doesn't mean the error was thrown inside the file itself,
but while it was running.
The latest test that might've caused the error is "Should ha
ndle a form submit".
It might mean one of the following:
- The error was thrown, while Vitest was running this test.
- This was the last recorded test before the error was throw
n,
if error originated after test finished its execution.
```

Why is this error happening? Why isn't TypeScript giving you an error here?

We'll give you a clue. We've hidden a nasty typo in there. Can you fix it?

See <https://totalts.link/essentials-4-4/>.

Solution

In this case, using `any` did not help at all. In fact, the `any` annotations seem to actually turn off type checking! With the `any` type, you're free to do anything you want to the variable, and TypeScript will not prevent it. Using `any` also disables useful features like autocompletion, which can help you

avoid typos. That's right—the error in the exercise code was caused by a typo of `e.target` instead of `e.target` when creating the `FormData`:

```
const handleFormData = (e: any) => {
  e.preventDefault();

  const data = new FormData(e.target); // e.target! Whoops!

  const value = Object.fromEntries(data.entries());

  return value;
};
```

If `e` had been properly typed, TypeScript would have caught the error right away. We'll come back to this example in the future to see the proper typing.

Using `any` may seem like a quick fix when you have trouble figuring out how to properly type something, but it can come back to bite you later.

Object Literal Types

Now that we've covered basic types, let's move on to object types. *Object types* are used to describe the shape of objects. Each property of an object can have its own type annotation. When defining an object type, you use curly brackets to contain the properties and their types:

```
const talkToAnimal = (animal: {name: string; type: string; age: number}) => {
  // rest of function body
};
```

This curly bracket syntax is called an *object literal type*.

Optional Object Properties

You can use the `?` operator to mark the `age` property as optional:

```
const talkToAnimal = (animal: {name: string; type: string; age?: number}) => {  
  // rest of function body  
};
```

This means you don't need to provide the age property when calling the function.

One cool thing about type annotations with object literals is that they provide autocompletion for the property names while you're typing. For instance, when calling `talkToAnimal`, it will provide an autocomplete drop-down menu with suggestions for the name, type, and age properties. This can save you a lot of time, and it also helps you avoid typos in situations when you have several properties with similar names.

Exercise 4-5: Object Literal Types

Here you have a `concatName` function that accepts a user object with `first` and `last` keys:

```
const concatName = (user) => {  
  // red squiggly line under user  
  return `${user.first} ${user.last}`;  
};
```

The test expects that the full name should be returned. Since the full name *is* being returned, the test is passing:

```
it("should return the full name", () => {  
  const result = concatName({  
    first: "John",  
    last: "Doe",  
  });  
  
  type test = Expect<Equal<typeof result, string>>;  
  
  expect(result).toEqual("John Doe");  
});
```

However, there is a familiar error on the `user` parameter in the `concatName` function:

Parameter 'user' implicitly has an 'any' type.

You can tell from the `concatName` function body that it expects `user.first` and `user.last` to be strings. How could you type the `user` parameter to ensure that it has these properties and that they are of the correct type?

See <https://totalts.link/essentials-4-5/>.

Solution

To annotate the `user` parameter as an object, you can use the curly bracket syntax. Let's start by annotating the `user` parameter as an empty object:

```
const concatName = (user: {}) => {  
  return `${user.first} ${user.last}`; // red squiggly line  
  under .first and .last  
};
```

The errors change, which is progress. The errors now show up under `.first` and `.last` in the function return:

Property 'first' does not exist on type '{}'.
Property 'last' does not exist on type '{}'.

To fix these errors, you need to add the `first` and `last` properties to the type annotation:

```
const concatName = (user: {first: string; last: string}) =>  
  {  
    return `${user.first} ${user.last}`;  
  };
```

Now TypeScript knows that both the `first` and `last` properties of `user` are strings, and the test passes.

Exercise 4-6: Optional Property Types

Here's a version of the `concatName` function that has been updated to return only the first name if a last name wasn't provided:

```
const concatName = (user: {first: string; last: string}) =>
{
  if (!user.last) {
    return user.first;
  }

  return `${user.first} ${user.last}`;
};
```

Like before, TypeScript gives you an error when testing that the function returns only the first name:

```
it("should return only the first name if last name not provided", () => {
  const result = concatName({
    first: "John",
  }); // red squiggly line under {first: "John"}

  type test = Expect<Equal<typeof result, string>>;

  expect(result).toEqual("John");
});
```

This time the entire `{first: "John"}` object is underlined in red, and the error message reads:

```
Argument of type '{first: string;}' is not assignable to
  parameter of type '{first: string; last: string;}'.
Property 'last' is missing in type '{first: string;}' but
  required in type '{first: string; last: string;}'.
```

The error tells you that you are missing a property, but the error is incorrect. You *do* want to support objects that include only a first

property. In other words, last needs to be optional.

How would you update this function to fix the errors?

See <https://totalts.link/essentials-4-6/>.

Solution

Similar to when you set a function parameter as optional, you can use the question mark (?) to specify that an object's property is optional:

```
function concatName(user: {first: string; last?: string}) {  
  // implementation  
}
```

Adding `?:` indicates to TypeScript that the property doesn't need to be present.

If you hover over the `last` property in the function body, you'll see that the `last` property is `string | undefined`:

```
// hovering over user.last shows:  
(property) last?: string | undefined
```

This means it's `string` or `undefined`. It's a useful bit of TypeScript syntax that you'll see more of in the future.

Type Aliases

So far, you've been declaring all of your types inline, which is fine for these simple examples, but in a real application, you're going to have types that repeat a lot across your app. They might be users, products, or other domain-specific types, and you won't want to have to repeat the same type definition in every file that needs it.

This is where the `type` keyword comes in. It allows you to define a type once and use it in multiple places:

```
type Animal = {  
  name: string;  
  type: string;
```

```
    age?: number;  
};
```

It's a way to give a name to a type—in other words, a *type alias*—and then use that name wherever you need to use that type.

To create a new variable with the `Animal` type, you'll add it as a type annotation after the variable name:

```
let pet: Animal = {  
  name: "Karma",  
  type: "cat",  
};
```

You can also use the `Animal` type alias in place of the object type annotation in a function

```
const getAnimalDescription = (animal: Animal) => {};
```

and call the function with your pet variable:

```
const desc = getAnimalDescription(pet);
```

Type aliases can be objects, but they can also use basic types:

```
type Id = string | number;
```

We'll cover this syntax in more detail later (it's called a *union type*), but it's basically saying that an `Id` can be either a `string` or a `number`.

Using a type alias is a great way to ensure that there's a single source of truth for a type definition, which makes it easier to make changes in the future.

Sharing Types Across Modules

Type aliases can be created in their own `.ts` files and imported into the files where you need them, which is useful when sharing types in multiple places

or when a type definition gets too large:

```
// In shared-types.ts

export type Animal = {
  width: number;
  height: number;
};

// In index.ts
import type {Animal} from "../shared-types";
```

As a convention, you can even create your own *.types.ts* files, which can help keep your type definitions separate from your other code.

Exercise 4-7: The type Keyword

The following code uses the same type in multiple places:

```
const getRectangleArea = (rectangle: {width: number; height:
number}) => {
  return rectangle.width * rectangle.height;
};

const getRectanglePerimeter = (rectangle: {
  width: number;
  height: number;
}) => {
  return 2 * (rectangle.width + rectangle.height);
};
```

The `getRectangleArea` and `getRectanglePerimeter` functions both take in a rectangle object with `width` and `height` properties.

Tests for each function pass as expected:

```
it("should return the area of a rectangle", () => {
  const result = getRectangleArea({
    width: 10,
    height: 20,
```



```
});

type test = Expect<Equal<typeof result, number>>;

expect(result).toEqual(200);
});

it("should return the perimeter of a rectangle", () => {
  const result = getRectanglePerimeter({
    width: 10,
    height: 20,
  });

  type test = Expect<Equal<typeof result, number>>;

  expect(result).toEqual(60);
});
```

Even though everything is working, there's an opportunity for refactoring to clean things up. How could you use the type keyword to make this code more readable?

See <https://totalts.link/essentials-4-7/>.

Solution

You can use the type keyword to create a Rectangle type with width and height properties:

```
type Rectangle = {
  width: number;
  height: number;
};
```

With the type alias created, you can update the getRectangleArea and getRectanglePerimeter functions to use the Rectangle type:

```
const getRectangleArea = (rectangle: Rectangle) => {
  return rectangle.width * rectangle.height;
};
```

```
const getRectanglePerimeter = (rectangle: Rectangle) => {  
    return 2 * (rectangle.width + rectangle.height);  
};
```

This makes the code a lot more concise and gives you a single source of truth for the `Rectangle` type.

Arrays

You can also describe the types of arrays in TypeScript. There are two different syntaxes for doing this. The first option is the square bracket syntax, which is similar to the type annotations you've made so far but with the addition of two square brackets (`[]`) at the end to indicate an array:

```
let albums: string[] = [  
    "Rubber Soul",  
    "Revolver",  
    "Sgt. Pepper's Lonely Hearts Club Band",  
];  
  
let dates: number[] = [1965, 1966, 1967];
```

The second option is to explicitly use the `Array` type with angle brackets (`< >`) containing the type of data the array will hold:

```
typescript  
let albums: Array<string> = [  
    "Rubber Soul",  
    "Revolver",  
    "Sgt. Pepper's Lonely Hearts Club Band",  
];
```

Both of these syntaxes are equivalent, but the square bracket syntax is a bit more concise when creating arrays. It's also the way that TypeScript presents error messages. Keep the angle bracket syntax in mind, though; you'll see more examples of it later in this chapter.

Arrays of Objects

You have several choices when annotating an array of objects. You can specify the object inline:

```
type ArrayOfAlbums = {artist: string; title: string; year: number}[]
```

Or you can use a type alias and wrap it with `Array<>` or append square brackets:

```
type Album = {artist: string; title: string; year: number}
type ArrayOfAlbums = Array<Album>; // option 1
type ArrayOfAlbums = Album[]; // option 2
```

Finally, you can use the type like so:

```
// create an array of Albums
let selectedDiscography: ArrayOfAlbums = [
  {
    artist: "The Beatles",
    title: "Rubber Soul",
    year: 1965,
  },
  {
    artist: "The Beatles",
    title: "Revolver",
    year: 1966,
  },
];
```

If you try to update the array with an item that doesn't match the type, TypeScript will give you an error:

```
selectedDiscography.push({name: "Karma", type: "cat"};) // red squiggly line under name
// error message:
```

```
// Argument of type '{name: string; type: string;}' is not  
assignable to parameter of type 'Album'.
```

Tuples

Tuples let you specify an array with a fixed number of elements, where each element has its own type. Creating a tuple is similar to an array's square bracket syntax, except the square brackets contain the types instead of abutting the variable name:

```
// Tuple  
let album: [string, number] = ["Rubber Soul", 1965];  
  
// Array  
let albums: string[] = [  
    "Rubber Soul",  
    "Revolver",  
    "Sgt. Pepper's Lonely Hearts Club Band",  
];
```

Tuples are useful for grouping related information together without having to create a new type. For example, if you wanted to group an album with its play count, you could do something like this:

```
let albumWithPlayCount: [Album, number] = [  
    {  
        artist: "The Beatles",  
        title: "Revolver",  
        year: 1965,  
    },  
    10000,  
];
```

Now the `albumWithPlayCount` tuple can be indexed similarly to an array—at index 0 is the `Album`, and at index 1 is 1000.

Named Tuples

To add more clarity to the tuple, you can add names for each of the types in the square brackets:

```
type MyTuple = [album: Album, playCount: number];
```

This can be helpful when you have a tuple with a lot of elements or when you want to make the code more readable.

NOTE

The names of the tuples don't do anything at runtime; they're just descriptions to make it clear what should go in each element.

Exercise 4-8: Array Type

Consider the following shopping cart code:

```
type ShoppingCart = {  
  userId: string;  
};  
  
const processCart = (cart: ShoppingCart) => {  
  // Do something with the cart in here  
};  
  
processCart({  
  userId: "user123",  
  items: ["item1", "item2", "item3"], // red squiggly line u  
nder items  
});
```

We have a type alias for `ShoppingCart` that currently has a `userId` property of type `string`. The `processCart` function takes in a `cart` parameter of type `ShoppingCart`. Its implementation doesn't matter at this point. What does matter is that when you call `processCart`, you are passing in an object with a `userId` and an `items` property that is an array of strings.

There is an error underneath `items` that reads:

Argument of type '{userId: string; items: string[];}' is not assignable to parameter of type 'ShoppingCart'.
Object literal may only specify known properties, and 'items' does not exist in type 'ShoppingCart'.

As the error message points out, there is not currently a property called `items` on the `ShoppingCart` type. How would you change the types to fix this error?

See <https://totalts.link/essentials-4-8/>.

Solution

For the `ShoppingCart` example, defining an array of item strings would look like the following when using the square bracket syntax:

```
type ShoppingCart = {  
  userId: string;  
  items: string[];  
};
```

With this in place, you must pass in `items` as an array. A single string or other type would result in a type error.

The other syntax is to write `Array` explicitly and pass it a type in the angle brackets:

```
type ShoppingCart = {  
  userId: string;  
  items: Array<string>;  
};
```

Exercise 4-9: Arrays of Objects

Consider this `processRecipe` function that takes in a `Recipe` type:

```
type Recipe = {  
  title: string;  
  instructions: string;  
};
```

```
const processRecipe = (recipe: Recipe) => {
  // Do something with the recipe in here
};

processRecipe({
  title: "Chocolate Chip Cookies",
  ingredients: [
    {name: "Flour", quantity: "2 cups"},
    {name: "Sugar", quantity: "1 cup"},
    // other ingredients here. . .
  ],
  instructions: ". . .",
});
```

The function is called with an object containing title, instructions, and ingredients properties, but there are errors because the Recipe type doesn't currently have an ingredients property:

```
Argument of type '{title: string; ingredients: {
  name: string;
  quantity: string;}[];
instructions: string;
}' is not assignable to parameter of type 'Recipe'.
```

Object literal may only specify known properties, and 'ingredients' does not exist in type 'Recipe'.

By combining what you've seen with typing object properties and working with arrays, how would you specify ingredients for the Recipe type?

See <https://totalts.link/essentials-4-9/>.

Solution

There are a few different ways to express an array of objects. One approach is to create a new Ingredient type that you can use to represent the objects in the array:

```
type Ingredient = {  
  name: string;  
  quantity: string;  
};
```

Then, the Recipe type can be updated to include an ingredients property of type Ingredient[]:

```
type Recipe = {  
  title: string;  
  instructions: string;  
  ingredients: Ingredient[];  
};
```

This solution reads nicely, fixes the errors, and helps create a mental map of your domain model.

As shown previously, using the Array<Ingredient> syntax would also work:

```
type Recipe = {  
  title: string;  
  instructions: string;  
  ingredients: Array<Ingredient>;  
};
```

It's also possible to specify the ingredients property as an inline object literal on the Recipe type using square brackets:

```
type Recipe = {  
  title: string;  
  instructions: string;  
  ingredients: {  
    name: string;  
    quantity: string;  
  }[];  
};
```

Or using `Array<>` with the object literal inside:

```
type Recipe = {  
  title: string;  
  instructions: string;  
  ingredients: Array<{  
    name: string;  
    quantity: string;  
  }>;  
};
```

The inline approaches are useful, but we prefer extracting them to a new type. This means that if another part of your application needs to use the `Ingredient` type, it can.

Exercise 4-10: Tuples

Here you have a `setRange` function that takes in an array of numbers:

```
const setRange = (range: Array) => {  
  const x = range[0];  
  const y = range[1];  
  
  // Do something with x and y in here  
  // x and y should both be numbers!  
  
  type tests = [  
    Expect<Equal<typeof x, number>>, // red squiggly line under  
    Equal<> statement  
    Expect<Equal<typeof y, number>>, // red squiggly line under  
    Equal<> statement  
  ];  
};
```

In the function, you grab the first element of the array and assign it to `x`, and you grab the second element of the array and assign it to `y`. Two tests in the `setRange` function are currently failing.

Using the `// @ts-expect-error` directive, you find a couple more errors that need fixing. Recall that this directive tells TypeScript you know there will be an error on the next line, and it should ignore it. However, if you say you expect an error but there isn't one, you will get the red squiggly line under the actual `// @ts-expect-error` line:

```
// both of these show a red squiggly line under the ts-expect-error directive

// @ts-expect-error too few arguments
setRange([0]);

// @ts-expect-error too many arguments
setRange([0, 10, 20]);
```

The code for the `setRange` function needs an updated type annotation to specify that it accepts only a tuple of two numbers.

See <https://totalts.link/essentials-4-10/>.

Solution

In this case, you would update the `setRange` function to use the tuple syntax instead of the array syntax:

```
const setRange = (range: [number, number]) => {
  // rest of function body
};
```

If you want to add more clarity to the tuple, you can add names for each of the types:

```
const setRange = (range: [x: number, y: number]) => {
  // rest of function body
};
```

Exercise 4-11: Optional Members of Tuples

This `goToLocation` function takes in an array of coordinates. Each coordinate has a latitude and longitude, which are both numbers, as well as an optional elevation, which is also a number:

```
const goToLocation = (coordinates: Array) => {
  const latitude = coordinates[0];
  const longitude = coordinates[1];
  const elevation = coordinates[2];

  // Do something with latitude, longitude, and elevation in
  here

  type tests = [
    Expect<Equal<typeof latitude, number>>, // red squiggly
    line under Equal<> statement
    Expect<Equal<typeof longitude, number>>, // red squiggly
    line under Equal<> statement
    Expect<Equal<typeof elevation, number | undefined>>,
  ];
};
```

Your challenge is to update the type annotation for the `coordinates` parameter to specify that it should be a tuple of three numbers, where the third number is optional.

See <https://totalts.link/essentials-4-11/>.

Solution

The fix here is to provide an optional modifier (?) to the member of the tuple:

```
const goToLocation = (
  coordinates: [latitude: number, longitude: number, elevati
on?: number],
) => {};
```

The values are clear, and using the ? modifier specifies that `elevation` is an optional number. It almost looks like an object, but it's still a tuple.

Alternatively, if you don't want to use named tuples, you can use the `?` modifier after the definition:

```
const goToLocation = (coordinates: [number, number, number?]) => {};
```

Passing Types to Functions

Let's take a quick look back at the `Array` type discussed earlier:

```
Array<string>;
```

This type describes an array of strings. To make that happen, you're passing a type (`string`) as an argument to another type (`Array`).

There are lots of other types that can receive types as arguments, like `Promise<string>`, `Record<string, string>`, and more. In each of them, you use the angle brackets to pass a type to another type, but you can also use that syntax to pass types to functions.

Passing Types to Set

A `Set` is a JavaScript feature that represents a collection of unique values. To create a `Set`, use the `new` keyword and call `Set`:

```
const formats = new Set();
```

If you hover over the `formats` variable, you can see that it is typed as `Set<unknown>`:

```
// hovering over formats shows:  
const formats: Set<unknown>;
```

That's because the `Set` doesn't know what type it's supposed to be! You haven't passed it any values, so it defaults to an unknown type. You'll look at `unknown` in much more depth in the next chapter.

One way to make TypeScript know what type you want the Set to hold is to pass in some initial values:

```
const formats = new Set(["CD", "DVD"]);
```

In this case, since you specified two strings when creating the Set, TypeScript knows that `formats` is a Set of strings:

```
// hovering over formats shows:  
const formats: Set<string>;
```

But it's not always the case that you know exactly what values you want to pass to a Set when you create it. You might want to create an empty Set that you know will hold strings later.

For this, you can pass a type to Set using the angle bracket syntax:

```
const formats = new Set<string>();
```

Now `formats` understands that it's a set of strings, and adding anything other than a string will fail:

```
formats.add("Digital"); // this works  
  
formats.add(8); // red squiggly line under 8  
  
// hovering over 8 shows:  
  
Argument of type 'number' is not assignable to parameter of  
type 'string'
```

This is a really important thing to understand in TypeScript. You can pass types as well as values to functions.

Not All Functions Can Receive Types

Most functions in TypeScript *can't* receive types. For example, let's look at `document.getElementById` that comes in from the DOM typing. A

common scenario where you might want to pass a type is when calling `document.getElementById`. Here you're trying to get an audio element:

```
const audioElement = document.getElementById("player");
```

You know that `audioElement` is going to be an `HTMLAudioElement`, so it seems like you should be able to pass it to `document.getElementById`. However, you can't:

```
const audioElement = document.getElementById<HTMLAudioElement>("player");  
// red squiggly line under HTMLAudioElement
```

When you hover over `HTMLAudioElement`, the error tells you that it expects zero type arguments, but you passed it one—in this case, `player`.

You can see whether a function can receive type arguments by hovering over it. Let's try hovering over `.getElementById`:

```
// hovering over .getElementById shows:  
(method) Document.getElementById(elementId: string): HTMLElement | null
```

Notice that `.getElementById` contains no angle brackets in its hover, which is why you can't pass a type to it.

Let's contrast it with a function that *can* receive type arguments, like `document.querySelector`:

```
const audioElement = document.querySelector("#player");  
  
// hovering over .querySelector shows:  
(method) ParentNode.querySelector<Element>(selectors: string): Element | null
```

This type definition shows you that `.querySelector` has some angle brackets before the parentheses. In the brackets is the default value—in this

case, `Element`. To fix the code, you could replace `.getElementById` with `.querySelector` and use the `#player` selector to find the audio element:

```
const audioElement = document.querySelector<HTMLAudioElement>("#player");
```

And everything works.

To tell whether a function can receive a type argument, hover over it and check whether it has any angle brackets.

Exercise 4-12: Passing Types to Map

Here you are creating a `Map`, a JavaScript feature that represents a dictionary. In this case, you want to pass in a number for the key and an object for the value:

```
const userMap = new Map();

userMap.set(1, {name: "Max", age: 30});

userMap.set(2, {name: "Manuel", age: 31});

// @ts-expect-error // red squiggly line under @ts-expect-error
userMap.set("3", {name: "Anna", age: 29});

// @ts-expect-error // red squiggly line under @ts-expect-error
userMap.set(3, "123");
```

There are red lines under the `@ts-expect-error` directives because currently any type of key and value is allowed in the `Map`:

```
// hovering over Map shows:
var Map: MapConstructor

new () => Map<any, any> (+3 overloads)
```

How would you type the `userMap` so that the key must be a number and the value is an object with `name` and `age` properties?

See <https://totalts.link/essentials-4-12/>.

Solution

There are a few different ways to solve this problem, but we'll start with the most straightforward one. The first thing to do is to create a `User` type:

```
type User = {  
  name: string;  
  age: number;  
};
```

Following the patterns you've seen so far, you can pass `number` and `User` as the types for the `Map`:

```
const userMap = new Map<number, User>();
```

That's right, some functions can receive *multiple* type arguments. In this case, the `Map` constructor can receive two types: one for the key and one for the value.

With this change, the errors go away, and you can no longer pass incorrect types into the `userMap.set` function.

You can also express the `User` type inline:

```
const userMap = new Map<number, {name: string; age: number}>  
( );
```

Exercise 4-13: JSON.parse() Can't Receive Type Arguments

Consider the following code, which uses `JSON.parse()` to parse some JSON:

```
const parsedData = JSON.parse(<  
  name: string; // red squiggly line under the full {} argument
```



```
ent
  age: number;
}>({'name': "Alice", "age": 30}');

```

There is currently an error under the type argument for `JSON.parse`:

Expected 0 type arguments, but got 1.

A test that checks the type of `parsedData` is currently failing, since it is typed as `any` instead of the expected type:

```
type test = Expect<
  Equal<
    // red squiggly line under the full Equal<>
    typeof parsedData,
    {
      name: string;
      age: number;
    }
  >
>;

it("Should be the correct shape", () => {
  expect(parsedData).toEqual({
    name: "Alice",
    age: 30,
  });
});

```

You've tried to pass a type argument to the `JSON.parse` function, but it doesn't appear to be working in this case. The test errors tell you that the type of `parsed` is not what you expect. The properties `name` and `age` are not being recognized.

Why is this happening? What would be a different way to correct these type errors?

See <https://totalts.link/essentials-4-13/>.

Solution

Let's look a bit more closely at the error message you get when passing a type argument to `JSON.parse`:

```
Expected 0 type arguments, but got 1.
```

This message indicates that TypeScript is not expecting anything in the angle brackets when calling `JSON.parse`. To resolve this error, you can remove the angle brackets:

```
const parsedData = JSON.parse('{ "name": "Alice", "age": 30 }');
```

Now that `.parse` is receiving the correct number of type arguments, TypeScript is happy. However, you want your parsed data to have the correct type. Hovering over `JSON.parse`, you can see its type definition:

```
JSON.parse(text: string, reviver?: ((this: any, key: string, value: any) => any) | undefined): any
```

It always returns `any`, which is a bit of a problem.

To get around this issue, you can give `parsedData` a variable type annotation with `name: string` and `age: number`:

```
const parsedData: {  
  name: string;  
  age: number;  
} = JSON.parse('{ "name": "Alice", "age": 30 }');
```

Now you have `parsedData` typed as you want it to be.

The reason this works is because `any` disables type checking. So, you can assign it any type you want. You could assign it something that doesn't make sense, like `number`, and TypeScript wouldn't complain:

```
const parsedData: number = JSON.parse('{ "name": "Alice", "age": 30 }');
```

This is more “type faith” than “type safe.” You are hoping that `parsedData` is the type you expect it to be. This relies on us keeping the type annotation up to date with the actual data.

Typing Functions

Let’s look at some additional ways to annotate function parameters.

Optional Parameters

For cases where a function parameter is optional, you can add the `?` modifier before the `:`. Say you wanted to add an optional `releaseDate` parameter to the `logAlbumInfo` function. You could do so like this:

```
const logAlbumInfo = (  
  title: string,  
  trackCount: number,  
  isReleased: boolean,  
  releaseDate?: string,  
) => {  
  // rest of function body  
};
```

Now you can call `logAlbumInfo` and include a release date string or leave it out:

```
logAlbumInfo("Midnights", 13, true, "2022-10-21");  
  
logAlbumInfo("American Beauty", 10, true);
```

Hovering over the optional `releaseDate` parameter shows that it is now typed as `string | undefined`.

We’ll discuss the `|` symbol later, but this means the parameter could be either a `string` or `undefined`. It would be acceptable to literally pass `undefined` as a second argument, or it can be omitted altogether.

Default Parameters

In addition to marking parameters as optional, you can set default values for parameters by using the `=` operator.

For example, you could set the `format` to default to `"CD"` if no `format` is provided:

```
const logAlbumInfo = (  
  title: string,  
  trackCount: number,  
  isReleased: boolean,  
  format: string = "CD",  
) => {  
  // rest of function body  
};
```

The annotation of `: string` can also be omitted since it can infer the type of the `format` parameter from the value provided, which is another nice example of type inference:

```
const logAlbumInfo = (  
  title: string,  
  trackCount: number,  
  isReleased: boolean,  
  format = "CD",  
) => {  
  // rest of function body  
};
```

Function Return Types

In addition to setting parameter types, you can also set the return type of a function. You can annotate the return type of a function by placing a `:` and the type after the closing parenthesis of the parameter list. For the `logAlbumInfo` function, you can specify that the function will return a `string`:

```
const logAlbumInfo = (  
  title: string,  
  trackCount: number,
```

```
    isReleased: boolean,  
  ): string => {  
    // rest of function body  
  };
```

If the value returned from a function doesn't match the type that was specified, TypeScript will show an error:

```
const logAlbumInfo = (  
  title: string,  
  trackCount: number,  
  isReleased: boolean,  
): string => {  
  return 123; // red squiggly line under 123  
};
```

Return types are useful for when you want to ensure that a function returns a specific type of value.

Rest Parameters

Just like in JavaScript, TypeScript supports rest parameters by using the `...` syntax for the final parameter, which allows you to pass in any number of arguments to a function. For example, this `printAlbumFormats` is set up to accept an album and any number of formats:

```
function getAlbumFormats(album: Album, ...formats: string  
[]) {  
  return `${album.title} is available in the following forma  
ts: ${formats.join(  
    ", ",  
  )}`;  
}
```

Declaring the parameter with the `...formats` syntax combined with an array of strings lets us pass in any number of strings to the function

```
getAlbumFormats(  
  {artist: "Radiohead", title: "OK Computer", year: 1997},  
  "CD",  
  "LP",  
  "Cassette",  
);
```

or even spread in an array of strings:

```
const albumFormats = ["CD", "LP", "Cassette"];  
  
getAlbumFormats(  
  {artist: "Radiohead", title: "OK Computer", year: 1997},  
  . . .albumFormats,  
);
```

This means that `formats` will be passed as an array to the function.

As an alternative, you can use the `Array<>` syntax:

```
function getAlbumFormats(album: Album, . . .formats: Array<s  
tring>) {  
  // function body  
}
```

Function Types

We've shown how to use type annotations to specify the types of function parameters, but you can also use TypeScript to describe the types of the functions themselves. You can do this using the following syntax:

```
type Mapper = (item: string) => number;
```

This is a type alias for a function that takes in a `string` and returns a `number`.

You could then use this to describe a callback function passed to another function:

```
const mapOverItems = (items: string[], map: Mapper) => {  
    return items.map(map);  
};
```

Or, you could declare it inline:

```
const mapOverItems = (items: string[], map: (item: string) =  
> number) => {  
    return items.map(map);  
};
```

This lets you pass a function to `mapOverItems` that changes the value of the items in the array:

```
const arrayOfNumbers = mapOverItems(["1", "2", "3"], (item)  
=> {  
    return parseInt(item) * 100;  
});
```

Function types are as flexible as function definitions. You can declare multiple parameters, rest parameters, and optional parameters:

```
// Optional parameters  
type WithOptional = (index?: number) => number;  
  
// Rest parameters  
type WithRest = (. . .rest: string[]) => number;  
  
// Multiple parameters  
  
type WithMultiple = (first: string, second: string) => number;
```

The void Type

Some functions don't return anything. They perform some kind of action, but they don't produce a value. A great example is `console.log`:

```
const logResult = console.log("Hello!");
```

What type do you expect `logResult` to be? In JavaScript, the value is undefined. If you were to use `console.log(logResult)`, that's what you'd see in the console. But TypeScript has a special type for these situations when a function's return value should be deliberately ignored. It's called `void`. If you hover over `.log` in `console.log`, you'll see that it returns `void`:

```
(method) Console.log(. . .data: any[]): void
```

So, `logResult` is also `void`. This is TypeScript's way of saying "ignore the result of this function call."

Async Functions

You've seen how to strongly type what a function returns via a return type:

```
const getUser = (id: string): User => {  
  // function body  
};
```

But what about when the function is asynchronous?

```
const getUser = async (id: string): User => {  
  // red squiggly line under User  
  // function body  
};  
// hovering over User shows:  
The return type of an async function or method must be  
the global Promise type. Did you mean to write 'Promise'?
```

Fortunately, TypeScript's error message is helpful here. It's telling you that the return type of an `async` function must be a `Promise`, so you can pass `User` to a `Promise`:

```
const getUser = async (id: string): Promise<User> => {  
  const user = await db.users.get(id);  
  
  return user;  
};
```

Now your function must return a Promise that resolves to a User.

Exercise 4-14: Optional Function Parameters

Here you have a concatName function whose implementation takes in two string parameters, first and last.

If there's no last name passed, the return would be just the first name. Otherwise, it would return first concatenated with last:

```
const concatName = (first: string, last: string) => {  
  if (!last) {  
    return first;  
  }  
  
  return `${first} ${last}`;  
};
```

When calling concatName with a first and last name, the function works as expected without errors:

```
const result = concatName("John", "Doe");
```

However, when calling concatName with just a first name, you get an error:

```
const result2 = concatName("John"); // red squiggly line under concatName("John")
```

The error message reads:

```
Expected 2 arguments, but got 1.
```

Try to use an optional parameter annotation to fix the error.
See <https://totalts.link/essentials-4-14/>.

Solution

By adding a question mark to the end of a parameter, it will be marked as optional:

```
function concatName(first: string, last?: string) {  
    // . . .implementation  
}
```

Exercise 4-15: Default Function Parameters

Here you have the same concatName function as before, where the last name is optional:

```
const concatName = (first: string, last?: string) => {  
    if (!last) {  
        return first;  
    }  
  
    return `${first} ${last}`;  
};
```

You also have a couple of tests. The first test checks that the function returns the full name when passed a first and last name:

```
it("should return the full name", () => {  
    const result = concatName("John", "Doe");  
  
    type test = Expect<Equal<typeof result, string>>;  
  
    expect(result).toEqual("John Doe");  
});
```

However, the second test expects that when concatName is called with just a first name as an argument, the function should use Pocock as the

default last name:

```
it("should return the first name", () => {  
  const result = concatName("John");  
  
  type test = Expect<Equal<typeof result, string>>;  
  
  expect(result).toEqual("John Pocock");  
});
```

This test currently fails, with the output from Vitest indicating the error is on the expect line:

```
AssertionError: expected 'John' to deeply equal 'John Pocock'  
k'
```

```
- Expected
```

```
+ Received
```

```
- John Pocock
```

```
+ John
```

```
expect(result).toEqual("John Pocock");
```

Update the concatName function to use Pocock as the default last name if one is not provided.

See <https://totalts.link/essentials-4-15/>.

Solution

To add a default parameter in TypeScript, you use the = syntax that is also used in JavaScript. In this case, you update last to default to "Pocock" if no value is provided:

```
export const concatName = (first: string, last?: string = "Pocock") => {  
  return `${first} ${last}`;  
};
```

While this passes your runtime tests, it actually fails in TypeScript:

Parameter cannot have question mark and initializer.

This is because TypeScript doesn't allow us to have both an optional parameter and a default value. The optionality is already implied by the default value.

To fix the error, remove the question mark from the last parameter:

```
export const concatName = (first: string, last = "Pocock") => {
  return `${first} ${last}`;
};
```

Exercise 4-16: Rest Parameters

Here you have a concatenate function that takes in a variable number of strings:

```
export function concatenate(...strings) {
  // red squiggly line under `...strings`
  return strings.join("");
}
```

The code runs as expected, but there's an error on the `...strings` rest parameter:

```
// hovering over ...strings shows:
Rest parameter 'strings' implicitly has an 'any[]' type.
```

How would you update the rest parameter to specify that it should be an array of strings?

See <https://totalts.link/essentials-4-16/>.

Solution

When using rest parameters, all of the arguments passed to the function will be collected into an array. This means that the `strings` parameter can be typed as an array of strings:

```
export function concatenate(. . .strings: string[]) {  
  return strings.join("");  
}
```

Or, of course, it can be typed using the `Array<>` syntax:

```
export function concatenate(. . .strings: Array<string>) {  
  return strings.join("");  
}
```

Exercise 4-17: Function Types

Here you have a `modifyUser` function that takes in an array of users, an `id` of the user that you want to change, and a `makeChange` function that makes that change:

```
type User = {  
  id: string;  
  name: string;  
};  
  
const modifyUser = (user: User[], id: string, makeChange) =>  
{  
  // red squiggly line under `makeChange`  
  
  return user.map((u) => {  
    if (u.id === id) {  
      return makeChange(u);  
    }  
  
    return u;  
  });  
};
```

Currently there is an error under makeChange:

Parameter `makeChange` implicitly has an `any` type.

Here's an example of how this function would be called:

```
const users: User[] = [
  {id: "1", name: "John"},
  {id: "2", name: "Jane"},
];

modifyUser(users, "1", (user) => {
  // red squiggly line under user
  return { . . .user, name: "Waqas"};
});
```

In the previous example, the user parameter to the error function also has the “implicit any” error.

The modifyUser type annotation for the makeChange function needs to be updated so that it returns a modified user. For example, you should not be able to return a name of 123 because in the User type, name is a string:

```
modifyUser(
  users,
  "1",
  // @ts-expect-error
  (user) => {
    return { . . .user, name: 123};
  },
);
```

How would you type makeChange as a function so that it takes in a User and returns a User?

See <https://totalts.link/essentials-4-17/>.

Solution

Let's start by annotating the `makeChange` parameter to be a function. For now, you'll specify that it returns `any`:

```
const modifyUser = (user: User[], id: string, makeChange: ()  
=> any) => {  
  return user.map((u) => {  
    if (u.id === id) {  
      return makeChange(u); // red squiggly line under `u`  
    }  
  
    return u;  
  });  
};
```

With this first change in place, you get an error under `u` when calling `makeChange` since you said that `makeChange` takes in no arguments:

```
// In the user.map() function  
  
return makeChange(u)  
  
// hovering over u shows:  
  
Expected 0 arguments, but got 1.
```

This says you need to add a parameter to the `makeChange` function type. In this case, you will specify that `user` is of type `User`:

```
const modifyUser = (  
  user: User[],  
  id: string,  
  makeChange: (user: User) => any,  
) => {  
  // function body  
};
```

This is pretty good, but you also need to make sure your `makeChange` function returns a `User`:

```
const modifyUser = (  
  user: User[],  
  id: string,  
  makeChange: (user: User) => User,  
) => {  
  // function body  
};
```

Now the errors are resolved, and you have autocompletion for the `User` properties when writing a `makeChange` function.

Optionally, you can clean up the code a bit by creating a type alias for the `makeChange` function type:

```
type MakeChangeFunc = (user: User) => User;  
  
const modifyUser = (user: User[], id: string, makeChange: Ma  
keChangeFunc) => {  
  // function body  
};
```

Both techniques behave the same, but if you need to reuse the `makeChange` function type, a type alias is the way to go.

Exercise 4-18: Functions Returning void

Here you explore a classic web development example. You have an `addClickListener` function that takes in a listener function and adds it to the document:

```
const addClickListener = (listener) => {  
  // red squiggly line under `listener`  
  
  document.addEventListener("click", listener);  
};  
  
addClickListener(() => {  
  console.log("Clicked!");  
});
```

Currently there is an error under `listener` because it doesn't have a type signature. You're also *not* getting an error when you pass an incorrect value to `addClickListener`:

```
addClickListener(  
  // @ts-expect-error // red squiggly line under @ts-expect-error  
  "abc",  
);
```

This is triggering your `@ts-expect-error` directive.

How should `addClickListener` be typed so that each error is resolved?

See <https://totalts.link/essentials-4-18/>.

Solution

Let's start by annotating the `listener` parameter to be a function. For now, you'll specify that it returns a string:

```
const addClickListener = (listener: () => string) => {  
  document.addEventListener("click", listener);  
};
```

The problem is that you now have an error when you call `addClickListener` with a function that returns nothing:

```
addClickListener(() => {  
  // red squiggly line under `() => {`  
  
  console.log("Clicked!");  
});
```

When you hover over the error, you see the following message:

```
Argument of type '() => void' is not assignable to parameter  
of type '() => string'.  
Type 'void' is not assignable to type 'string'.
```

The error message tells you that the `listener` function is returning `void`, which is not assignable to `string`. This suggests that instead of typing the `listener` parameter as a function that returns a string, you should type it as a function that returns `void`:

```
const addClickListener = (listener: () => void) => {  
  document.addEventListener("click", listener);  
};
```

This is a great way to tell TypeScript that you don't care about the return value of the `listener` function.

Exercise 4-19: void vs. undefined

Let's say you've got a function that accepts a callback and calls it. The callback doesn't return anything, so you've typed it as `() => undefined`:

```
const acceptsCallback = (callback: () => undefined) => {  
  callback();  
};
```

But you're getting an error when you try to pass in `returnString`, a function that *does* return something:

```
const returnString = () => {  
  return "Hello!";  
};  
  
// Argument of type '() => string' is not  
// assignable to parameter of type '() => undefined'.  
acceptsCallback(returnString); // red squiggly line under `r  
eturnString`
```

Why is this happening? Can you alter the type of `acceptsCallback` to fix this error?

See <https://totalts.link/essentials-4-19/>.

Solution

The solution is to change the type of callback to `() => void`:

```
const acceptsCallback = (callback: () => void) => {  
    callback();  
};
```

Now you can pass in `returnString` without any issues. This is because `returnString` returns a string, and `void` tells TypeScript to ignore the return value when comparing them. If you really don't care about the result of a function, type it as `() => void`.

Exercise 4-20: Async Functions

This `fetchData` function awaits the response from a call to `fetch` and then gets the data by calling `response.json()`:

```
async function fetchData() {  
    const response = await fetch("https://api.example.com/data");  
  
    const data = await response.json();  
  
    return data;  
}
```

It's worth noting a couple of things here. Hovering over `response`, you can see that it has a type of `Response`, which is a globally available type:

```
// hovering over response shows:  
const response: Response;
```

When hovering over `response.json()`, you can see that it returns a `Promise<any>`:

```
// hovering over response.json() shows:  
const response.json(): Promise<any>
```

If you were to remove the `await` keyword from the call to `fetch`, the return type would also become `Promise<any>`:

```
const response = fetch("https://api.example.com/data");

// hovering over response shows:
const response: Promise<any>;
```

Consider this example and its test:

```
typescript
const example = async () => {
  const data = await fetchData();

  type test = Expect<Equal<typeof data, number>>;
};
```

The test is currently failing because `data` is typed as `any` instead of `number`.

How can you type `data` as a `number` without changing the calls to `fetch` or `response.json()`? There are two possible solutions.

See <https://totalts.link/essentials-4-20/>.

Solutions

You might be tempted to try passing a type argument to `fetch`, similar to how you would with `Map` or `Set`. However, hovering over `fetch`, you can see that it doesn't accept type arguments:

```
// this won't work!

const response = fetch<number>("https://api.example.com/data");
// red squiggly line under number

// hovering over fetch shows:
```

```
function fetch(  
  input: RequestInfo | URL,  
  init?: RequestInit | undefined,  
): Promise;
```

You also can't add a type annotation to `response.json()` because it doesn't accept type arguments either:

```
// this won't work!  
  
const data: number = await response.json<number>();  
// red squiggly line under number  
  
// hovering over number shows:  
  
Expected 0 type arguments, but got 1.
```

One thing that will work is to specify that `data` is a number:

```
const data: number = await response.json();
```

It works because `data` was any before, and `await response.json()` returns any. So, now you're putting any into a slot that requires a number.

However, the best way to solve this problem is to add a return type to the function. In this case, it should be a number:

```
// red squiggly line under number  
async function fetchData(): number {  
  // function body  
}
```

Now `data` is typed as a number, except you have an error under your return type annotation:

The return type of an async function or method must be the global `Promise` type.
Did you mean to write `'Promise'`?

Therefore, you should change the return type to `Promise<number>`:

```
async function fetchData(): Promise<number> {  
    const response = await fetch("https://api.example.com/data");  
  
    const data = await response.json();  
  
    return data;  
}
```

By wrapping the number in `Promise<>`, you make sure that the data is awaited before the type is figured out.

Summary

This chapter introduced the foundational concepts of TypeScript's type system, focusing on type annotations and inference. You learned when to explicitly annotate your code and when TypeScript can figure out types automatically.

TypeScript provides basic types like `string`, `number`, `boolean`, `symbol`, `bigint`, `null`, and `undefined` that correspond to JavaScript's primitive values. These form the building blocks for more complex type annotations.

Function parameters always require type annotations since TypeScript can't infer what types they should accept. Variable annotations are often optional because TypeScript can infer types from assigned values, but function parameters need explicit typing.

The `any` type disables TypeScript's type checking entirely, allowing anything to be assigned or accessed without errors. While useful as an escape hatch, overusing `any` defeats the purpose of using TypeScript.

Object literal types use curly bracket syntax to describe object shapes, with optional properties marked using the `?` operator. Type aliases created with the `type` keyword let you define reusable types and share them across modules.

Arrays can be typed using square bracket syntax, like `string[]`, or using the generic `Array<string>` syntax. Tuples specify fixed-length arrays

where each position has its own type, which is useful for grouping related data without creating new types.

Some functions, like `Set` and `Map`, can receive type arguments using angle brackets, while others, like `JSON.parse`, cannot. You can check by hovering over functions to see if they accept type parameters.

Function types describe the shape of functions themselves, including parameter and return types. Optional parameters use `?`, default parameters use `=`, and rest parameters use the spread syntax with array types.

The `void` type indicates functions that don't return meaningful values, while async functions must return `Promise<T>` where `T` is the resolved value type. These concepts form the essential foundation for working effectively with TypeScript's type system.

5

UNIONS, LITERALS, AND NARROWING



In this chapter, you'll see how TypeScript can help when a value is one of many possible types. You'll first look at declaring those types using union types, and then you'll see how TypeScript can narrow down the type of a value based on your runtime code.

Union Types

A *union type* is TypeScript's way of saying that a value can be “either this type or that type.” This situation comes up in JavaScript all the time. Imagine you have a value that is a `string` on Tuesdays but `null` the rest of the time:

```
const message = Date.now() % 2 === 0 ? "Hello Tuesdays!" : null;
```

If you hover over `message`, you'll see that TypeScript has inferred its type as `string | null`:

```
// hovering over message shows:  
const message: string | null;
```

This is a union type. It means that message can be either a string or null.

Declaring Union Types

To create a union type, use the `|` operator to separate the types. Each type you specify is called a *member* of the union.

For example, say you have a `logId` function that accepts an `id`:

```
const logId = (id: string | number) => {  
  console.log(id);  
};
```

This syntax means that `logId` can accept a string or a number as an argument but not a boolean or any other type:

```
logId("abc"); // "abc"  
  
logId(123); // 123  
  
logId(true); // red squiggly line under true
```

Union types also work when creating your own type aliases. For example, you can refactor your earlier definition into a type alias:

```
type Id = number | string;  
  
function logId(id: Id) {  
  console.log(id);  
}
```

It's possible for union types to contain many different types; they don't have to all be primitives, and they don't need to be related. If the union type gets too large, you can use this syntax (with the `|` before the first member of the union) to make it more readable:

```
type AllSortsOfStuff =  
  | string  
  | number
```

```
| boolean
| object
| null
| {
  name: string;
  age: number;
};
```

Union types are a great tool for creating flexible type definitions.

Literal Types

Just as TypeScript allows for creating union types from multiple types, it also allows you to create types that represent a specific primitive value. These are called *literal types*, and they can be used to represent strings, numbers, or Booleans that have specific values, like so:

```
type YesOrNo = "yes" | "no";
type StatusCode = 200 | 404 | 500;
```

In the `YesOrNo` type, the `|` operator is used to create a union of the string literals `"yes"` and `"no"`. This means that a value of type `YesOrNo` can be only one of these two strings.

This feature is what powers the autocomplete you've seen in functions like `document.addEventListener`:

```
document.addEventListener(
  // autocomplete will suggest DOMContentLoaded, mouseover,
  // and so on.
  "click",
  () => {},
);
```

The first argument to `addEventListener` is a union of string literals, which is why you get autocompletion for the different event types.

Combining Unions with Unions

What happens when you make a union of two union types? They combine to make one big union. For example, you can create `DigitalFormat` and `PhysicalFormat` types that contain a union of literal values:

```
type DigitalFormat = "MP3" | "FLAC";

type PhysicalFormat = "LP" | "CD" | "Cassette";
```

You can then specify `AlbumFormat` as a union of `DigitalFormat` and `PhysicalFormat`:

```
type AlbumFormat = DigitalFormat | PhysicalFormat;
```

With these union types specified, you can ensure that each function handles only the cases it's supposed to: The `DigitalFormat` type can be used for functions that handle digital formats, the `PhysicalFormat` type for functions that handle analog formats, and the `AlbumFormat` type for functions that handle all cases.

If you try to pass an incorrect format to a function, TypeScript will throw an error:

```
const getAlbumFormats = (format: PhysicalFormat) => {
  // function body
};

getAlbumFormats("MP3"); // red squiggly line under "MP3"
```

Exercise 5-1: string or null

Here you have a function called `getUsername` that takes in a username string. If the username is not equal to `null`, you return a new interpolated string. Otherwise, you return `"Guest"`:

```
function getUsername(username: string) {
  if (username !== null) {
    return `User: ${username}`;
  }
}
```

```
} else {  
    return "Guest";  
}  
}
```

In the first test, you'll call `getUsername` and pass in a string of "Alice", which passes as expected. However, in the second test, you'll have a red squiggly line under `null` when passing it into `getUsername`:

```
const result = getUsername("Alice");  
  
type test = Expect<Equal<typeof result, string>>;  
  
const result2 = getUsername(null); // red squiggly line under null  
  
type test2 = Expect<Equal<typeof result2, string>>;
```

Hovering over `null` shows the following error message:

```
Argument of type 'null' is not assignable to parameter of type 'string'.
```

Normally you wouldn't explicitly call the `getUsername` function with `null`, but in this case, it's important to handle `null` values. For example, you might be getting the username from a user record in a database, and the user might or might not have a name, depending on how they signed up.

Currently, the username parameter accepts only a `string` type, and the check for `null` isn't doing anything. Update the function parameter's type so that the errors are resolved and the function can handle `null`.

See <https://totalts.link/essentials-5-1/>.

Solution

The solution involves updating the username parameter to be a union of `string` and `null`:

```
function getUsername(username: string | null) {  
    // function body  
}
```

With this change, the `getUsername` function will now accept `null` as a valid value for the `username` parameter, and the errors will be resolved.

Exercise 5-2: Restricting Function Parameters

Here's a `move` function that takes in a `direction` of type `string` and a `distance` of type `number`:

```
function move(direction: string, distance: number) {  
    // Move the specified distance in the given direction.  
}
```

The implementation of the function is not important, but the idea is that you want to be able to move up, down, left, or right.

Here's what calling the `move` function might look like:

```
move("up", 10);  
  
move("left", 5);
```

To test this function, you have some `@ts-expect-error` directives that tell TypeScript you expect the following lines to throw an error. However, since the `move` function takes in a `string` for the `direction` parameter, you can pass in any string you want, even if it's not a valid direction.

This leads to TypeScript drawing a red squiggly line under the `@ts-expect-error` directives:

```
move(  
    // @ts-expect-error - "up-right" is not a valid direction  
    // red squiggly line under @ts-expect-error  
    "up-right",  
    10,  
);
```

```
move(  
  // @ts-expect-error - "down-left" is not a valid direction  
  // red squiggly line under @ts-expect-error  
  "down-left",  
  20,  
);
```

Update the move function so that it accepts only the strings "up", "down", "left", and "right". This way, TypeScript will throw an error when any other string is passed in.

See <https://totalts.link/essentials-5-2/>.

Solution

To restrict what the direction can be, use a union type of literal string values. Here's what this looks like:

```
function move(direction: "up" | "down" | "left" | "right", distance: number) {  
  // Move the specified distance in the given direction.  
}
```

With this change, there are now autocomplete suggestions for the possible direction values. To clean things up, you could create a new type alias called `Direction` and update the parameter accordingly:

```
type Direction = "up" | "down" | "left" | "right";  
  
function move(direction: Direction, distance: number) {  
  // Move the specified distance in the given direction.  
}
```

Wide and Narrow Types

TypeScript has a notion of “wide” and “narrow” types.

Some types are wider versions of other types. For example, `string` is wider than the literal string `"small"`. This is because `string` can be any

string, while "small" can be only the string "small".

In reverse, you could say that "small" is a narrower type than string because it's a more specific version of a string. Similarly, 404 is a narrower type than number, and true is a narrower type than boolean.

Note that this is only true of types that have some kind of shared relationship. For example, "small" is not a narrower version of number because "small" itself is not a number.

In TypeScript, the narrower version of a type can always take the place of the wider version. For example, if a function accepts a string, you can pass in "small":

```
const logSize = (size: string) => {  
  console.log(size.toUpperCase());  
};  
  
logSize("small");
```

But if a function accepts a literal "small" type, you can't pass any random string:

```
const recordOfSizes = {  
  small: "small",  
  large: "large",  
};  
  
const logSize = (size: "small" | "large") => {  
  console.log(recordOfSizes[size]);  
};  
  
logSize("medium"); // red squiggly line under "medium"
```

If you're familiar with the concept of *subtypes* and *supertypes* in set theory, this is a similar idea: "small" is a subtype of string (it's more specific), and string is a supertype of "small".

Unions Are Wider Than Their Members

A union type is a wider type than its members. For example, `string | number` is wider than `string` or `number` on its own. This means you can pass a `string` or a `number` to a function that accepts `string | number`:

```
function logId(id: string | number) {  
    console.log(id);  
}  
  
logId("abc");  
logId(123);
```

However, this doesn't work in reverse. You can't pass `string | number` to a function that only accepts `string`. For example, if you change the `logId` function to accept only a `number`, TypeScript throws an error when you try to pass `string | number` to it:

```
function logId(id: number) {  
    console.log(`The id is ${id}`);  
}  
  
type User = {  
    id: string | number;  
};  
  
const user: User = {  
    id: 123,  
};  
  
logId(user.id); // red squiggly line under user.id
```

Hovering over `user.id` shows the following:

```
Argument of type 'string | number' is not assignable to parameter of type 'number'.  
  Type 'string' is not assignable to type 'number'.
```

So, it's important to think of a union type as a wider type than its members.

The Process of Narrowing

TypeScript lets you take a wider type and make it narrower using runtime code. This can be useful when you want to do different things based on the type of a value. For example, you might handle a string differently from a number, or "small" differently from "large".

One way you can narrow down the type of a value is to use the `typeof` operator combined with an `if` statement. Consider a function `getAlbumYear` that takes in a parameter `year`, which can be either a string or a number. Here's how the `typeof` operator could be used to narrow down the type of `year`:

```
const getAlbumYear = (year: string | number) => {  
  if (typeof year === "string") {  
    console.log(`The album was released in ${year.toUpperCase()}.`); // year is string  
  } else if (typeof year === "number") {  
    console.log(`The album was released in ${year.toFixed(0)}.`); // year is number  
  }  
};
```

It looks straightforward, but there are some important things to realize about what's happening behind the scenes.

Scoping plays a big role in narrowing. In the first `if` block, TypeScript understands that `year` is a string because we've used the `typeof` operator to check its type. In the `else if` block, TypeScript understands that `year` is a number because we've used the `typeof` operator to check its type.

Thanks to this narrowing, TypeScript will let you call `toUpperCase` on `year` when it's a string, and `toFixed` on `year` when it's a number. However, anywhere outside of the conditional block, the type of `year` is still the union `string | number`. This is because narrowing applies only within the block's scope.

For the sake of illustration, if you add a boolean to the year union, the first if block will still end up with a type of string, but the else block will end up with a type of number | boolean:

```
const getAlbumYear = (year: string | number | boolean) => {
  if (typeof year === "string") {
    console.log(`The album was released in ${year}.`); // year is string
  } else if (typeof year === "number") {
    console.log(`The album was released in ${year}.`); // year is number
  }

  console.log(year); // year is string | number | boolean
};
```

The typeof operator is just one way to narrow types. TypeScript can also use conditional operators like && and ||, taking the truthiness into account for coercing the boolean value. It's also possible to use other operators like instanceof and in for checking object properties. You can even throw errors or use early returns to narrow types.

You'll take a closer look at these in the following exercises.

Exercise 5-3: Narrowing with if Statements

Here's a function called validateUsername that takes in a string or null and will always return a boolean:

```
function validateUsername(username: string | null): boolean
{
  return username.length > 5; // red squiggly line under username

  return false;
}
```

Tests for checking the length of the username pass as expected:

```
it("should return true for valid usernames", () => {
  expect(validateUsername("Matt1234")).toBe(true);

  expect(validateUsername("Alice")).toBe(false);

  expect(validateUsername("Bob")).toBe(false);
});
```

However, there is an error underneath username in the function body because you are attempting to access a property off of an object that could possibly be `null`.

This means that the following test fails:

```
it("Should return false for null", () => {
  expect(validateUsername(null)).toBe(false);
});
```

Rewrite the `validateUsername` function to add narrowing so that the `null` case is handled and the tests all pass.

See <https://totalts.link/essentials-5-3/>.

Solutions

You can validate the username length in several different ways:

Option 1

You could use an `if` statement to check if `username` is truthy. If it is, you can return `username.length > 5`. Otherwise, you can return `false`:

```
function validateUsername(username: string | null): boolean
{
  // Rewrite this function to make the error go away.
  if (username) {
    return username.length > 5;
  }

  return false;
}
```

There is a catch to this piece of logic: If username is an empty string, it will return false because an empty string is falsy. This matches the behavior you want for this exercise, but it's important to bear that in mind.

Option 2

You could use `typeof` to check if the username is a string:

```
function validateUsername(username: string | null): boolean
{
  if (typeof username === "string") {
    return username.length > 5;
  }
  return false;
}
```

This avoids the issue with empty strings.

Option 3

Similar to the previous option, you could check if `typeof username !== "string"`. In this case, if username is not a string, you know it's null and could return false right away. Otherwise, you'd return the check for length being greater than 5:

```
function validateUsername(username: string | null | undefined): boolean {
  if (typeof username !== "string") {
    return false;
  }
  return username.length > 5;
}
```

This shows that TypeScript understands the *reverse* of a check. Very smart.

Option 4

An odd JavaScript quirk is that the type of `null` is equal to `"object"`. TypeScript knows this, so you can use it to your advantage. Check if

username is an object, and if it is, you can return false:

```
function validateUsername(username: string | null): boolean
{
  if (typeof username === "object") {
    return false;
  }
  return username.length > 5;
}
```

Option 5

Finally, for readability and reusability purposes, you could store the check in its own variable `isUsernameOK`. TypeScript is smart enough to understand that the value of the variable corresponds to whether username is a string:

```
function validateUsername(username: string | null): boolean
{
  const isUsernameOK = typeof username === "string";
  if (isUsernameOK) {
    return username.length > 5;
  }

  return false;
}
```

All of these options use `if` statements to perform checks by narrowing types with `typeof`. No matter which option you go with, remember that you can use an `if` statement to narrow your type and add code to the case that the condition passes.

Exercise 5-4: Throwing Errors to Narrow

Here's a line of code that uses `document.getElementById` to fetch an HTML element, which can return either an `HTMLElement` or `null`:

```
const appElement = document.getElementById("app");
```

Currently, a test to see if the `appElement` is an `HTMLElement` fails:

```
type Test = Expect<Equal<typeof appElement, HTMLElement>>;  
// red squiggly line under Equal<>
```

Use `throw` to narrow down the type of `appElement` before it's checked by the test.

See <https://totalts.link/essentials-5-4/>.

Solution

The issue with this code is that `document.getElementById` returns `null` | `HTMLElement`. You need to make sure that `appElement` is an `HTMLElement` before attempting to use it.

There needs to be a check to make sure `appElement` exists. Add an `if` statement that checks if `appElement` is falsy, throwing an error as appropriate:

```
const appElement = document.getElementById("app");  
if (!appElement) {  
  throw new Error("Could not find app element");  
}
```

By adding this error condition, you can ensure that the application will never reach any subsequent code if `appElement` is `null`. If you hover over `appElement` after the `if` statement, you can see that TypeScript now knows that `appElement` is an `HTMLElement`; it's no longer `null`. This means the test now passes:

```
console.log(appElement);  
  
// hovering over appElement shows:  
const appElement: HTMLElement;  
  
type Test = Expect<Equal<typeof appElement, HTMLElement>>;  
// passes
```

In this case, TypeScript narrows down the code *outside* of the immediate `if` statement scope and throws an error to help identify issues at runtime.

Exercise 5-5: Using `in` to Narrow

Here's a `handleResponse` function that takes in a type of `APIResponse`, which is a union of two types of objects. The goal of the `handleResponse` function is to check whether the provided object has a `data` property. If it does, the function should return the `id` property. If not, it should throw an `Error` with the message from the `error` property:

```
type APIResponse =
  | {
    data: {
      id: string;
    };
  }
  | {
    error: string;
  };

const handleResponse = (response: APIResponse) => {
  if (true) {
    return response.data.id;
  } else {
    throw new Error(response.error);
  }
};
```

Currently, several errors are being thrown, as shown in the following tests. The first error is `Property 'data' does not exist on type 'APIResponse'`:

```
test("passes the test even with the error", () => {
  // there is no error in runtime, so this test passes:
  expect(() =>
    handleResponse({
```

```
        error: "Invalid argument",
    }),
).not.toThrowError();

// but the data is returned instead of an error, so this test fails:
expect(
    handleResponse({
        error: "Invalid argument",
    }),
).toEqual("Should this be 'Error'?");
});
```

Then there's the inverse error, in which Property 'error' does not exist on type 'APIResponse':

Property data does not exist on type 'APIResponse'.

Find the correct syntax for narrowing down the types within the `handleResponse` function's `if` condition. The changes should happen in the function without modifying other parts of the code.

See <https://totalts.link/essentials-5-5/>.

Solutions

Your first instinct will be to check if `response.data` is truthy, like so:

```
const handleResponse = (response: APIResponse) => {
    // Property 'data' does not exist on type 'APIResponse'.
    // red squiggly line under response.data
    if (response.data) {
        return response.data.id;
    } else {
        throw new Error(response.error);
    }
};
```

But you'll get an error. This is because `response.data` is available on only one of the members of the union. TypeScript doesn't know that

response is the one with data on it. There are a couple ways to handle this:

Option 1

It may be tempting to change the `APIResponse` type to add a `data` property to both members of the union type:

```
type APIResponse =  
  | {  
    data: {  
      id: string;  
    };  
  }  
  | {  
    data?: undefined;  
    error: string;  
  };  
;
```

This is one way to handle it, but there is also a built-in way to do a check for a specific key.

Option 2

You can use the `in` operator to check if a specific key exists on response.

In this example, it would check for the key `data`:

```
const handleResponse = (response: APIResponse) => {  
  if ("data" in response) {  
    return response.data.id;  
  } else {  
    throw new Error(response.error);  
  }  
};  
;
```

If the response isn't the one with data on it, then it must be the one with error, so you can throw an `Error` with the error message.

You can check this out by hovering over `.data` and `.error` in each of the branches of the `if` statement. TypeScript will show that it knows the type of response in each case. Using `in` here gives you a great way

to narrow down objects that might have different keys from one another.

The unknown and never Types

Let's introduce a couple more types that play an important role in TypeScript, particularly when discussing wide and narrow types.

The Widest Type: unknown

TypeScript's widest type is unknown, which represents something you don't know.

If you imagine a scale in which the widest types are at the top and the narrowest types are at the bottom, unknown is at the top. All other types, like string, number, boolean, null, undefined, and their respective literals, are assignable to unknown, as shown in its assignability chart ([Figure 5-1](#)).

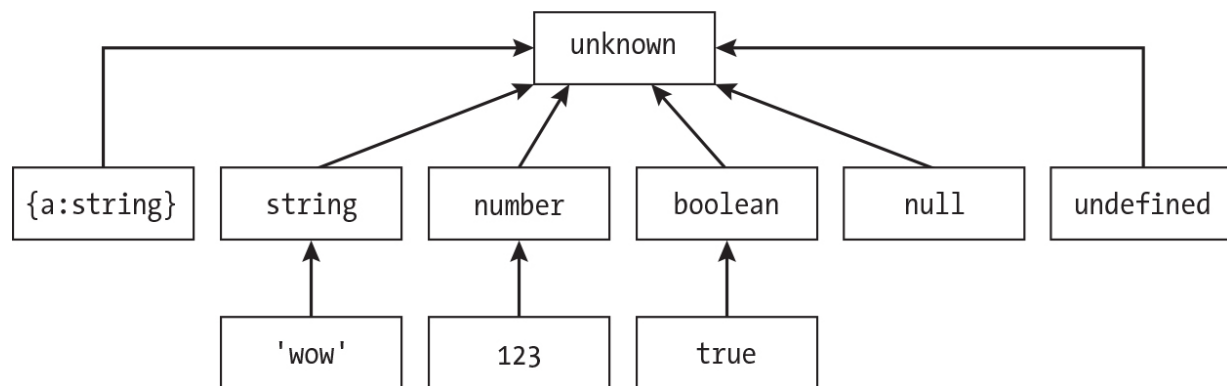


Figure 5-1: The assignability chart shows the types that are assignable to unknown.

Consider this example function, `fn`, that takes in an input parameter of type `unknown`:

```
const fn = (input: unknown) => {};  
  
// Anything is assignable to unknown!  
fn("hello");  
fn(42);  
fn(true);  
fn({});
```

```
fn([]);  
fn(() => {});  
fn(unknown);  
fn(null);
```

All these function calls are valid because `unknown` is assignable to any other type. When you want to represent something that's truly unknown in JavaScript, the `unknown` type is the preferred choice. For example, it's extremely useful when you have things from outside sources coming into your application, like input from a form or a call to a webhook.

The Difference Between `unknown` and `any`

You might wonder what the difference is between `unknown` and `any`. They're both wide types, but there's a key difference. `any` doesn't fit into your definition of "wide" and "narrow" types; it actually disables the type system because it's not really a type at all. Instead, it's a way of opting out of TypeScript's type checking.

`any` can be assigned to anything, and anything can be assigned to `any`. `any` is narrower and wider than every other type. `unknown`, on the other hand, plays by the rules of TypeScript's type system. It will show errors if you attempt to access any property on it. You can see this with an example:

```
const handleWebhookInput = (input: unknown) => {  
  // red squiggly line under input  
  
  input.toUpperCase();  
};  
// hovering over input shows:  
// 'input' is of type 'unknown'.  
  
const handleWebhookInputWithAny = (input: any) => {  
  // no error  
  input.toUpperCase();  
};
```

In `handleWebhookInput`, you can't do anything with `input` because it's `unknown`. TypeScript gives you a warning if you access any properties on it

or call any of its methods. But as you've seen with any in previous examples, it lets you do *anything* with input.

This means that unknown is a safe type, but any is not. Think of it as unknown meaning "I don't know what this is," while any means "I don't care what this is."

The Narrowest Type: never

If unknown is the widest type in TypeScript, never is the narrowest. never represents something that will *never* happen. It's at the bottom of the type hierarchy.

You'll rarely use a never type annotation yourself. Instead, it'll pop up in error messages and hovers, often when narrowing (which you'll see later). But first, let's look at a simple example of a never type.

Let's consider a function that never returns anything:

```
const getNever = () => {  
  // This function never returns!  
};
```

When you hover over this function, TypeScript will infer that it returns void, indicating that it essentially returns nothing:

```
// hovering over getNever shows:  
  
const getNever: () => void;
```

However, if you throw an error in the function, the function will *never* return:

```
const getNever = () => {  
  throw new Error("This function never returns");  
};
```

With this change, TypeScript will infer that the function's type is never:

```
// hovering over getNever shows:
```

```
const getNever: () => never;
```

The `never` type represents something that can never happen. There are some weird implications for the `never` type; you cannot assign anything to `never`, except for `never` itself:

```
const fn = (input: never) => {};
```

```
// red squiggly line under everything in parens:
```

```
fn("hello");  
fn(42);  
fn(true);  
fn({});  
fn([]);  
fn(() => {});
```

```
// No error here, since we're assigning never to never:  
fn(getNever());
```

However, you can assign `never` to anything:

```
const str: string = getNever();
```

```
const num: number = getNever();
```

```
const bool: boolean = getNever();
```

```
const arr: string[] = getNever();
```

This behavior looks odd at first, but you'll see why it's useful later. Let's update your chart to include `never`, as shown in [Figure 5-2](#).

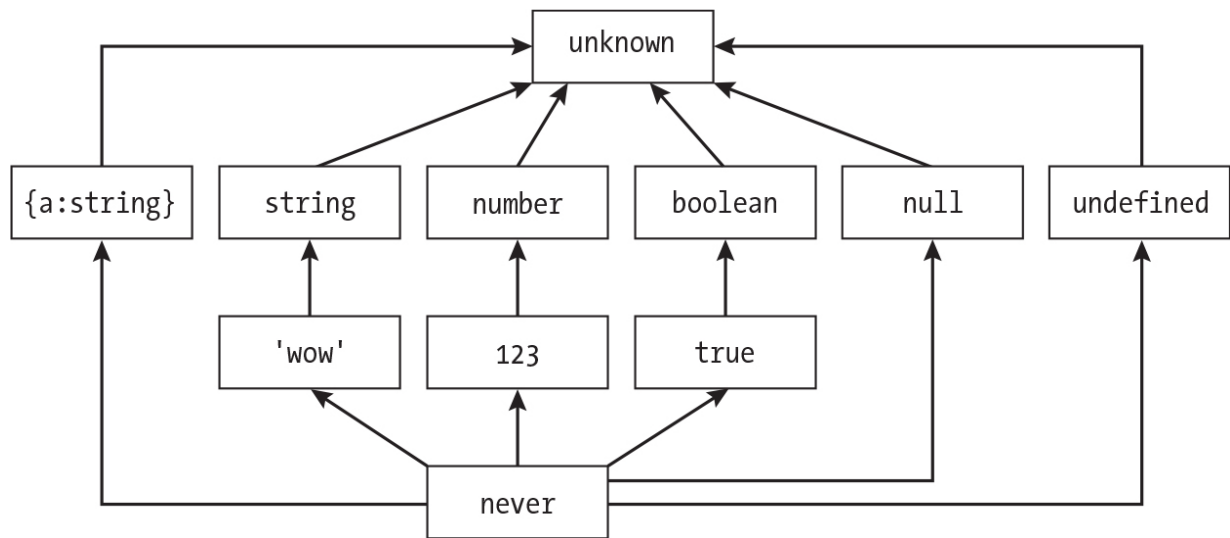


Figure 5-2: The assignability chart with never added

This gives you the full picture of TypeScript's type hierarchy.

Exercise 5-6: Narrowing Errors with instanceof

In TypeScript, one of the most common places you'll encounter the unknown type is when using a try . . . catch statement to handle potentially dangerous code. Let's consider an example:

```

const somethingDangerous = () => {
  if (Math.random() > 0.5) {
    throw new Error("Something went wrong");
  }

  return "all good";
};

try {
  somethingDangerous();
} catch (error) {
  if (true) {
    console.error(error.message); // red squiggly line under
    error in error.message
  }
}

```

In the previous code snippet, you have a function called `somethingDangerous` that has a 50/50 chance of either throwing an error or completing successfully. Notice that the error variable in the catch clause is typed as unknown.

Now let's say you want to log the error using `console.error()` only if the error contains a message attribute. Assume that errors come with a message attribute, like in the following example:

```
const error = new Error("Some error message");

console.log(error.message);
```

Update the `if` statement to have the proper condition to check if the error has a message attribute before logging it. Check the title of the exercise to get a hint . . . and remember, `Error` is a class.

See <https://totalts.link/essentials-5-6/>.

Solution

You can solve this challenge by narrowing the error using the `instanceof` operator. Update the error message check to see if `error` is an instance of the class `Error`:

```
if (error instanceof Error) {
  console.log(error.message);
}
```

The `instanceof` operator covers other classes that inherit from the `Error` class as well, such as `TypeError`. In this case, you're logging the error message to the console, but you could use this to display something different in your applications or log the error to an external service. Even though it works in this example for all kinds of `Errors`, it won't cover you for the strange case in which someone throws a non-`Error` object, as shown here:

```
throw "This is not an error!";
```

To stay safe from these edge cases, include an else block that would throw the error variable, like so:

```
if (error instanceof Error) {
  console.log(error.message);
} else {
  throw error;
}
```

Using this technique, you can handle the error in a safe way and avoid potential runtime errors.

Exercise 5-7: Narrowing unknown to a Value

Here you have a `parseValue` function that takes in a value of type `unknown`:

```
const parseValue = (value: unknown) => {
  if (true) {
    return value.data.id; // red squiggly line under value
  }

  throw new Error("Parsing error!");
};
```

This function returns the `id` property of the `data` property of the `value` object. If the `value` object doesn't have a `data` property, then it should throw an error.

Here are some tests for the function that show you the amount of narrowing that needs to be done in the `parseValue` function:

```
it("Should handle a {data: {id: string}}", () => {
  const result = parseValue({
    data: {
      id: "123",
    },
  });

  type test = Expect<Equal<typeof result, string>>;
```



```
    expect(result).toBe("123");
  });

  it("Should error when anything else is passed in", () => {
    expect(() => parseValue("123")).toThrow("Parsing error!");

    expect(() => parseValue(123)).toThrow("Parsing error!");
  });
```

Modify the `parseValue` function so that the tests pass and the errors go away. Challenge yourself by narrowing *only* the type of value in the function without changes to the types. This will require a large `if` statement!

See <https://totalts.link/essentials-5-7/>.

Solution

Here's your starting point:

```
const parseValue = (value: unknown) => {
  if (true) {
    return value.data.id; // red squiggly line under value
  }

  throw new Error("Parsing error!");
};
```

To fix the error, you'll need to narrow the type using conditional checks. Let's take it step-by-step.

First, check if the type of value is an object by replacing the `true` with a type check:

```
if (typeof value === "object") {
  return value.data.id; // red squiggly line under value and data
}
```

Then, check if the value argument has a data attribute using the `in` operator:

```
if (typeof value === "object" && "data" in value) {  
    // red squiggly line under value  
  
    return value.data.id; // red squiggly line under value and  
    data  
}
```

With this change, TypeScript complains that `value` is possibly `null`. This is because, of course, `typeof null` is `"object"`. Thanks, JavaScript!

To fix this, add `&& value` to your first condition to make sure it isn't `null`:

```
if (typeof value === "object" && value && "data" in value) {  
    return value.data.id; // red squiggly line under value and  
    data  
}
```

Now your condition check passes, but you're still getting an error on `value.data` being typed as `unknown`.

You'll need to narrow the type of `value.data` to an object and make sure it isn't `null`. At this point, you'll also want to specify a return type of `string` to avoid returning an unknown type:

```
const parseValue = (value: unknown): string => {  
    if (  
        typeof value === "object" &&  
        value !== null &&  
        "data" in value &&  
        typeof value.data === "object" &&  
        value.data !== null  
    ) {  
        return value.data.id; // red squiggly line under id  
    }  
}
```

```
    throw new Error("Parsing error!");  
};
```

Finally, add a check to ensure that the `id` is a string. If it isn't, TypeScript will throw an error:

```
const parseValue = (value: unknown): string => {  
  if (  
    typeof value === "object" &&  
    value !== null &&  
    "data" in value &&  
    typeof value.data === "object" &&  
    value.data !== null &&  
    "id" in value.data &&  
    typeof value.data.id === "string"  
  ) {  
    return value.data.id;  
  }  
  
  throw new Error("Parsing error!");  
};
```

Now when you hover over `parseValue`, you can see that it takes in an unknown input and always returns a string:

```
// hovering over parseValue shows:  
const parseValue: (value: unknown) => string;
```

Thanks to this huge conditional, the tests pass, and the error messages disappear. Usually, this is *not* the way you'd want to write your code since it's a bit of a mess. You could use a library like `Zod` to do this with a much nicer API. However, using the large conditional is a great way to understand how unknown and narrowing work in TypeScript.

Discriminated Unions

In this section, you'll look at *discriminated unions*, which are a common pattern TypeScript developers use to structure their code. To understand

what a discriminated union is, let's first look at the problem it solves.

The Problem: The Bag of Optionals

Imagine modeling a data fetch. Here you have a `State` type with a `status` property that can be in one of three states—loading, success, or error:

```
type State = {  
  status: "loading" | "success" | "error";  
};
```

This is useful, but extra data needs to be captured: the data coming back from the fetch, or the error message if the fetch fails. To handle this, you could add optional error and data properties to the `State` type:

```
type State = {  
  status: "loading" | "success" | "error";  
  error?: string;  
  data?: string;  
};
```

Say you have a `renderUI` function that takes in `State` and returns a string based on the input:

```
const renderUI = (state: State) => {  
  if (state.status === "loading") {  
    return "Loading. . .";  
  }  
  
  if (state.status === "error") {  
    return `Error: ${state.error.toUpperCase()}`; // red squiggly line under state.error  
  }  
  
  if (state.status === "success") {  
    return `Data: ${state.data}`;  
  }  
};
```

This looks good, except there's an error underneath `state.error`. Here, TypeScript tells you that `state.error` could be undefined, and you can't call `toUpperCase` on undefined.

You get this error because the `State` type's `error` and `data` properties are *not related to the statuses in which they occur*. In other words, the `State` type has been declared in such a way that it's possible to create types that will never happen in your app. For example, a state could be created that contains both an error and data, which shouldn't be allowed:

```
const state: State = {
  status: "loading",
  error: "This is an error", // should not happen on "loading!"
  data: "This is data", // should not happen on "loading!"
};
```

The `State` type is too loose and could be described as a “bag of optionals.” It needs to be tightened so that error can happen only on error, and data can happen only on success.

The Solution: Discriminated Unions

To correct this issue, you need to fix the `State` type to be a discriminated union. A *discriminated union* is a type that has a common property, the *discriminant*, which is a literal type that is unique to each member of the union. In this case, the `status` property is the discriminant.

Let's take each status and separate them into separate object literals:

```
type State =
  | {
    status: "loading";
  }
  | {
    status: "error";
  }
  | {
    status: "success";
  };
```

Now you can associate the error and data properties with the error and success statuses, respectively:

```
type State =  
  | {  
    status: "loading";  
  }  
  | {  
    status: "error";  
    error: string;  
  }  
  | {  
    status: "success";  
    data: string;  
  };  
};
```

Now, when hovering over `state.error` in the `renderUI` function, you can see that TypeScript knows that `state.error` is a string:

```
const renderUI = (state: State) => {  
  if (state.status === "loading") {  
    return "Loading. . .";  
  }  
  
  if (state.status === "error") {  
    return `Error: ${state.error.toUpperCase()}`;  
  }  
  
  if (state.status === "success") {  
    return `Data: ${state.data}`;  
  }  
};
```

This happens because of TypeScript's narrowing; it knows that `state.status` is "error", so it knows that `state.error` is a string in the `if` block.

With the type working, it could be cleaned up further to use a type alias for each of the statuses:

```
type LoadingState = {
  status: "loading";
};

type ErrorState = {
  status: "error";
  error: string;
};

type SuccessState = {
  status: "success";
  data: string;
};

type State = LoadingState | ErrorState | SuccessState;
```

If you're noticing that your types resemble the bag of optionals problem, consider reworking them to use a discriminated union instead.

Exercise 5-8: Deconstructing a Discriminated Union

Consider a discriminated union called `Shape` that is made up of two types: `Circle` and `Square`. Both types have a `kind` property that acts as the discriminant:

```
type Circle = {
  kind: "circle";
  radius: number;
};

type Square = {
  kind: "square";
  sideLength: number;
};

type Shape = Circle | Square;
```

This `calculateArea` function destructures the `kind`, `radius`, and `sideLength` properties from the `Shape` that is passed in and calculates the area of the shape accordingly:

```
function calculateArea({kind, radius, sideLength}: Shape) {  
  // red squiggly line under radius and sideLength  
  
  if (kind === "circle") {  
    return Math.PI * radius * radius;  
  } else {  
    return sideLength * sideLength;  
  }  
}
```

However, TypeScript shows errors below `radius` and `sideLength`:

```
// hovering over radius shows:  
Property 'radius' does not exist on type 'Shape'.  
  
// hovering over sideLength shows:  
Property 'sideLength' does not exist on type 'Shape'.
```

Update the implementation of the `calculateArea` function so that destructuring properties from the passed-in `Shape` work without errors. Hint: The examples shown earlier in the chapter *didn't* use destructuring, but some destructuring is possible.

See <https://totalts.link/essentials-5-8/>.

Solution

Before you look at the working solution, let's look at an attempt that doesn't work out.

Since you know that `kind` is present in all branches of the discriminated union, you can try using the rest parameter syntax to bring along the other properties:

```
function calculateArea({kind, . . .shape}: Shape) {  
  // Rest of function
```

```
}
```

Then, in the conditional branches, you can specify the kind and destructure from the shape object:

```
if (kind === "circle") {  
  const {radius} = shape; // red squiggly line under radius  
  
  return Math.PI * radius * radius;  
} else {  
  const {sideLength} = shape; // red squiggly line under sideLength  
  
  return sideLength * sideLength;  
}
```

However, this approach doesn't work because the kind property has been separated from the rest of the shape. As a result, TypeScript can't track the relationship between kind and the other properties of shape.

Both radius and sideLength have errors:

```
Property 'radius' does not exist on type '{radius: number;} | {sideLength: number;}'.  
Property 'sideLength' does not exist on type '{radius: number;} | {sideLength: number;}'.
```

TypeScript gives these errors because it still can't guarantee properties in the function parameters since it doesn't yet know whether it's dealing with a Circle or a Square.

The Working Destructuring Solution

Instead of destructuring at the function parameter level, you will revert the function parameter to shape:

```
function calculateArea(shape: Shape) {  
  // Rest of function  
}
```

Then, you will move the destructuring to take place in the conditional branches:

```
if (shape.kind === "circle") {
    const {radius} = shape;

    return Math.PI * radius * radius;
} else {
    const {sideLength} = shape;

    return sideLength * sideLength;
}
```

Within the `if` condition, TypeScript recognizes that `shape` is indeed a `Circle` and allows you to access the `radius` property safely. A similar approach is taken for the `Square` in the `else` condition.

This approach works because TypeScript can track the relationship between `kind` and the other properties of `shape` when the destructuring takes place in the conditional branches. Note that it's common to avoid destructuring when working with discriminated unions, but if you want to destructure, you should do it *within* the conditional branches.

Exercise 5-9: Narrowing a Discriminated Union with a `switch` Statement

Here is the `calculateArea` function from the previous exercise, but without any destructuring:

```
function calculateArea(shape: Shape) {
    if (shape.kind === "circle") {
        return Math.PI * shape.radius * shape.radius;
    } else {
        return shape.sideLength * shape.sideLength;
    }
}
```

Your task is to refactor this function to use a switch statement instead of the if. . .else statement. The switch statement should narrow the type of shape and calculate the area accordingly.

See <https://totalts.link/essentials-5-9/>.

Solution

First, clear out the calculateArea function, add the switch keyword, and specify shape.kind as the switch condition:

```
function calculateArea(shape: Shape) {
  switch (shape.kind) {
    case "circle": {
      return Math.PI * shape.radius * shape.radius;
    }
    case "square": {
      return shape.sideLength * shape.sideLength;
    }
    // Potential additional cases for more shapes
  }
}
```

Notice that TypeScript offers autocomplete on the cases for the switch statement. This is a great way to ensure that you're handling all the cases for your discriminated union.

As an experiment, comment out the case where the kind is square:

```
function calculateArea(shape: Shape) {
  switch (shape.kind) {
    case "circle": {
      return Math.PI * shape.radius * shape.radius;
    }
    // case "square": {
    //   return shape.sideLength * shape.sideLength;
    // }
    // Potential additional cases for more shapes
  }
}
```

Now, when you hover over the function, you'll see that the return type is `number | undefined`. That's because TypeScript knows that if you don't return a value for the square case, the output will be undefined for any square shape:

```
// hovering over calculateArea shows:  
function calculateArea(shape: Shape): number | undefined;
```

switch statements work well with discriminated unions!

Exercise 5-10: Discriminated Tuples

Here's a `fetchData` function that returns a promise that resolves to an `ApiResponse` tuple consisting of two elements. The first element is a string that indicates the type of the response. The second element uses an array of `User` objects in the case of successful data retrieval, or a string in the event of an error:

```
type ApiResponse = [string, User[] | string];
```

Here's what the `fetchData` function looks like:

```
async function fetchData(): Promise<ApiResponse>{  
  try {  
    const response = await fetch("https://api.example.com/data");  
  
    if (!response.ok) {  
      return [  
        "error",  
        // Imagine some improved error handling here  
        "An error occurred",  
      ];  
    }  
  
    const data = await response.json();  
  
    return ["success", data];  
  }  
}
```

```
    } catch (error) {  
      return ["error", "An error occurred"];  
    }  
  }  
}
```

However, as shown in these tests, the `ApiResponse` type will allow for other combinations that aren't what you want. For example, it allows for passing an error message when data is being returned:

```
async function exampleFunc() {  
  const [status, value] = await fetchData();  
  
  if (status === "success") {  
    console.log(value);  
  
    type test = Expect<Equal<typeof value, User[]>>; // red squiggly line under Equal<>  
  } else {  
    console.error(value);  
  
    type test = Expect<Equal<typeof value, string>>; // red squiggly line under Equal<>  
  }  
}
```

The problem stems from the `ApiResponse` type being shaped like a bag of optionals. The `ApiResponse` type needs to be updated so that there are two possible combinations for the returned tuple:

- If the first element is "error", then the second element should be the error message.
- If the first element is "success", then the second element should be an array of `User` objects.

Redefine the `ApiResponse` type to be a tuple that allows only for the specific combinations for the success and error states defined previously.

See <https://totalts.link/essentials-5-10/>.

Solution

The solution is to make the `ApiResponse` type look like this:

```
type ApiResponse = ["error", string] | ["success", User[]];
```

This syntax uses tuples instead of objects to create two possible combinations for the `ApiResponse` type: an error state and a success state.

The `ApiResponse` type is now a discriminated tuple where the discriminant is the first element.

In the `exampleFunc` function, the array destructuring syntax can be used to pull out the status and value from the `ApiResponse` tuple:

```
const [status, value] = await fetchData();
```

Even though the status and value variables are separate, TypeScript keeps track of the relationships behind them. If status is checked and is equal to "success", TypeScript can narrow down the value to be of the `User[]` type automatically:

```
// hovering over status shows:  
const status: "error" | "success";
```

Note that this intelligent behavior is specific to discriminated tuples and won't work with discriminated objects, as shown in the previous exercise.

Exercise 5-11: Handling Defaults with a Discriminated Union

You're back with your `calculateArea` function:

```
function calculateArea(shape: Shape) {  
  if (shape.kind === "circle") {  
    return Math.PI * shape.radius * shape.radius;  
  } else {  
    return shape.sideLength * shape.sideLength;  
  }  
}
```

```
}  
}
```

Until now, the test cases involved checking if the kind of the Shape is a circle or a square and then calculating the area accordingly. However, here's a new test case for a situation in which no kind has been passed into the function:

```
it("Should calculate the area of a circle when no kind is passed", () => {  
  const result = calculateArea({  
    radius: 5, // red squiggly line under radius  
  });  
  
  expect(result).toBe(78.53981633974483);  
  
  type test = Expect<Equal<typeof result, number>>;  
});
```

TypeScript shows errors under radius in the test:

```
// hovering over radius shows:  
Argument of type '{radius: number;}' is not assignable to parameter of type 'Shape'.  
Property 'kind' is missing in type '{radius: number;}' but required in type 'Circle'.
```

The test expects that if a kind isn't passed in, the shape should be treated as a circle. However, the current implementation doesn't account for this.

Your challenge is to do the following:

1. Update the Shape discriminated union so that it will allow for kind to be omitted.
2. Adjust the calculateArea function to ensure that TypeScript's type narrowing works properly within the function.

See <https://totalts.link/essentials-5-11/>.

Solution

Before you view the working solution, take a look at a couple of approaches that don't quite work out:

Attempt 1

One possible first step is to create an `OptionalCircle` type by discarding the `kind` property:

```
type OptionalCircle = {  
  radius: number;  
};
```

Then, update the `Shape` type to include the new type:

```
type Shape = Circle | OptionalCircle | Square;
```

This solution appears to work since it resolves the error in the `radius` test case. However, this approach reintroduces errors in the `calculateArea` function. The discriminated union is broken since not every member has a `kind` property:

```
function calculateArea(shape: Shape) {  
  if (shape.kind === "circle") {  
    // red squiggly line under s  
    hape.kind  
    return Math.PI * shape.radius * shape.radius;  
  } else {  
    return shape.sideLength * shape.sideLength;  
  }  
}
```

Attempt 2

Rather than developing a new type, the `Circle` type could be modified to make the `kind` property optional:

```
type Circle = {  
  kind?: "circle";  
  radius: number;
```



```
};  
type Square = {  
  kind: "square";  
  sideLength: number;  
};  
type Shape = Circle | Square;
```

This modification allows you to distinguish between circles and squares. The discriminated union remains intact while also accommodating the optional case in which kind is not specified. However, now there's a new error in the calculateArea function:

```
// In the calculateArea function  
  
if (shape.kind === "circle") {  
  return Math.PI * shape.radius * shape.radius;  
} else {  
  return shape.sideLength * shape.sideLength; // red squiggly  
  line under sideLength  
}
```

The error tells you that TypeScript is no longer able to narrow down the type of shape to a Square because there is not a check to see if shape.kind is undefined.

This error could be fixed by adding checks for the kind, but instead you could just swap the order of the conditional checks to look for square first and then fall back to a circle:

```
if (shape.kind === "square") {  
  return shape.sideLength * shape.sideLength;  
} else {  
  return Math.PI * shape.radius * shape.radius;  
}
```

By inspecting square first, all shape cases that aren't squares default to circles. The circle is treated as optional, which preserves the discriminated

union and keeps the function flexible. Sometimes, flipping the runtime logic makes TypeScript happy!

Summary

This chapter explored TypeScript's union types and type narrowing, two fundamental concepts that help manage values that can be multiple possible types.

Union types use the `|` operator to specify that a value can be “either this type or that type.” You can declare them directly, as in `string | number`, or you can create type aliases for reusability. Literal types take this further by representing specific primitive values like `"yes" | "no"` or `200 | 404 | 500`.

TypeScript has a hierarchy of “wide” and “narrow” types. The `unknown` type sits at the top as the widest type, accepting any value but requiring narrowing before use. The `never` type occupies the bottom as the narrowest, representing impossible values. Union types are wider than their individual members.

Type narrowing allows you to take wider types and make them narrower using runtime checks. You can narrow with `typeof` operators, truthiness checks, the `in` operator for object properties, or `instanceof` for class instances, or you can narrow by throwing errors. Each narrowing technique works within specific scopes.

Discriminated unions solve the “bag of optionals” problem by using a common literal property as a discriminant. Instead of having optional properties that could create impossible states, you create separate object types for each case and then union them together.

The chapter demonstrated these concepts through practical exercises involving error handling, API responses, and shape calculations. `switch` statements work particularly well with discriminated unions, and even tuples can be discriminated using their first element as the discriminant.

PART III

OBJECTS, CLASSES, AND MUTABILITY

6

OBJECTS



So far, we've covered object types only in the context of object literals, defined using `{}` with type aliases. However, TypeScript has many other tools that let you be more expressive with object types. In this chapter, you'll learn several techniques for working with objects that combine syntax with built-in utility types.

You can use intersection types to combine properties from multiple object types into a single type. This feature is similar to modeling inheritance between interfaces, though there are some nuances to be aware of. There are a variety of utility types for working with object types that allow you to quickly transform existing types to fit your needs, as well as special syntax for working with dynamic property names.

Extending Objects

Let's start the investigation by looking at how to build object types from other object types in TypeScript. As applications grow, it's common for more objects to be introduced. Instead of repeating yourself in your code, you have a variety of options for creating object types from other object types, including intersection types and using the `extends` keyword when working with interfaces.

Intersection Types

An intersection type uses the & operator and lets you combine multiple object types into a single type. You can think of it like the reverse of the | operator. Instead of representing an “or” relationship between types, the & operator signifies an “and” relationship.

Consider these types for Album and SalesData:

```
type Album = {  
  title: string;  
  artist: string;  
  releaseYear: number;  
};  
  
type SalesData = {  
  unitsSold: number;  
  revenue: number;  
};
```

On their own, each type represents a distinct set of properties. While the SalesData type on its own could be used to represent sales data for any product, using the & operator to create an intersection type allows you to combine the two types into a single type that represents an album’s sales data:

```
type AlbumSales = Album & SalesData;
```

The AlbumSales type now requires objects to include all the properties from both Album and SalesData:

```
const wishYouWereHereSales: AlbumSales = {  
  title: "Wish You Were Here",  
  artist: "Pink Floyd",  
  releaseYear: 1975,  
  unitsSold: 13000000,  
  revenue: 65000000,  
};
```

In the previous example, `wishYouWereHereSales` includes all the required properties. If any of these properties were missing, the contract of the `AlbumSales` type wouldn't be fulfilled, and TypeScript would raise an error.

It's also possible to intersect more than two types. If you wanted to add a genre to the `AlbumSales` type, a simple object could be added to the intersection:

```
type AlbumSales = Album & SalesData & {genre: string};
```

Intersection types can also be used with primitives, like `string` and `number`, though this often produces odd results. For instance, let's try intersecting `string` and `number`:

```
type StringAndNumber = string & number;
```

What type do you think `StringAndNumber` is? It's actually `never` because `string` and `number` can't be combined. While an object can be both `Album` and `SalesData`, there is no value in JavaScript that can be both a `string` and a `number`.

Whenever you intersect two object types with an incompatible property, you will end up with the `never` type:

```
type User1 = {
  age: number;
};

type User2 = {
  age: string;
};

type User = User1 & User2;

// hovering over User shows:
type User = {
  age: never;
};
```

In this case, the age property resolves to never because it's impossible for a single property to be both a number and a string.

Interfaces

So far, we've shown only how to use the type keyword to define object types. Experienced TypeScript programmers will likely be tearing their hair out thinking, "Why aren't we talking about interfaces?" Interfaces are one of TypeScript's most famous features. They shipped with the first versions of TypeScript and are considered a core part of the language.

Interfaces let you declare object types using a slightly different syntax from type. Let's compare the syntax:

```
type Album = {  
  title: string;  
  artist: string;  
  releaseYear: number;  
};
```

```
interface Album {  
  title: string;  
  artist: string;  
  releaseYear: number;  
}
```

The syntax looks similar, except for the interface keyword and lack of an equal sign. This leads to the common mistake of thinking of them as interchangeable, but they're not! Interfaces have quite different capabilities from types.

One of interface's most powerful features is its ability to extend other types. This allows you to create new interfaces that inherit properties from existing ones.

In this example, you have a base Album interface that will be extended into StudioAlbum and LiveAlbum interfaces that allow you to provide more specific details about an album:

```
interface Album {
  title: string;
  artist: string;
  releaseYear: number;
}

interface StudioAlbum extends Album {
  studio: string;
  producer: string;
}

interface LiveAlbum extends Album {
  concertVenue: string;
  concertDate: Date;
}
```

Using extends to create new interfaces allows you to create more specific album representations with a clear inheritance relationship:

```
const americanBeauty: StudioAlbum = {
  title: "American Beauty",
  artist: "Grateful Dead",
  releaseYear: 1970,
  studio: "Wally Heider Studios",
  producer: "Grateful Dead and Stephen Barncard",
};

const oneFromTheVault: LiveAlbum = {
  title: "One from the Vault",
  artist: "Grateful Dead",
  releaseYear: 1991,
  concertVenue: "Great American Music Hall",
  concertDate: new Date("1975-08-13"),
};
```

In the previous examples, it's clear to see how `americanBeauty` and `oneFromTheVault` relate to `StudioAlbum`, `LiveAlbum`, and the base `Album` interface.

Just as adding extra & operators adds types to an intersection, it's also possible for an interface to extend multiple other interfaces by separating them with commas:

```
interface BonusConcertEdition extends StudioAlbum, LiveAlbum
{
  numberOfDiscs: number;
}
```

Here the BonusConcertEdition interface would take on all the properties of both the StudioAlbum and LiveAlbum interfaces.

Intersections vs. interface extends

We've now covered two separate TypeScript syntaxes for extending object types: & and interface extends. So, which is better? You should choose interface extends for two reasons.

Better Errors When Merging Incompatible Types

You saw earlier that when you intersect two object types with an incompatible property, TypeScript resolves the property to never:

```
type User1 = {
  age: number;
};

type User2 = {
  age: string;
};

type User = User1 & User2;
```

When using interface extends, TypeScript raises an error when you try to extend an interface with an incompatible property:

```
interface User1 {
  age: number;
}
```

```
// red squiggly line under User
interface User extends User1 {
  age: string;
}
```

Hovering over User shows an error message:

```
Interface 'User' incorrectly extends interface 'User1'.
  Types of property 'age' are incompatible.
```

This behavior is different from what it would be with intersections; the error would show when trying to access the age property, instead of when it is defined.

Better TypeScript Performance

When you're working in TypeScript, the performance of your types should be at the back of your mind. In large projects, how you define your types can have a big impact on how fast your IDE feels and how long it takes for tsc to check your code.

`interface extends` is much better for TypeScript performance than intersections. With intersections, the intersection is recomputed every time it's used. This can be slow, especially when you're working with complex types.

Interfaces are faster. TypeScript can cache the resultant type of an interface based on its name. This means that if you use `interface extends`, TypeScript has to compute the type only once, and then it can reuse it every time you use the interface.

To summarize, using `interface extends` is better for catching errors and for TypeScript performance. This doesn't mean you need to define all your object types using `interface` (we'll get to that later), but if you need to make one object type extend another, you should use `interface extends` where possible.

Types vs. Interfaces

Now that you know how good `interface` extends is for extending object types, a natural question arises. Should you use `interface` for all of your types by default? Let's look at a few comparison points between types and interfaces.

Type aliases are a lot more flexible than interfaces. A type can represent anything—union types, object types, intersection types, and more:

```
type Union = string | number;
```

When you declare a type alias, you're just giving a name (or alias) to an existing type. On the other hand, an `interface` can represent only object types and functions.

There's another odd property that interfaces have. When multiple interfaces with the same name in the same scope are created, TypeScript automatically merges them. This is known as *declaration merging*.

Say you create an `Album` interface with properties for the `title` and `artist`. Now imagine that further down in the same file, you accidentally declare another `Album` interface with properties for the `releaseYear` and `genres`:

```
interface Album {  
  title: string;  
  artist: string;  
}  
  
interface Album {  
  releaseYear: number;  
  genres: string[];  
}
```

TypeScript automatically merges these two declarations into a single interface that includes all the properties from both declarations:

```
// Under the hood:  
interface Album {  
  title: string;
```

```
    artist: string;
    releaseYear: number;
    genres: string[];
}
```

Because of declaration merging, TypeScript now expects that the Album interface will include title, artist, releaseYear, and genres properties. This is very different from type, which would give you an error if you tried to declare the same type twice:

```
// red squiggly line under both Albums
// duplicate identifier Album
type Album = {
    title: string;
    artist: string;
};

type Album = {
    releaseYear: number;
    genres: string[];
};
```

Coming from a JavaScript point of view, this behavior of interfaces feels pretty weird. We have lost hours of our lives to having two interfaces with the same name in the same file with 2,000+ lines. It's a bit of a gotcha, but the feature exists for a good reason that we'll cover further in [Chapter 9](#).

The big question here is whether you should use type or interface for declaring simple object types. We tend to default to type unless we specifically need to use interface extends. Using type is more flexible and doesn't declaration merge unexpectedly. But it's a close call. Many folks coming from a more object-oriented background will prefer interface because it's more familiar to them from other languages.

Exercise 6-1: Creating an Intersection Type

Here you have a User type and a Product type, both with some common properties like id and createdAt:

```
type User = {  
  id: string;  
  createdAt: Date;  
  name: string;  
  email: string;  
};  
  
type Product = {  
  id: string;  
  createdAt: Date;  
  name: string;  
  price: number;  
};
```

Your task is to create a new BaseEntity type that includes the id and createdAt properties. Then, use the & operator to create User and Product types that intersect with BaseEntity. See <https://totalts.link/essentials-6-1/>.

Solution

To solve this challenge, let's create a new BaseEntity type with the common properties:

```
type BaseEntity = {  
  id: string;  
  createdAt: Date;  
};
```

Once the BaseEntity type is created, you can intersect it with the User and Product types:

```
type User = {  
  id: string;  
  createdAt: Date;  
  name: string;  
  email: string;  
} & BaseEntity;  
  
type Product = {
```

```
    id: string;
    createdAt: Date;
    name: string;
    price: number;
} & BaseEntity;
```

Then, you can remove the common properties from User and Product:

```
type User = {
    name: string;
    email: string;
} & BaseEntity;
```

```
type Product = {
    name: string;
    price: number;
} & BaseEntity;
```

Now User and Product have the same behavior that they did before, but with less duplicated code.

Exercise 6-2: Extending Interfaces

After the previous exercise, you'll have a BaseEntity type along with User and Product types that intersect with it. This time, your task is to refactor the types to be interfaces and use the extends keyword to extend the BaseEntity type. For extra credit, try creating and extending multiple smaller interfaces. See <https://totalts.link/essentials-6-2/>.

Solution

Instead of using the type keyword, the BaseEntity, User, and Product can be declared as interfaces. Remember, interfaces do not use an equal sign like type does:

```
interface BaseEntity {
    id: string;
    createdAt: Date;
}
```

```
interface User {  
    name: string;  
    email: string;  
}  
  
interface Product {  
    name: string;  
    price: number;  
}
```

Once the interfaces are created, you can use the `extends` keyword to extend the `BaseEntity` interface:

```
interface User extends BaseEntity {  
    name: string;  
    email: string;  
}  
  
interface Product extends BaseEntity {  
    name: string;  
    price: number;  
}
```

For extra credit, you can take this further by creating `WithId` and `WithCreatedAt` interfaces that represent objects with `id` and `createdAt` properties. Then, you can have `User` and `Product` extend from these interfaces by adding commas:

```
interface WithId {  
    id: string;  
}  
  
interface WithCreatedAt {  
    createdAt: Date;  
}  
  
interface User extends WithId, WithCreatedAt {  
    name: string;
```

```
    email: string;
  }

  interface Product extends WithId, WithCreatedAt {
    name: string;
    price: number;
  }
```

You've now refactored your intersections to use `interface extends`, and your TypeScript compiler will thank you.

Dynamic Object Keys

When using objects, you often won't know exactly which keys will be used. This can happen when fetching data from an API, when working with user input, or even when creating your own objects. TypeScript gives you a few different ways to support dynamic object keys while still remaining type-safe, including index signatures, the `Record` utility type for expressing key-value relationships, and the `PropertyKey` type. Let's start by comparing how JavaScript and TypeScript behave.

In JavaScript, you can start with an empty object and add keys and values to it dynamically:

```
// JavaScript example
const albumAwards = {};

albumAwards.Grammy = true;
albumAwards.MercuryPrize = false;
albumAwards.Billboard = true;
```

Here, there's no problem with `albumAwards` starting empty and having the `Grammy` and `MercuryPrize` keys and values added to it. However, when you try to add keys dynamically to an object in TypeScript, you'll get errors:

```
// TypeScript example
const albumAwards = {};
```



```
albumAwards.Grammy = true; // red squiggly line under Grammy
albumAwards.MercuryPrize = false; // red squiggly line under
MercuryPrize
albumAwards.Billboard = true; // red squiggly line under Bil
lboard

// hovering over Grammy shows:
Property 'Grammy' does not exist on type '{}'.

```

This can feel unhelpful. You might think that TypeScript, based on its ability to narrow your code, should be able to figure out that you're adding keys to an object. In this case, TypeScript prefers to be conservative. It's not going to let you add keys to an object that it doesn't know about because TypeScript is trying to prevent you from making a mistake. You need to tell TypeScript that you want to be able to dynamically add keys. Let's look at some ways to do that.

Index Signatures

Let's return to the previous example:

```
const albumAwards = {};

albumAwards.Grammy = true; // red squiggly line under Grammy

```

The technical term for what you're doing here is *indexing*. You're indexing into `albumAwards` with a string key, `Grammy`, and assigning it a value. To support this behavior, you want to tell TypeScript that whenever you try to index into `albumAwards` with a string, you should expect a Boolean value. To do that, you can use an *index signature*.

Here's how you would specify an index signature for the `albumAwards` object:

```
const albumAwards: {
  [index: string]: boolean;
} = {};

albumAwards.Grammy = true;

```

```
albumAwards.MercuryPrize = false;  
albumAwards.Billboard = true;
```

The `[index: string]: boolean` syntax is an index signature. It tells TypeScript that `albumAwards` can have any string key, and the value will always be a `Boolean`.

You can choose any name for the index. It's just a description:

```
const albumAwards: {  
  [iCanBeAnything: string]: boolean;  
} = {};
```

You also can use the same syntax with types and interfaces:

```
interface AlbumAwards {  
  [index: string]: boolean;  
}  
  
const beyonceAwards: AlbumAwards = {  
  Grammy: true,  
  Billboard: true,  
};
```

Index signatures are one way to handle dynamic keys, but there's a utility type that some argue is even better.

The Record Type

The `Record` utility type is another option for supporting dynamic keys.

Here's how you would use `Record` for the `albumAwards` object, where the key will be a string and the value will be a `Boolean`:

```
const albumAwards: Record<string, boolean> = {};  
albumAwards.Grammy = true;
```

The first type argument is the key, and the second type argument is the value. This is a more concise way to achieve a similar result as an index signature.

The Record type can also support a union type as keys, while an index signature can't:

```
const albumAwards: Record<"Grammy" | "MercuryPrize" | "Billboard", boolean> = {
  Grammy: true,
  MercuryPrize: false,
  Billboard: true,
};
```

```
const albumAwardsWithIndexSignature: {
  // red squiggly line under index

  [index: "Grammy" | "MercuryPrize" | "Billboard"]: boolean;
} = {
  Grammy: true,
  MercuryPrize: false,
  Billboard: true,
};
```

// hovering over index shows:

An index signature parameter type cannot be a literal type or generic type.

Consider using a mapped object type instead.

As shown by the error message in `albumAwardsWithIndexSignature`, a union of possible strings can't be passed in for the index signature. This is because index signatures don't support literal types. You'll learn why this is when we cover mapped types in [Chapter 15](#).

The Record type helper is flexible enough to support literal types and provides you with a repeatable pattern that's easy to read and understand. It should be your go-to for dynamic keys.

A Combination of Known and Dynamic Keys

In many cases there will be a base set of keys you know you want to include, but you also want to allow for additional keys to be added dynamically. For example, say you are working with a base set of awards you know were nominations, but you don't know what other awards are in play. You can use the Record type to define a base set of awards and then

use an intersection to extend it with an index signature for additional awards:

```
type BaseAwards = "Grammy" | "MercuryPrize" | "Billboard";

type ExtendedAlbumAwards = Record<BaseAwards, boolean> & {
  [award: string]: boolean;
};

const extendedNominations: ExtendedAlbumAwards = {
  Grammy: true,
  MercuryPrize: false,
  Billboard: true, // Additional awards can be dynamically added.
  "American Music Awards": true,
};
```

In the previous example, the `ExtendedAlbumAwards` type combines a `Record` of known awards while supporting additional, unknown awards that can be added dynamically. The same technique would also work when using an interface and the `extends` keyword:

```
interface BaseAwards {
  Grammy: boolean;
  MercuryPrize: boolean;
  Billboard: boolean;
}

interface ExtendedAlbumAwards extends BaseAwards {
  [award: string]: boolean;
}
```

While both the type and interface examples result in combining known and dynamic keys, in this case, `interface extends` is preferable to intersections. Being able to support both default and dynamic keys in your data structures allows a lot of flexibility to adapt to changing requirements in your applications.

The PropertyKey Type

A useful type for working with dynamic keys is `PropertyKey`. This global type represents the set of all possible keys that can be used on an object, including `string`, `number`, and `symbol`. You can find its type definition in TypeScript's ES5 type definitions file, which we'll cover more closely in [Chapter 13](#):

```
// in lib.es5.d.ts
declare type PropertyKey = string | number | symbol;
```

Because `PropertyKey` works with all possible keys, it's great for working with dynamic keys where you aren't sure what the type of the key will be. For example, when using an index signature, you could set the key type to `PropertyKey` to allow for any valid key type:

```
type Album = {
  [key: PropertyKey]: string;
};
```

Exercise 6-3: Using an Index Signature for Dynamic Keys

Here you have an object called `scores`, and you are trying to assign several different properties to it:

```
const scores = {};
```



```
scores.math = 95; // red squiggly line under math
scores.english = 90; // red squiggly line under english
scores.science = 85; // red squiggly line under science
```

Your task is to give `scores` a type annotation to support the dynamic subject keys. There are four ways to do this: an inline index signature, a type, an interface, or a `Record`. See <https://totalts.link/essentials-6-3/>.

Solutions

This exercise has four solutions:

- You can use an inline index signature:

```
const scores: {  
  [key: string]: number;  
} = {};
```

- You can use an interface:

```
interface Scores {  
  [key: string]: number;  
}
```

- You can use a type:

```
type Scores = {  
  [key: string]: number;  
};
```

- Finally, you can use a record:

```
const scores: Record<string, number> = {};
```

Exercise 6-4: Default Properties with Dynamic Keys

In this exercise, you're trying to model a situation where you want some required keys—math, english, and science—on your Scores object, but you also want to add dynamic properties, in this case, athletics, french, and spanish:

```
interface Scores {}  
  
// @ts-expect-error science should be provided // red squiggly line under @ts-expect-error  
const scores: Scores = {  
  math: 95,  
  english: 90,
```

```
};

scores.athletics = 100; // red squiggly line under athletics
scores.french = 75; // red squiggly line under french
scores.spanish = 70; // red squiggly line under spanish
```

The definition of Scores should be erroring because science is missing, but it's not, since your definition of Scores is currently an empty object.

Your task is to update the Scores interface to specify default keys for math, english, and science while allowing for any other subject to be added. Once you've updated the type correctly, the red squiggly line below @ts-expect-error will go away because science will be required but missing. Try using interface extends to achieve this. See <https://totalts.link/essentials-6-4/>.

Solution

Here's how to add an index signature to the Scores interface to support dynamic keys along with the required keys:

```
interface Scores {
  [subject: string]: number;
  math: number;
  english: number;
  science: number;
}
```

Creating a RequiredScores interface and extending it looks like this:

```
interface RequiredScores {
  math: number;
  english: number;
  science: number;
}

interface Scores extends RequiredScores {
  [key: string]: number;
}
```

These two solutions are functionally equivalent, except that you get access to the `RequiredScores` interface if you need to use that separately.

Exercise 6-5: Restricting Object Keys with Records

For this example, you have a `configurations` object, typed as `Configurations`, which is currently unknown. The object holds keys for `development`, `production`, and `staging`, and each respective key is associated with configuration details such as `apiBaseUrl` and `timeout`. There is also a `notAllowed` key, which is decorated with an `@ts-expect-error` comment, but currently it's not erroring in TypeScript as expected:

```
type Environment = "development" | "production" | "staging";

type Configurations = unknown;

const configurations: Configurations = {
  development: {
    apiBaseUrl: "http://localhost:8080",
    timeout: 5000,
  },
  production: {
    apiBaseUrl: "https://api.example.com",
    timeout: 10000,
  },
  staging: {
    apiBaseUrl: "https://staging.example.com",
    timeout: 8000,
  },
  // @ts-expect-error // red squiggly line under @ts-expect-
  error
  notAllowed: {
    apiBaseUrl: "https://staging.example.com",
    timeout: 8000,
  },
};
```

Update the Configurations type so that only the keys from Environment are allowed on the configurations object. Once you've updated the type correctly, the red squiggly line below `@ts-expect-error` will go away because `notAllowed` will be disallowed properly. See <https://totalts.link/essentials-6-5/>.

Solution

First, let's consider an attempt at using `Record`. You know that the values of the configurations object will be `apiBaseUrl`, which is a string, and `timeout`, which is a number. It may be tempting to use a `Record` to set the key as a string and the value as an object with the properties `apiBaseUrl` and `timeout`:

```
type Configurations = Record<
  string,
  {
    apiBaseUrl: string;
    timeout: number;
  }
>;
```

However, having the key as `string` still allows for the `notAllowed` key to be added to the object. You need to make the keys dependent on the `Environment` type.

Instead, the correct approach is to specify the key as `Environment` in the `Record`:

```
type Configurations = Record<
  Environment,
  {
    apiBaseUrl: string;
    timeout: number;
  }
>;
```

Now TypeScript will throw an error when the object includes a key that doesn't exist in `Environment`, like `notAllowed`.

Exercise 6-6: Dynamic Key Support

Consider this `hasKey` function that accepts an object and a key and then calls `object.hasOwnProperty` on that object:

```
const hasKey = (obj: object, key: string) => {  
  return obj.hasOwnProperty(key);  
};
```

Note that the `object` type represents more than just object literals; it represents any nonprimitive type, including arrays and functions.

There are several test cases for the `hasKey` function.

The first test case checks that it works on string keys, which doesn't present any issues. As anticipated, `hasKey(obj, "foo")` would return `true`, and `hasKey(obj, "bar")` would return `false`:

```
it("Should work on string keys", () => {  
  const obj = {  
    foo: "bar",  
  };  
  
  expect(hasKey(obj, "foo")).toBe(true);  
  expect(hasKey(obj, "bar")).toBe(false);  
});
```

A test case that checks for numeric keys does have issues because the function is expecting a string key:

```
it("Should work on number keys", () => {  
  const obj = {  
    1: "bar",  
  };  
  
  expect(hasKey(obj, 1)).toBe(true); // red squiggly line under 1  
  expect(hasKey(obj, 2)).toBe(false); // red squiggly line under 2  
});
```

Because an object can also have a symbol as a key, there is also a test for that case. It currently has type errors for `fooSymbol` and `barSymbol` when calling `hasKey`:

```
it("Should work on symbol keys", () => {
  const fooSymbol = Symbol("foo");
  const barSymbol = Symbol("bar");

  const obj = {
    [fooSymbol]: "bar",
  };

  expect(hasKey(obj, fooSymbol)).toBe(true); // red squiggly
line under fooSymbol
  expect(hasKey(obj, barSymbol)).toBe(false); // red squiggly
y line under barSymbol
});
```

Your task is to update the `hasKey` function so that all of these tests pass. Try to be as concise as possible! See <https://totalts.link/essentials-6-6/>.

Solution

The obvious answer is to change `key`'s type to `string | number | symbol`:

```
const hasKey = (obj: object, key: string | number | symbol)
=> {
  return obj.hasOwnProperty(key);
};
```

However, there's a much more succinct solution. Hovering over `hasOwnProperty` shows you the type definition:

```
(method) Object.hasOwnProperty(v: PropertyKey): boolean
```

Recall that the `PropertyKey` type represents every possible value a key can have. This means you can use it as the type for the `key` parameter:

```
const hasKey = (obj: object, key: PropertyKey) => {  
  return obj.hasOwnProperty(key);  
};
```

Beautiful.

Reducing Duplication with Utility Types

We've described how using `interface extends` can help you model inheritance, but TypeScript also provides tools to manipulate object types directly. With its built-in utility types, you can remove properties from types, make them optional or required, and more.

The Partial Type

The `Partial` utility type lets you create a new object type from an existing one, except all of its properties are optional.

Consider an `Album` interface that contains detailed information about an album:

```
interface Album {  
  title: string;  
  artist: string;  
  releaseYear: number;  
  genre: string;  
}
```

When you want to update an album's information, you might not have all the information at once. For example, it can be difficult to decide what genre to assign to an album before it's released. Using the `Partial` utility type and passing in `Album`, you can create a type that allows you to update any subset of an album's properties:

```
type PartialAlbum = Partial<Album>;
```

Now you have a `PartialAlbum` type where `title`, `artist`, `releaseYear`, and `genre` are all optional, which means you can create a

function that receives only a subset of the album's properties:

```
const updateAlbum = (album: PartialAlbum) => {  
    // . . .  
};  
  
updateAlbum({title: "Geogaddi", artist: "Boards of Canada"});
```

In this example, TypeScript won't complain about missing the `id`, `releaseYear`, or `genre` properties thanks to using the `Partial` type helper.

The Required Type

On the opposite side of `Partial` is the `Required` type, which makes sure all the properties of a given object type are required.

This `Album` interface has the `releaseYear` and `genre` properties marked as optional:

```
interface Album {  
    title: string;  
    artist: string;  
    releaseYear?: number;  
    genre?: string;  
}
```

You can use the `Required` utility type to create a new `RequiredAlbum` type:

```
type RequiredAlbum = Required<Album>;
```

With `RequiredAlbum`, all of the original `Album` properties become required, and omitting any of them would result in an error:

```
const doubleCup: RequiredAlbum = {  
    title: "Double Cup",  
    artist: "DJ Rashad",  
    releaseYear: 2013,
```

```
    genre: "Juke",  
};
```

The `doubleCup` example defines all the properties of the original `Album` interface, so there are no errors. If it were missing `releaseYear` or `genre`, TypeScript would show an error.

It's important to note that both `Required` and `Partial` work only one level deep. For example, if the `Album`'s `genre` contained nested properties, `Required<Album>` would not make the children required:

```
type Album = {  
    title: string;  
    artist: string;  
    releaseYear?: number;  
    genre?: {  
        parentGenre?: string;  
        subGenre?: string;  
    };  
};  
  
type RequiredAlbum = Required<Album>;  
// hovering over RequiredAlbum shows:  
type RequiredAlbum = {  
    title: string;  
    artist: string;  
    releaseYear: number;  
    genre: {  
        parentGenre?: string;  
        subGenre?: string;  
    };  
};
```

In the previous code snippet, the `parentGenre` and `subGenre` properties are optional even though the `Required` type helper was used.

NOTE

If you find yourself in a situation where you need a deeply Required type, check out the type-fest library by Sindre Sorhus at <https://github.com/sindresorhus/type-fest>.

The Pick Type

The `Pick` utility type allows you to create a new object type by picking certain properties from an existing object.

For example, say you want to create a new type that includes only the title and artist properties from the `Album` type:

```
type AlbumData = Pick<Album, "title" | "artist">;
```

This results in `AlbumData` being a type that includes only those properties. This utility is extremely useful when you want to have one object that relies on the shape of another object. We'll explore this more in [Chapter 10](#).

The Omit Type

The `Omit` helper type is kind of like the opposite of `Pick`. It allows you to create a new type by excluding a subset of properties from an existing type.

For example, you could use `Omit` to create the same `AlbumData` type you created with `Pick`, but this time by excluding the `id`, `releaseYear`, and `genre` properties:

```
type AlbumData = Omit<Album, "id" | "releaseYear" | "genre">;
```

A common use case is to create a type without `id`, for situations where the `id` has not yet been assigned:

```
type AlbumData = Omit<Album, "id">;
```

This means that as `Album` gains more properties, they will flow down to `AlbumData` too.

On the surface, using `omit` is straightforward, but there is a small quirk to be aware of. When using `omit`, you are able to exclude properties that don't exist on an object type. For example, creating an `AlbumWithoutProducer` type with your `Album` type would not result in an error, even though `producer` doesn't exist on `Album`:

```
type Album = {
  id: string;
  title: string;
  artist: string;
  releaseYear: number;
  genre: string;
};

type AlbumWithoutProducer = Omit<Album, "producer">;
```

If you tried to create an `AlbumWithOnlyProducer` type using `Pick`, you would get an error because `producer` doesn't exist on `Album`:

```
type AlbumWithOnlyProducer = Pick<Album, "producer">; // red
squiggly line under "producer"

// hovering over "producer" shows:
Type '"producer"' does not satisfy the constraint 'keyof Album'.
```

Why do these two utility types behave differently? When the TypeScript team was originally implementing `omit`, they were faced with a decision to create a strict or loose version of `omit`. The strict version would permit only the omission of valid keys (`id`, `title`, `artist`, `releaseYear`, and `genre`), whereas the loose version wouldn't have this constraint. At the time, it was a more popular idea in the community to implement a loose version, so that's the one the team went with. Given that global types in TypeScript are globally available and don't require an `import` statement, the looser version is seen as a safer choice, as it is more compatible and less likely to cause unforeseen errors.

This loose version of `omit` should be sufficient for most cases. Just keep an eye out, since it may error in ways you don't expect. We'll look at a strict version of `omit` in [Chapter 15](#).

NOTE

For more insights into the decisions behind `omit`, refer to the TypeScript team's original discussion (<https://github.com/microsoft/TypeScript/issues/30455>) and pull request adding `omit` (<https://github.com/microsoft/TypeScript/pull/30552>), as well as their final note on the topic (<https://github.com/microsoft/TypeScript/issues/30825#issuecomment-523668235>).

Union Types with `Omit` and `Pick`

The `omit` and `Pick` types exhibit some odd behavior when used with union types. For example, consider a scenario where you have three interface types for `Album`, `CollectorEdition`, and `DigitalRelease`:

```
type Album = {
  id: string;
  title: string;
  genre: string;
};

type CollectorEdition = {
  id: string;
  title: string;
  limitedEditionFeatures: string[];
};

type DigitalRelease = {
  id: string;
  title: string;
  digitalFormat: string;
};
```

These types share two common properties, `id` and `title`, but each also has unique attributes. The `Album` type includes `genre`, `CollectorEdition`

includes `limitedEditionFeatures`, and `DigitalRelease` has `digitalFormat`.

After creating a `MusicProduct` type that is a union of these three types, say you want to create a `MusicProductWithoutId` type, mirroring the structure of `MusicProduct` but excluding the `id` field:

```
type MusicProduct = Album | CollectorEdition | DigitalRelease;  
  
type MusicProductWithoutId = Omit<MusicProduct, "id">;
```

You might assume that `MusicProductWithoutId` would be a union of the three types minus the `id` field. However, what you get instead is a simplified object type containing only `title`:

```
// Expected:  
type MusicProductWithoutId =  
  | Omit<Album, "id">  
  | Omit<CollectorEdition, "id">  
  | Omit<DigitalRelease, "id">;  
  
// Actual:  
type MusicProductWithoutId = {  
  title: string;  
};
```

The reason for this is that `id` and `title` were the only properties shared by all members of the union. So, `Omit` seems to collapse the union into its common properties (`id` and `title`), and then remove `id`.

This is particularly annoying given that `Partial` and `Required` work as expected with union types:

```
type PartialMusicProduct = Partial<MusicProduct>;  
  
// hovering over PartialMusicProduct shows:  
type PartialMusicProduct =  
  | Partial<Album>
```

```
| Partial<CollectorEdition>  
| Partial<DigitalRelease>;
```

This issue stems from how `Omit` processes union types. Rather than iterating over each union member, it amalgamates them into a single structure it can understand. The technical reason for this is that `Omit` and `Pick` are not distributive, which means when you use them with a union type, they don't operate individually on each union member.

To address this, you can create a `DistributiveOmit` type. It's defined similarly to `Omit` but operates individually on each union member. Note the inclusion of `PropertyKey` in the type definition to allow for any valid key type:

```
type DistributiveOmit<T, K extends PropertyKey> = T extends  
  any  
    ? Omit<T, K>  
    : never;
```

When you apply `DistributiveOmit` to your `MusicProduct` type, you get the anticipated result—a union of `Album`, `CollectorEdition`, and `DigitalRelease` with the `id` field omitted:

```
type MusicProductWithoutId = DistributiveOmit<MusicProduct,  
  "id">;  
  
// hovering over MusicProductWithoutId shows:  
type MusicProductWithoutId =  
  | Omit<Album, "id">  
  | Omit<CollectorEdition, "id">  
  | Omit<DigitalRelease, "id">;
```

Structurally, that's the same as this:

```
type MusicProductWithoutId =  
  | {  
    title: string;  
    genre: string;
```

```
    }  
  | {  
    title: string;  
    limitedEditionFeatures: string[];  
  }  
  | {  
    title: string;  
    digitalFormat: string;  
  };  
};
```

In situations where you need to use `Omit` with union types, using a distributive version will give you a much more predictable result.

For completeness, the `DistributivePick` type can be defined in a similar way:

```
type DistributivePick<T, K extends keyof T> = T extends any  
  ? Pick<T, K>  
  : never;
```

This `DistributivePick` type would apply the `Pick` operation to each member of a union type individually.

Exercise 6-7: Expecting Certain Properties

In this exercise, you have a `fetchUser` function that uses `fetch` to access an endpoint named `api/user` and returns a `Promise<User>`:

```
interface User {  
  id: string;  
  name: string;  
  email: string;  
  role: string;  
}  
  
const fetchUser = async (): Promise<User> => {  
  const response = await fetch("/api/user");  
  const user = await response.json();  
  return user;  
};
```

```
const example = async () => {
  const user = await fetchUser();

  type test = Expect<Equal<typeof user, {name: string; email: string}>>>;
  // red squiggly line under Equal<>
};
```

Since you're in an asynchronous function, you do want to use a Promise, but there's a problem with this User type. In the example function that calls `fetchUser`, you're expecting to receive only the name and email fields. These fields are only part of what exists in the User interface.

Your task is to update the typing so that only the name and email fields are expected to be returned from `fetchUser`. You can use the helper types you've looked at to accomplish this, but for extra practice try using just interfaces. See <https://totalts.link/essentials-6-7/>.

Solution

There are quite a few ways to solve this problem. Using the `Pick` utility type, you can create a new type that includes only the name and email properties from the User interface:

```
type PickedUser = Pick<User, "name" | "email">;
```

Then, you can update the `fetchUser` function to return a Promise of `PickedUser`:

```
const fetchUser = async (): Promise<PickedUser> => {
  . . .
```

You can also use the `Omit` utility type to create a new type that excludes the `id` and `role` properties from the User interface:

```
type OmittedUser = Omit<User, "id" | "role">;
```

Then, you can update the `fetchUser` function to return a `Promise` of `OmittedUser`:

```
const fetchUser = async (): Promise<OmittedUser> => {  
  . . .  
}
```

Another possibility would be to create an interface `NameAndEmail` that contains a `name` property and an `email` property, along with updating the `User` interface to remove those properties in favor of extending them:

```
interface NameAndEmail {  
  name: string;  
  email: string;  
}  
  
interface User extends NameAndEmail {  
  id: string;  
  role: string;  
}
```

Then, the `fetchUser` function could return a `Promise` of `NameAndEmail`:

```
const fetchUser = async (): Promise<NameAndEmail> => {  
  // . . .  
};
```

Using `omit` will mean that the object grows as the source object grows, and using `Pick` and `interface extends` will mean that the object will stay the same size. Choose the best approach depending on your requirements.

Exercise 6-8: Updating a Product

Here you have a function, `updateProduct`, that takes two arguments—an `id` and a `productInfo` object derived from the `Product` type, excluding the `id` field:

```
interface Product {
  id: number;
  name: string;
  price: number;
  description: string;
}

const updateProduct = (id: number, productInfo: Omit<Produc
t, "id">) => {
  // Do something with the productInfo.
};
```

The twist here is that during a product update, you might not want to modify all of its properties at the same time. Because of this, not all properties have to be passed into the function.

This means you have several different test scenarios. For example, update just the name, just the price, or just the description. Combinations like updating the name and the price or the name and the description are also tested:

```
updateProduct(1, {
  // red squiggly line under the entire object
  name: "Book",
});

updateProduct(1, {
  // red squiggly line under the entire object
  price: 12.99,
});

updateProduct(1, {
  // red squiggly line under the entire object
  description: "A book about Dragons",
});

updateProduct(1, {
  // red squiggly line under the entire object
  name: "Book",
  price: 12.99,
```

```
});

updateProduct(1, {
  // red squiggly line under the entire object
  name: "Book",
  description: "A book about Dragons",
});
```

Your challenge is to modify the `productInfo` parameter to reflect these requirements. The `id` should remain absent from `productInfo`, but you also want all other properties in `productInfo` to be optional. See <https://totalts.link/essentials-6-8/>.

Solution

Using a combination of `Omit` and `Partial` will allow you to create a type that excludes the `id` field from `Product` and makes all other properties optional.

In this case, wrapping `Omit<Product, "id">` in `Partial` will remove the `id` while making all of the remaining properties optional:

```
const updateProduct = (
  id: number,
  productInfo: Partial<Omit<Product, "id">>,
) => {
  // Do something with the productInfo.
};
```

Summary

This chapter explored TypeScript's powerful features for working with object types, focusing on extending objects and managing dynamic properties.

TypeScript provides two main approaches for extending object types: intersection types using the `&` operator and interface extension with the `extends` keyword. Intersection types combine multiple object types into one, while `interface extends` creates inheritance relationships between interfaces.

The interface extends approach offers significant advantages over intersections: It provides better error messages when properties conflict and delivers superior TypeScript performance since interfaces can be cached by name rather than recomputed each time.

When choosing between type and interface for object definitions, type aliases offer more flexibility, supporting unions and other complex types. However, interfaces have a quirk, declaration merging, where multiple interfaces with the same name automatically combine their properties.

For dynamic object keys, TypeScript offers several solutions. Index signatures using `[key: string]: valueType` syntax allow objects to accept any string key. The `Record<KeyType, ValueType>` utility type provides a cleaner alternative and supports union types as keys, unlike index signatures.

You can combine known and dynamic keys by intersecting a `Record` type with an index signature or by extending an interface that includes both specific properties and an index signature. The global `PropertyKey` type represents all possible key types: `string`, `number`, and `symbol`.

TypeScript's built-in utility types help reduce code duplication. `Partial<T>` makes all properties optional, while `Required<T>` makes them all required. `Pick<T, K>` selects specific properties, and `Omit<T, K>` excludes them.

When `Omit` and `Pick` are used with union types, they don't distribute over each union member as expected. Creating distributive versions using conditional types solves this limitation, ensuring that these utilities work predictably with unions.

7

MUTABILITY



The way you declare variables and types has an effect on TypeScript's behavior. In this chapter, we'll cover the relationship between mutability and inference from several different angles. You'll learn how variable declarations affect narrowing, how to use helpers to prevent mutations at the type level, and how to create deeply immutable data structures.

Mutability and Inference

How you declare variables in TypeScript affects whether they can be changed later. This includes whether you use `let` or `const` keywords, as well as the syntax you use when creating objects and arrays.

How TypeScript Infers `let`

When using the `let` keyword, the variable is *mutable* and can be reassigned.

Consider this `AlbumGenre` type—a union of literal values representing possible genres for an album:

```
type AlbumGenre = "rock" | "country" | "electronic";
```

Using `let`, you can declare a variable `albumGenre` and assign it the value `"rock"`. Then, you can attempt to pass `albumGenre` to a function that expects an `AlbumGenre`:

```
let albumGenre = "rock";

const handleGenre = (genre: AlbumGenre) => {
  // . . .
};

handleGenre(albumGenre); // red squiggly line under albumGenre
// hovering over albumGenre shows:
Argument of type 'string' is not assignable to parameter of
type 'AlbumGenre'.
```

In this code example, TypeScript understands that the value of `albumGenre` can later be changed because `let` was used when declaring the variable. Because the value might be changed, TypeScript widens it to a wider type. In this case, it infers `albumGenre` as a `string` rather than the specific literal type `"rock"`.

The error could be fixed by assigning a specific type to the `let`:

```
let albumGenre: AlbumGenre = "rock";
const handleGenre = (genre: AlbumGenre) => {
  // . . .
};

handleGenre(albumGenre); // No more error
```

With this change, the `albumGenre` variable *can* be reassigned, but only to a value that is a member of the `AlbumGenre` union. So, there will no longer be an error when passed to `handleGenre`.

How TypeScript Infers const

Let's look at the same example again, but this time using `const` instead of `let`.

When using `const`, the variable is *immutable* and cannot be reassigned. When you change the variable declaration to use `const`, TypeScript will infer the type more narrowly:

```
const albumGenre = "rock";
const handleGenre = (genre: AlbumGenre) => {
  // . . .
};

handleGenre(albumGenre); // No error
```

There is no longer an error in the assignment, and hovering over `albumGenre` in the `albumDetails` object shows that TypeScript has inferred it as the literal type `"rock"`.

If you try to change the value of `albumGenre` after declaring it as `const`, TypeScript will show an error:

```
albumGenre = "country"; // red squiggly line under albumGenre
// hovering over albumGenre shows:
Cannot assign to 'albumGenre' because it is a constant.
```

TypeScript is mirroring JavaScript's treatment of `const` to prevent possible runtime errors. When you declare a variable with `const`, TypeScript infers it as the literal type you specified.

So, TypeScript uses how JavaScript works to its advantage. This will often encourage you to use `const` over `let` when declaring variables, as it's a little stricter.

Object Property Inference

The situation with `const` and `let` becomes a bit more complicated when it comes to object properties.

Objects are mutable in JavaScript, meaning their properties can be changed after they are created.

For this example, you have an `AlbumAttributes` type that includes a `status` property with a union of literal values representing possible album statuses:

```
type AlbumAttributes = {  
  status: "new-release" | "on-sale" | "staff-pick";  
};
```

Say you had an `updateStatus` function that takes an `AlbumAttributes` object:

```
const updateStatus = (attributes: AlbumAttributes) => {  
  // . . .  
};  
  
const albumAttributes = {  
  status: "on-sale",  
};  
  
updateStatus(albumAttributes); // red squiggly line under al  
bumAttributes
```

TypeScript gives you an error after `albumAttributes` in the `updateStatus` function call, with messages similar to what you saw before:

```
// hovering over albumAttributes shows:  
Argument of type '{status: string;}' is not assignable to pa  
rameter of  
type 'AlbumAttributes'.  
Types of property 'status' are incompatible.  
Type 'string' is not assignable to type '"new-release" | "on  
-sale" | "staff-pick"'.  

```

This is happening because TypeScript has inferred the `status` property as a string rather than the specific literal type `"on-sale"`. Similar to `let`, TypeScript understands that the property could later be reassigned:

```
albumAttributes.status = "new-release";
```

This is true even though the `albumAttributes` object was declared with `const`. You get the error when calling `updateStatus` because `status: string` can't be passed to a function that expects `status: "new-release" | "on-sale" | "staff-pick"`. TypeScript is trying to protect you from potential runtime errors.

Let's look at a couple of ways to fix this issue.

One approach is to inline the object when calling the `updateStatus` function instead of declaring it separately:

```
updateStatus({  
  status: "on-sale",  
}); // No error
```

When inlining the object, TypeScript knows that there is no way that `status` could be changed before it is passed into the function, so it infers it more narrowly.

Another option is to explicitly declare the type of the `albumAttributes` object to be `AlbumAttributes`:

```
const albumAttributes: AlbumAttributes = {  
  status: "on-sale",  
};  
  
updateStatus(albumAttributes); // No error
```

This works similarly to how it did with `let`. While `albumAttributes.status` can still be reassigned, it can be reassigned only to a valid value:

```
albumAttributes.status = "new-release"; // No error
```

This behavior works the same for all object-like structures, including arrays and tuples. We'll cover them later in the exercises.

Readonly Object Properties

In JavaScript, as you've seen, object properties are mutable by default. But TypeScript lets you be more specific about whether a property of an object can be mutated.

To make a property read-only (not writable), you can use the `readonly` modifier.

Consider this `Album` interface, where the `title` and `artist` are marked as `readonly`:

```
interface Album {  
  readonly title: string;  
  readonly artist: string;  
  status?: "new-release" | "on-sale" | "staff-pick";  
  genre?: string[];  
}
```

Once an `Album` object is created, its `title` and `artist` properties are locked in and cannot be changed. However, the optional `status` and `genre` properties can still be modified.

Note that this occurs only on the *type* level. At runtime, the properties are still mutable. TypeScript is just helping you catch potential errors.

If you want to specify that *all* properties of an object should be read-only, TypeScript provides a type helper called `Readonly`.

To use it, you simply wrap the object type with `Readonly`.

Here's an example of using `Readonly` to create an `Album` object:

```
const readonlyWhiteAlbum: Readonly<Album> = {  
  title: "The Beatles (White Album)",  
  artist: "The Beatles",  
  status: "staff-pick",  
};
```

Because the `readonlyWhiteAlbum` object was created using the `Readonly` type helper, none of the properties can be modified:

```
readOnlyWhiteAlbum.genre = ["rock", "pop", "unclassifiabl  
e"]; // red squiggly line under genre  
// hovering over genre shows:  
Cannot assign to 'genre' because it is a read-only property.
```

Note that like many of TypeScript's type helpers, the immutability enforced by `Readonly` operates only on the first level. It won't make properties read-only recursively.

As with object properties, arrays and tuples can also be made immutable by using the `readonly` modifier.

Here's how the `readonly` modifier can be used to create a read-only array of genres. Once the array is created, its contents cannot be modified:

```
const readOnlyGenres: readonly string[] = ["rock", "pop", "u  
nclassifiable"];
```

Similar to the `Array` syntax, TypeScript also offers a `ReadonlyArray` type helper that functions in the same way as using the previous syntax:

```
const readOnlyGenres: ReadonlyArray<string> = ["rock", "po  
p", "unclassifiable"];
```

Both of these approaches are functionally the same. Hovering over the `readOnlyGenres` variable shows that TypeScript has inferred it as a read-only array:

```
// hovering over readOnlyGenres shows:  
const readOnlyGenres: readonly string[];
```

Read-only arrays disallow the use of array methods that cause mutations, such as `push` and `pop`:

```
readOnlyGenres.push("experimental"); // red squiggly line un  
der push  
// Property 'push' does not exist on type 'readonly string  
[]'.
```

However, methods like `map` and `reduce` will still work, as they create a copy of the array and do not mutate the original:

```
const uppercaseGenres = readOnlyGenres.map((genre) => genre.  
  toUpperCase()); // No error  
  
readOnlyGenres.push("experimental"); // red squiggly line un  
der push  
  
// hovering over push shows:  
Property 'push' does not exist on type 'readonly string[]'
```

Note that, just like the `readonly` for object properties, this doesn't affect the runtime behavior of the array. It's just a way to help catch potential errors.

How Read-Only and Mutable Arrays Work Together

To help drive the concept home, let's see how read-only and mutable arrays work together.

Here are two `printGenre` functions that are functionally identical, except `printGenresReadOnly` takes a read-only array of genres as a parameter, whereas `printGenresMutable` takes a mutable array:

```
function printGenresReadOnly(genres: readonly string[]) {  
  // . . .  
}  
  
function printGenresMutable(genres: string[]) {  
  // . . .  
}
```

When you create a mutable array of genres, it can be passed as an argument to both of these functions without error:

```
const mutableGenres = ["rock", "pop", "unclassifiable"];
```

```
printGenresReadOnly(mutableGenres);  
printGenresMutable(mutableGenres);
```

This works because specifying `readonly` on the `printGenresReadOnly` function parameter guarantees only that the function won't alter the array's content. Thus, it doesn't matter if you pass a mutable array because it won't be changed.

However, the reverse is not true. If you declare a read-only array, you can pass it only to `printGenresReadOnly`. Attempting to pass it to `printGenresMutable` will yield an error:

```
const readonlyGenres: readonly string[] = ["rock", "pop", "unclassifiable"];  
  
printGenresReadOnly(readonlyGenres);  
printGenresMutable(readonlyGenres); // red squiggly line under readonlyGenres  
  
// hovering over readonlyGenres shows:  
Error: Argument of type 'readonly ["rock", "pop", "unclassifiable"]' is not  
assignable to parameter of type 'string[]'.
```

We can't pass a read-only array to `printGenresMutable` because TypeScript isn't sure if you might be mutating the array in the function.

Essentially, read-only arrays can be assigned only to other read-only types. This can spread virally throughout your application: If an array is marked as `readonly`, then every function that handles it must also expect a `readonly` array. But doing so brings benefits. It ensures that the array won't be mutated in any manner as it moves down the stack. Very useful.

The takeaway here is that even though you can assign mutable arrays to read-only arrays, you cannot assign read-only arrays to mutable arrays.

The readonly Object Gap

While TypeScript is quite strict with assignability rules around arrays, it is less strict with objects. An object with a `readonly` property can freely be passed to a function that then mutates it. Let's show this with an example.

First, let's declare two types. One has a readonly property, and another has a mutable property:

```
type ReadonlyAlbum = {  
  readonly genre: string;  
};
```

```
type MutableAlbum = {  
  genre: string;  
};
```

Next, let's declare a readonly album:

```
const readonlyAlbum: ReadonlyAlbum = {  
  genre: "Jazz Rap",  
};
```

Let's create a function that takes in a mutable album and mutates its genre property:

```
const processAlbum = (album: MutableAlbum) => {  
  album.genre = "Pop Punk";  
};
```

Finally, let's call processAlbum with the new album:

```
processAlbum(readonlyAlbum);
```

Using the rules of arrays we've discussed, this *should* throw an error. We've said that genre is readonly, yet we've modified it in a function.

However, this doesn't show an error. This is a known limitation of TypeScript. The TypeScript team made a decision early on to allow readonly properties to be passed into places they could be mutated. Enforcing this would be more correct.

Recently, readonly object properties have come under more scrutiny, and this behavior may be changed in a future version. We'll have to wait

and see.

For now, be a little careful with readonly object properties—they are occasionally mutable.

Exercise 7-1: Inference with an Array of Objects

Here you have a `modifyButtons` function that takes in an array of objects with type properties that are either `"button"`, `"submit"`, or `"reset"`.

When attempting to call `modifyButtons` with an array of objects that seem to meet the contract, TypeScript gives you an error:

```
type ButtonAttributes = {
  type: "button" | "submit" | "reset";
};

const modifyButtons = (attributes: ButtonAttributes[]) =>
  {};

const buttonsToChange = [
  {
    type: "button",
  },
  {
    type: "submit",
  },
];

modifyButtons(buttonsToChange); // red squiggly line under b
uttonsToChange
```

Your task is to determine why this error shows up and then resolve it. See <https://totalts.link/essentials-7-1/>.

Solution

Hovering over the `buttonsToChange` variable shows you that it is being inferred as an array of objects with a type property of type string:

```
// hovering over buttonsToChange shows:
const buttonsToChange: {
  type: string;
}[];
```

This inference is happening because your array is mutable. You could change the type of the first element in the array to something different:

```
buttonsToChange[0].type = "something strange";
```

This wider type is incompatible with the `ButtonAttributes` type, which expects the `type` property to be one of "button", "submit", or "reset".

The fix here is to specify that `buttonsToChange` is an array of `ButtonAttributes`:

```
type ButtonAttributes = {
  type: "button" | "submit" | "reset";
};

const modifyButtons = (attributes: ButtonAttributes[]) =>
  {};

const buttonsToChange: ButtonAttributes[] = [
  {
    type: "button",
  },
  {
    type: "submit",
  },
];

modifyButtons(buttonsToChange); // No error
```

Or you could pass the array directly to the `modifyButtons` function:

```
modifyButtons([
  {
    type: "button",
  },
  {
    type: "submit",
  },
]); // No error
```

By doing this, TypeScript will infer the type property more narrowly, and the error will go away.

Exercise 7-2: Avoiding Array Mutation

This `printNames` function accepts an array of name strings and logs them to the console. However, there are also nonworking `@ts-expect-error` comments that should not allow for names to be added or changed:

```
function printNames(names: string[]) {
  for (const name of names) {
    console.log(name);
  }

  // @ts-expect-error // red squiggly line under unused "@ts-
  // expect-error" directive
  names.push("John");

  // @ts-expect-error // red squiggly line under unused "@ts-
  // expect-error" directive
  names[0] = "Billy";
}
```

Your task is to update the type of the `names` parameter so that the array cannot be mutated.

See <https://totalts.link/essentials-7-2/>.

Solutions

Here are a couple of ways to solve this problem:

Option 1

The first approach is to add the `readonly` keyword before the `string[]` array. It applies to the entire `string[]` array, converting it into a read-only array:

```
function printNames(names: readonly string[]) {  
    . . .  
}
```

With this setup, TypeScript won't allow you to add items with `.push()` or perform any other modifications on the array.

Option 2

Alternatively, you could use the `ReadonlyArray` type helper:

```
function printNames(names: ReadonlyArray<string>) {  
    . . .  
}
```

Regardless of which of these two methods you use, TypeScript will still display `readonly string[]` when hovering over the `names` parameter:

```
// hovering over names shows:  
(parameter) names: readonly string[]
```

Both work equally well at preventing the array from being modified.

Exercise 7-3: An Unsafe Tuple

Here you have a `dangerousFunction` that accepts an array of numbers as an argument:

```
const dangerousFunction = (arrayOfNumbers: number[]) => {  
    arrayOfNumbers.pop();  
    arrayOfNumbers.pop();  
};
```

Additionally, you've defined a variable `myHouse`, which is a tuple representing a `Coordinate`:

```
type Coordinate = [number, number];  
const myHouse: Coordinate = [0, 0];
```

Our tuple `myHouse` contains two elements, and the `dangerousFunction` is structured to pop two elements from the given array.

Given that `pop` removes the last element from an array, calling `dangerousFunction` with `myHouse` will remove its contents.

Currently, TypeScript does not alert you to this potential issue, as shown by the error line under `@ts-expect-error`:

```
dangerousFunction(  
  // @ts-expect-error // red squiggly line under @ts-expect-  
  error  
  myHouse,  
);
```

Your task is to adjust the type of `Coordinate` such that TypeScript triggers an error when you attempt to pass `myHouse` into `dangerousFunction`.

Note that you should change only `Coordinate` and leave the function untouched.

See <https://totalts.link/essentials-7-3/>.

Solution

The best way to prevent unwanted changes to the `Coordinate` tuple is to make it a readonly tuple:

```
type Coordinate = readonly [number, number];
```

Now `dangerousFunction` throws a TypeScript error when you try to pass `myHouse` to it:

```
const dangerousFunction = (arrayOfNumbers: number[]) => {
  arrayOfNumbers.pop();
  arrayOfNumbers.pop();
};

dangerousFunction(
  // @ts-expect-error
  myHouse, // red squiggly line under myHouse
);

// hovering over myHouse shows:
Argument of type 'Coordinate' is not assignable to parameter
of type 'number[]'.
The type 'Coordinate' is 'readonly' and cannot be assigned t
o the mutable type 'number[]'.
```

You get an error because the function's signature expects a modifiable array of numbers, but `myHouse` is a read-only tuple. TypeScript is protecting you against unwanted changes.

It's a good practice to use `readonly` tuples as much as possible to avoid problems like the one in this exercise.

Deep Immutability with `as const`

You've seen so far that objects and arrays are mutable in JavaScript. This leads to their properties being inferred *widely* by TypeScript.

You can get around this by giving the property a type annotation. But it still doesn't infer the literal type of the property:

```
const albumAttributes: AlbumAttributes = {
  status: "on-sale",
};

// hovering over albumAttributes shows:
const albumAttributes: {
  status: "new-release" | "on-sale" | "staff-pick";
};
```

Instead of `albumAttributes.status` being inferred as `"on-sale"`, it's inferred as `"new-release" | "on-sale" | "staff-pick"`.

One way you could get TypeScript to infer it properly would be to somehow mark the entire object, and all its properties, as immutable. This would tell TypeScript that the object and its properties can't be changed, so it would be free to infer the literal types of the properties.

This is where the `as const` assertion comes in. You can use it to mark an object and all of its properties as constants, meaning that they can't be changed once they are created:

```
const albumAttributes = {  
  status: "on-sale",  
} as const;  
  
// hovering over albumAttributes shows:  
const albumAttributes: {  
  readonly status: "on-sale";  
};
```

The `as const` assertion has made the entire object deeply read-only, including all of its properties. This means that `albumAttributes.status` is now inferred as the literal type `"on-sale"`.

Attempting to change the `status` property will result in an error:

```
albumAttributes.status = "new-release"; // red squiggly line  
under status
```

This makes `as const` ideal for large config objects that you don't expect to change.

Just like the `readonly` modifier, `as const` affects only the type level. At runtime, the object and its properties are still mutable.

as const vs. Variable Annotation

You might be wondering what would happen if you combined `as const` with a variable annotation. How would it be inferred?

```
const albumAttributes: AlbumAttributes = {
  status: "on-sale",
} as const;
```

You can think of this code as a competition between two forces: the `as const` assertion operating on the value, and the annotation operating on the variable.

When you have a competition like this, the variable annotation wins. The variable *owns* the value and forgets whatever the explicit value was before.

This means, curiously, that the `status` property is inferred as being mutable:

```
albumAttributes.status = "new-release"; // No error
```

The `as const` assertion is being overridden by the variable annotation. Not fun.

We'll cover this interaction between variables and values further in [Chapter 11](#).

as const vs. Object.freeze

In JavaScript, the `Object.freeze` method is a way to create immutable objects at runtime. There are some significant differences between `Object.freeze` and `as const`.

For this example, you'll create a `shelfLocations` object that uses `Object.freeze`:

```
const shelfLocations = Object.freeze({
  entrance: {
    status: "on-sale",
  },
  frontCounter: {
    status: "staff-pick",
  },
  endCap: {
    status: "new-release",
  },
});
```

```
    },  
  });  
};
```

Hovering over `shelfLocations` shows that the object has the `Readonly` modifier applied to it:

```
// hovering over shelfLocations shows:  
const shelfLocations: Readonly<{  
  entrance: {  
    status: string;  
  };  
  frontCounter: {  
    status: string;  
  };  
  endCap: {  
    status: string;  
  };  
}>;
```

Recall that the `Readonly` modifier works only on the *first level* of an object. If you try to modify the `frontCounter` property, TypeScript will throw an error:

```
shelfLocations.frontCounter = {status: "on-sale"}; // red squiggly line under frontCounter  
  
// hovering over frontCounter shows:  
Cannot assign to 'frontCounter' because it is a read-only property.
```

However, you are able to change the nested `status` property of a specific location:

```
shelfLocations.entrance.status = "new-release";
```

This is in line with how `Object.freeze` works in JavaScript. It makes the object and its properties read-only solely at the first level. It doesn't

make the entire object deeply read-only.

Using `as const` makes the entire object deeply read-only, including all nested properties:

```
const shelfLocations = {
  entrance: {
    status: "on-sale",
  },
  frontCounter: {
    status: "staff-pick",
  },
  endCap: {
    status: "new-release",
  },
} as const;

// hovering over shelfLocations shows:
const shelfLocations: {
  readonly entrance: {
    readonly status: "on-sale";
  };
  readonly frontCounter: {
    readonly status: "staff-pick";
  };
  readonly endCap: {
    readonly status: "new-release";
  };
};
```

You can see from hovering that all the properties of `shelfLocations` now have the `readonly` modifier.

To summarize, `Object.freeze` gives you runtime immutability, while `as const` gives you type-level immutability. While both enforce immutability, `as const` is the more convenient and efficient choice since it means less work has to be done at runtime. Unless you specifically need the top level of an object to be frozen at runtime, you should stick with `as const`.

Exercise 7-4: Inferring Literal Values in Arrays

Let's revisit a previous exercise and evolve your solution.

The `modifyButtons` function accepts an array of objects with a `type` property:

```
type ButtonAttributes = {  
  type: "button" | "submit" | "reset";  
};  
  
const modifyButtons = (attributes: ButtonAttributes[]) =>  
  {};  
  
const buttonsToChange = [  
  {  
    type: "button",  
  },  
  {  
    type: "submit",  
  },  
];  
  
modifyButtons(buttonsToChange); // red squiggly line under b  
uttonsToChange
```

Previously, the error was solved by updating `buttonsToChange` to be specified as an array of `ButtonAttributes`:

```
const buttonsToChange: ButtonAttributes[] = [  
  {  
    type: "button",  
  },  
  {  
    type: "submit",  
  },  
];
```

This time, your challenge is to solve the error by finding a different solution. Specifically, you should modify the `buttonsToChange` array so that TypeScript infers the literal type of the `type` property.

You should not alter the `ButtonAttributes` type definition or the `modifyButtons` function.

See <https://totalts.link/essentials-7-4/>.

Solutions

Let's look at some different options for solving this challenge:

Option 1

The `as const` assertion can be used to solve this problem. By annotating the entire array with `as const`, TypeScript will infer the literal type of the `type` property:

```
const buttonsToChange = [
  {
    type: "button",
  },
  {
    type: "submit",
  },
] as const;
```

However, this won't quite work as expected. With just this change, TypeScript shows a different error:

```
modifyButtons(buttonsToChange); // red squiggly line under b
uttonsToChange
// The type 'readonly [{readonly type: "button";}, {readonly
type:
"submit";}]' is 'readonly' and cannot be assigned to the mut
able type 'ButtonAttributes[]'.
```

This is because `readonly` arrays aren't assignable to mutable ones. So, you'll need to change the declaration of `attributes` to match:

```
const modifyButtons = (attributes: readonly ButtonAttributes  
[]) => {};
```

Now the error will disappear.

Option 2

A simpler way to solve this problem is to annotate each member of the array with `as const`:

```
const buttonsToChange = [  
  {  
    type: "button",  
  } as const,  
  {  
    type: "submit",  
  } as const,  
];
```

Hovering over `buttonsToChange` shows something interesting. Each object is now inferred as `readonly`, but the array itself is not:

```
// hovering over buttonsToChange shows:  
const buttonsToChange: (  
  | {  
    readonly type: "button";  
  }  
  | {  
    readonly type: "submit";  
  }  
)[];
```

The `buttonsToChange` array is also no longer being inferred as a tuple with a fixed length, so you can modify it by pushing new objects to it:

```
buttonsToChange.push({  
  type: "button",  
});
```

This behavior stems from tagging the individual members of the array with `as const`, instead of the entire array.

However, this inference is good enough to satisfy `modifyButtons` because it matches the `ButtonAttributes` type.

Option 3

The last solution we'll present is using `as const` on the string literals to infer the literal type:

```
const buttonsToChange = [
  {
    type: "button" as const,
  },
  {
    type: "submit" as const,
  },
];
```

Now when you hover over `buttonsToChange`, you've lost the `readonly` modifier because `as const` is being targeted only at the string in the object, not at the object itself:

```
// hovering over buttonsToChange shows:
const buttonsToChange: (
  | {
    type: "button";
  }
  | {
    type: "submit";
  }
)[];
```

But again, this is still typed strongly enough to satisfy `modifyButtons`.

Using `as const` like this hints to TypeScript that it should infer a literal type where it wouldn't otherwise. This can be occasionally

useful for when you want to allow mutation but still want to infer a literal type.

Summary

This chapter explored how mutability affects TypeScript's type inference and provides tools for controlling when data can be changed.

When you declare variables with `let`, TypeScript infers wider types because the values might be reassigned later. Using `const` creates more precise literal types since the variable can't be changed.

Object properties present a more complex situation. Even when declared with `const`, object properties remain mutable, so TypeScript infers them as wider string types rather than specific literals.

You can solve this by explicitly typing objects, inlining them in function calls, or using the `readonly` modifier to prevent mutations at the type level.

The `Readonly` type helper makes all properties of an object read-only, while `readonly` arrays prevent mutating methods like `push` but allow nonmutating ones like `map`.

Read-only arrays can be passed only to functions expecting read-only parameters, but mutable arrays work with both mutable and read-only function signatures.

The `as const` assertion provides deep immutability, making entire objects and all nested properties read-only while inferring precise literal types. This differs from `Object.freeze`, which provides only shallow runtime immutability.

Understanding these concepts helps you write more precise TypeScript code by being intentional about where mutability is allowed in your applications.

8

CLASSES



Classes allow you to encapsulate data and behavior into a single unit, making them a fundamental feature of object-oriented programming. In this chapter, you will practice creating classes with methods, properties, and constructors. We'll also cover the concept of inheritance, including using modifiers to change behavior.

Creating a Class

To create a class, you use the `class` keyword followed by the name of the class.

Let's start by creating an `Album` class, with syntax that is also similar to how a type or interface is created:

```
class Album {  
    title: string; // red squiggly line under title  
    artist: string; // red squiggly line under artist  
    releaseYear: number; // red squiggly line under releaseYea  
r  
}
```

At this point, even though it looks like a type or interface, TypeScript gives an error for each property in the class:

```
// hovering over title shows:
```

```
Property 'title' has no initializer and is not definitely as  
signed in the constructor.
```

How do you fix these errors?

Adding a Constructor

To fix these errors, you need to add a constructor to the class. The constructor is a special method that runs when a new instance of the class is created. It's where you can set up the initial state of the object.

To start, you'll add a constructor that assigns values for the properties of the Album class:

```
class Album {  
  title: string;  
  artist: string;  
  releaseYear: number;  
  
  constructor() {  
    this.title = "Loop Finding Jazz Records";  
    this.artist = "Jan Jelinek";  
    this.releaseYear = 2001;  
  }  
}
```

Now, when you create a new instance of the Album class, you can access the properties and values you've set in the constructor:

```
const loopFindingJazzRecords = new Album();  
  
console.log(loopFindingJazzRecords.title); // Output: Loop F  
inding Jazz Records
```

The new keyword creates a new instance of the Album class, and the constructor sets the initial values of your class's properties. In this case, because the properties are hardcoded, every instance of the Album class will have the same values.

Just as in other areas, TypeScript can do some really smart inference with classes.

It's able to infer the types of the properties from where you assign them in the constructor, so you can actually drop some of the type annotations:

```
class Album {
  title;
  artist;
  releaseYear;

  constructor() {
    this.title = "Loop Finding Jazz Records";
    this.artist = "Jan Jelinek";
    this.releaseYear = 2001;
  }
}
```

However, it's common to see the types specified in the class body as well since they act as a form of documentation for the class that's quick to read.

Adding Arguments to the Constructor

You can use the constructor to declare arguments for the class. This allows you to pass in values when creating a new instance of the class.

Update the constructor to accept an opts argument that includes the properties of the Album class:

```
// In the Album class
constructor(opts: {title: string; artist: string; releaseYear: number}) {
  // . . .
}
```

Then, in the body of the constructor, you'll assign `this.title`, `this.artist`, and `this.releaseYear` to the values of the `opts` argument:

```
// In the Album class
constructor(opts: {title: string; artist: string; releaseYear: number}) {
  this.title = opts.title;
  this.artist = opts.artist;
  this.releaseYear = opts.releaseYear;
}
```

The `this` keyword refers to the instance of the class, and it's used to access the properties and methods of the class.

Now, when you create a new instance of the `Album` class, you can pass an object with the properties you want to set:

```
const loopFindingJazzRecords = new Album({
  title: "Loop Finding Jazz Records",
  artist: "Jan Jelinek",
  releaseYear: 2001,
});

console.log(loopFindingJazzRecords.title); // Output: Loop Finding Jazz Records
```

Using a Class as a Type

An interesting property of classes in TypeScript is that they can be used as types for variables and function parameters. The syntax is similar to how you would use any other type or interface.

In this case, you'll use the `Album` class to type the `album` parameter of a `printAlbumInfo` function:

```
function printAlbumInfo(album: Album) {
  console.log(
    `${album.title} by ${album.artist}, released in ${album.releaseYear}.`,
  );
}
```

```
    );  
}
```

Album here refers to the instance of the class (with its `title`, `artist`, and `releaseYear` properties in place), not the class itself. That means when you call the function, you pass in an instance of the Album class:

```
printAlbumInfo(sixtyNineLoveSongsAlbum);
```

```
// Output: 69 Love Songs by The Magnetic Fields, released in  
1999.
```

Properties in Classes

Now that you've seen how to create a class and create new instances of it, let's look a bit more closely at how properties work.

Class Property Initializers

You can set default values for properties directly in the class body. These are called *class property initializers*:

```
class Album {  
    title = "Unknown Album";  
    artist = "Unknown Artist";  
    releaseYear = 0;  
}
```

Just like with variables, the type of the property (that is, `string`) can be inferred from the value (that is, `Unknown Album`). If you want to annotate the type as well, you can do this:

```
class Album {  
    title: string = "Unknown Album";  
    artist: string = "Unknown Artist";  
    releaseYear: number = 0;  
}
```

In the previous code example, class property initializers are used to set default string values for `title` and `artist`, as well as a number for `releaseYear`.

Importantly, class property initializers are run *before* the constructor is called. This means you can override the default values by assigning a different value in the constructor:

```
class User {
  name = "Unknown User";

  constructor() {
    this.name = "Matt Pocock";
  }
}

const user = new User();

console.log(user.name); // Output: Matt Pocock
```

Here, the name being set in the constructor overrides the default name value from the class property initializer.

readonly Class Properties

As you've seen with types and interfaces, the `readonly` keyword can be used to make a property immutable. This means that once the property is set in the constructor, it cannot be changed:

```
class Album {
  readonly title: string;
  readonly artist: string;
  readonly releaseYear: number;
}
```

Optional Class Properties

You can also mark properties as optional in the same way as objects, using the `?:` modifier:

```
class Album {  
  title?: string;  
  artist?: string;  
  releaseYear?: number;  
}
```

As you can see from the lack of errors here, this also means they don't need to be set in the constructor.

public and private Properties

The `public` and `private` keywords are used to control the visibility and accessibility of class properties.

By default, properties are `public`, which means they can be accessed from outside the class.

If you want to restrict access to certain properties, you can mark them as `private`. This means that they can be accessed only from within the class itself.

For example, say you want to add a `rating` property to the `album` class that will be used only in the class:

```
class Album {  
  private rating = 0;  
}
```

Now if you try to access the `rating` property from outside the class, TypeScript will give you an error:

```
const loopFindingJazzRecords = new Album();  
console.log(loopFindingJazzRecords.rating); // red squiggly  
      line under rating  
  
// hovering over rating shows:  
Property 'rating' is private and only accessible within clas  
s 'Album'.
```

However, this doesn't actually prevent it from being accessed at runtime; `private` is just a compile-time annotation. You could suppress the error using an `@ts-ignore` and still access the property:

```
// @ts-ignore
console.log(loopFindingJazzRecords.rating); // Output: 0
```

Using the `@ts-ignore` directive tells TypeScript to ignore any errors that might happen on the following line. In this case, even though the `rating` property is private and should be accessible only within an instance of the `Album` class, the `console.log()` call will run as expected. You'll learn more about this and other directives in [Chapter 11](#).

To really make a property private, you can use the `#` prefix:

```
class Album {
  #rating = 0;
}
```

The `#` syntax behaves the same as `private`, but it's a newer feature that's part of the ECMAScript standard. This means it can be used in JavaScript as well as TypeScript.

Attempting to access a `#`-prefixed property from outside the class will result in a syntax error:

```
console.log(loopFindingJazzRecords.#rating); // red squiggly
line under #rating
// hovering over #rating shows:
Property '#rating' is not accessible outside class 'Album' b
ecause it has a private identifier.
```

Attempting to cheat by accessing it with a dynamic string will return `undefined`:

```
console.log(loopFindingJazzRecords["#rating"]); // red squig
gly line under loopFindingJazzRecords["#rating"]
// hovering over loopFindingJazzRecords["#rating"] shows:
```

Element implicitly has an 'any' type because expression of type '"#rating"' can't be used to index type 'Album'.
Property '#rating' does not exist on type 'Album'.

Unlike what you saw when using the `private` keyword, using the `#` syntax won't allow for the `console.log()` call to complete.

Class Methods

Along with properties, classes can also contain methods. These functions help express the behaviors of a class and can be used to interact with both public and private properties.

Let's add a `printAlbumInfo` method to the `Album` class that will log the album's title, artist, and release year.

There are a couple of techniques for adding methods to a class. The first is to follow the same pattern as the constructor and directly add the method to the class body:

```
// In the Album class
printAlbumInfo() {
  console.log(`${this.title} by ${this.artist}, released in
    ${this.releaseYear}.`);
}
```

Another option is to use an arrow function to define the method:

```
// In the Album class
printAlbumInfo = () => {
  console.log(
    `${this.title} by ${this.artist}, released in ${this.releaseYear}.`,
  );
};
```

Once the `printAlbumInfo` method has been added, you can call it to log the album's information:

```
loopFindingJazzRecords.printAlbumInfo();
```

```
// Output: Loop Finding Jazz Records by Jan Jelinek, released in 2001.
```

No matter which version of method definition you use, the `printAlbumInfo()` call will work as expected. However, there are differences in behavior between the two in the way that `this` is handled at runtime. To illustrate, here is an example class that includes a regular method and an arrow function method:

```
class MyClass {
  location = "Class";

  arrow = () => {
    console.log("arrow", this);
  };

  method() {
    console.log("method", this);
  }
}

const myObj = {
  location: "Object",
  arrow: new MyClass().arrow,
  method: new MyClass().method,
};

myObj.arrow(); // {location: 'Class'}
myObj.method(); // {location: 'Object'}
```

In the arrow method, `this` is bound to the instance of the class where it was defined. In the method method, `this` is bound to the object where it was called.

While this difference is in runtime JavaScript behavior, it is worth being aware of when working with classes. It's hard to make a blanket recommendation here, since there are other differences between the two that

are beyond the scope of the book to explore. But in general, using an arrow function when defining methods will help avoid issues with `this`.

Class Inheritance

Similar to how you can extend types and interfaces, you can also extend classes. This allows you to create a hierarchy of classes that can inherit properties and methods from one another, making your code more organized and reusable.

For this example, you'll go back to your basic `Album` class that will act as your base class:

```
class Album {
  title: string;
  artist: string;
  releaseYear: number;

  constructor(opts: {title: string; artist: string; releaseYear: number}) {
    this.title = opts.title;
    this.artist = opts.artist;
    this.releaseYear = opts.releaseYear;
  }

  displayInfo() {
    console.log(
      `${this.title} by ${this.artist}, released in ${this.releaseYear}.`,
    );
  }
}
```

The goal is to create a `SpecialEditionAlbum` class that extends the `Album` class and adds a `bonusTracks` property.

Extending a Class

The first step is to use the `extends` keyword to create the `SpecialEditionAlbum` class:

```
class SpecialEditionAlbum extends Album {}
```

Once the `extends` keyword is added, any new properties or methods added to the `SpecialEditionAlbum` class will be in addition to what it inherits from the `Album` class. For example, you can add a `bonusTracks` property to the `SpecialEditionAlbum` class:

```
class SpecialEditionAlbum extends Album {  
  bonusTracks: string[];  
}
```

Next, you need to add a constructor that includes all the properties from the `Album` class as well as the `bonusTracks` property. There are a couple of important things to note about the constructor when extending a class.

First, the arguments to the constructor should match the shape used in the parent class. In this case, that's an `opts` object with the properties of the `Album` class along with the new `bonusTracks` property.

Second, you need to include a call to `super()`. This is a special method that calls the constructor of the parent class and sets up the properties it defines. This is crucial to ensure that the base properties are initialized properly. You'll pass in `opts` to the `super()` method and then set the `bonusTracks` property:

```
class SpecialEditionAlbum extends Album {  
  bonusTracks: string[];  
  
  constructor(opts: {  
    title: string;  
    artist: string;  
    releaseYear: number;  
    bonusTracks: string[];  
  }) {  
    super(opts);  
    this.bonusTracks = opts.bonusTracks;  
  }  
}
```

Now that you have the `SpecialEditionAlbum` class set up, you can create a new instance similarly to how you would with the `Album` class:

```
const plasticOnoBandSpecialEdition = new SpecialEditionAlbum({
  title: "Plastic Ono Band",
  artist: "John Lennon",
  releaseYear: 2000,
  bonusTracks: ["Power to the People", "Do the Oz"],
});
```

This pattern can be used to add more methods, properties, and behavior to the `SpecialEditionAlbum` class, while still maintaining the properties and methods of the `Album` class.

protected Properties

In addition to `public` and `private`, there's a third visibility modifier called `protected`. This is similar to `private`, but it allows the property to be accessed from within classes that extend the class.

For example, if you wanted to make the `title` property of the `Album` class `protected`, you could do so like this:

```
class Album {
  protected title = ""; // . . .
}
```

Now the `title` property can be accessed from within the `SpecialEditionAlbum` class, not from outside the class:

```
class Album {
  protected title = "Brown Sugar";
  doesTitleHaveASpace = () => {
    return this.title.includes(" "); // Permitted inside the
  }
}
```

```
const album = Album();  
album.title; // Errors outside the class
```

Safe Overrides with override

You can run into trouble when extending classes if you try to override a method in a subclass. Let's say your Album class implements a `displayInfo` method:

```
class Album {  
  // . . .  
  displayInfo() {  
    console.log(  
      `${this.title} by ${this.artist}, released in ${this.releaseYear}.`,  
    );  
  }  
}
```

And your `SpecialEditionAlbum` class also implements a `displayInfo` method:

```
class SpecialEditionAlbum extends Album {  
  // . . .  
  displayInfo() {  
    console.log(  
      `${this.title} by ${this.artist}, released in ${this.releaseYear}.`,  
    );  
    console.log(`Bonus tracks: ${this.bonusTracks.join(",  
    ")} `);  
  }  
}
```

This overrides the `displayInfo` method from the Album class, adding an extra log for the bonus tracks.

But what happens if you change the `displayInfo` method in `Album` to `displayAlbumInfo`? `SpecialEditionAlbum` won't automatically get updated, and its override will no longer work.

To prevent this, you can use the `override` keyword in the subclass to indicate that you're intentionally overriding a method from the parent class:

```
class SpecialEditionAlbum extends Album {  
    // . . .  
    override displayInfo() {  
        console.log(  
            `${this.title} by ${this.artist}, released in ${this.releaseYear}.`,  
        );  
        console.log(`Bonus tracks: ${this.bonusTracks.join(",  
    ")}`);  
    }  
}
```

Now, if the `displayInfo` method in the `Album` class is changed, TypeScript will give an error in the `SpecialEditionAlbum` class, letting you know that the method is no longer being overridden.

You can also enforce this by setting `noImplicitOverride` to `true` in your `tsconfig.json` file. This will force you to always specify `override` when you're overriding a method:

```
{  
    "compilerOptions": {  
        "noImplicitOverride": true  
    }  
}
```

The implements Keyword

There are some situations where you want to enforce that a class adheres to a specific structure. To do that, you can use the `implements` keyword.

The `SpecialEditionAlbum` class you created in the previous example adds a `bonusTracks` property to the `Album` class, but there is no `trackList`

property for the regular Album class.

Let's create an interface to enforce that any class that implements it must have a trackList property.

You'll call the interface IAlbum and include properties for the title, artist, releaseYear, and trackList properties:

```
interface IAlbum {  
  title: string;  
  artist: string;  
  releaseYear: number;  
  trackList: string[];  
}
```

Note that the I prefix is used to indicate an interface, while a T indicates a type. It isn't required to use these prefixes, but it's a common convention called *Hungarian notation* that makes it clearer what the interface will be used for when reading the code. We don't recommend doing this for all your interfaces and types—only when they conflict with a class of the same name.

With the interface created, you can use the implements keyword to associate it with the Album class:

```
class Album implements IAlbum {  
  // red squiggly line under Album  
  title: string;  
  artist: string;  
  releaseYear: number;  
  
  constructor(opts: {title: string; artist: string; releaseYear: number}) {  
    this.title = opts.title;  
    this.artist = opts.artist;  
    this.releaseYear = opts.releaseYear;  
  }  
}
```

// hovering over Album shows:
Class 'Album' incorrectly implements interface 'IAlbum'.

Property 'trackList' is missing in type 'Album' but required in type 'IAlbum'.

Because the `trackList` property is missing from the `Album` class, TypeScript now gives you an error. To fix it, the `trackList` property needs to be added to the `Album` class. Once the property is added, you could update the interface or set up getters and setters accordingly:

```
class Album implements IAlbum {
  title: string;
  artist: string;
  releaseYear: number;
  trackList: string[];

  constructor(opts: {
    title: string;
    artist: string;
    releaseYear: number;
    trackList: string[];
  }) {
    this.title = opts.title;
    this.artist = opts.artist;
    this.releaseYear = opts.releaseYear;
    this.trackList = opts.trackList;
  }

  // . . .
}
```

This lets you define a contract for the `Album` class that enforces the structure of the class and helps catch errors early.

Abstract Classes

Another pattern you can use for defining base classes is the `abstract` keyword. Abstract classes blur the line between types and runtime. You can declare an abstract class like this:

```
abstract class AlbumBase {}
```

You can then define methods and behavior on it, like you can on a regular class:

```
abstract class AlbumBase {
  title: string;
  artist: string;
  releaseYear: number;
  trackList: string[] = [];

  constructor(opts: {title: string; artist: string; releaseYear: number}) {
    this.title = opts.title;
    this.artist = opts.artist;
    this.releaseYear = opts.releaseYear;
  }

  addTrack(track: string) {
    this.trackList.push(track);
  }
}
```

In the previous code, the abstract AlbumBase class is declared with an addTrack method.

But if you try to create an instance of the AlbumBase class in the same way you did before, TypeScript will give you an error:

```
const albumBase = new AlbumBase({
  title: "Unknown Album",
  artist: "Unknown Artist",
  releaseYear: 0,
}); // red squiggly line under AlbumBase

// hovering over AlbumBase shows:
Cannot create an instance of an abstract class.
```

Abstract classes don't allow you to directly create instances of them. Instead, you'd need to create a class that extends the AlbumBase class:

```
class Album extends AlbumBase {  
    // Any extra functionality you want  
}  
  
const album = new Album({  
    title: "Unknown Album",  
    artist: "Unknown Artist",  
    releaseYear: 0,  
});
```

In this code, the Album class extends the AlbumBase class and can include additional functionality. You'll notice that this idea is similar to implementing interfaces, except that abstract classes can also include implementation details. This means you can blur the line a little between types and runtime. You can define a type contract for a class but make it more reusable.

Abstract Methods

On your abstract class, you can use the abstract keyword before a method to indicate that it must be implemented by any class that extends the abstract class:

```
abstract class AlbumBase {  
    // . . .Other properties and methods  
  
    abstract addReview(author: string, review: string): void;  
}
```

Now any class that extends AlbumBase must implement the addReview method:

```
class Album extends AlbumBase {  
    // . . .Other properties and methods  
  
    addReview(author: string, review: string) {  
        // . . .Implementation  
    }
```

```
}  
}
```

This gives you another tool for expressing the structure of your classes and ensuring that they adhere to a specific contract.

Exercise 8-1: Creating a Class

Here is a class called `CanvasNode` that currently functions identically to an empty object:

```
class CanvasNode {}
```

In a test case, the class is instantiated by calling `new CanvasNode()`.

However, there are some errors since the test expects it to house two properties, specifically `x` and `y`, each with a default value of `0`:

```
it("Should store some basic properties", () => {  
  const canvasNode = new CanvasNode();  
  
  expect(canvasNode.x).toEqual(0); // red squiggly line unde  
r x  
  expect(canvasNode.y).toEqual(0); // red squiggly line unde  
r y  
  
  // @ts-expect-error Property is readonly  
  canvasNode.x = 10;  
  
  // @ts-expect-error Property is readonly  
  canvasNode.y = 20;  
});  
// hovering over x or y shows:  
Property 'x' (or 'y') does not exist on type 'CanvasNode'.
```

As shown with the `@ts-expect-error` directives, you also expect these properties to be `readonly`.

Your challenge is to implement the `CanvasNode` class to satisfy these requirements. For extra practice, solve the challenge with and without the

use of a constructor.

See <https://totalts.link/essentials-8-1/>.

Solution

Here's an example of a CanvasNode class with a constructor that meets the requirements:

```
class CanvasNode {
  readonly x: number;
  readonly y: number;

  constructor() {
    this.x = 0;
    this.y = 0;
  }
}
```

Without a constructor, the CanvasNode class can be implemented by assigning the properties directly:

```
class CanvasNode {
  readonly x = 0;
  readonly y = 0;
}
```

Exercise 8-2: Implementing Class Methods

In this exercise, you've simplified your CanvasNode class so that it no longer has read-only properties:

```
class CanvasNode {
  x = 0;
  y = 0;
}
```

There is a test case for being able to move the CanvasNode object to a new location:

```
it("Should be able to move to a new location", () => {
  const canvasNode = new CanvasNode();

  expect(canvasNode.x).toEqual(0);
  expect(canvasNode.y).toEqual(0);

  canvasNode.move(10, 20); // red squiggly line under move

  expect(canvasNode.x).toEqual(10);
  expect(canvasNode.y).toEqual(20);
});

// hovering over move shows:
Property 'move' does not exist on type 'CanvasNode'.
```

Currently, there is an error under the move method call because the CanvasNode class does not have a move method.

Your task is to add a move method to the CanvasNode class that will update the x and y properties to the new location.

See <https://totalts.link/essentials-8-2/>.

Solution

The move method can be implemented either as a regular method or as an arrow function.

Here's the regular method:

```
class CanvasNode {
  x = 0;
  y = 0;

  move(x: number, y: number) {
    this.x = x;
    this.y = y;
  }
}
```

And here's the arrow function:

```
class CanvasNode {
  x = 0;
  y = 0;

  move = (x: number, y: number) => {
    this.x = x;
    this.y = y;
  };
};
```

Exercise 8-3: Implementing a Getter

Let's continue working with the CanvasNode class, which now has a constructor that accepts an optional argument, renamed position. This position is an object that replaces the individual x and y you had before:

```
class CanvasNode {
  x: number;
  y: number;

  constructor(position?: {x: number; y: number}) {
    this.x = position?.x ?? 0;
    this.y = position?.y ?? 0;
  }

  move(x: number, y: number) {
    this.x = x;
    this.y = y;
  }
}
```

In these test cases, there are errors accessing the position property since it is not currently a property of the CanvasNode class:

```
it("Should be able to move", () => {
  const canvasNode = new CanvasNode();

  expect(canvasNode.position).toEqual({x: 0, y: 0}); // red
  squiggly line under position
});
```

```
canvasNode.move(10, 20);

expect(canvasNode.position).toEqual({x: 10, y: 20}); // red squiggly line under position
});

it("Should be able to receive an initial position", () => {
  const canvasNode = new CanvasNode({
    x: 10,
    y: 20,
  });

  expect(canvasNode.position).toEqual({x: 10, y: 20}); // red squiggly line under position
});
```

Your task is to update the CanvasNode class to include a position getter that will allow the test cases to pass. Hint: Use the get keyword to specify that a method is a getter.

See <https://totalts.link/essentials-8-3/>.

Solution

Here's how the CanvasNode class can be updated to include a getter for the position property:

```
class CanvasNode {
  x: number;
  y: number;

  constructor(position?: {x: number; y: number}) {
    this.x = position?.x ?? 0;
    this.y = position?.y ?? 0;
  }

  move(x: number, y: number) {
    this.x = x;
    this.y = y;
  }
}
```

```
    get position() {  
        return {x: this.x, y: this.y};  
    }  
}
```

With the getter in place, the test cases will pass.

Remember, when using a getter, you can access the property as if it were a regular property on the class instance:

```
const canvasNode = new CanvasNode();  
console.log(canvasNode.position.x); // 0  
console.log(canvasNode.position.y); // 0
```

Exercise 8-4: Implementing a Setter

The CanvasNode class has been updated so that x and y are now private properties:

```
class CanvasNode {  
    #x: number;  
    #y: number;  
  
    constructor(position?: {x: number; y: number}) {  
        this.#x = position?.x ?? 0;  
        this.#y = position?.y ?? 0;  
    }  
  
    // Your `position` getter method here  
  
    // Move method as before  
}
```

The # in front of the x and y properties means they are private and can't be modified directly outside the class.

Your task is to write a setter for the position property that will allow the test case to pass. Hint: Use the set keyword in your solution.

See <https://totalts.link/essentials-8-4/>.

Solution

Here's how a position setter can be added to the CanvasNode class:

```
class CanvasNode {  
    // In the CanvasNode class  
    set position(pos) {  
        this.x = pos.x;  
        this.y = pos.y;  
    }  
}
```

Note that you don't have to add a type to the pos parameter since TypeScript is smart enough to infer it based on the getter's return type.

Exercise 8-5: Extending a Class

Here you have a more complex version of the CanvasNode class.

In addition to the x and y properties, the class now has a viewMode property that is typed as ViewMode and can be set to hidden, visible, or selected:

```
type ViewMode = "hidden" | "visible" | "selected";  
  
class CanvasNode {  
    x = 0;  
    y = 0;  
    viewMode: ViewMode = "visible";  
  
    constructor(options?: {x: number; y: number; viewMode?: Vi  
ewMode}) {  
        this.x = options?.x ?? 0;  
        this.y = options?.y ?? 0;  
        this.viewMode = options?.viewMode ?? "visible";  
    }  
  
    /* getter, setter, and move methods as before */
```

Imagine if your application had a Shape class that needed only the x and y properties and the ability to move around. It wouldn't need the viewMode property or the logic related to it.

Your task is to refactor the CanvasNode class to split the x and y properties into a separate class called Shape. Then, the CanvasNode class should extend the Shape class, adding the viewMode property and the logic related to it. If you like, you can use an abstract class to define Shape.

See <https://totalts.link/essentials-8-5/>.

Solution

The new Shape class would look similar to the original CanvasNode class:

```
class Shape {
  #x: number;
  #y: number;

  constructor(options?: {x: number; y: number}) {
    this.#x = options?.x ?? 0;
    this.#y = options?.y ?? 0;
  }

  // Position getter and setter methods

  move(x: number, y: number) {
    this.#x = x;
    this.#y = y;
  }
}
```

The CanvasNode class would then extend the Shape class and add the viewMode property. The constructor would also be updated to accept the viewMode and call super() to pass the x and y properties to the Shape class:

```
class CanvasNode extends Shape {
  #viewMode: ViewMode;

  constructor(options?: {x: number; y: number; viewMode?: ViewMode}) {
```

```
    super(options);  
    this.#viewMode = options?.viewMode ?? "visible";  
  }  
}
```

Summary

This chapter introduced classes as a fundamental TypeScript feature for encapsulating data and behavior. Classes use the `class` keyword with PascalCase naming and require property initialization through constructors or default values.

Properties can be marked as `readonly` or optional with `?`, or they can have visibility controlled through `public`, `private`, and `protected` keywords. The `#` syntax provides true private properties that can't be accessed at runtime.

Methods define class behavior and can use regular functions or arrow functions. Arrow functions maintain this binding to the class instance, preventing common runtime issues.

Class inheritance uses `extends` to create hierarchies where child classes inherit from parents. Child constructors must call `super()` to initialize properly. The `override` keyword helps catch method signature changes.

The `implements` keyword enforces interface contracts, while abstract classes provide shared implementation without allowing direct instantiation. Getters and setters using `get` and `set` keywords provide controlled access to private properties.

Classes can be used as types for variables and function parameters, representing instances rather than the class itself. This makes them powerful tools for creating reusable, type-safe object-oriented code.

9

TYPESCRIPT-ONLY FEATURES



Based on what we’ve told you so far, you might be thinking of TypeScript as just “JavaScript with types.” JavaScript handles the runtime code, and TypeScript describes it with types.

But TypeScript actually has a few runtime features that don’t exist in JavaScript. These features are compiled into JavaScript, but they are not part of the JavaScript language itself.

In this chapter, you’ll learn about several of these TypeScript-only features, including parameter properties, enums, and namespaces. Along the way, we’ll discuss benefits and trade-offs, as well as when you might want to stick with JavaScript.

Class Parameter Properties

One TypeScript feature that doesn’t exist in JavaScript is class parameter properties. These allow you to declare and initialize class members directly from the constructor parameters.

Consider this `Rating` class:

```
class Rating {  
  constructor(public value: number, private max: number) {}  
}
```

Note that the constructor includes `public` before the `value` parameter and `private` before the `max` parameter. In JavaScript, this compiles to code that assigns the parameters to properties on the class:

```
class Rating {
  constructor(value, max) {
    this.value = value;
    this.max = max;
  }
}
```

Compared to handling the assignment manually, this saves a lot of code and keeps the class definition concise.

But unlike other TypeScript features, the generated JavaScript is not a direct representation of the TypeScript code. This can make it difficult to understand what's happening if you're not familiar with the feature.

Enums

You can use the `enum` keyword to define a set of named constants. These can be used as types or values.

Enums were added in the first version of TypeScript, but they haven't yet been added to JavaScript. This means it's a TypeScript-only runtime feature. And, as you'll see, it comes with some quirky behavior.

A good use case for enums is when there are a limited set of related values that aren't expected to change.

Numeric Enums

Numeric enums group together a set of related members and automatically assign them numeric values starting from 0. For example, consider this `AlbumStatus` enum:

```
enum AlbumStatus {
  NewRelease,
  OnSale,
  StaffPick,
}
```

In this case, `AlbumStatus.NewRelease` would be 0, `AlbumStatus.OnSale` would be 1, and so on.

To use the `AlbumStatus` as a type, you could use its name:

```
function logStatus(genre: AlbumStatus) {  
    console.log(genre); // 0  
}
```

Now `logStatus` can receive values only from the `AlbumStatus` enum object:

```
logStatus(AlbumStatus.NewRelease);
```

Numeric Enums with Explicit Values

You can also assign specific values to each member of the enum. For example, if you wanted to assign the value 1 to `NewRelease`, 2 to `OnSale`, and 3 to `StaffPick`, you could do so like this:

```
enum AlbumStatus {  
    NewRelease = 1,  
    OnSale = 2,  
    StaffPick = 3,  
}
```

Now `AlbumStatus.NewRelease` would be 1, `AlbumStatus.OnSale` would be 2, and so on.

Auto-Incrementing Numeric Enums

If you choose to assign only *some* numeric values to the enum, TypeScript will automatically increment the rest of the values from the last assigned value. For example, if you assign a value only to `NewRelease`, then `OnSale` and `StaffPick` will be 2 and 3, respectively:

```
enum AlbumStatus {  
    NewRelease = 1,  
    OnSale,  
}
```

```
    StaffPick,  
}
```

String Enums

String enums allow you to assign string values to each member of the enum. For example:

```
enum AlbumStatus {  
    NewRelease = "NEW_RELEASE",  
    OnSale = "ON_SALE",  
    StaffPick = "STAFF_PICK",  
}
```

The same `logStatus` function from the previous section would now log the string value instead of the number:

```
function logStatus(genre: AlbumStatus) {  
    console.log(genre); // "NEW_RELEASE"  
}  
  
logStatus(AlbumStatus.NewRelease);
```

Enums Are Strange

There is no equivalent syntax in JavaScript to the `enum` keyword. This means enums need to be *transpiled* into JavaScript code to work at runtime. The way this transpilation works can feel slightly unexpected.

How Numeric Enums Transpile

For example, the enum `AlbumStatus`

```
enum AlbumStatus {  
    NewRelease,  
    OnSale,  
    StaffPick,  
}
```

would be transpiled into the following JavaScript:

```
var AlbumStatus;  
(function (AlbumStatus) {  
    AlbumStatus[(AlbumStatus["NewRelease"] = 0)] = "NewRelease";  
    AlbumStatus[(AlbumStatus["OnSale"] = 1)] = "OnSale";  
    AlbumStatus[(AlbumStatus["StaffPick"] = 2)] = "StaffPick";  
})(AlbumStatus || (AlbumStatus = {}));
```

This rather opaque piece of JavaScript does several things in one go. It creates an object with properties for each enum value, and it also creates a reverse mapping of the values to the keys.

The result would then be similar to the following:

```
var AlbumStatus = {  
    0: "NewRelease",  
    1: "OnSale",  
    2: "StaffPick",  
    NewRelease: 0,  
    OnSale: 1,  
    StaffPick: 2,  
};
```

This reverse mapping means there are more keys available on an enum than you might expect. So, performing an `Object.keys` call on an enum will return both the keys and the values:

```
console.log(Object.keys(AlbumStatus)); // ["0", "1", "2", "NewRelease", "OnSale", "StaffPick"]
```

This can be a real gotcha if you're not expecting it.

How String Enums Transpile

String enums don't have the same behavior as numeric enums. When you specify string values, the transpiled JavaScript is much simpler. This is the TypeScript:

```
enum AlbumStatus {  
    NewRelease = "NEW_RELEASE",  
    OnSale = "ON_SALE",  
    StaffPick = "STAFF_PICK",  
}
```

This is the transpiled JavaScript:

```
var AlbumStatus;  
(function (AlbumStatus) {  
    AlbumStatus["NewRelease"] = "NEW_RELEASE";  
    AlbumStatus["OnSale"] = "ON_SALE";  
    AlbumStatus["StaffPick"] = "STAFF_PICK";  
})(AlbumStatus || (AlbumStatus = {}));
```

Now there is no reverse mapping, and the object contains only the enum values. An `Object.keys` call will return only the keys, as you might expect:

```
console.log(Object.keys(AlbumStatus)); // ["NewRelease", "On  
Sale", "StaffPick"]
```

This difference between numeric and string enums feels inconsistent, and it can be a source of confusion.

Numeric and String Enums Don't Behave the Same

Another odd feature of enums is that string enums and numeric enums behave differently when used as types.

Let's redefine your `logStatus` function with a numeric enum:

```
enum AlbumStatus {  
    NewRelease = 0,  
    OnSale = 1,  
    StaffPick = 2,  
}  
  
function logStatus(genre: AlbumStatus) {
```

```
    console.log(genre);  
}
```

Now you can call `logStatus` with a member of the enum:

```
logStatus(AlbumStatus.NewRelease);
```

But you can also call it with a plain number:

```
logStatus(0);
```

If you call it with a number that isn't a member of the enum, TypeScript will report an error:

```
logStatus(3); // Argument of type '3' is not assignable to p  
arameter of type 'AlbumStatus'.
```

This is different from string enums, which only allow the enum members to be used as types:

```
enum AlbumStatus {  
    NewRelease = "NEW_RELEASE",  
    OnSale = "ON_SALE",  
    StaffPick = "STAFF_PICK",  
}  
  
function logStatus(genre: AlbumStatus) {  
    console.log(genre);  
}  
  
logStatus(AlbumStatus.NewRelease);  
logStatus("NEW_RELEASE"); // Argument of type '"NEW_RELEAS  
E"' is not  
assignable to parameter of type 'AlbumStatus'.
```

The way string enums behave feels more natural: It matches how enums work in other languages like C# and Java.

But the fact that they're not consistent with numeric enums can be a source of confusion. In fact, string enums are unusual in TypeScript because they're compared *nominally*. All other types in TypeScript are compared *structurally*, meaning that two types are considered the same if they have the same structure. But string enums are compared based on their name (nominally), not their structure.

This means that two string enums with the same members are considered different types if they have different names:

```
enum AlbumStatus2 {  
    NewRelease = "NEW_RELEASE",  
    OnSale = "ON_SALE",  
    StaffPick = "STAFF_PICK",  
}  
  
logStatus(AlbumStatus2.NewRelease); // Argument of type 'AlbumStatus2' is not  
assignable to parameter of type 'AlbumStatus'.
```

For those of us used to structural typing, this can be a bit of a surprise. But to developers used to enums in other languages, string enums will feel the most natural.

const Enums

A const enum is declared similarly to the other enums but with the `const` keyword first:

```
const enum AlbumStatus {  
    NewRelease = "NEW_RELEASE",  
    OnSale = "ON_SALE",  
    StaffPick = "STAFF_PICK",  
}
```

You can use const enums to declare numeric or string enums, since they have the same behavior as regular enums.

The major difference is that const enums disappear when the TypeScript is transpiled to JavaScript. Instead of creating an object with the

enum's values, the transpiled JavaScript will use the enum's values directly.

For instance, if an array is created that accesses the enum's values, the transpiled JavaScript will end up with those values:

```
let albumStatuses = [  
  AlbumStatus.NewRelease,  
  AlbumStatus.OnSale,  
  AlbumStatus.StaffPick,  
];  
  
// The above transpiles to:  
let albumStatuses = ["NEW_RELEASE", "ON_SALE", "STAFF_PICK"];
```

const enums do have some limitations, especially when declared in declaration files (which we'll cover later). The TypeScript team actually recommends avoiding const enums in the library code because they can behave unpredictably for consumers of the library.

Should You Use Enums?

Enums are a useful feature, but they have some quirks that can make them difficult to work with.

There are some alternatives to enums that you might want to consider, such as plain union types. But our preferred alternative uses some syntax we haven't covered yet.

We'll discuss whether you should use enums in general in "Using as const for JavaScript-Style Enums" on [page 203](#).

Namespaces

Namespaces were an early feature of TypeScript that tried to solve a big problem in JavaScript at the time: the lack of a module system. They were introduced before ES6 modules were standardized, and they were TypeScript's attempt to organize the code.

Namespaces let you create mini-modules where you can export functions and types. This allows you to use names that wouldn't conflict with other things declared in the global scope.

Consider a scenario where you are building a TypeScript application to manage a music collection. There could be functions to add an album, calculate sales, and generate reports. Using namespaces, you can group these functions logically:

```
namespace RecordStoreUtils {
  export namespace Album {
    export interface Album {
      title: string;
      artist: string;
      year: number;
    }
  }

  export function addAlbum(title: string, artist: string, year: number) {
    // Implementation to add an album to the collection
  }

  export namespace Sales {
    export function recordSale(
      albumTitle: string,
      quantity: number,
      price: number,
    ) {
      // Implementation to record an album sale
    }

    export function calculateTotalSales(albumTitle: string):
    number {
      // Implementation to calculate total sales for an album
      return 0; // Placeholder return
    }
  }
}
```

In this example, AlbumCollection is the main namespace, with Sales as a nested namespace. This structure helps organize the code by

functionality and makes it clear which part of the application each function pertains to.

The contents of the AlbumCollection can be used as values or types:

```
const odelay: AlbumCollection.Album.Album = {
  title: "Odelay!",
  artist: "Beck",
  year: 1996,
};

AlbumCollection.Sales.recordSale("Odelay!", 1, 10.99);
```

How Namespaces Compile

Namespaces compile into relatively simple JavaScript. For instance, a simpler version of the RecordStoreUtils namespace, like

```
namespace RecordStoreUtils {
  export function addAlbum(title: string, artist: string, year: number) {
    // Implementation to add an album to the collection
  }
}
```

would be transpiled into the following JavaScript:

```
var RecordStoreUtils;
(function (RecordStoreUtils) {
  function addAlbum(title, artist, year) {
    // Implementation to add an album to the collection
  }
  RecordStoreUtils.addAlbum = addAlbum;
})(RecordStoreUtils || (RecordStoreUtils = {}));
```

Similarly to an enum, this code creates an object with properties for each function and type in the namespace. This means the namespace can be accessed as an object, and its properties can be accessed as methods or properties.

Merging Namespaces

Just like interfaces, namespaces can be merged through declaration merging. This allows you to combine two or more separate declarations into a single definition.

Here you have two declarations of `RecordStoreUtils`—one with an `Album` namespace and another with a `Sales` namespace:

```
namespace RecordStoreUtils {
  export namespace Album {
    export interface Album {
      title: string;
      artist: string;
      year: number;
    }
  }
}

namespace RecordStoreUtils {
  export namespace Sales {
    export function recordSale(
      albumTitle: string,
      quantity: number,
      price: number,
    ) {
      // Implementation to record an album sale
    }

    export function calculateTotalSales(albumTitle: string):
number {
      // Implementation to calculate total sales for an album
      return 0; // Placeholder return
    }
  }
}
```

Because namespaces support declaration merging, the two declarations are automatically combined into a single `RecordStoreUtils` namespace.

Both the Album and Sales namespaces can be accessed as before:

```
const loaded: RecordStoreUtils.Album.Album = {
  title: "Loaded",
  artist: "The Velvet Underground",
  year: 1970,
};

RecordStoreUtils.Sales.calculateTotalSales("Loaded");
```

Merging Interfaces Within Namespaces

It's also possible for interfaces within namespaces to be merged. If you had two different RecordStoreUtils namespaces, each with their own Album interface, TypeScript would automatically merge them into a single Album interface that includes all the properties:

```
namespace RecordStoreUtils {
  export interface Album {
    title: string;
    artist: string;
    year: number;
  }
}

namespace RecordStoreUtils {
  export interface Album {
    genre: string[];
    recordLabel: string;
  }
}

const madvillainy: RecordStoreUtils.Album = {
  title: "Madvillainy",
  artist: "Madvillain",
  year: 2004,
  genre: ["Hip Hop", "Experimental"],
  recordLabel: "Stones Throw",
};
```

This information will become crucial later when you look at the namespace's key use case: globally scoped types.

Should You Use Namespaces?

Imagine ECMAScript modules, with `import` and `export`, never existed. In this world, everything you declare is in the global scope. You'd have to be careful about naming things, and you'd have to come up with a way to organize your code.

This is the world that TypeScript was born into. Module systems like CommonJS (`require`) and ES Modules (`import`, `export`) weren't popular yet. So, namespaces were a crucial way to avoid naming conflicts and organize your code.

But now that ECMAScript modules are widely supported, you should use them over namespaces. Runtime namespaces are a thing of the past.

Type-Only Namespaces

However, it's important to distinguish between type-only namespaces and runtime namespaces:

```
// Runtime namespace
namespace Utility {
  export function log(message: string) {
    console.log(message);
  }
}

// Type-only namespace
namespace DB {
  export type User = {
    id: number;
    name: string;
  };
}
```

When to Use ECMAScript vs. TypeScript

In this chapter, you’ve seen several TypeScript-only features. These features have two things in common. First, they don’t exist in JavaScript. Second, they are *old*.

In 2010, when TypeScript was being built, JavaScript was seen as a problematic language that needed fixing. Enums, namespaces, and class parameter properties were added in an atmosphere where new runtime additions to JavaScript were seen as a good thing.

But now, JavaScript itself is in a much healthier place. The TC39 committee, the body that decides what features get added to JavaScript, is more active and efficient. New features are being added to the language every year, and the language is evolving rapidly.

The TypeScript team now sees its role very differently. Instead of adding new features to TypeScript, they cleave to JavaScript as closely as possible. Daniel Rosenwasser, the program manager for TypeScript, is co-chair of the TC39 committee.

The right way to think about TypeScript today is as “JavaScript with types.”

Given this attitude, it’s clear how you should treat these TypeScript-only features: as relics of the past. If enums, namespaces, and class parameter properties were proposed today, they would not even be considered.

But the question remains: Should you use them? TypeScript will likely never stop supporting these features. Doing so would break too much existing code. So, they’re safe to continue using.

We prefer writing code in the spirit of the language we’re using. Writing “JavaScript with types” keeps the relationship between TypeScript and JavaScript crystal-clear.

However, this is our personal preference. If you’re working on a large codebase that already uses these features, it is *not* worth the effort to remove them. Reaching a decision as a team, and staying consistent, is the key.

The Future of TypeScript: Erasable Syntax Only

The TypeScript team’s evolving philosophy is perhaps best exemplified by the introduction of the `--erasableSyntaxOnly` flag in TypeScript 5.8. This

flag explicitly disallows enums, namespaces, and class parameter properties, marking them as errors rather than valid TypeScript code. The term *erasable* refers to syntax that can be completely removed without affecting runtime behavior, like type annotations that simply disappear during compilation. In contrast, the TypeScript-only features we've discussed require complex transformations and generate actual runtime JavaScript code.

By encouraging developers to avoid these features through `--erasableSyntaxOnly`, TypeScript is preparing for a world where the line between TypeScript and JavaScript becomes even thinner. In that world, the language would serve purely as a compile-time type checker rather than a runtime code generator.

Summary

This chapter covered TypeScript-only runtime features that compile to JavaScript but don't exist in JavaScript. Class parameter properties use public and private modifiers in constructors to declare and initialize members concisely, though the compiled output differs significantly from the source.

Enums define named constants with quirky behavior. Numeric enums create reverse mappings and allow plain numbers as values, while string enums are simpler but nominally typed. `const` enums disappear at runtime with values inlined.

Namespaces were TypeScript's early module system before ECMAScript modules existed. They create exportable functions and types and support declaration merging, but they are now largely obsolete.

The TypeScript team's modern philosophy is "JavaScript with types," treating these features as relics from when JavaScript needed fixing. The new `--erasableSyntaxOnly` flag can disable them entirely.

While still supported for compatibility, modern TypeScript should prefer JavaScript-native approaches over legacy features.

PART IV

WORKING WITH THE COMPILER

10

DERIVING TYPES



One of the most common pieces of advice for writing maintainable code is to “Keep code DRY,” or more explicitly, “Don’t Repeat Yourself.” One way to do this in JavaScript is to take repeating code and capture it in functions or variables. These variables and functions can be reused, composed, and combined in different ways to create new functionality. In TypeScript, you can apply this same principle to types.

In this chapter, you’ll learn about deriving types from other types, which lets you reduce repetition in your code and create a single source of truth for your types. This allows you to make changes in one type and have those changes propagate throughout your application without needing to manually update every instance.

We’ll even cover how you can derive types from *values* so that your types always represent the runtime behavior of your application.

Deriving Types from Other Types

You can create derived types using some of the tools you’ve used so far.

You can use `interface extends` to make one interface inherit from another:

```
interface Album {
  title: string;
  artist: string;
  releaseYear: number;
}

interface AlbumDetails extends Album {
  genre: string;
}
```

`AlbumDetails` inherits all the properties of `Album`. This means that any changes to `Album` will trickle down to `AlbumDetails`. `AlbumDetails` is derived from `Album`.

Another example is a union type:

```
type Triangle = {
  type: "triangle";
  sideLength: number;
};

type Rectangle = {
  type: "rectangle";
  width: number;
  height: number;
};

type Shape = Triangle | Rectangle;
```

A derived type represents a relationship. That relationship is one-way. `Shape` can't go back and modify `Triangle` or `Rectangle`, but any changes to `Triangle` and `Rectangle` will flow through to `Shape`.

When well-designed, derived types can deliver huge gains in productivity. You can make changes in one place and have them propagate throughout your application. This is a powerful way to keep your code DRY and to leverage TypeScript's type system to its fullest.

This has trade-offs. You can think of deriving as a form of coupling. If you change a type that other types depend on, you need to be aware of the impact of that change. We'll compare deriving to decoupling in more detail in "Deriving vs. Decoupling" on [page 221](#).

For now, let's look at some of the tools TypeScript provides for deriving types.

The keyof Operator

The keyof operator allows you to extract the keys from an object type into a union type.

Let's start with the familiar Album type:

```
interface Album {  
  title: string;  
  artist: string;  
  releaseYear: number;  
}
```

You can use keyof Album and end up with a union of the "title", "artist", and "releaseYear" keys:

```
type AlbumKeys = keyof Album; // "title" | "artist" | "releaseYear"
```

Since keyof tracks the keys from a source, any changes made to the type will automatically be reflected in the AlbumKeys type:

```
interface Album {  
  title: string;  
  artist: string;  
  releaseYear: number;  
  genre: string; // Added "genre"  
}  
  
type AlbumKeys = keyof Album; // "title" | "artist" | "releaseYear" | "genre"
```

The `AlbumKeys` type can then be used to help ensure that a key being used to access a value in an `Album` is valid, as shown in this function:

```
function getAlbumDetails(album: Album, key: AlbumKeys) {  
    return album[key];  
}
```

If the key passed to `getAlbumDetails` is not a valid key of `Album`, TypeScript will show an error:

```
getAlbumDetails(album, "producer"); // red squiggly line under producer
```

`keyof` is an important building block when creating new types from existing types. You'll see later how you can use it with `as const` to build your own type-safe enums.

The typeof Operator

The `typeof` operator allows you to extract a type from a value.

Say you have an `albumSales` object containing a few album title keys and some sales statistics:

```
const albumSales = {  
    "Kind of Blue": 5000000,  
    "A Love Supreme": 1000000,  
    "Mingus Ah Um": 3000000,  
};
```

You can use `typeof` to extract the type of `albumSales`, which will turn it into a type with the original keys as strings and their inferred types as values:

```
type AlbumSalesType = typeof albumSales;  
  
// hovering over AlbumSalesType shows:  
type AlbumSalesType = {
```

```
"Kind of Blue": number;  
"A Love Supreme": number;  
"Mingus Ah Um": number;  
};
```

Now that you have the `AlbumSalesType` type, you can create *another* derived type from it. For example, you can use `keyof` to extract the keys from the `albumSales` object:

```
type AlbumTitles = keyof AlbumSalesType; // "Kind of Blue" |  
"A Love Supreme" | "Mingus Ah Um"
```

A common pattern is to combine `keyof` and `typeof` to create a new type from an existing object type's keys and values:

```
type AlbumTitles = keyof typeof albumSales;
```

You can use this in a function to ensure that the `title` parameter is a valid key of `albumSales`, perhaps to look up the sales for a specific album:

```
function getSales(title: AlbumTitles) {  
    return albumSales[title];  
}
```

It's worth noting that `typeof` is not the same as the `typeof` operator used at runtime. TypeScript can tell the difference based on whether it's used in a type context or a value context:

```
// Runtime typeof  
const albumSalesType = typeof albumSales; // "object"  
  
// Type typeof  
type AlbumSalesType = typeof albumSales; // {"Kind of Blue":  
number; "A Love Supreme": number; "Mingus Ah Um": number;}
```

Use the `typeof` keyword whenever you need to extract types based on TypeScript's understanding of the runtime value. This includes objects, functions, classes, and more. It's a powerful tool for deriving types from values, and it's a key building block for other patterns that you'll explore later.

You Can't Create Values from Types

You've seen that `typeof` can create types from runtime values, but it's important to note that there is no way to create a value from a type.

In other words, there is no `valueof` operator:

```
type Album = {
  title: string;
  artist: string;
  releaseYear: number;
};

// This will not work!
const album = valueof Album; // red squiggly line under valueof
```

This is a common question from beginners. But how would this even work? How would TypeScript guess what value you intended to add for title, artist, or release year?

TypeScript's types disappear at runtime, so there's no built-in way to create a value from a type. In other words, you can move from the "value world" to the "type world," but not the other way around.

Indexed Access Types

Indexed access types in TypeScript allow you to access a property of another type. This is similar to how you would access the value of a property in an object at runtime, but instead, it operates at the type level.

For example, you could use an indexed access type to extract the type of the `title` property from `AlbumDetails`:

```
interface Album {  
  title: string;  
  artist: string;  
  releaseYear: number;  
}
```

If you try to use dot notation to access the `title` property from the `Album` type, TypeScript will throw an error:

```
type AlbumTitle = Album.title; // red squiggly line under Album.title
```

// hovering over `Album.title` shows:
Cannot access 'Album.title' because 'Album' is a type, but not a namespace.
Did you mean to retrieve the type of the property 'title' in 'Album' with 'Album["title"]'?

In this case, the error message has a helpful suggestion—use `Album["title"]` to access the type of the `title` property in the `Album` type:

```
type AlbumTitle = Album["title"];
```

// hovering over `AlbumTitle` shows:
`type AlbumTitle = string;`

Using this indexed access syntax, the `AlbumTitle` type is equivalent to `string` because that's the type of the `title` property in the `Album` interface.

This same approach can be used to extract types from a tuple, where the index is used to access the type of a specific element in the tuple:

```
type AlbumTuple = [string, string, number];  
type AlbumTitle = AlbumTuple[0];
```

Once again, the `AlbumTitle` will be a `string` type because that's the type of the first element in the `AlbumTuple`.

Chaining Multiple Indexed Access Types

Indexed access types can be chained together to access nested properties. This is useful when working with complex types that have nested structures.

For example, you could use indexed access types to extract the type of the name property from the artist property in the Album type:

```
interface Album {  
  title: string;  
  artist: {  
    name: string;  
  };  
}  
  
type ArtistName = Album["artist"]["name"];
```

In this case, the ArtistName type will be equivalent to string because that's the type of the name property in the artist object.

Passing a Union to an Indexed Access Type

If you want to access multiple properties from a type, you might be tempted to create a union type containing multiple indexed accesses:

```
type Album = {  
  title: string;  
  isSingle: boolean;  
  releaseYear: number;  
};  
  
type AlbumPropertyTypes =  
  | Album["title"]  
  | Album["isSingle"]  
  | Album["releaseYear"];
```

This will work, but you can do one better. You can pass a union type to an indexed access type directly:

```
type AlbumPropertyTypes = Album["title" | "isSingle" | "releaseYear"];  
// string | boolean | number
```

This is a more concise way to achieve the same result.

Getting an Object's Values with keyof

You may have noticed that you have another opportunity to reduce repetition here. You can use `keyof` to extract the keys from the `Album` type and use them as the union type:

```
type AlbumPropertyTypes = Album[keyof Album]; // string | boolean | number
```

This is a great pattern to use when you want to extract all the values from an object type. `keyof Obj` will give you a union of all the *keys* in `Obj`, and `Obj[keyof Obj]` will give you a union of all the *values* in `Obj`.

Using `as const` for JavaScript-Style Enums

In [Chapter 9](#), you looked at the `enum` keyword. You saw that `enum` is a powerful way to create a set of named constants, but it has some downsides.

You now have all the tools available to you to see an alternative approach to creating enum-like structures in TypeScript.

First, let's use the `as const` assertion you saw in [Chapter 7](#). This forces an object to be treated as read-only and infers literal types for its properties:

```
const albumTypes = {  
  CD: "cd",  
  VINYL: "vinyl",  
  DIGITAL: "digital",  
} as const;
```

You can now *derive* the types you need from `albumTypes` using `keyof` and `typeof`. For instance, you can grab the keys using `keyof`:

```
type UppercaseAlbumType = keyof typeof albumTypes; // "CD" | "VINYL" | "DIGITAL"
```

You can also grab the values using `Obj[keyof Obj]`:

```
type AlbumType = typeof albumTypes[keyof typeof albumTypes];  
// "cd" | "vinyl" | "digital"
```

You can now use your `AlbumType` type to ensure that a function accepts only one of the values from `albumTypes`:

```
function getAlbumType(type: AlbumType) {  
    // . . .  
}
```

This approach is sometimes called a *POJO*, or *Plain Old JavaScript Object*. While it takes a bit of TypeScript magic to get the types set up, the result is simple to understand and easy to work with.

Let's now compare this to the enum approach.

Enums Require You to Pass the Enum Value

The `getAlbumType` function behaves differently than if it accepted an enum. Because `AlbumType` is just a union of strings, you can pass a raw string to `get AlbumType`. But if you pass the incorrect string, TypeScript will show an error:

```
getAlbumType(albumTypes.CD); // No error  
getAlbumType("vinyl"); // No error  
getAlbumType("cassette"); // red squiggly line under cassette
```

This is a trade-off. With enum, you have to pass the enum value, which is more explicit. With your `as const` approach, you can pass a raw string. This can make refactoring a little harder.

Enums Are Nominal

One of the biggest differences between enum and the `as const` approach is that enum is *nominal*, while the `as const` approach is *structural*.

This means that with enum, the type is based on the name of the enum, so enums with the same values that come from different enums are not compatible:

```
enum AlbumType {
  CD = "cd",
  VINYL = "vinyl",
  DIGITAL = "digital",
}

enum MediaType {
  CD = "cd",
  VINYL = "vinyl",
  DIGITAL = "digital",
}

getAlbumType(AlbumType.CD); // No error
getAlbumType(MediaType.CD); // red squiggly line under Media
                              Type.CD
```

If you're used to enums from other languages, this is probably what you expect. But for developers used to JavaScript, this can be surprising.

With a POJO, where the value comes from doesn't matter. If two POJOs have the same values, they are compatible:

```
const albumTypes = {
  CD: "cd",
  VINYL: "vinyl",
  DIGITAL: "digital",
} as const;

const mediaTypes = {
  CD: "cd",
  VINYL: "vinyl",
  DIGITAL: "digital",
} as const;
```

```
getAlbumType(albumTypes.CD); // No error  
getAlbumType(mediaTypes.CD); // No error
```

This is a trade-off. Nominal typing can be more explicit and help catch bugs, but it can also be more restrictive and harder to work with.

Which Approach Should You Use?

The enum approach is more explicit and can help you refactor your code. It's also more familiar to developers coming from other languages.

The `as const` approach is more flexible and easier to work with. It's also more familiar to JavaScript developers.

If you're particularly forward-thinking, there are also in-progress proposals to add enums to JavaScript. Since TypeScript's enums are relatively flawed, it's likely the JavaScript implementation will work quite differently.

In general, if you're working with a team that's used to enum, you should use enum. But if you were starting a project today, you would use `as const` instead of enums.

Exercise 10-1: Reducing Key Repetition

Here you have an interface named `FormValues`:

```
interface FormValues {  
  name: string;  
  email: string;  
  password: string;  
}
```

This `inputs` variable is typed as a `Record` that specifies a key of either `name`, `email`, or `password` and a value that is an object with an `initialValue` and `label` properties that are both strings:

```
const inputs: Record<  
  "name" | "email" | "password", // Change me!  
  {
```

```
        initialValue: string;
        label: string;
    }
> = {
    name: {
        initialValue: "",
        label: "Name",
    },
    email: {
        initialValue: "",
        label: "Email",
    },
    password: {
        initialValue: "",
        label: "Password",
    },
};
```

Notice there is a lot of duplication here. Both the `FormValues` interface and the `inputs` record contain `name`, `email`, and `password`.

Your task is to modify the `inputs` record so that its keys are derived from the `FormValues` interface.

See <https://totalts.link/essentials-10-1/>.

Solution

The solution is to use `keyof` to extract the keys from the `FormValues` interface and use them as the keys for the `inputs` record:

```
const inputs: Record<
    keyof FormValues, // "name" | "email" | "password"
    {
        initialValue: string;
        label: string;
    }
> = {
    // Object as before
};
```

Now, if the `FormValues` interface changes, the `inputs` record will automatically be updated to reflect those changes. `inputs` is derived from `FormValues`.

Exercise 10-2: Deriving a Type from a Value

Here, you have an object named `configurations` that comprises a set of deployment environments for development, production, and staging.

Each environment has its own URL and timeout settings:

```
const configurations = {
  development: {
    apiUrl: "http://localhost:8080",
    timeout: 5000,
  },
  production: {
    apiUrl: "https://api.example.com",
    timeout: 10000,
  },
  staging: {
    apiUrl: "https://staging.example.com",
    timeout: 8000,
  },
};
```

An `Environment` type has been declared as follows:

```
type Environment = "development" | "production" | "staging";
```

You want to use the `Environment` type across your application. However, the `configurations` object should be used as the source of truth.

Your task is to update the `Environment` type so that it is derived from the `configurations` object.

See <https://totalts.link/essentials-10-2/>.

Solution

The solution is to use the `typeof` keyword in combination with `keyof` to create the `Environment` type.

You could use them together in a single line:

```
type Environment = keyof typeof configurations;
```

Or you could first create a type from the `configurations` object and then update `Environment` to use `keyof` to extract the names of the keys:

```
type Configurations = typeof configurations;
type Environment = keyof Configurations;

// hovering over Configurations shows:
type Configurations = {
  development: {
    apiUrl: string;
    timeout: number;
  };
  production: {
    apiUrl: string;
    timeout: number;
  };
  staging: {
    apiUrl: string;
    timeout: number;
  };
};

// hovering over Environment shows:
type Environment = "development" | "production" | "staging";
```

Exercise 10-3: Accessing Specific Values

Here you have a `programModeEnumMap` object that keeps different groupings in sync. There is also a `ProgramModeMap` type that uses `typeof` to represent the entire enum mapping:

```
export const programModeEnumMap = {
  GROUP: "group",
  ANNOUNCEMENT: "announcement",
  ONE_ON_ONE: "1on1",
  SELF_DIRECTED: "selfDirected",
  PLANNED_ONE_ON_ONE: "planned1on1",
  PLANNED_SELF_DIRECTED: "plannedSelfDirected",
} as const;

type ProgramModeMap = typeof programModeEnumMap;
```

The goal is to have a Group type that is always in sync with the Program ModeEnumMap's group value. Currently it is typed as unknown:

```
type Group = unknown;
type test = Expect<Equal<Group, "group">>; // red squiggly line under Equal<>
```

Your task is to find the proper way to type Group so that the test passes as expected.

See <https://totalts.link/essentials-10-3/>.

Solution

Using an indexed access type, you can access the GROUP property from the ProgramModeMap type:

```
type Group = ProgramModeMap["GROUP"];

// hovering over Group shows:
type Group = "group";
```

With this change, the Group type will be in sync with the ProgramModeEnum Map's group value. This means your test will pass as expected.

Exercise 10-4: Unions with Indexed Access Types

This exercise starts with the same `programModeEnumMap` and `ProgramModeMap` as the previous exercise:

```
export const programModeEnumMap = {
  GROUP: "group",
  ANNOUNCEMENT: "announcement",
  ONE_ON_ONE: "1on1",
  SELF_DIRECTED: "selfDirected",
  PLANNED_ONE_ON_ONE: "planned1on1",
  PLANNED_SELF_DIRECTED: "plannedSelfDirected",
} as const;

type ProgramModeMap = typeof programModeEnumMap;

type PlannedPrograms = unknown;
type test = Expect<
  Equal<PlannedPrograms, "planned1on1" | "plannedSelfDirected">
>; // red squiggly line under Equal<>
```

This time, your challenge is to update the `PlannedPrograms` type to use an indexed access type to extract a union of the `ProgramModeMap` values that include "planned".

See <https://totalts.link/essentials-10-4/>.

Solution

To create the `PlannedPrograms` type, you can use an indexed access type to extract a union of the `ProgramModeMap` values that include `planned`:

```
type Key = "PLANNED_ONE_ON_ONE" | "PLANNED_SELF_DIRECTED";
type PlannedPrograms = ProgramModeMap[Key];
```

With this change, the `PlannedPrograms` type will be a union of `planned1on1` and `plannedSelfDirected`, which means your test will pass as expected.

Exercise 10-5: Extract a Union of All Values

You're back with the `programModeEnumMap` and the `ProgramModeMap` type:

```
export const programModeEnumMap = {
  GROUP: "group",
  ANNOUNCEMENT: "announcement",
  ONE_ON_ONE: "1on1",
  SELF_DIRECTED: "selfDirected",
  PLANNED_ONE_ON_ONE: "planned1on1",
  PLANNED_SELF_DIRECTED: "plannedSelfDirected",
} as const;

type ProgramModeMap = typeof programModeEnumMap;
```

This time you're interested in extracting all of the values from the `programModeEnumMap` object:

```
type AllPrograms = unknown;

type test = Expect<
  Equal<
    AllPrograms,
    | "group"
    | "announcement"
    | "1on1"
    | "selfDirected"
    | "planned1on1"
    | "plannedSelfDirected"
  >
>; // red squiggly line under Equal<>
```

Using what you've learned so far, your task is to update the `AllPrograms` type to use an indexed access type to create a union of all the values from the `programModeEnumMap` object.

See <https://totalts.link/essentials-10-5/>.

Solution

Using `keyof` and `typeof` together is the solution to this problem.

The most condensed solution looks like this:

```
type AllPrograms = (typeof programModeEnumMap)[keyof typeof  
programModeEnumMap];
```

Using an intermediate type, you could first use `typeof` `programModeEnumMap` to create a type from the `programModeEnumMap` object and then use `keyof` to extract the keys:

```
type ProgramModeMap = typeof programModeEnumMap;  
type AllPrograms = ProgramModeMap[keyof ProgramModeMap];
```

Either solution results in a union of all values from the `programModeEnumMap` object, which means your test will pass as expected:

```
// hovering over AllPrograms shows:  
type AllPrograms =  
  | "group"  
  | "announcement"  
  | "1on1"  
  | "selfDirected"  
  | "planned1on1"  
  | "plannedSelfDirected";
```

Exercise 10-6: Creating a Union from an `as const` Array

Here's an array of `programModes` wrapped in an `as const`:

```
export const programModes = [  
  "group",  
  "announcement",  
  "1on1",  
  "selfDirected",  
  "planned1on1",  
  "plannedSelfDirected",  
] as const;
```

A test has been written to check if an `AllPrograms` type is a union of all the values in the `programModes` array:

```
type test = Expect<
  Equal<
    AllPrograms,
    | "group"
    | "announcement"
    | "1on1"
    | "selfDirected"
    | "planned1on1"
    | "plannedSelfDirected"
  >
>; // red squiggly line under Equal<>
```

Your task is to determine how to create the `AllPrograms` type for the test to pass as expected.

Note that just using `keyof` and `typeof` in an approach similar to the previous exercise's solution won't quite work to solve this one! This is tricky to find, but as a hint, you can pass primitive types to indexed access types.

See <https://totalts.link/essentials-10-6/>.

Solution

When using `typeof` and `keyof` with indexed access types, you can extract all the values, but you also get some unexpected values, like a `6` and an `IterableIterator` function:

```
type AllPrograms = (typeof programModes)[keyof typeof programModes];

// hovering over AllPrograms shows:
type AllPrograms =
  | "group"
  | "announcement"
  | "1on1"
  | "selfDirected"
  | "planned1on1"
  | "plannedSelfDirected"
  | 6
  | (() => IterableIterator<"group" | "announcement" | "1on
```

```
1" |  
"selfDirected" | "planned1on1" | "plannedSelfDirected">)  
  | . . . 23 more . . .  
  | ((index: number) => "group" | . . . 5 more . . . | undef  
ined);
```

The additional content being extracted causes the test to fail because it is expecting only the original values instead of numbers and functions.

Recall that you can access the first element using `programModes[0]`, the second element using `programModes[1]`, and so on. This means you could use a union of all possible index values to extract the values from the `programModes` array:

```
type AllPrograms = (typeof programModes)[0 | 1 | 2 | 3 | 4 |  
5];
```

This solution makes the test pass, but it doesn't scale well. If the `programModes` array were to change, you would need to update the `AllPrograms` type manually.

Instead, you can use the `number` type as the argument to the indexed access type to represent all possible index values:

```
type AllPrograms = (typeof programModes)[number];
```

Now new items can be added to the `programModes` array without needing to update the `AllPrograms` type manually. This solution makes the test pass as expected and is a great pattern to apply in your own projects.

Deriving Types from Functions

So far, we've covered deriving types only from objects and arrays. But deriving types from functions can help solve some common problems in TypeScript.

Parameters

The Parameters utility type extracts the parameters from a given function type and returns them as a tuple.

For example, this sellAlbum function takes in an Album, a price, and a quantity and then returns a number representing the total price:

```
function sellAlbum(album: Album, price: number, quantity: number) {  
  return price * quantity;  
}
```

Using the Parameters utility type, you can extract the parameters from the sellAlbum function and assign them to a new type:

```
type SellAlbumParams = Parameters;  
  
// hovering over SellAlbumParams shows:  
type SellAlbumParams = [Album, number, number];
```

Note that you need to use typeof to create a type from the sellAlbum function. Passing sellAlbum directly to Parameters won't work on its own because sellAlbum is a value instead of a type:

```
type SellAlbumParams = Parameters<sellAlbum>; // red squiggly line under sellAlbum
```

This SellAlbumParams type is a tuple type that holds the Album, price, and quantity parameters from the sellAlbum function.

If you need to access a specific parameter from the SellAlbumParams type, you can use indexed access types:

```
type Price = SellAlbumParams[1]; // Number
```

ReturnType

The ReturnType utility type extracts the return type from a given function:

```
type SellAlbumReturn = ReturnType<typeof sellAlbum>;

// hovering over SellAlbumReturn shows:
type SellAlbumReturn = number;
```

In this case, the `SellAlbumReturn` type is a `number`, which is derived from the `sellAlbum` function.

Awaited

Earlier in the book, you used the `Promise` type when working with asynchronous code.

The `Awaited` utility type is used to unwrap the `Promise` type and provide the type of the resolved value. Think of it as a shortcut similar to using the `await` or `.then()` methods.

This can be particularly useful for deriving the return types of `async` functions. To use `Awaited`, you would pass it a `Promise` type, and it would return the type of the resolved value:

```
type AlbumPromise = Promise<Album>;

type AlbumResolved = Awaited<AlbumPromise>;
```

Why Derive Types from Functions?

Being able to derive types from functions might not seem useful at first. After all, if you control the functions, then you can just write the types yourself and reuse them as needed:

```
type Album = {
  title: string;
  artist: string;
  releaseYear: number;
};

const sellAlbum = (album: Album, price: number, quantity: number) => {
```

```
    return price * quantity;
};
```

There's no reason to use `Parameters` or `ReturnType` on `sellAlbum`, since you defined the `Album` type and the return type yourself.

But what about functions you don't control?

A common example is a third-party library. A library might export a function that you can use but might not export the accompanying types. An example we recently came across was a type from the library `@monaco-editor/react`:

```
tsx twoslash
import {Editor} from "@monaco-editor/react";

// This is a JSX component, for your purposes equivalent to.
. .
  onMount={(editor) => {
    // . . .
  }}
/>;

// . . .Calling the function directly with an object
Editor({
  onMount: (editor) => {
    // . . .
  },
});
```

In this case, we wanted to know the type of `editor` so that we could reuse it in a function elsewhere. But the `@monaco-editor/react` library didn't export its type.

First, we extracted the type of the object the component expected:

```
type EditorProps = Parameters<typeof Editor>[0];
```

Then, we used an indexed access type to extract the type of the `onMount` property:

```
type OnMount = EditorProps["onMount"];
```

Finally, we extracted the first parameter from the `OnMount` type to get the type of editor:

```
type Editor = Parameters<OnMount>[0];
```

This allowed us to reuse the `Editor` type in a function elsewhere in our code.

By combining indexed access types with TypeScript's utility types, you can work around the limitations of third-party libraries and ensure that your types stay in sync with the functions you're using.

Exercise 10-7: A Single Source of Truth

Here you have a `makeQuery` function that takes two parameters—a `url` and an optional `opts` object:

```
const makeQuery = (  
  url: string,  
  opts?: {  
    method?: string;  
    headers?: {  
      [key: string]: string;  
    };  
    body?: string;  
  },  
) => {};
```

You want to specify these parameters as a tuple called `MakeQueryParameters` where the first argument of the tuple would be the string, and the second member would be the optional `opts` object. This might be so that you can reference one of the parameters later.

Manually specifying the `MakeQueryParameters` would look something like this:

```
type MakeQueryParameters = [  
  string,  
  {  
    method?: string;  
    headers?: {  
      [key: string]: string;  
    };  
    body?: string;  
  }?,  
];
```

In addition to being a bit annoying to write and read, the other problem with this code is that you now have two sources of truth: One is the `MakeQueryParameters` type, and the other is in the `makeQuery` function.

Your task is to use a utility type to fix this problem.

See <https://totalts.link/essentials-10-7/>.

Solution

The `Parameters` utility type is key to this solution, but there is an additional step to follow.

Passing `makeQuery` directly to `Parameters` won't work on its own because `makeQuery` is a value instead of a type:

```
type MakeQueryParameters = Parameters<makeQuery>; // red squiggly line under makeQuery
```

// hovering over `makeQuery` shows:
'makeQuery' refers to a value, but is being used as a type here.
Did you mean 'typeof makeQuery'?

As the error message suggests, you need to use `typeof` to create a type from the `makeQuery` function and then pass that type to `Parameters`:

```
type MakeQueryParameters = Parameters<typeof makeQuery>;
```

// hovering over `MakeQueryParameters` shows:

```
type MakeQueryParameters = [  
  url: string,  
  opts?:  
    | {  
      method?: string | undefined;  
      headers?:  
        | {  
          [key: string]: string;  
        }  
        | undefined;  
      body?: string | undefined;  
    }  
    | undefined,  
];
```

You now have `MakeQueryParameters` representing a tuple where the first member is a `url` string and the second is the optional `opts` object.

Indexing into the type would allow you to create an `Opts` type that represents the `opts` object:

```
type Opts = MakeQueryParameters[1];
```

Exercise 10-8: Typing Based on a Return Value

Say you're working with a `createUser` function from some code another team has control over:

```
const createUser = (id: string) => {  
  return {  
    id,  
    name: "John Doe",  
    email: "example@email.com",  
  };  
};
```

For the sake of this exercise, assume you don't have access to the implementation of the function.

The goal is to create a User type that represents the return type of the createUser function. A test has been written to check if the User type is a match:

```
type User = unknown;

type test = Expect<
  Equal<
    User,
    {
      id: string;
      name: string;
      email: string;
    }
  >
>; // red squiggly line under Equal<>
```

Your task is to update the User type so that the test passes as expected. See <https://totalts.link/essentials-10-8/>.

Solution

Using the ReturnType utility type, you can extract the return type from the createUser function and assign it to a new type. Remember that since createUser is a value, you need to use typeof to create a type from it:

```
type User = ReturnType<typeof createUser>;

// hovering over User shows:
type User = {
  id: string;
  name: string;
  email: string;
};
```

This User type is a match for the expected type, which means your test will pass as expected.

Exercise 10-9: Unwrapping a Promise

This time the `createUser` function from the third-party library is asynchronous:

```
const fetchUser = async (id: string) => {
  return {
    id,
    name: "John Doe",
    email: "example@email.com",
  };
};

type test = Expect<
  Equal<
    // red squiggly line under Equal<>
    User,
    {
      id: string;
      name: string;
      email: string;
    }
  >
>; // red squiggly line under Equal<>
```

Like before, assume you do not have access to the implementation of the `fetchUser` function.

Your task is to update the `User` type so that the test passes as expected.

See <https://totalts.link/essentials-10-9/>.

Solution

When using the `ReturnType` utility type with an `async` function, the resultant type will be wrapped in a `Promise`:

```
type User = ReturnType;

// hovering over User shows:
type User = Promise<{
  id: string;
```

```
    name: string;
    email: string;
  }>;
```

To unwrap the `Promise` type and provide the type of the resolved value, you can use the `Awaited` utility type:

```
type User = Awaited>;
```

Like before, the `User` type is now a match for the expected type, which means your test will pass as expected.

It would also be possible to create intermediate types, but combining operators and type derivation gives you a more succinct solution.

Transforming Derived Types

In the previous section, you looked at how to derive types from functions you don't control. Sometimes, you'll also need to do the same with *types* you don't control.

Exclude

The `Exclude` utility type is used to remove types from a union. Let's imagine you have a union of different states your album can be in:

```
type AlbumState =
  | {
    type: "released";
    releaseDate: string;
  }
  | {
    type: "recording";
    studio: string;
  }
  | {
    type: "mixing";
    engineer: string;
  };
```

You want to create a type that represents the states that are not “released.” You can use the `Exclude` utility type to achieve this:

```
type UnreleasedState = Exclude<AlbumState, {type: "released"}>;
// hovering over UnreleasedState shows:
type UnreleasedState =
  | {
    type: "recording";
    studio: string;
  }
  | {
    type: "mixing";
    engineer: string;
  };

```

In this case, the `UnreleasedState` type is a union of the recording and mixing states, which are the states that are not “released.” `Exclude` filters out any member of the union with a type of `released`.

You could have done it by checking for a `releaseDate` property instead:

```
type UnreleasedState = Exclude<AlbumState, {releaseDate: string}>;

```

This is because `Exclude` works by a process similar to pattern matching. It will remove any type from the union that is a subtype (a narrower type) of the pattern you provide.

This means you can use it to remove all strings from a union:

```
type Example = "a" | "b" | 1 | 2;

type Numbers = Exclude<Example, string>;
// hovering over Numbers shows:
type Numbers = 1 | 2;

```

NonNullable

NonNullable is used to remove null and undefined from a type. This can be useful when extracting a type from a partial object:

```
type Album = {
  artist?: {
    name: string;
  };
};

type Artist = NonNullable<Album["artist"]>;
// hovering over Artist shows:
type Artist = {
  name: string;
};
```

This operates similarly to Exclude:

```
type Artist = Exclude<Album["artist"], null | undefined>;
```

But NonNullable is more explicit and easier to read.

Extract

Extract is the opposite of Exclude. It's used to extract types from a union. For example, you can use Extract to extract the recording state from the AlbumState type:

```
type RecordingState = Extract<AlbumState, {type: "recordin
g"}>;
// hovering over RecordingState shows:
type RecordingState = {
  type: "recording";
  studio: string;
};
```

This is useful when you want to extract a specific type from a union you don't control.

Similarly to `Exclude`, `Extract` works by pattern matching. It will extract any type from the union that matches the pattern you provide.

This means that, to reverse your `Extract` example earlier, you can use it to extract all strings from a union:

```
type Example = "a" | "b" | 1 | 2 | true | false;

type Strings = Extract;
// hovering over Strings shows:
type Strings = "a" | "b";
```

It’s worth noting the similarities between `Exclude/Extract` and `Omit/Pick`. A common mistake is to think that you can `Pick` from a union or use `Exclude` on an object. [Table 10-1](#) will help you remember.

Table 10-1: `Exclude`, `Extract`, `Omit`, and `Pick`

Name	Used on	Action	Example
Exclude	Unions	Excludes union members	<code>Exclude<'a' 1, string></code>
Extract	Unions	Extracts union members	<code>Extract<'a' 1, string></code>
Omit	Objects	Excludes properties	<code>Omit<UserObj, 'id'></code>
Pick	Objects	Extracts properties	<code>Pick<UserObj, 'id'></code>

Deriving vs. Decoupling

Thanks to the tools in these chapters, you now know how to derive types from all sorts of sources: functions, objects, and types. But there’s a trade-off to consider when deriving types: coupling.

When you derive a type from a source, you’re coupling the derived type to that source. If you derive a type from another derived type, this can create long chains of coupling throughout your app that can be hard to manage.

When Decoupling Makes Sense

Let's imagine you have a `User` type in a `db.ts` file:

```
export type User = {  
  id: string;  
  name: string;  
  imageUrl: string;  
  email: string;  
};
```

We'll say for this example that we're using a component-based framework like React, Vue, or Svelte. We have an `AvatarImage` component that renders an image of the user. We could pass in the `User` type directly:

```
import {User} from "../db";  
  
export const AvatarImage = (props: {user: User}) => {  
  return <img src={props.user.imageUrl} alt={props.user.name} />;  
};
```

But as it turns out, we're using only the `imageUrl` and `name` properties from the `User` type. It's a good idea to make your functions and components require only the data they need to run. This helps prevent you from passing around unnecessary data.

Let's try deriving. You'll create a new type called `AvatarImageProps` that includes only the properties you need:

```
import {User} from "../db";  
  
type AvatarImageProps = Pick<User, "imageUrl" | "name">;
```

But let's think for a moment. You've now coupled the `AvatarImageProps` type to the `User` type. `AvatarImageProps` now not only depends on the shape of `User` but also depends on its *existence* in the `db.ts` file. This means if you ever move the location of the `User` type or split it into separate interfaces, you'll need to think about `AvatarImageProps`.

Let's try the other way around. Instead of deriving `AvatarImageProps` from `User`, you'll decouple them. You'll create a new type that has just the properties you need:

```
type AvatarImageProps = {  
  imageUrl: string;  
  name: string;  
};
```

Now `AvatarImageProps` is decoupled from `User`. You can move `User` around, split it into separate interfaces, or even delete it, and `AvatarImageProps` will be unaffected.

In this particular case, decoupling feels like the right choice. This is because `User` and `AvatarImage` are separate concerns. `User` is a data type, while `AvatarImage` is a UI component. They have different responsibilities and different reasons to change. By decoupling them, `AvatarImage` becomes more portable and easier to maintain.

What can make decoupling a difficult decision is that deriving can make you feel “clever.” `Pick` tempts you because it uses a more advanced feature of TypeScript, which makes you feel good for applying the knowledge you've gained. But often, it's smarter to do the simple thing and keep your types decoupled.

When Deriving Makes Sense

Deriving makes the most sense when the code you're coupling shares a common concern. The examples in this chapter are good examples of this. Our `as const` object illustrates this well:

```
const albumTypes = {  
  CD: "cd",  
  VINYL: "vinyl",  
  DIGITAL: "digital",  
} as const;  
  
type AlbumType = (typeof albumTypes)[keyof typeof albumTypes];
```

Here, `AlbumType` is derived from `albumTypes`. If you were to decouple it, you'd have to maintain two closely related sources of truth:

```
type AlbumType = "cd" | "vinyl" | "digital";
```

Because both `AlbumType` and `albumTypes` are closely related, deriving `AlbumType` from `albumTypes` makes sense.

Another example is when one type is directly related to another. For instance, your `User` type might have a `UserWithoutId` type derived from it:

```
type User = {  
  id: string;  
  name: string;  
  imageUrl: string;  
  email: string;  
};  
  
type UserWithoutId = Omit;  
  
const updateUser = (id: string, user: UserWithoutId) => {  
  // . . .  
};
```

Again, these concerns are closely related. Decoupling them would make your code harder to maintain and introduce more busywork into your codebase.

The decision to derive or decouple is all about reducing your future workload:

- Are the two types so related that updates to one will need to ripple to the other? Derive.
- Are they so unrelated that coupling them could result in more work down the line? Decouple.

Summary

This chapter explored deriving types from other types to reduce repetition and create single sources of truth. The `keyof` operator extracts object keys

into unions, while `typeof` derives types from runtime values.

Indexed access types like `Obj["property"]` extract specific property types. You can pass unions to access multiple properties at once and use `Obj[keyof Type]` to get all values from an object type.

The `as const` assertion creates JavaScript-style enums with literal types, offering more flexibility than TypeScript's `enum` keyword through structural rather than nominal typing.

Function utilities include `Parameters` for extracting parameter tuples, `ReturnType` for return types, and `Awaited` for unwrapping Promises. These help when working with third-party libraries that don't export types.

Transformation utilities like `Exclude` and `Extract` work on unions, while `Pick` and `Omit` work on objects. `NonNullable` removes `null` and `undefined` from types.

The key decision is when to derive versus decouple. Derive when types share common concerns and changes should propagate. Decouple when types serve different purposes to avoid unnecessary coupling and maintenance overhead.

11

ANNOTATIONS AND ASSERTIONS



Throughout this book, you've been using relatively simple type annotations to specify what types your variables and functions should have, or you've allowed TypeScript to infer types on its own. But sometimes you need to lie to TypeScript. In this chapter, you will learn different ways to communicate with the compiler with annotations and assertions: techniques that allow you to affect type inference and force TypeScript to treat values as certain types.

Annotating Variables vs. Values

There's a difference in TypeScript between annotating *variables* and annotating *values*. The way they conflict can be confusing, but here's a rule of thumb: When you annotate a variable, the variable wins.

Let's look at an example that demonstrates how you've seen annotations so far:

```
type Color =  
  | string
```

```
| {  
  r: number;  
  g: number;  
  b: number;  
};  
  
const config: Record<string, Color> = {  
  foreground: {r: 255, g: 255, b: 255},  
  background: {r: 0, g: 0, b: 0},  
  border: "transparent",  
};
```

Here, you're annotating the config variable as a Record with a string key and a Color value. This is useful because if you specify a Color that doesn't match the type, TypeScript will show an error:

```
const config: Record<string, Color> = {  
  border: {incorrect: 0, g: 0, b: 0}, // red squiggly line u  
  under incorrect  
};  
// hovering over incorrect shows:  
Object literal may only specify known properties,  
and 'incorrect' does not exist in type '{r: number; g: numbe  
r; b: number;}'.
```

In this code, you get the error you expect from trying to use incorrect as a key instead of r. However, there's a problem with this approach.

You can create a config variable that is properly typed without errors, but if you try to access any of the keys, TypeScript gets confused:

```
const config: Record<string, Color> = {  
  foreground: {r: 255, g: 255, b: 255},  
  background: {r: 0, g: 0, b: 0},  
  border: "transparent",  
};  
  
config.foreground.r; // red squiggly line under r  
// hovering over r shows:
```

```
Property 'r' does not exist on type 'Color'.
Property 'r' does not exist on type 'string'.
```

TypeScript won't let you access `config.foreground.r` because it doesn't know that `foreground` is defined on the object. It doesn't know whether `foreground` is the string version of the `Color` type or the object version.

This is because you've told TypeScript that `config` is a `Record` with any number of string keys. You annotated the variable, but the actual *value* got discarded. This is an important point; when you annotate a variable, TypeScript will:

1. Ensure that the value passed to the variable matches the annotation.
2. Forget about the value's type.

This has some benefits. You can add new keys to `config`, and TypeScript won't complain:

```
config.primary = "red";
```

But this isn't really what you want; this is a `config` object that shouldn't be changed.

One way to get around this would be to drop the variable annotation. When there is no annotation, the value wins:

```
const config = {
  foreground: {r: 255, g: 255, b: 255},
  background: {r: 0, g: 0, b: 0},
  border: "transparent",
};
```

Because there's no variable annotation, `config` is inferred as the type of the value provided.

But now you've lost the ability to check that the `Color` type is correct. You can add a number to the `foreground` key, and TypeScript won't complain:

```
const config = {  
  foreground: 123,  
};
```

So, it seems you're at an impasse. You want to infer the type of the value but also constrain it to be a certain shape.

Annotating Values with `satisfies`

The `satisfies` operator is a way to tell TypeScript that a value must satisfy certain criteria but still allow TypeScript to infer the type.

Let's use it to make sure your `config` object has the right shape:

```
const config = {  
  foreground: {r: 255, g: 255, b: 255},  
  background: {r: 0, g: 0, b: 0},  
  border: "transparent",  
} satisfies Record<string, Color>;
```

Now you get the best of both worlds. This means you can access the keys without any issues:

```
config.foreground.r;  
config.border.toUpperCase();
```

TypeScript recognizes that `foreground.r` leads to a number and that `border` is a string. But you've also told TypeScript that `config` must be a `Record` with a string key and a `Color` value. If you try to add a key that doesn't match this shape, TypeScript will show an error:

```
const config = {  
  primary: 123, // red squiggly line under primary  
} satisfies Record<string, Color>;  
  
// hovering over primary shows:  
Type 'number' is not assignable to type 'Color'.
```

There is no primary property in the `Color` type, so the error is expected. You have also lost the ability to add new keys to `config` without TypeScript complaining:

```
config.somethingNew = "red"; // red squiggly line under somethingNew
```

```
// hovering over somethingNew shows:  
Property 'somethingNew' does not exist on type '{}'.  

```

The `somethingNew` property can't be added to `config` because TypeScript is now inferring `config` as *just* an object with a fixed set of keys.

Let's recap:

- When you use a variable annotation, the variable's type wins.
- When you don't use a variable annotation, the value's type wins.
- When you use `satisfies`, you can tell TypeScript that a value must satisfy certain criteria but still allow TypeScript to infer the type.

Narrowing Values with satisfies

A common misconception about `satisfies` is that it doesn't affect the type of the value. This is not quite true; in certain situations, `satisfies` does help narrow down a value to a particular type.

Let's take this example:

```
const album = {  
  format: "Vinyl",  
};  

```

Here you have an `album` object with a `format` key. As you know from [Chapter 7](#), TypeScript will infer `album.format` as `string`. You want to make sure that the `format` is one of three values: `CD`, `Vinyl`, or `Digital`.

You could give it a variable annotation:

```
type Album = {  
  format: "CD" | "Vinyl" | "Digital";  
};  
  
const album: Album = {  
  format: "Vinyl",  
};
```

But now, `album.format` is `"CD" | "Vinyl" | "Digital"`. This might be a problem if you want to pass it to a function that accepts only `"Vinyl"`. Instead, you can use `satisfies`:

```
const album = {  
  format: "Vinyl",  
} satisfies Album;
```

Now `album.format` is inferred as `"Vinyl"` because you've told TypeScript that `album` satisfies the `Album` type. So, `satisfies` is narrowing down the value of `album.format` to a specific type.

Assertions: Forcing the Type of Values

Sometimes, the way TypeScript infers types isn't quite what you want. You can use assertions in TypeScript to force values to be inferred as a certain type.

The as Assertion

The `as` assertion is a way to tell TypeScript that you know more about a value than it does. It's a way to override the type of any variable or property and tell TypeScript to treat it as a different type.

Let's look at an example. Imagine you're building a web page that has some information in the search query string of the URL. You happen to know that the user can't navigate to this page without passing `?id=some-id` to the URL:

```
const searchParams = new URLSearchParams(window.location.sea  
rch);
```

```
const id = searchParams.get("id");

// hovering over id shows:
const id: string | null;
```

But TypeScript doesn't know that the `id` will always be a string. It thinks that `id` could be a string or `null`.

So, let's force it. You can use `as` on the result of `searchParams.get("id")` to tell TypeScript that you know it will always be a string:

```
const id = searchParams.get("id") as string;

// hovering over id shows:
const id: string;
```

Now TypeScript knows that `id` will always be a string, and you can use it as such.

This `as` is a little unsafe! If `id` is somehow not actually passed in the URL, it will be `null` at runtime but `string` at compile time. This means if you called `.toUpperCase()` on `id`, you'd crash your app. But it's useful in cases where you truly know more than TypeScript can about the behavior of your code.

Note that there is an alternative syntax to `as`, where you prefix the value with the type wrapped in angle brackets:

```
const id = <string>searchParams.get("id");
```

This behaves in the same way, but it's best to stick with using `as`. The angle brackets syntax works only in `.ts` files (not `.tsx` files) and is not recommended by the TypeScript team.

The Limits of `as`

`as` has some limits on how it can be used. It can be used to convert only between types that have "sufficient overlap."

Consider this example where `as` is used to assert that a string should be treated as a number:

```
const albumSales = "Heroes" as number; // red squiggly line  
under "Heroes" as number
```

TypeScript realizes that even though we're using `as`, you might have made a mistake:

```
// hovering over "Heroes" as number shows:  
Conversion of type 'string' to type 'number' may be a mistake  
because neither type sufficiently  
overlaps with the other. If this was intentional, convert  
the expression to 'unknown' first.
```

The error message is telling us that a string and a number don't share any common properties, but if you really want to go through with it, you could double up on the `as` assertions to first assert the string as `unknown` and then as a number:

```
const albumSales = "Heroes" as unknown as number; // No error
```

When using `as` to assert as `unknown as number`, the red squiggly line goes away, but that doesn't mean the operation is safe. This operation should be considered as dangerous as the `as` any operation, just with an extra step.

The same behavior applies to other types as well.

In this example, an `Album` interface and a `SalesData` interface don't share any common properties:

```
interface Album {  
  title: string;  
  artist: string;  
  releaseYear: number;  
}
```

```
interface SalesData {  
    sales: number;  
    certification: string;  
}  
  
const paulsBoutique: Album = {  
    title: "Paul's Boutique",  
    artist: "Beastie Boys",  
    releaseYear: 1989,  
};  
  
const paulsBoutiqueSales = paulsBoutique as SalesData;  
// red squiggly line under paulsBoutique as SalesData
```

Trying to use the `as SalesData` annotation on `paulsBoutique` causes TypeScript to show you a warning about the lack of common properties:

```
// hovering over paulsBoutique as SalesData shows:  
Conversion of type 'Album' to type 'SalesData' may be a mistake  
because neither type sufficiently overlaps with the other. If this  
was intentional, convert the expression to 'unknown' first.  
Type 'Album' is missing the following properties from type  
'SalesData': sales, certification.
```

So, `as` does have some built-in safeguards. But by using `as unknown as X`, you can easily bypass them. And because `as` does nothing at runtime, it's a convenient way to lie to TypeScript about the type of a value.

The Non-Null Assertion

Another assertion you can use is the non-null assertion, which is specified by using the `!` operator. This provides a quick way to tell TypeScript that a value is not `null` or `undefined`.

Heading back to your `searchParams` example from earlier, you can use the non-null assertion to tell TypeScript that `id` will never be `null`:

```
const searchParams = new URLSearchParams(window.location.search);

const id = searchParams.get("id")!;
```

This forces TypeScript to treat `id` as a string, even though it could be `null` at runtime. It's the equivalent of using `as string` but is a little more convenient.

You can also use it when accessing a property that may or may not be defined:

```
type User = {
  name: string;
  profile?: {
    bio: string;
  };
};

const logUserBio = (user: User) => {
  console.log(user.profile!.bio);
};
```

In this example, the non-null assertion tells TypeScript you aren't sure if `user.profile` will be defined. This also works for function calls, as shown when trying to call `logger.log` in `main`:

```
type Logger = {
  log?: (message: string) => void;
};

const main = (logger: Logger) => {
  logger.log!("Hello, world!");
};
```

Both of these examples will fail at runtime if the values are not defined. However, the non-null assertion is a convenient way for you to lie to TypeScript that you're sure they will be defined.

The non-null assertion, like other assertions, is a dangerous tool. It's particularly nasty because it's one character long, so it is easier to miss than `as`.

As an alternative, it's worth considering optional chaining, a JavaScript feature that uses the `?` operator instead:

```
const main = (logger: Logger) => {  logger.log?("Hello, world!");
```

Now `logger.log` will be called only if it exists. If not, it'll resolve to `undefined`, and the function will not be called. This is a safer way to handle the problem, since it won't fail at runtime.

For fun, consider using at least three or four in a row to make sure other developers know that there is danger:

```
const id = searchParams.get("id")!!!!;
```

Yes, this syntax is legal!

Error Suppression Directives

Assertions are not the only way you can lie to TypeScript. There are several comment directives that can be used to suppress errors. You can tell the compiler that there will be an error on the next line, that it should ignore any errors on the next line, or even that it should skip checking altogether.

@ts-expect-error

Throughout the book's exercises, you've seen several examples of `@ts-expect -error`. This directive gives you a way to tell TypeScript that you expect an error to occur on the next line of code.

In this example, you're creating an error by passing a string into a function that expects a number:

```
function addOne(num: number) {  
    return num + 1;  
}
```

```
// @ts-expect-error
const result = addOne("one");
```

But the error doesn't show up in the editor because you told TypeScript to expect it.

However, if you pass a number into the function, the error will show up:

```
// @ts-expect-error // red squiggly line under @ts-expect-error
const result = addOne(1);

// hovering over addOne(1) shows:
Unused @ts-expect-error directive.
```

So, TypeScript expects every `@ts-expect-error` directive to be *used*—that is, to be followed by an error.

Frustratingly, `@ts-expect-error` doesn't let you expect a specific error, but only that an error will occur.

`@ts-ignore`

The `@ts-ignore` directive behaves a bit differently than `@ts-expect-error`. Instead of *expecting* an error, it *ignores* any errors that do occur.

Going back to your `addOne` example, you can use `@ts-ignore` to ignore the error that occurs when passing a string into the function:

```
// @ts-ignore
const result = addOne("one");
```

But if you later fix the error, `@ts-ignore` won't tell you that it's unused:

```
// @ts-ignore
const result = addOne(1); // No errors here!
```

In general, `@ts-expect-error` is more useful than `@ts-ignore` because it tells you when you've fixed the error. This means you can get a warning to remove the directive.

`@ts-nocheck`

Finally, the `@ts-nocheck` directive will completely remove type checking for a file. To use it, add the directive at the top of your file:

```
// @ts-nocheck
```

With all checking disabled, TypeScript won't show you any errors, but it also won't be able to protect you from any runtime issues that might show up when you run your code.

Generally speaking, you shouldn't use `@ts-nocheck`. We've personally lost hours of our lives working in large files where we didn't notice that `@ts-nocheck` was at the top.

Suppressing Errors vs. as any

There's one tool in a TypeScript developers' toolkit that *also* suppresses errors but isn't a comment directive: `as any`.

The `as any` tool is extremely powerful because it combines a lie to TypeScript (`as`) with a type that disables all type checking (`any`).

This means you can use it to suppress nearly any error. Our previous example? No problem:

```
const result = addOne({} as any);
```

Use of `as any` turns the empty object into `any`, which disables all type checking. This means `addOne` will happily accept it.

When there's a choice in how to suppress an error, we prefer using `as any`. Error suppression directives are too broad: They target the entire line of code. This can lead to accidentally suppressing errors that you didn't mean to:

```
// @ts-ignore
const result = addone("one");
```

Here, you're calling `addone` instead of `addOne`. The error suppression directive will suppress the error, but it will also suppress any other errors that might occur on that line. Using `as any` instead is more precise:

```
const result = addone("one" as any); // red squiggly line under addone
// hovering over addone shows:
Cannot find name 'addone'. Did you mean 'addOne'?
```

Now you'll suppress only the error that you intended to.

When to Suppress Errors

Each of the error suppression tools you've seen essentially tells TypeScript to "keep quiet." TypeScript doesn't attempt to limit how often you try to silence it. It's perfectly possible that every time you encounter an error, you could suppress it with `@ts-ignore` or `as any`.

Taking this approach limits how useful TypeScript can be. Your code will compile, but you will likely get many more runtime errors.

But sometimes suppressing errors is a good idea. Let's explore a few different scenarios.

When You Know More Than TypeScript

The important thing to remember about TypeScript is that really you're writing JavaScript.

This disconnect between compile time and runtime means that types *can sometimes be wrong*. This can mean you know more about the runtime code than TypeScript does.

This can happen when third-party libraries don't have good type definitions or when you're working with a complex pattern that TypeScript struggles to understand.

Error suppression directives exist for this reason. They let you patch over the differences that sometimes crop up between TypeScript and the

JavaScript it produces.

But this feeling of superiority over TypeScript can be dangerous. So, let's compare it to a similar feeling.

When TypeScript Is Being “Dumb”

Some patterns lend themselves better to being typed than others. More dynamic patterns can be harder for TypeScript to understand and will lead you to suppressing more errors.

A simple example is constructing an object. In JavaScript, there's no real difference between these two patterns:

```
// Static
const obj = {
  a: 1,
  b: 2,
};

// Dynamic
const obj2 = {};

obj2.a = 1; // red squiggly line under a
obj2.b = 2; // red squiggly line under b
// hovering over a or b shows:
Property 'a' (or 'b') does not exist on type '{}'.

```

In the first pattern, you construct an object by passing in the keys and values. In the second, you construct an empty object and add the keys and values later. The first pattern is static; the second is dynamic.

In TypeScript, the first pattern is much easier to work with. TypeScript can infer the type of `obj` as `{a: number, b: number}`. But it can't infer the type of `obj2` because it's just an empty object. In fact, you'll get errors when you try to do this.

If you're used to constructing your objects in a dynamic way, this can be frustrating. You know that `obj2` will have an `a` and a `b` key, but TypeScript doesn't.

In these cases, it's tempting to bend the rules a little by using an `as` to tell TypeScript that you know what you're doing:

```
const obj2 = {} as {a: number; b: number};
```

This is subtly different from the first scenario, where you know more than TypeScript does. In this case, there's a simple runtime refactor you can use to make TypeScript happy and avoid suppressing errors.

The more experienced you are with TypeScript, the more often you'll be able to spot these patterns. You'll be able to spot the times when TypeScript lacks crucial information, requiring an `as`, or when the patterns you're using aren't letting TypeScript do its job properly.

So, if you're tempted to suppress an error, see if there's a way you can refactor your code to a pattern that TypeScript understands better. After all, it's easier to swim with the current than against it.

When You Don't Understand the Error

Let's say you've been coding for a few hours. An unread Slack message notification is blinking at you. The feature is all but finished, except for some types you need to add. You've got a call in 20 minutes. And then, TypeScript shows an error that you don't understand.

TypeScript errors can be extremely hard to read. They can be long, multilayered, and filled with references to types you've never heard of.

It's at this moment that TypeScript can feel most frustrating. It's enough to turn many developers off TypeScript for good.

So, you suppress the error. You add an `@ts-ignore` or an `as any` and move on.

Weeks later, a bug gets reported. You end up back in the same area of the codebase. And you track the error down to the exact line you suppressed.

The time you save by suppressing errors will, eventually, come back to bite you. You're not saving time but borrowing it.

It's in this situation, when you don't understand the error, that we recommend sticking it out. TypeScript is attempting to communicate with you. Try refactoring your runtime code. Use all the tools mentioned in [Chapter 2](#) to investigate the types the errors mention.

Think of the time you invest in fixing TypeScript errors as an investment in yourself. You're both fixing potential bugs in the future and

leveling up your own understanding.

Exercise 11-1: Providing Additional Info to TypeScript

This `handleFormData` function accepts an argument `e` typed as `SubmitEvent`, which is a global type from the DOM typings that is emitted when a form is submitted.

Within the function, you use the method `e.preventDefault()`, available on `SubmitEvent`, to stop the form from its default submission action. Then, you attempt to create a new `FormData` object, `data`, with `e.target`:

```
const handleFormData = (e: SubmitEvent) => {  
  e.preventDefault();  
  const data = new FormData(e.target); // red squiggly line  
  under e.target  
  const value = Object.fromEntries(data.entries());  
  return value;  
};
```

At runtime, this code works flawlessly. However, at the type level, TypeScript shows an error under `e.target`:

```
// hovering over e.target shows:  
Argument of type 'EventTarget | null' is not assignable to p  
arameter of  
type 'HTMLFormElement | undefined'.  
Type 'null' is not assignable to type 'HTMLFormElement | und  
efined'.
```

Your task is to provide TypeScript with additional information to resolve the error.

See <https://totalts.link/essentials-11-1/>.

Solution

The error you encountered in this challenge was that the `EventTarget | null` type was incompatible with the required parameter of type

HTMLFormElement. The problem stems from the fact that these types don't match, and null is not permitted:

```
const data = new FormData(e.target); // red squiggly line under e.target
```

// hovering over e.target shows:
Argument of type 'EventTarget | null' is not assignable to parameter of type 'HTMLFormElement | undefined'.
Type 'null' is not assignable to type 'HTMLFormElement | undefined'.

First, it's necessary to ensure that e.target is not null:

Option 1

You can use the as keyword to recast e.target to a specific type. However, if you recast it as EventTarget, an error will continue to occur:

```
const data = new FormData(e.target as EventTarget);  
// red squiggly line under e.target as EventTarget
```

The error message states that the argument of type EventTarget is not assignable to the parameter of type HTMLFormElement:

```
// hovering over e.target shows:  
Argument of type 'EventTarget' is not assignable to parameter of type 'HTMLFormElement'.
```

Since you know that the code works at runtime and has tests covering it, you can force e.target to be of type HTMLFormElement:

```
const data = new FormData(e.target as HTMLFormElement);
```

Optionally, you can create a new variable, target, and assign the casted value to it:

```
const target = e.target as HTMLFormElement;  
const data = new FormData(target);
```

Either way, this change resolves the error, `target` is now inferred as an `HTMLFormElement`, and the code functions as expected.

Option 2

A quicker solution would be to use `as any` for the `e.target` variable to tell TypeScript that you don't care about the type of the variable:

```
const data = new FormData(e.target as any);
```

While using `as any` can get you past the error message more quickly, it does have its drawbacks.

For example, you wouldn't be able to leverage autocompletion or have type checking for other `e.target` properties that would come from the `HTMLFormElement` type.

When faced with a situation like this, it's better to use the most specific `as` assertion you can. This communicates that you have a clear understanding of what `e.target` is not only to TypeScript but also to other developers who might read your code.

Exercise 11-2: Solving Issues with Assertions

Here we'll revisit a previous exercise but solve it in a different way.

The `findUsersByName` function takes in some `searchParams` as its first argument, where `name` is an optional string property. The second argument is `users`, which is an array of objects with `id` and `name` properties:

```
const findUsersByName = (  
  searchParams: {name?: string},  
  users: {  
    id: string;  
    name: string;  
  }[],  
) => {
```

```
    if (searchParams.name) {
        return users.filter((user) => user.name.includes(searchP
arams.name));
    }
    // red squiggly line under searchParams.name

    return users;
};

// hovering over searchParams.name shows:
Argument of type 'string | undefined' is not assignable to p
arameter of type 'string'.
```

If `searchParams.name` is defined, you want to filter the `users` array using this name.

However, this currently has an error:

```
Argument of type 'string | undefined' is not assignable to p
arameter of type 'string'.
  Type 'undefined' is not assignable to type 'string'.
```

Your challenge is to adjust the code so that the error disappears.

We'll give you a clue: There are two “casting” solutions, which use the tricks we've covered in this chapter, but there is also one non-casting solution. Can you find them all?

See <https://totalts.link/essentials-11-2/>.

Solution

In the `findUsersByName` function, TypeScript is complaining about `searchParams.name`. It's not sure that it's a string in the `filter` function, and `.includes` expects a string:

```
if (searchParams.name) {
    return users.filter((user) => user.name.includes(searchPar
ams.name));
}
```

The reason why is that it doesn't know that the `.filter` function will be called synchronously. The function could be registered as an event handler and called later, by which time `search.Params.name` might be undefined.

The way to solve this *without* an assertion is to extract `searchParams.name` into a variable and perform the check against that:

```
const name = searchParams.name;
if (name) {
  return users.filter((user) => user.name.includes(name));
}
```

Since `const` can't be reassigned, TypeScript knows that `name` is a string in the filter callback. However, finding this solution is not obvious. In production code, you would be much more likely to use an assertion. Here are two ways to do it:

Option 1

One way to solve this is to add as `string` to `searchParams.name`:

```
// In findUsersByName function
return users.filter((user) => user.name.includes(searchParams.name as string));
```

This removes undefined, and it's now just a string.

Option 2

Another way to solve this is to add a non-null assertion to `searchParams.name`. This is done by adding a `!` postfix operator to the property you are trying to access:

```
// In findUsersByName function
return users.filter((user) => user.name.includes(searchParams.name!));
```

The `!` operator tells TypeScript to remove any `null` or `undefined` types from the variable. This would leave you with just `string`.

Both of these solutions will remove the error and allow the code to work as expected. But neither protects you against the insidious `get` function that returns `string | undefined` at random.

Exercise 11-3: Enforcing a Valid Configuration

You're back to the `configurations` object that includes `development`, `production`, and `staging`. Each of these members contains specific settings relevant to its environment:

```
const configurations = {
  development: {
    apiUrl: "http://localhost:8080",
    timeout: 5000,
  },
  production: {
    apiUrl: "https://api.example.com",
    timeout: 10000,
  },
  staging: {
    apiUrl: "https://staging.example.com",
    timeout: 8000,
    // @ts-expect-error // red squiggly line under @ts-expec
    t-error
    notAllowed: true,
  },
};
```

You also have an `Environment` type along with a passing test case that checks if `Environment` is equal to `"development" | "production" | "staging"`:

```
type Environment = keyof typeof configurations;

type test = Expect<
  Equal<Environment, "development" | "production" | "staging">
>;
```

Even though the test case passes, you have an error in the staging object in configurations. You're expecting an error on `notAllowed: true`, but the `@ts-expect-error` directive is not working because TypeScript is not recognizing that `notAllowed` is not allowed.

Your task is to determine an appropriate way to annotate your configurations object to retain accurate Environment inference from it while simultaneously throwing an error for members that are not allowed. Hint: Consider using a helper type that allows you to specify a data shape.

See <https://totalts.link/essentials-11-3/>.

Solution

The first step is to determine the structure of the configurations object.

In this case, it makes sense for it to be a Record where the keys will be string and the values will be an object with `apiBaseUrl` and `timeout` properties:

```
const configurations: Record<
  string,
  {
    apiBaseUrl: string;
    timeout: number
  }
> = {
  . . .
```

This change makes the `@ts-expect-error` directive work as expected, but you now have an error related to the Environment type not being inferred correctly:

```
type Environment = keyof typeof configurations;

// hovering over Environment shows:
type Environment = string

// The test now fails:
type test = Expect<
  Equal<Environment, "development" | "production" | "staging"
```

```
g">  
// red squiggly line under Equal<>
```

You need to make sure that configurations is still being inferred as its type, while also type checking the thing being passed to it.

This is the perfect application for the satisfies keyword.

Instead of annotating the configurations object as a Record, we'll use the satisfies keyword for the type constraint:

```
const configurations = {  
  development: {  
    apiUrl: "http://localhost:8080",  
    timeout: 5000,  
  },  
  production: {  
    apiUrl: "https://api.example.com",  
    timeout: 10000,  
  },  
  staging: {  
    apiUrl: "https://staging.example.com",  
    timeout: 8000,  
    // @ts-expect-error  
    notAllowed: true,  
  },  
} satisfies Record<  
  string,  
  {  
    apiUrl: string;  
    timeout: number;  
  }  
>;
```

This allows you to specify that the values you pass to your configuration object must adhere to the criteria defined in the type, while still allowing the type system to infer the correct types for your development, production, and staging environments.

Exercise 11-4: Variable Annotation vs. as vs. satisfies

In this exercise, you are going to examine three different types of setups in TypeScript: variable annotations, `as`, and `satisfies`.

The first scenario consists of declaring a `const obj` as an empty object and then applying the keys `a` and `b` to it. Using `as Record<string, number>`, you're expecting the type of `obj` or `a` to be a number:

```
const obj = {} as Record<string, number>;
obj.a = 1;
obj.b = 2;

type test = Expect<Equal<typeof obj.a, number>>;
```

In the second scenario, you have a `menuConfig` object that is assigned a `Record` type with `string` as the keys. The `menuConfig` is expected to have either an object containing `label` and `link` properties or an object with a `label` and `children` properties, which include arrays of objects that have `labels` and `links`:

```
const menuConfig: Record<
  string,
  | {
    label: string;
    link: string;
  }
  | {
    label: string;
    children: {
      label: string;
      link: string;
    }[];
  }
> = {
  home: {
    label: "Home",
    link: "/home",
  },
  services: {
    label: "Services",
```

```

    children: [
      {
        label: "Consulting",
        link: "/services/consulting",
      },
      {
        label: "Development",
        link: "/services/development",
      },
    ],
  },
};
type tests = [
  Expect<Equal<typeof menuConfig.home.label, string>>,
  Expect<
    Equal<
      typeof menuConfig.services.children, // red squiggly line under children
      {
        label: string;
        link: string;
      }[]
    >,
  >,
];

// hovering over children shows:
Property 'children' does not exist on type '{label: string; link: string;} | {label: string; children: {label: string; link: string;}[];}'.

```

In the third scenario, you're trying to use `satisfies` with `document.getElementById('app')` and `HTMLElement`, but it's resulting in errors:

```

const element = document.getElementById("app") satisfies HTMLElement;
// red squiggly line under satisfies

```

```
type test3 = Expect<Equal<typeof element, HTMLElement>>; //  
red squiggly line under Equal<>
```

Your job is to rearrange the annotations to correct these issues.
At the end of this exercise, you should have used `as`, `variable` annotations, and `satisfies` once each.

See <https://totalts.link/essentials-11-4/>.

Solutions

Let's work through the solutions for `satisfies`, `as`, and `variable` annotations:

Attempt 1

For the first scenario that uses a `Record`, the `satisfies` keyword won't work because you can't add dynamic members to an empty object:

```
const obj = {} satisfies Record<string, number>;  
obj.a = 1; // red squiggly line under a  
// hovering over a shows:  
Property 'a' does not exist on type '{}'.  

```

In the second scenario with the `menuConfig` object, you started with errors about `menuConfig.home` and `menuConfig.services` not existing on both members.

This is a clue that you need to use `satisfies` to make sure a value is checked without changing the inference:

```
const menuConfig = {  
  home: {  
    label: "Home",  
    link: "/home",  
  },  
  services: {  
    label: "Services",  
    children: [  
      {  
        label: "Consulting",  

```

```

    link: "/services/consulting",
  },
  {
    label: "Development",
    link: "/services/development",
  },
],
},
} satisfies Record<
  string,
  | {
    label: string;
    link: string;
  }
  | {
    label: string;
    children: {
      label: string;
      link: string;
    }[];
  }
>;

```

With this use of `satisfies`, the tests pass as expected.

Just to check the third scenario, `satisfies` doesn't work with `document.getElementById("app")` because it's inferred as `HTMLElement | null`:

```

const element = document.getElementById("app") satisfies HTML
  Element;
// red squiggly line under satisfies
// hovering over satisfies shows:
Type 'HTMLElement | null' does not satisfy the expected type
'HTMLElement'.
  Type 'null' is not assignable to type 'HTMLElement'.

```

Attempt 2

If you try to use a variable annotation in the third example, you get the same error as with `satisfies`:

```
const element: HTMLElement = document.getElementById("app");  
// red squiggly line under element  
  
// hovering over element shows:  
Type 'HTMLElement | null' is not assignable to type 'HTMLElement'.  
  Type 'null' is not assignable to type 'HTMLElement'.
```

By process of elimination, `as` is the correct choice for this scenario:

```
const element = document.getElementById("app") as HTMLElement;
```

You know that there is an HTML element with an id of `app`, and TypeScript doesn't. So, `as` serves as a useful hint to TypeScript, rather than a way to get around the compiler.

With this change, `element` is inferred as `HTMLElement`.

Attempt 3

This takes you to the first scenario, where using variable annotations is the correct choice:

```
const obj: Record<string, number> = {};
```

Note that you could use `as` here, but it's less safe and may lead to complications as we're forcing a value to be of a certain type. A variable annotation simply denotes a variable as that certain type and checks anything that's passed to it, which is the more correct, safer approach.

Generally, when you do have a choice between `as` or a variable annotation, opt for the variable annotation.

The key takeaway in this exercise is to grasp the mental model for when to use `as`, `satisfies`, and variable annotations:

- Use `as` when you want to tell TypeScript that you know more than it does.
- Use `satisfies` when you want to make sure a value is checked without changing the inference on that value.

The rest of the time, use variable annotations.

Exercise 11-5: Creating a Deeply Read-Only Object

Here you have a routes object:

```
const routes = {
  "/": {
    component: "Home",
  },
  "/about": {
    component: "About",
    // @ts-expect-error // red squiggly line under @ts-expect-error
    search: "?foo=bar",
  },
};

// @ts-expect-error // red squiggly line under @ts-expect-error
routes["/"].component = "About";
```

You want to restrict this object to contain only a certain set of keys that you specify. For instance, when adding a search field under the `/about` key, it should raise an error, but it currently doesn't. You also expect that once the routes object is created, it should not be able to be modified. For example, assigning `About` to the `Home` component should cause an error, but the `@ts-expect-error` directive tells us there is no problem.

In the tests, you expect that accessing properties of the routes object should return `Home` and `About` instead of interpreting these as literals, but those are both currently failing:

```
type tests = [  
    Expect<Equal<(typeof routes)["/"]["component"], "Home">  
>,  
    // red squiggly line under Equal<>  
    Expect<Equal<(typeof routes)["/about"]["component"], "About">>,  
    // red squiggly line under Equal<>  
];
```

Your task is to update the routes object typing so that all errors are resolved. This will require you to use `satisfies` as well as another annotation that ensures that the object is deeply read-only.

See <https://totalts.link/essentials-11-5/>.

Solution

You started with an `@ts-expect-error` directive in routes that was not working as expected.

Because you wanted a configuration object to be in a certain shape while still being able to access certain pieces of it, this was a perfect use case for `satisfies`.

At the end of the routes object, add a `satisfies` that will be a `Record` of string and an object with a `component` property that is a string:

```
const routes = {  
    "/": {  
        component: "Home",  
    },  
    "/about": {  
        component: "About",  
        // @ts-expect-error  
        search: "?foo=bar",  
    },  
} satisfies Record<  
    string,  
    {  
        component: string;  
    }  
>;
```

This change solves the issue of the `@ts-expect-error` directive in the `routes` object, but you still have an error related to the `routes` object not being read-only.

To address this, you need to apply `as const` to the `routes` object. This will make `routes` read-only and add the necessary immutability.

If you try adding `as const` after the `satisfies`, you'll get the following error:

```
A 'const' assertion can only be applied to references to enum members,
or string, number, boolean, array, or object literals.
```

In other words, `as const` can be applied only to certain values (primitives, arrays, or object literals) and not types.

The correct way to use `as const` is to put it before the `satisfies`:

```
const routes = {
  // Routes as before
} as const satisfies Record<
  string,
  {
    component: string;
  }
>;
```

Now your tests pass as expected.

This setup of combining `as const` and `satisfies` is ideal when you need a particular shape for a configuration object while enforcing immutability.

Summary

This chapter covered TypeScript's annotations and assertions, the tools for overriding the compiler's type inference when you know more than it does.

Variable annotations like `const config: Record<string, Color>` make the variable's type win, but TypeScript forgets the specific value

structure. The `satisfies` operator provides both type checking and precise inference, perfect for configuration objects.

Assertions like `as` and the non-null assertion `!` let you force TypeScript to treat values as different types. The `as` assertion has safeguards against unrelated type conversions, though these can be bypassed with `as unknown as TargetType`.

You learned about the error suppression directives include `@ts-expect-error` (expects an error and warns when fixed) and `@ts-ignore` (silently suppresses errors). The `@ts-nocheck` directive disables all checking for a file.

The key is recognizing when you truly know more than TypeScript versus when you should refactor your code to patterns TypeScript understands better. Use these tools judiciously—they're powerful but can mask real problems if overused.

12

THE WEIRD PARTS



Now that you have a good understanding of most of TypeScript’s features, it’s time to take it to the next level. In this chapter, you’ll see some of the more unusual and lesser-known parts of TypeScript. Variables with changing types, seemingly inconsistent object property treatments, and overlaps between the type and value worlds are all in store.

The Evolving any Type

While most of the time you want your types to remain static, it is possible to create variables whose types can change dynamically, as in JavaScript. This can be done with a technique called the “evolving any,” which takes advantage of how variables are declared and inferred when no type is specified.

To start, use `let` to declare the variable without a type, and TypeScript will infer it as `any`:

```
let myVar;
```

```
// hovering over myVar shows:  
let myVar: any;
```

Now the myVar variable will take on the inferred type of whatever is assigned to it.

For example, you can assign it a number and then call number methods like `toExponential()` on it. Later, you could change it to a string and convert it to all caps:

```
myVar = 659457206512;  
  
console.log(myVar.toExponential()); // Logs "6.59457206512e+11"  
  
myVar = "mf doom";  
  
console.log(myVar.toUpperCase()); // Logs "MF DOOM"
```

Here myVar started as any, then became a number, and then became a string, while allowing for the type's specific methods to be called along the way.

Crucially, this doesn't work like a regular any. If, after you assign a number to myVar, you try to call a method that doesn't exist on number, TypeScript will throw an error:

```
myVar = 659457206512;  
myVar.toUpperCase(); // Error: Property 'toUpperCase' does not exist on type 'number'.
```

This is like an advanced form of narrowing, where the type of the variable is narrowed based on the value assigned to it.

This technique of using the evolving any also works with arrays. When you declare an array without a specific type, you can push various types of elements to it:

```
const evolvingArray = []; // any[]
```

```
evolvingArray.push("abc"); // string[];
evolvingArray.push(123); // (string | number)[];
evolvingArray.push("do re mi"); // (string | number)[];
evolvingArray.push({easy: true}); // (string | number | {easy: boolean})[];
```

Even without specifying types, TypeScript is incredibly smart about picking up on your actions and the behavior you're pushing to evolving any types.

Excess Property Warnings

For beginners, a deeply confusing part of TypeScript is how it handles excess properties in objects. In many situations, TypeScript won't show errors you might expect when working with objects.

Let's create an Album interface that includes title and releaseYear properties:

```
interface Album {
  title: string;
  releaseYear: number;
}
```

Here you create an untyped rubberSoul object that includes an excess label property:

```
const rubberSoul = {
  title: "Rubber Soul",
  releaseYear: 1965,
  label: "Parlophone",
};
```

Now if you create a processAlbum function that accepts an Album and logs it, you can pass in the rubberSoul object without any issues:

```
const processAlbum = (album: Album) => console.log(album);
```



```
processAlbum(rubberSoul); // No error!
```

This seems strange! It would seem that TypeScript would show an error for the excess `label` property, but it doesn't.

Even more strangely, when you pass the object *inline*, you do get an error:

```
processAlbum({
  title: "Rubber Soul",
  releaseYear: 1965,
  label: "Parlophone", // red squiggly line under label
});

// hovering over label shows:
Object literal may only specify known properties, and 'label' does not exist in type 'Album'.
```

When calling `processAlbum` with an inline object that includes an additional property, TypeScript gives you an error that the object literal can specify only known properties. In this case, the `Album` type does not include a `label`.

No Excess Property Checks on Variables

In the first example, you assigned the album to a variable and then passed the variable into your function. In this situation, TypeScript won't check for excess properties.

The reason is that you might be using that variable in other places where the excess property is needed. TypeScript doesn't want to get in the way of that.

But when you inline the object, TypeScript knows that you're not going to use it elsewhere, so it checks for excess properties.

This can make you *think* that TypeScript cares about excess properties, but it doesn't. It checks for them only in certain situations.

This behavior can be frustrating when you misspell the name of an optional parameter. Imagine you misspell `timeout` as `timeOut`:

```
const fetch = (options: {url: string; timeout?: number}) =>
{
  // Implementation
};

const options = {
  url: "/",
  timeout: 1000,
};

fetch(options); // No error!
```

In this case, TypeScript won't show an error, and you won't get the runtime behavior you expect. The only way to source the error would be to provide a type annotation for the options object:

```
const options: {timeout?: number; url: string} = {
  url: "/",
  timeout: 1000, // red squiggly line under timeout
};
```

Now you're comparing an inline object to a type, and TypeScript will check for excess properties.

No Excess Property Checks When Comparing Functions

Another situation where TypeScript won't check for excess properties is when comparing functions.

Let's imagine you have a `remapAlbums` function that itself accepts a function:

```
const remapAlbums = (albums: Album[], remap: (album: Album)
=> Album) => {
  return albums.map(remap);
};
```

This function takes an array of Albums and a function that remaps each Album. This can be used to change the properties of each Album in the array.

You can call it like this to increment the `releaseYear` of each album by one:

```
const newAlbums = remapAlbums(albums, (album) => ({
  . . .album,
  releaseYear: album.releaseYear + 1,
})));
```

But as it turns out, you can pass an excess property to the return type of the function without TypeScript complaining:

```
const newAlbums = remapAlbums(albums, (album) => ({
  . . .album,
  releaseYear: album.releaseYear + 1,
  strangeProperty: "This is strange",
})));
```

Now your `newAlbums` array will have an excess `strangeProperty` property on each `Album` object, without TypeScript even knowing about it. It thinks that the return type of the function is `Album[]`, but at runtime it'll also return `strangeProperty`.

The way you'd get this "working" is to add a return type annotation to your inline function:

```
const newAlbums = remapAlbums(
  albums,
  (album): Album => ({
    . . .album,
    releaseYear: album.releaseYear + 1,
    strangeProperty: "This is strange", // red squiggly line
    under strangeProperty
  }),
);
```

This will cause TypeScript to show an error for the excess `strangeProperty` property.

This works because in this situation, you're comparing an inline object (the value you're returning) directly to a type. TypeScript will check for excess properties in this case.

Without a return type annotation, TypeScript ends up trying to compare two functions, and it doesn't really mind if a function returns too many properties.

Open vs. Closed Object Types

TypeScript, by default, treats all objects as *open*. At any time, it expects that other properties might be present on objects.

Other languages, like Flow, treat objects as *closed* by default. Flow is Meta's internal type system and by default requires objects to be exact (Meta's term for "closed"):

```
function method(obj: {foo: string}) {  
  /* . . . */  
}  
  
method({foo: "test", bar: 42}); // Error!
```

You can opt in to open (or inexact) objects in Flow with a . . . syntax:

```
function method(obj: {foo: string, . . .}) {  
  /* . . . */  
}  
  
method({foo: "test", bar: 42}); // No more error!
```

But Flow recommends you use closed objects by default. The Flow developers think that, especially when working with spread operators, it's better to err on the side of caution.

However, we think TypeScript's approach makes more sense. Open objects more closely reflect how JavaScript actually works. Given how dynamic JavaScript is, any type system claiming to represent it has to be relatively cautious about how "safe" it can truly be.

Fresh and Stale Objects

The TypeScript team aimed for a compromise. A dogmatic approach to open objects would be counterproductive. You would be able to add any properties to any object:

```
type Album = {
  title: string;
  artist: string;
};

const album: Album = {
  title: "The Dark Side of the Moon",
  artist: "Pink Floyd",
  // In this imaginary world, excess properties would be allowed!
  year: 1973,
  anythingElse: "Anarchy!!!",
};
```

So, the team decided to use two different behaviors based on an object's freshness. A *fresh* object is one created directly at the assignment site. This can be when declaring a variable

```
// Fresh object!
const album: Album = {
  title: "The Dark Side of the Moon",
  artist: "Pink Floyd",
};
```

or when passing to a function:

```
const processAlbum = (album: Album) => {};

// Fresh object!
processAlbum({
  title: "The Dark Side of the Moon",
  artist: "Pink Floyd",
});
```

When using fresh objects, TypeScript will warn you about any excess properties passed in. This helps you catch common errors, like passing the wrong object to a function.

A *stale* object is one assigned to a variable or returned from a function:

```
// Stale object!
const staleAlbum = {
  title: "The Dark Side of the Moon",
  artist: "Pink Floyd",
};

const album: Album = staleAlbum;
```

Stale objects are not checked for excess properties and are treated like open objects.

In our opinion, this approach is the best of both worlds. It's more lenient than Flow's closed object types, while giving the warnings you expect when declaring fresh objects.

Object Keys Are Loosely Typed

A consequence of TypeScript having open object types is that iterating over the keys of an object can be frustrating.

In JavaScript, calling `Object.keys` with an object will return an array of strings representing the keys:

```
const yetiSeason = {
  title: "Yeti Season",
  artist: "El Michels Affair",
  releaseYear: 2021,
};

const keys = Object.keys(yetiSeason); // string[]
```

In theory, you can then use those keys to access the values of the object:

```
keys.forEach((key) => {  
    console.log(yetiSeason[key]); // red squiggly line under k  
    ey  
});  
  
// hovering over key shows:  
Element implicitly has an 'any' type because expression of t  
ype 'string'  
can't be used to index type '{title: string; artist: string;  
releaseYear: number;}'.
```

But you're getting an error. TypeScript is telling you that you can't use `string` to access the properties of `yetiSeason`.

The only way this would work would be if `key` was typed as `'title' | 'artist' | 'releaseYear'`—in other words, as `keyof typeof yetiSeason`. But it's not: It's typed as `string`.

The reason for this comes back to `Object.keys`. It returns `string[]`, not `(keyof typeof obj)[]`:

```
const keys = Object.keys(yetiSeason); // string[]
```

By the way, the same behavior happens with `for...in` loops:

```
for (const key in yetiSeason) {  
    console.log(yetiSeason[key]); // red squiggly line under k  
    ey  
}
```

This is a consequence of TypeScript's open object types. TypeScript can't know the exact keys of an object at compile time, so it has to assume that there are unspecified keys on every object.

It's possible to work around this by using `keyof typeof obj` to get the keys of an object, but it's not always advisable. TypeScript's caution protects you from a nasty error:

```
const yetiSeason = {
  title: "Yeti Season",
  artist: "Yeti",
  releaseYear: 2024,
};

type Album = {
  title: string;
  artist: string;
};

const staleAlbum: Album = yetiSeason;

for (const key in staleAlbum) {
  // No error at compile time,
  // but errors at runtime with:
  // "staleAlbum[key].toLocaleLowerCase is not a function"
  console.log(staleAlbum[key as keyof Album].toLocaleLowerCase());
}
```

When you're enumerating the keys of an object, the safest thing for it to do is to treat them all as `string`. However, you'll find a few ways to work around that in this chapter's exercises.

The Empty Object Type

Another consequence of open object types is that the empty object type `{}` doesn't behave the way you might expect.

To set the stage, let's revisit the type assignability chart shown in [Figure 12-1](#).

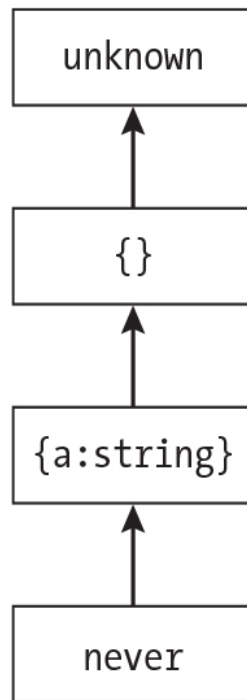


Figure 12-1: The type assignability chart for unknown

At the top of the chart is the unknown type, which can accept all other types. At the bottom is the never type, which no other type can be assigned to, but the never type itself can be assigned to any other type.

Between the never and unknown types is a universe of types. The empty object type `{}` has a unique place in this universe. Instead of representing an empty object, as you might imagine, it actually represents *anything that isn't null or undefined*.

This means it can accept a number of other types: `string`, `number`, `boolean`, `function`, `symbol`, and objects containing properties.

All of the following are valid assignments:

```
const coverArtist: {} = "Guy-Manuel De Homem-Christo";
const upcCode: {} = 724384260910;
const submit = (homework: {}) => console.log(homework);
submit("Oh Yeah");
```

In this example, all the variables are typed with empty objects. However, trying to call `submit` with `null` or `undefined` will result in a

TypeScript error:

```
submit(null); // red squiggly line under null

// hovering over null shows:
Argument of type 'undefined' is not assignable to parameter
of type '{}'.

```

This might feel a bit strange, but it makes sense when you remember that TypeScript's objects are *open*. Imagine your success function actually took an object containing message. TypeScript would be happy if you passed it an excess property:

```
const success = (response: {message: string}) =>
  console.log(response.message);

const messageWithExtra = {message: "Success!", extra: "This
  is extra"};

success(messageWithExtra); // No Error!

```

An empty object is really the “most open” object. Strings, numbers, and Booleans can all be considered objects in JavaScript. They each have properties and methods. So, TypeScript is happy to assign them to an empty object type.

The only things in JavaScript that don't have properties are `null` and `undefined`. Attempting to access a property on either of these will result in a runtime error. So, they don't fit the definition of an object in TypeScript.

When you consider this, the empty object type `{}` is a rather elegant solution to the problem of representing anything that isn't `null` or `undefined`.

The Type and Value Worlds

For the most part, TypeScript can be separated into two syntactical spaces: the type world and the value world. These two worlds can live side by side in the same line of code:

e here.

Did you mean 'typeof processAlbum'?

In this case, TypeScript suggests using `typeof processAlbum` instead of `processAlbum` to fix the error.

These boundaries are very clear, except in a few cases. Some entities can exist in both the type and value worlds.

Classes Can Cross Between Worlds

Consider this `Song` class that uses the shortcut of declaring properties in the constructor:

```
class Song {
  title: string;
  artist: string;

  constructor(title: string, artist: string) {
    this.title = title;
    this.artist = artist;
  }
}
```

You can use the `Song` class as a type—for instance, to type a function's parameter:

```
const playSong = (song: Song) =>
  console.log(`Playing ${song.title} by ${song.artist}`);
```

This type refers to the *instance* of the `Song` class, not the class itself:

```
const song1 = new Song("Song 1", "Artist 1");

playSong(song1);

playSong(Song); // red squiggly line under Song

// hovering over Song shows:
```

Argument of type 'typeof Song' is not assignable to parameter of type 'Song'.

In this case, TypeScript shows an error when you try to pass the Song class itself to the playSong function. This is because Song is a class, not an instance of the class.

So, classes exist in both the type and value worlds, and they represent an instance of the class when used as a type.

Enums Can Cross Between Worlds

Enums can also cross between worlds. Consider the following AlbumStatus enum and function that determines whether a discount is available:

```
enum AlbumStatus {
    NewRelease = 0,
    OnSale = 1,
    StaffPick = 2,
    Clearance = 3,
}

function logAlbumStatus(status: AlbumStatus) {
    if (status === AlbumStatus.NewRelease) {
        console.log("No discount available.");
    } else {
        console.log("Discounted price available.");
    }
}
```

You could use `typeof AlbumStatus` to refer to the entire structure of the enum itself:

```
function logAlbumStatus(status: typeof AlbumStatus) {
    // . . .Implementation
}
```

But then you'd need to pass in a structure matching the enum to the function:

```
logAlbumStatus({  
    NewRelease: 0,  
    OnSale: 1,  
    StaffPick: 2,  
    Clearance: 3,  
});
```

When used as a type, enums refer to the members of the enum, not the entire enum itself.

The this Keyword Can Cross Between Worlds

The `this` keyword can also cross between the type and value worlds. To illustrate, you'll work with this `Song` class that has a slightly different implementation than the one you saw earlier:

```
class Song {  
    playCount: number;  
  
    constructor(title: string) {  
        this.playCount = 0;  
    }  
  
    play(): this {  
        this.playCount += 1;  
        return this;  
    }  
}
```

In the `play` method, `this.playCount` uses `this` as a value to access the `this.playCount` property but also as a type to type the return value of the method.

When the `play` method returns `this`, in the type world it signifies that the method returns an instance of the current class.

This means you can create a new Song instance and chain multiple calls to the play method:

```
const earworm = new Song("Mambo No. 5").play().play().play();
```

This is a rare case where `this` and `typeof this` are the same thing. You could replace the `this` return type with `typeof this`, and the code would still work the same way:

```
class Song {  
  // . . .Implementation  
  
  play(): typeof this {  
    this.playCount += 1;  
    return this;  
  }  
}
```

Both point to the current instance of the class.

Naming Types and Values the Same

Finally, it's possible to name types and values the same thing. This can be useful when you want to use a type as a value or a value as a type.

Consider this Track object that has been created as a constant, and note the capital T:

```
export const Track = {  
  play: (title: string) => {  
    console.log(`Playing: ${title}`);  
  },  
  pause: () => {  
    console.log("Song paused");  
  },  
  stop: () => {  
    console.log("Song stopped");  
  }  
};
```

```
    },  
};
```

Next, you'll create a `Track` type that mirrors the `Track` constant:

```
export type Track = typeof Track;
```

You now have two entities being exported with the same name: One is a value, and the other is a type. This allows `Track` to serve as both when you go to use it.

Pretending you are in a different file, you can import `Track` and use it in a function that plays only “Mambo No. 5”:

```
import {Track} from "../other-file";  
  
const mamboNumberFivePlayer = (track: Track) => {  
    track.play("Mambo No. 5");  
};  
  
mamboNumberFivePlayer(Track);
```

Here, you've used `Track` as a type to type the `track` parameter and as a value to pass into the `mamboNumberFivePlayer` function.

Hovering over `{Track}` shows you that it is both a type and a value:

```
// hovering over {Track} shows:  
  
(alias) type Track = {  
    play: (title: string) => void;  
    pause: () => void;  
    stop: () => void;  
}  
  
(alias) const Track = {  
    play: (title: string) => void;  
    pause: () => void;  
    stop: () => void;  
}
```

As you can see, TypeScript has aliased `Track` to both the type and the value. This means it's available in both worlds.

A simple example would be to assert `Track` as `Track`:

```
console.log(Track as Track);  
//          ^^^^^^  ^^^^^^  
//          Value    type
```

Reading left to right, the first `Track` is a value, and the second `Track` is a type. TypeScript can seamlessly switch between the two, and this can be quite useful when you want to reuse types as values or values as types.

This double-duty functionality can prove quite helpful, especially when you have things that feel like types that you want to reuse elsewhere in your code.

You can even create your own enum-like functionality:

```
export const Direction = {  
  Up: "up",  
  Down: "down",  
  Left: "left",  
  Right: "right",  
} as const;  
  
export type Direction = (typeof Direction)[keyof typeof Direction];
```

You end up with a `Direction` type that represents `"up" | "down" | "left" | "right"` and a `Direction` constant that enumerates the values.

Using this in Functions

You've seen how `this` can be used in classes to refer to the current instance of the class. `this` can also be used in functions and objects, but there are some nuances to be aware of.

Here you have an object representing an album that includes a `sellAlbum` function written with the `function` keyword:

```
const solidAir = {
  title: "Solid Air",
  artist: "John Martyn",
  sales: 40000,
  price: 12.99,
  sellAlbum: function () {
    this.sales++;
    console.log(`${this.title} has sold ${this.sales} copie
s.`);
  },
};
```

Note that in the `sellAlbum` function, `this` is used to access the `sales` and `title` properties of the album object.

When you call the `sellAlbum` function, it will increment the `sales` property and log the expected message:

```
album.sellAlbum(); // Logs "Solid Air has sold 40001 copie
s."
```

This works because when declaring a function with the `function` keyword, `this` will always refer to the object that the function is a part of. Even when the function implementation is written outside the object, `this` will still refer to the object when the function is called:

```
function sellAlbum() {
  this.sales++;
  console.log(`${this.title} has sold ${this.sales} copies.
`);
}

const album = {
  title: "Solid Air",
  artist: "John Martyn",
  sales: 40000,
  price: 12.99,
  sellAlbum,
};
```

Here, `sellAlbum` is a function defined outside the album object but is included as a property of album.

While the `sellAlbum` function works, currently the `this.title` and `this.sales` properties are typed as `any`. You'll also get type errors to warn you about this. Fortunately, you can type `this` as a parameter in the function signature:

```
function sellAlbum(this: {title: string; sales: number}) {
  this.sales++;
  console.log(`${this.title} has sold ${this.sales} copies.
`);
}
```

Note that `this` is not a parameter that needs to be passed in when calling the function. It just refers to the object that the function is called on.

For the sake of demonstration, add the `sellAlbum` function to an album object as its only property:

```
const album = {
  sellAlbum,
};
```

When trying to call `album.sellAlbum()`, TypeScript gives you an error. The type checking works in an unintuitive way here; instead of checking `this` immediately, it checks it when the function is called:

```
album.sellAlbum(); // red squiggly line under album
// hovering over album shows:
The 'this' context of type '{sellAlbum: (this: {title: string; sales: number;}) => void;}' is not assignable to method's 'this' of type '{title: string; sales: number;}'.
```

In this case, you are told that the `title` and `sales` properties are missing. Adding them to the album object will resolve the issue:

```
const album = {  
  title: "Solid Air",  
  sales: 40000,  
  sellAlbum,  
};
```

Now when you call the `sellAlbum` function, TypeScript will know that `this` refers to an object with a `title` property of type `string` and a `sales` property of type `number`.

Arrow Functions Don't Support this

Unlike functions created with the `function` keyword, arrow functions can't be annotated with a `this` parameter. This matches JavaScript's behavior: You can't pass a `this` parameter into arrow functions in JavaScript.

The code shown here might look like a valid way to implement `sellAlbum` as an arrow function:

```
const sellAlbum = (this: {title: string; sales: number}) =>  
  {  
    this.sales++;  
    console.log(`${this.title} has sold ${this.sales} copies.  
  `);  
  };  
  
// red squiggly line under this parameter  
// hovering over this parameter shows:  
An arrow function cannot have a 'this' parameter.
```

However, as TypeScript points out, an arrow function can't have a `this` parameter. This is because arrow functions can't inherit `this` from the scope where they're called. Instead, they inherit `this` from the scope where they're *defined*. The only time an arrow function can access `this` is when it is defined in a class.

Function Assignability

While comparing functions might seem straightforward, there are some quirks. The first thing to be aware of is that when checking if a function is assignable to another function, not all function parameters need to be implemented. This can be a little surprising.

Imagine you're working in a music player application that includes a `handlePlayer` function that listens to a music player and calls a user-defined callback when certain events occur. The implementation of the function is unimportant; what matters is how it is called.

It should be able to accept a callback that has a single parameter for a filename:

```
handlePlayer((filename: string) => console.log(`Playing ${filename}`));
```

It should also handle a callback with a filename and volume:

```
handlePlayer((filename: string, volume: number) =>
  console.log(`Playing ${filename} at volume ${volume}`),
);
```

Finally, it should be able to handle a callback with a filename, volume, and bassBoost:

```
handlePlayer((filename: string, volume: number, bassBoost: boolean) => {
  console.log(`Playing ${filename} at volume ${volume} with
  bass boost on!`);
});
```

Assuming all of these examples are valid ways to call the function, how would you go about typing it?

It might be tempting to type `CallbackType` as a union of the three different function types:

```
type CallbackType =
  | ((filename: string) => void)
```

```
| ((filename: string, volume: number) => void)
| ((filename: string, volume: number, bassBoost: boolean)
=> void);

const handlePlayer = (callback: CallbackType) => {
  // Implementation
}
```

In this case, each member of the union is typed to match the function signatures. However, this would result in an implicit any error when calling handlePlayer with both the single and double parameter callbacks:

```
handlePlayer((filename) => console.log(`Playing ${filename}
`));
// red squiggly line under both instances of filename

handlePlayer((filename, volume) => console.log(`Playing ${fi
lename} at volume ${volume}`));
// red squiggly line under filename and volume

handlePlayer((filename, volume, bassBoost) => {
  console.log(`Playing ${filename} at volume ${volume} with
bass boost on!`);
}); // No errors
// hovering over filename shows:
Parameter 'filename' implicitly has an 'any' type.
```

This union of functions obviously serves your purposes, but it turns out there's a simple solution. You can actually remove the first two members of the union and include only the member with all three parameters:

```
type CallbackType = (
  filename: string,
  volume: number,
  bassBoost: boolean,
) => void;
```

By specifying only the types for the function call with the most parameters, the implicit any errors with the other two callback versions will disappear:

```
handlePlayer((filename) => console.log(`Playing ${filename}`)); // No error

handlePlayer((filename, volume) =>
  console.log(`Playing ${filename} at volume ${volume}`),
); // No error
```

This might seem weird, so let's break it down.

The callback passed to `handlePlayer` will be called with three arguments. If the callback accepts only one or two arguments, this is fine! No runtime bugs will be caused by the callback ignoring the arguments.

However, if the callback accepts more arguments than are passed, TypeScript would show an error. For example, let's add an extra parameter called `extra` to the `handlePlayer` function:

```
handlePlayer((filename, volume, bassBoost, extra) => {
  console.log(`Playing ${filename} at volume ${volume} with
  bass boost on!`);
}); // red squiggly line under filename, volume, bassBoost,
  extra
```

// hovering over filename shows:
Argument of type '(filename: any, volume: any, bassBoost: any, extra: any) => void'
is not assignable to parameter of type 'CallbackType'.
Target signature provides too few arguments. Expected 4 or more, but got 3.

As shown in the error messages, the `CallbackType` has only three properties, and adding `extra` brings the count to four. Since `extra` will never be passed to the callback, TypeScript shows the error.

But again, implementing fewer parameters than expected is fine. To illustrate this further, you can see this concept in action when calling `map` on

an array:

```
["macarena.mp3", "scatman.wma", "cotton-eye-joe.ogg"].map((file) =>
  file.toUpperCase(),
);
```

The `.map` method is always called with three arguments: the current element, the index, and the full array. But you don't have to use all of them—in this case, you care only about the `file` parameter.

Remember, just because a function can receive a certain number of parameters doesn't mean it has to use them all in its implementation.

Unions of Functions Intersect Parameters

When creating a union of functions, TypeScript will do something that might be unexpected: It will create an intersection of the parameters.

Consider this `formatterFunctions` object:

```
const formatterFunctions = {
  title: (album: {title: string}) => `Title: ${album.title}`,
  artist: (album: {artist: string}) => `Artist: ${album.artist}`,
  releaseYear: (album: {releaseYear: number}) =>
    `Release Year: ${album.releaseYear}`,
};
```

Each function in the `formatterFunctions` object accepts an album object with a specific property and returns a string.

Now let's create a `getAlbumInfo` function that accepts an album object and a key that will be used to call the appropriate function from the `formatterFunctions` object:

```
const getAlbumInfo = (album: any, key: keyof typeof formatterFunctions) => {
  const functionToCall = formatterFunctions[key];
```

```
    return functionToCall(album);  
};
```

You've annotated album as any for now, but let's take a moment to think: What should it be annotated with?

You can get a clue by hovering over functionToCall:

```
// hovering over functionToCall shows:  
const functionToCall:  
  | ((album: {title: string}) => string)  
  | ((album: {artist: string}) => string)  
  | ((album: {releaseYear: number}) => string);
```

functionToCall is being inferred as a union of the three different functions from the formatterFunctions object.

Surely, this means you should call it with a union of the three different types of album objects, right?

```
const getAlbumInfo = (  
  album: {title: string} | {artist: string} | {releaseYear:  
    number},  
  key: keyof typeof formatterFunctions,  
) => {  
  const functionToCall = formatterFunctions[key];  
  
  return functionToCall(album); // red squiggly line under a  
  lbum  
};  
  
// hovering over album shows:  
Argument of type '{title: string;} | {artist: string;} | {re  
leaseYear: number;}' is  
not assignable to parameter of type '{title: string;}  
& {artist: string;} & {releaseYear: number;}'.
```

You can see where you've gone wrong from the error. Instead of needing to be called with a union of the three different types of album

objects, `functionToCall` actually needs to be called with an *intersection* of them.

This makes sense. To satisfy every function, you need to provide an object that has all three properties: `title`, `artist`, and `releaseYear`. If you miss one of the properties, you'll fail to satisfy one of the functions.

So, you can provide a type that is an intersection of the three different types of album objects:

```
const getAlbumInfo = (  
  album: {title: string} & {artist: string} & {releaseYear:  
    number},  
  key: keyof typeof formatterFunctions,  
) => {  
  const functionToCall = formatterFunctions[key];  
  
  return functionToCall(album);  
};
```

This can itself be simplified to a single object type:

```
const getAlbumInfo = (  
  album: {title: string; artist: string; releaseYear: number},  
  key: keyof typeof formatterFunctions,  
) => {  
  const functionToCall = formatterFunctions[key];  
  
  return functionToCall(album);  
};
```

Now when you call `getAlbumInfo`, TypeScript will know that `album` is an object with `title`, `artist`, and `releaseYear` properties:

```
const formatted = getAlbumInfo(  
  {  
    title: "Solid Air",  
    artist: "John Martyn",  
    releaseYear: 1973,
```

```
},  
  "title",  
);
```

This situation is relatively easy to resolve because each parameter is compatible with the others. But when dealing with incompatible parameters, things can get a bit more complicated. You'll investigate that more in the exercises.

Exercise 12-1: Accepting Anything Except null and undefined

Here you have a function, `acceptAnythingExceptNullOrUndefined`, that hasn't been assigned a type annotation yet:

```
const acceptAnythingExceptNullOrUndefined = (input) => {};  
// red squiggly line under input
```

This function can be called with a variety of inputs: strings, numbers, Boolean expressions, symbols, objects, arrays, functions, regex, and an Error class instance:

```
acceptAnythingExceptNullOrUndefined("hello");  
acceptAnythingExceptNullOrUndefined(42);  
acceptAnythingExceptNullOrUndefined(true);  
acceptAnythingExceptNullOrUndefined(Symbol("foo"));  
acceptAnythingExceptNullOrUndefined({});  
acceptAnythingExceptNullOrUndefined([]);  
acceptAnythingExceptNullOrUndefined(() => {});  
acceptAnythingExceptNullOrUndefined(/foo/);  
acceptAnythingExceptNullOrUndefined(new Error("foo"));
```

None of these inputs should throw an error.

However, as the name of the function suggests, if you pass `null` or `undefined` to the function, you want it to throw an error:

```
acceptAnythingExceptNullOrUndefined(  
  // @ts-expect-error // red squiggly line under @ts-expect-  
  error  
    null,  
);  
acceptAnythingExceptNullOrUndefined(  
  // @ts-expect-error // red squiggly line under @ts-expect-  
  error  
    undefined,  
);
```

Your task is to add a type annotation to the `acceptAnythingExceptNullOrUndefined` function that will allow it to accept any value except `null` or `undefined`.

See <https://totalts.link/essentials-12-1/>.

Solution

The solution is to add an empty object annotation to the input parameter:

```
const acceptAnythingExceptNullOrUndefined = (input: {}) =>  
  {};
```

Since the input parameter is typed as an empty object, it will accept any value except `null` or `undefined`.

Exercise 12-2: Detecting Excess Properties in an Object

In this exercise, you're dealing with an options object along with the `FetchOptions` interface that specifies `url`, `method`, `headers`, and `body`:

```
interface FetchOptions {  
  url: string;  
  method?: string;  
  headers?: Record<string, string>;  
  body?: string;  
}  
  
const options = {
```

```
url: "/",
method: "GET",
headers: {
  "Content-Type": "application/json",
},
// @ts-expect-error // red squiggly line under @ts-expect-
error
search: new URLSearchParams({
  limit: "10",
}),
};
```

Note that the options object has an excess property search that is not specified in the FetchOptions interface, along with an @ts-expect-error directive that currently isn't working.

There is also a myFetch function, which accepts as its argument a FetchOptions typed object that doesn't have any errors when called with the options object:

```
const myFetch = async (options: FetchOptions) => {};

myFetch(options);
```

Your challenge is to determine why the @ts-expect-error directive doesn't work and restructure the code so that it does. Try to solve it multiple ways!

See <https://totalts.link/essentials-12-2/>.

Solutions

You aren't seeing an error in the starting point of the exercise because TypeScript's objects are open, not closed. The options object has all the required properties of the FetchOptions interface, so TypeScript considers it to be assignable to FetchOptions and doesn't care that additional properties were added.

Let's look at a few ways to make the excess property error work as expected:

Option 1

Adding a type annotation to the options object will result in an error for the excess property:

```
const options: FetchOptions = {
  url: "/",
  method: "GET",
  headers: {
    "Content-Type": "application/json",
  },
  // @ts-expect-error
  search: new URLSearchParams({
    limit: "10",
  }),
};
```

This triggers the excess property error because TypeScript is comparing an object literal to a type directly.

Option 2

Another way to trigger excess property checking is to add the satisfies keyword at the end of the variable declaration:

```
const options = {
  url: "/",
  method: "GET",
  headers: {
    "Content-Type": "application/json",
  },
  // @ts-expect-error
  search: new URLSearchParams({
    limit: "10",
  }),
} satisfies FetchOptions;
```

This works for the same reason.

Option 3

Finally, TypeScript will also check for excess properties if the variable is inlined into the function call:

```
const myFetch = async (options: FetchOptions) => {};  
  
myFetch({  
  url: "/",  
  method: "GET",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  // @ts-expect-error  
  search: new URLSearchParams({  
    limit: "10",  
  }),  
});
```

In this case, TypeScript will provide an error because it knows that `search` is not part of the `FetchOptions` interface.

Exercise 12-3: Detecting Excess Properties in a Function

Here's another exercise where TypeScript does not trigger excess property warnings where you might expect.

Here you have a `User` interface with `id` and `name` properties and a `users` array containing two user objects, "Waqas" and "Zain":

```
interface User {  
  id: number;  
  name: string;  
}  
  
const users = [  
  {  
    name: "Waqas",  
  },  
  {  
    name: "Zain",
```

```
    },  
  ];  
}
```

A `usersWithIds` variable is typed as an array of `Users`. A `map()` function is used to spread the user into a newly created object, with an `id` and an `age` of 30:

```
const usersWithIds: User[] = users.map((user, index) => ({  
  . . .user,  
  id: index,  
  // @ts-expect-error // red squiggly line under @ts-expect-  
  error  
  age: 30,  
})));
```

Despite TypeScript not expecting an `age` on `User`, it doesn't show an error, and at runtime the object will indeed contain an `age` property.

Your task is to determine why TypeScript isn't raising an error in this case and to find two different solutions to make it error appropriately when an unexpected property is added.

See <https://totalts.link/essentials-12-3/>.

Solutions

There are two solutions that we'll look at for this exercise:

Option 1

The first way to solve this issue is to annotate the `map` function.

In this case, the mapping function will take in a `user` that is an object with a `name` string and an `index` that will be a number.

Then, for the return type, we'll specify that it must return a `User` object:

```
const usersWithIds: User[] = users.map(  
  (user, index): User => ({  
    . . .user,  
    id: index,  
    // @ts-expect-error
```

```
    age: 30,  
  }},  
);
```

With this setup, there will be an error on age because it is not part of the User type.

Option 2

For this solution, you'll use the `satisfies` keyword to ensure that the object returned from the `map` function satisfies the User type:

```
const usersWithIds: User[] = users.map(  
  (user, index) =>  
    ({  
      . . .user,  
      id: index,  
      // @ts-expect-error  
      age: 30,  
    } satisfies User),  
);
```

TypeScript's excess property checks can sometimes lead to unexpected behavior, especially when dealing with function returns. To avoid this issue, always declare the types for variables that may contain excess properties or indicate the expected return types in your functions.

Exercise 12-4: Iterating over Objects

Consider an interface `User` with properties `id` and `name`, and a `printUser` function that accepts a `User` as its argument:

```
interface User {  
  id: number;  
  name: string;  
}  
  
function printUser(user: User) {}
```

In a test setup, you want to call the `printUser` function with an `id` of 1 and a name of "Waqas". The expectation is that the spy on `console.log` will first be called with 1 and then with "Waqas":

```
it("Should log all the keys of the user", () => {
  const consoleSpy = vitest.spyOn(console, "log");

  printUser({
    id: 1,
    name: "Waqas",
  });

  expect(consoleSpy).toHaveBeenCalledWith(1);
  expect(consoleSpy).toHaveBeenCalledWith("Waqas");
});
```

Your task is to implement the `printUser` function so that the test case passes as expected. Obviously, you could manually log the properties in the `printUser` function, but the goal here is to iterate over every property of the object.

Try to solve this exercise with a `for` loop for one solution and `Object.keys().forEach()` for another. For extra credit, try widening the type of the function parameter beyond `User` for a third solution. Remember, `Object.keys()` is typed to always return an array of strings.

See <https://totalts.link/essentials-12-4/>.

Solutions

Let's look at both looping approaches for the `printUser` function:

Option 1

The first approach is to use `Object.keys().forEach()` to iterate over the object keys. In the `forEach` callback, you'll access the value of the key by using the key variable:

```
function printUser(user: User) {
  Object.keys(user).forEach((key) => {
    console.log(user[key]); // red squiggly line under user
```

```
[key]
  });
}
```

This change will have the test case passing, but TypeScript raises a type error on `user[key]`:

```
// hovering over user[key] shows:
Element implicitly has an 'any' type because expression of t
ype 'string'
can't be used to index type 'User'.
```

The issue is that the `User` interface doesn't have an index signature. To get around the type error without modifying the `User` interface, you can use a type assertion on `key` to tell TypeScript that it is of type `keyof User`:

```
console.log(user[key as keyof User]);
```

The `keyof User` will be a union of the property names, such as `id` or `name`. And by using `as`, you are telling TypeScript that `key` is like a more precise string.

Option 2

The `for` loop approach is similar to the `Object.keys().forEach()` approach. You can use a `for` loop and pass in an object instead of a `user`:

```
function printUser(user: User) {
  for (const key in user) {
    console.log(user[key as keyof typeof user]);
  }
}
```

Like before, you need to use `keyof` `typeof` because of how TypeScript handles excess property checking.

Option 3

Another approach is to widen the type in the `printUser` function. In this case, you'll specify that the user being passed in is a `Record` with a string key and an unknown value.

In this case, the object being passed in doesn't have to be a user since you're just going to be mapping over every key that it receives:

```
function printUser(obj: Record<string, unknown>) {  
  Object.keys(obj).forEach((key) => {  
    console.log(obj[key]);  
  });  
}
```

This works on both the runtime and type levels without error.

Option 4

Another way to iterate over the object is to use `Object.values`:

```
function printUser(user: User) {  
  Object.values(user).forEach(console.log);  
}
```

This approach avoids the whole issue with the keys because `Object.values` will return an array of the values of the object. When this option is available, it's a nice way to avoid needing to deal with the issue of loosely typed keys.

When it comes to iterating over object keys, there are two main choices for handling this issue: You can either make key access slightly unsafe via `as keyof typeof` or loosen the type being indexed. Both approaches will work, so it's up to you to decide which one is best for your use case.

Exercise 12-5: Function Parameter Comparisons

Here you have a `listenToEvent` function that takes a callback that can handle a varying number of parameters based on how it's called. Currently the `CallbackType` is set to `unknown`:

```
type Event = "click" | "hover" | "scroll";

type CallbackType = unknown;

const listenToEvent = (callback: CallbackType) => {};
```

For example, you might want to call `listenToEvent` and pass a function that accepts no parameters—in this case, there's no need to worry about arguments at all:

```
listenToEvent(() => {});
```

Alternatively, you could pass a function that expects a single parameter, `event`:

```
listenToEvent((event) => {
  // red squiggly line under event
  type tests = [Expect<Equal<typeof event, Event>>]; // red
  squiggly line under Equal<>
});
```

Increasing the complexity, you could call it with an event, `x`, and `y`:

```
listenToEvent((event, x, y) => {
  // red squiggly line under event, x, and y
  type tests = [
    Expect<Equal<typeof event, Event>>, // red squiggly line
    under Equal<>
    Expect<Equal<typeof x, number>>, // red squiggly line un
    der Equal<>
    Expect<Equal<typeof y, number>>, // red squiggly line un
    der Equal<>
  ];
});
```

```
// hovering over event, x, or y shows:
Parameter implicitly has an 'any' type.
```

Finally, the function could take parameters event, x, y, and screenId:

```
listenToEvent((event, x, y, screenId) => {  
  // red squiggly line under event, x, y, and screenId  
  type tests = [  
    Expect<Equal<typeof event, Event>>, // red squiggly line  
under Equal<>  
    Expect<Equal<typeof x, number>>, // red squiggly line un  
der Equal<>  
    Expect<Equal<typeof y, number>>, // red squiggly line un  
der Equal<>  
    Expect<Equal<typeof screenId, number>>, // red squiggly  
line under Equal<>  
  ];  
});
```

In almost every case, TypeScript is giving you errors. Your task is to update the `CallbackType` to ensure that it can handle all of these cases.

See <https://totalts.link/essentials-12-5/>.

Solution

The solution is to type `CallbackType` as a function that specifies each of the possible parameters:

```
type CallbackType = (  
  event: Event,  
  x: number,  
  y: number,  
  screenId: number,  
) => void;
```

Recall that when implementing a function, it doesn't have to pay attention to every argument that has been passed in. However, it can't use a parameter that doesn't exist in its definition.

By typing `CallbackType` with each of the possible parameters, the test cases will pass regardless of how many parameters are passed in.

Exercise 12-6: Unions of Functions with Object Params

Here you are working with two functions: `logId` and `logName`. The `logId` function logs an `id` from an object to the console, while `logName` does the same with a `name`.

These functions are grouped into an array called `loggers`:

```
const logId = (obj: {id: string}) => {
  console.log(obj.id);
};

const logName = (obj: {name: string}) => {
  console.log(obj.name);
};

const loggers = [logId, logName];
```

In a `logAll` function, a currently untyped object is passed as a parameter. Each logger function from the `loggers` array is then invoked with this object:

```
const logAll = (obj) => {
  // red squiggly line under obj
  loggers.forEach((func) => func(obj));
};
```

Your task is to determine how to type the `obj` parameter to the `logAll` function. Look closely at the type signatures for the individual logger functions to understand what type this object should be.

See <https://totalts.link/essentials-12-6/>.

Solution

Hovering over the `loggers.forEach()`, you can see that `func` is a union between two different types of functions:

```
const logAll = (obj) => { // red squiggly line under obj
  loggers.forEach((func) => func(obj));
};
```

```
// hovering over forEach shows:
```

```
(parameter) func: ((obj: {  
  id: string;  
}) => void) | ((obj: {  
  name: string;  
}) => void)
```

One function takes in an id string, and the other takes in a name string. This makes sense because when you call the array, you don't know which one you're getting at which time.

You can use an intersection type with objects for id and name:

```
const logAll = (obj: {id: string} & {name: string}) => {  
  loggers.forEach((func) => func(obj));  
};
```

Alternatively, you could just pass in a regular object with id string and name string properties. As you've seen, having an extra property won't cause runtime issues, and TypeScript won't complain about it:

```
const logAll = (obj: {id: string; name: string}) => {  
  loggers.forEach((func) => func(obj));  
};
```

In both cases, the result is func being a function that contains all the possibilities of things being passed into it:

```
// hovering over (func) shows:  
(parameter) func: (obj: {  
  id: string;  
} & {  
  name: string;  
}) => void
```

This behavior makes sense, and this pattern is useful when working with functions that have varied requirements.

Exercise 12-7: Unions of Functions with Incompatible Parameters

Here you're working with an object called `objOfFunctions`, which contains functions keyed by `string`, `number`, or `boolean`. Each key has an associated function to process an input of that type:

```
const objOfFunctions = {
  string: (input: string) => input.toUpperCase(),
  number: (input: number) => input.toFixed(2),
  boolean: (input: boolean) => (input ? "true" : "false"),
};
```

A format function accepts an input that can be either a `string`, `number`, or `boolean`. From this input, it extracts the type via the regular `typeof` operator, but it asserts the operator to `string`, `number`, or `boolean`.

Here's how it looks:

```
const format = (input: string | number | boolean) => {
  // 'typeof' isn't smart enough to know that.
  // It can only be 'string', 'number', or 'boolean'.
  const inputType = typeof input as "string" | "number" | "boolean";
  const formatter = objOfFunctions[inputType];

  return formatter(input); // red squiggly line under input
};
```

The formatter that is extracted from `objOfFunctions` ends up typed as a union of functions. This happens because it can be any one of the functions that take either a `string`, `number`, or `boolean`:

```
// hovering over the formatter variable shows:
const formatter:
```

```
| ((input: string) => string)
| ((input: number) => string)
| ((input: boolean) => "true" | "false");
```

Currently there's an error on input in the return statement of the format function:

```
// hovering over input shows:
Argument of type 'string | number | boolean' is not assignable
to parameter of type 'never'.
```

Your challenge is to resolve this error on the type level, even though the code works at runtime. Try to use an assertion for one solution and a type guard for another.

A useful tidbit: any is not assignable to never.

See <https://totalts.link/essentials-12-7/>.

Solution

Hovering over the formatter function shows you that its input is typed as never because it's a union of incompatible types:

```
// hovering over formatter shows:
const formatter: (input: never) => string;
```

To fix the type-level issue, you can use the `as never` assertion to tell TypeScript that input is of type never:

```
// In the format function
return formatter(input as never);
```

This is a little unsafe, but you know from the runtime behavior that input will always be a string, number, or boolean.

Funnily enough, `as any` won't work here because any is not assignable to never:

```
// In the format function
return formatter(input as any); // red squiggly line under i
nput
// Argument of type 'any' is not assignable to parameter of
type 'never'.
```

Another way to solve this issue is to give up on your union of functions by narrowing down the type of input before calling formatter:

```
const format = (input: string | number | boolean) => {
  if (typeof input === "string") {
    return objOfFunctions.string(input);
  } else if (typeof input === "number") {
    return objOfFunctions.number(input);
  } else {
    return objOfFunctions.boolean(input);
  }
};
```

This solution is more verbose and won't compile down as nicely as `never`, but it will fix the error as expected.

Summary

This chapter discussed TypeScript's "weird" parts and found them to be less weird than expected.

You learned how evolving anys allow type safety when variables change type over the course of a program.

You learned how TypeScript's decision to use open objects ripples into object keys, excess property warnings, and the empty object type.

You investigated the type and value worlds and saw how enums, classes, and this can cross from one to the other.

Finally, you explored typing this and function assignability, observing how unions of functions behave.

We hope that this chapter feels mistitled. We hope that the features you looked at don't look weird but instead look like inevitable decisions of TypeScript's design.

PART V

UNDERSTANDING THE ENVIRONMENT

13

MODULES, SCRIPTS, AND DECLARATION FILES



In this chapter, you'll be diving deeper into modules, scripts, and declaration files. First, you'll look at how TypeScript figures out whether your code is a module or a script and how this distinction affects your application. Then, we'll talk about how declaration files—the ones that end in *.d.ts*—describe JavaScript code and add types to the global scope.

Understanding Modules and Scripts

TypeScript has two ways of understanding what a *.ts* file is. It can be treated either as a module, which contains `imports` and `exports`, or as a script, which executes in the global scope. Let's look at each of these a bit more closely.

Modules Have Local Scope

A *module* is an isolated piece of code that can be imported to other modules as needed. Modules have their own scope, meaning that variables,

functions, and types defined within a module are not accessible from other files unless they are explicitly exported.

Consider this *constants.ts* module that defines a `DEFAULT_VOLUME` constant:

```
const DEFAULT_VOLUME = 90;
```

Without being imported, the `DEFAULT_VOLUME` constant is not accessible from other files:

```
// In index.ts
console.log(DEFAULT_VOLUME); // red squiggly line under DEFAULT_VOLUME
```

To use the `DEFAULT_VOLUME` constant in the *index.ts* file, it must be imported from the *constants.ts* module:

```
// In index.ts
import {DEFAULT_VOLUME} from "./constants";

console.log(DEFAULT_VOLUME); // 90
```

TypeScript has a built-in understanding of modules and, by default, will treat any file that contains an `import` or `export` statement as a module.

Scripts Have Global Scope

Scripts, on the other hand, execute in the global scope. Any variables, functions, or types defined in a script file are accessible from anywhere in the project without the need for explicit imports. This behavior is similar to traditional JavaScript, where scripts are included in HTML files and executed in the global scope.

If a file does not contain any `import` or `export` statements, TypeScript will treat it as a script. If you remove the `export` keyword from the `DEFAULT_VOLUME` constant in the *constants.ts* file, it will be treated as a script:

```
// In constants.ts
const DEFAULT_VOLUME = 90;
```

Now you no longer need to import the `DEFAULT_VOLUME` constant in the *index.ts* file to use it:

```
// In index.ts

console.log(DEFAULT_VOLUME); // 90
```

This behavior might be surprising to you. Let's figure out why TypeScript does this.

TypeScript Guesses Which to Use

TypeScript is, at this point, pretty old. It's actually older than the standardized use of `import` and `export` statements in JavaScript. When TypeScript was first created, it was mostly used to create *scripts*, not modules.

So, TypeScript's default behavior is to *guess* whether your file is supposed to be treated like a module or a script. As you've seen, it does this by looking for `import` and `export` statements.

But whether your code is treated like a module or a script is not actually decided by TypeScript; it's decided by the environment in which the code executes.

Even in the browser, you can opt in to using modules by adding the `type="module"` attribute to your script tag:

```
<script type="module" src="index.js"></script>
```

This means your JavaScript file will be treated as a module. But remove the `type="module"` attribute, and your JavaScript file will be treated as a script.

TypeScript's default is relatively sensible, seeing as it can't know how your code will be executed. But these days, 99 percent of the code you'll be

writing will be in modules. This automatic detection can lead to frustrating situations.

Let's imagine you create a new TypeScript file, *utils.ts*, and add a name constant:

```
// In utils.ts
const name = "Alice"; // red squiggly line under name

// hovering over name shows:
Cannot redeclare block-scoped variable 'name'.
```

You'll be greeted with a surprising error telling you that you can't declare name because it's already been declared.

A curious way to fix this is to add an empty export statement at the end of the file:

```
const name = "Alice";

export {};
```

The error disappears. Why?

Let's use what you've already learned to figure this out. You don't have any import or export statements in *utils.ts*, so TypeScript treats it as a script. This means that name is declared in the global scope.

It turns out that in the DOM, there is already a global variable called name (<https://developer.mozilla.org/en-US/docs/Web/API/Window/name>). This lets you set targets for hyperlinks and forms. So, when TypeScript sees name in a script, it gives you an error because it thinks you're trying to redeclare the global name variable.

By adding the export {} statement, you're telling TypeScript that *utils.ts* is a module, and name is now scoped to the module, not the global scope.

This accidental collision is a good example of why it's a good idea to treat all your files as modules. Fortunately, TypeScript gives you a way to do it.

Forcing Modules with moduleDetection

TypeScript's `moduleDetection` configuration setting determines how functions and variables are scoped in your project. There are three different options available: `auto`, `force`, and `legacy`.

By default, it's set to `auto`, which corresponds to the behavior you saw in the previous section. The `force` setting will treat all files as modules, regardless of the presence of `import` or `export` statements. `legacy` can be safely ignored, as it's used only for compatibility with older versions of TypeScript.

Updating *tsconfig.json* to specify `moduleDetection` to `force` is straightforward:

```
// tsconfig.json
{
  "compilerOptions": {
    // . . .Other options. . .
    "moduleDetection": "force"
  }
}
```

After adding this setting to the configuration, all files in the project will be treated as modules, so you will need to use `import` and `export` statements to access functions and variables across files. This helps align your development environment more closely with real-world scenarios while reducing unexpected errors.

Declaration Files

Declaration files in TypeScript have a special extension: *.d.ts*. These files are used for two main purposes in TypeScript: describing JavaScript code and adding types to the global scope. You'll explore both in this section.

Declaration Files Describe JavaScript

Let's say part of the codebase is written in JavaScript, and you want to keep it that way. You have a *musicPlayer.js* file that exports a `playTrack` function:

```
// musicPlayer.js

export const playTrack = (track) => {
  // Complicated logic to play the track. . .
  console.log(`Playing: ${track.title}`);
};
```

If you try to import the *musicPlayer.ts* file into a TypeScript file, you'll get an error:

```
// In app.ts

import {playTrack} from "../musicPlayer"; // red squiggly line under ./musicPlayer
// hovering over ./musicPlayer shows:
Could not find a declaration file for module './musicPlayer'.
```

In the example *app.js* file, you get an error that a declaration file can't be found. In other words, TypeScript doesn't have any type information for the *musicPlayer.js* file. To fix this, you can create a declaration file with the same name as the JavaScript file, but with a *.d.ts* extension:

```
// musicPlayer.d.ts
export function playTrack(track: {
  title: string;
  artist: string;
  duration: number;
}): void;
```

It's important to notice that this new *musicPlayer.d.ts* file doesn't contain any implementation code. It describes only the types of the functions and variables in the JavaScript file.

Now when you import the *musicPlayer.js* file into the *app.ts* TypeScript file, the error will be resolved, and you can use the *playTrack* function as expected:

```
// In app.ts

import {playTrack} from "../musicPlayer";

const track = {
  title: "Otha Fish",
  artist: "The Pharcyde",
  duration: 322,
};

playTrack(track);
```

It's also possible for types and interfaces to be declared and exported in declaration files. Here's how you could refactor the *musicPlayer.d.ts* file to demonstrate:

```
// In musicPlayer.d.ts
export interface Track {
  title: string;
  artist: string;
  duration: number;
}

export function playTrack(track: Track): void;
```

Now that the *Track* interface is being exported, it can be imported and used in other TypeScript files:

```
// In app.ts

import {Track, playTrack} from "../musicPlayer";
```

It's important to note that declaration files are not checked against the JavaScript files they describe. You can easily make a mistake in your declaration file, such as changing *playTrack* to *playTRACK*, and TypeScript won't complain.

So, describing JavaScript files by hand can be error-prone and is not usually recommended.

Declaration Files Can Add to the Global Scope

Just like regular TypeScript files, declaration files can be treated as either modules or scripts based on whether the `export` keyword is used. In the previous example, *musicPlayer.d.ts* is treated as a module because it includes the `export` keyword.

This means that without an `export`, declaration files can be used to add types to the global scope. Even setting `moduleDetection` to `force` won't change this behavior; `moduleDetection` is always set to `auto` for *.d.ts* files.

For example, you could create an `Album` type in a *global.d.ts* file that you want to be used across the entire project:

```
// In global.d.ts

type Album = {
  title: string;
  artist: string;
  releaseDate: string;
};
```

Now the `Album` type is available globally and can be used in any TypeScript file without importing it, but there are implications that will be discussed later in this chapter.

Declaration Files Can't Contain Implementations

Earlier you saw that a declaration file can contain only descriptions of the types of functions and variables. But what would happen if you tried to write normal TypeScript in a *.d.ts* file?

```
// musicPlayer.d.ts

export function playTrack(track: {
  title: string;
  artist: string;
  duration: number;
```

```
    }) {  
      // red squiggly line under {  
      console.log(`Playing: ${track.title}`);  
    }  
  
    // hovering over {shows:  
    An implementation cannot be declared in ambient contexts.
```

(It's worth noting here that this error occurs only when you have `skipLibCheck: false` in your TypeScript configuration file. We'll discuss this in more detail later in this chapter.)

Declaration files completely disappear at runtime, so they can't contain any code that would be executed. In this case, there's an error in the `playTrack` function telling you that an implementation can't be declared in an ambient context. TypeScript uses "ambient" to mean "without implementation" (<https://github.com/Microsoft/TypeScript-Handbook/issues/180#issuecomment-195446760>). Since declaration files can't contain implementations, everything in them is considered "ambient." The big takeaway here is that declaration files can't contain implementations.

The declare Keyword

The `declare` keyword lets you define ambient values in TypeScript. It can be used to declare variables, define a global scope with `declare global`, or augment module types with `declare module`. The `declare` keyword can also be used to define values that don't have an implementation. This can be useful in a variety of ways. Let's look at how it can help with typing.

Typing Global Variables

Let's say you have a global variable `MUSIC_API` that is available in the environment via a script tag:

```
<script src="/music-api.js"></script>
```

You can create a *musicApi.d.ts* declaration file and use `declare const` to describe it:

```
// In musicApi.d.ts

type Album = {
  title: string;
  artist: string;
  releaseDate: string;
};

declare const ALBUM_API: {
  getAlbumInfo(upc: string): Promise<Album>;
  searchAlbums(query: string): Promise<Album[]>;
};
```

Because the file doesn't include any imports or exports, this file is treated as a script. This means that the `ALBUM_API` variable is now available globally in your project.

Scoping Global Variables to One File

The `declare` keyword can also limit scope to just a single file. For example, if you didn't want `MUSIC_API` to be globally available, you could move the `declare const` statement out of the declaration file and into *musicUtils.ts*:

```
// In musicUtils.ts

type Album = {
  title: string;
  artist: string;
  releaseDate: string;
};

declare const ALBUM_API: {
  getAlbumInfo(upc: string): Promise<Album>;
  searchAlbums(query: string): Promise<Album[]>;
};

export function getAlbumTitle(upc: string) {
  return ALBUM_API.getAlbumInfo(upc).then((album) => album.title);
}
```

The `declare` keyword defines the value within the scope it's currently in. So, because you're now in a module (due to the `export` statement), `ALBUM_API` is scoped to the *musicUtils.ts* module.

More Ways to declare

You might have noticed that you used `declare const` in the previous examples. But you can also use `declare` for other things, like `var`, `let`, and `function`. Here are some examples of the syntax:

```
declare const MY_CONSTANT: number;
declare var MY_VARIABLE: string;
declare let MY_LET: boolean;
declare function myFunction(): void;
```

This is not an exhaustive list. You can use `declare` on enums, classes, class fields, and more. But all of them do the same thing: They declare a value without an implementation.

declare global

Using `declare global` lets you add things to the global scope from within modules. This can be useful when you want to colocate global types with the code that uses them. Let's try wrapping the `declare const` statement in a `declare global` block:

```
// In musicUtils.ts
declare global {
  declare const ALBUM_API: { // red squiggly line under declare
    getAlbumInfo(upc: string): Promise<Album>;
    searchAlbums(query: string): Promise<Album[]>;
  };
}

// hovering over declare shows:
A 'declare' modifier cannot be used in an already ambient context.
```

TypeScript shows us an error under the `declare` for the `ALBUM_API` that says you can't use `declare` in an already ambient context. Since the `declare global` block is already ambient, you can just remove the `declare` keyword in favor of using `const`:

```
// In musicUtils.ts

declare global {
  const ALBUM_API: {
    getAlbumInfo(upc: string): Promise<Album>;
    searchAlbums(query: string): Promise<Album[]>;
  };
}
```

Now the error is gone, and the `ALBUM_API` variable has been put into the global scope. This means it can be accessed from any file, not just in *musicUtils.ts*.

declare module

There are some situations where you need to declare types for a module that either doesn't have type definitions or is not included in the project directly. In these cases, you can use the `declare module` syntax to define types for the module.

For example, say you are working with a `duration-utils` module that doesn't have type definitions.

The first step would be to create a new file named *duration-utils.d.ts*. Then, at the top of the file, the `declare module` syntax is used to define the types for the module:

```
// In duration-utils.d.ts
declare module "duration-utils" {
  export function formatDuration(seconds: number): string;
}
```

In this example, `export` is used to define what is being exported from the module. Like before, you are not including any implementation code in

the *.d.ts* file; it's just the types that are being declared.

Once the *duration-utils.d.ts* file is created, the module can be imported and used as usual:

```
// In an app.ts file
import {formatDuration} from "duration-utils";

const formattedTime = formatDuration(309);
```

Just like normal declaration files, the types you add are not checked against the actual module, so it's important to keep them up to date.

Module Augmentation vs. Module Overriding

When using `declare module`, you can either augment an existing module or override it completely. Augmenting a module means appending new types to an existing module. Overriding a module means replacing the existing types with new ones.

Whether you augment or override depends on whether you're in a module or a script.

If you're in a module, `declare module` will augment the targeted module. For instance, you can add a new type to the already existing `express` module:

```
// In express.d.ts
declare module "express" {
  export interface MyType {
    hello: string;
  }
}

export {}; // Adding an export turns this .d.ts file into a
module.
```

In this example, a new `MyType` interface has been declared. Note the `export {}` statement at the bottom, which tells TypeScript that it should treat *express.d.ts* as a module. Now, across your project, you can import `MyType` from the `express` module:

```
// anywhere.ts
import {MyType} from "express";
```

Note that in this case you don't actually need to put this in a declaration file. You can get exactly the same behavior by changing *express.d.ts* to *express.ts*.

This example is a little bit silly; there's no real use case for adding your own type to a module. But you'll see later that augmenting the types of modules can be extremely useful.

If you remove the `export {}` statement from the *express.d.ts* file, it will be treated as a script:

```
// In express.d.ts

declare module "express" {
  export interface MyType {
    hello: string;
  }
}
```

With just that change, you've completely overridden the `express` module! This means the `express` module no longer has any exports except for `MyType`:

```
// anywhere.ts
import {express} from "express"; // red squiggly line under
"express"
```

Just like module augmentation, you can get the same behavior by changing *express.d.ts* to *express.ts* (assuming `moduleDetection` is set to `auto` in the *tsconfig.json* file).

Overriding is occasionally useful when you want to completely replace the types of a module, perhaps when a third-party library has incorrect types. The presence or absence of an `export` statement can radically change the behavior of `declare module`.

Declaration Files You Don't Control

You might think that declaration files are a relatively niche feature of TypeScript. But in every project you create, you're likely using hundreds of declaration files. They either ship with libraries or come bundled with TypeScript itself.

TypeScript's Types

Whenever you use TypeScript, you're also using JavaScript. JavaScript has many built-in constants, functions, and objects that TypeScript needs to know about. Array methods are a classic example:

```
const numbers = [1, 2, 3];

numbers.map((n) => n * 2);
```

Let's step back for a minute. How does TypeScript know that `.map` exists on an array? How does it know that `.map` exists but `.transform` doesn't?

As it turns out, TypeScript ships with a bunch of declaration files that describe the JavaScript environment. In VS Code, using the Go to Definition feature on `.map` will show you where that is:

```
// In lib.es5.d.ts

interface Array<T> {
  // . . . Other methods . . .
  map<U>(
    callbackfn: (value: T, index: number, array: T[]) => U,
    thisArg?: any,
  ): U[];
}
```

We've ended up in a file called *lib.es5.d.ts*. This file is part of TypeScript and describes what JavaScript looked like in ES5, the version of JavaScript from 2009 when `.map` was introduced.

Let's look at another example, this time with the `.replaceAll` string method:

```
const str = "hello world";

str.replaceAll("hello", "goodbye");
```

Doing a “go to definition” on `.replaceAll` will take you to a file called *lib.es2021.string.d.ts*. This file describes the string methods that were introduced in ES2021.

Looking at the code in *node_modules/typescript/lib*, you'll see dozens of declaration files that describe the JavaScript environment.

Understanding how to navigate these declaration files can be useful for fixing type errors. Take a few minutes to explore what's in *lib.es5.d.ts* by using Go to Definition to navigate around.

Choosing Your JavaScript Version with `lib`

The `lib` setting in *tsconfig.json* lets you choose which *.d.ts* files are included in your project. Choosing `es2022` will give you all the JavaScript features up to ES2022. Choosing `es5` will give you all the features up to ES5:

```
// In tsconfig.json
{
  "compilerOptions": {
    "lib": ["es2022"]
  }
}
```

By default, `lib` inherits from the `target` setting, which we'll cover in [Chapter 14](#).

DOM Types

Another set of declaration files that ship with TypeScript are the DOM types. These describe the browser environment and include types for `document`, `window`, and all the other browser globals:

```
document.querySelector("h1");
```

If you use Go to Definition on `document`, you'll end up in a file called *lib.dom.d.ts*:

```
// In lib.dom.d.ts
declare var document: Document;
```

This file declares the `document` variable as type `Document`, using the `declare` keyword you saw earlier. To include these in your project, you can specify them in the `lib` setting, along with the JavaScript version:

```
// In tsconfig.json
{
  "compilerOptions": {
    "lib": ["es2022", "dom", "dom.iterable"]
  }
}
```

`dom.iterable` includes the types for the iterable DOM collections, like `NodeList`. If you don't specify `lib`, TypeScript will include `dom` by default alongside the JavaScript version chosen in `target`:

```
// In tsconfig.json
{
  "compilerOptions": {
    "target": "es2022"
    // "lib": ["es2022", "dom", "dom.iterable"] is implied
  }
}
```

Different browsers support different features. A quick browse of <https://caniuse.com> will show how patchy browser support can be for certain features. But TypeScript ships only one set of DOM types. So, how does it know what to include?

TypeScript's policy is that if a feature is supported in two major browsers, it's included in the DOM types. This is a good balance between including everything and including nothing. At the time of writing, the DOM types are more than 28,000 lines long, but understanding what's in there over a period of time can be useful.

Types That Ship with Libraries

When you install a library with npm, you're downloading JavaScript to your filesystem. To make that JavaScript work with TypeScript, authors will often include declaration files alongside them.

For example, we'll discuss Zod, a popular library that allows for validating data at runtime.

After running the installation command `pnpm i zod`, a new *zod* subdirectory will be created in *node_modules*. In it, you'll find a *package.json* file with a *types* key that points to the type definitions for the library:

```
// In node_modules/zod/package.json
{
  "types": "index.d.ts",
  // Other keys. . .
}
```

In *index.d.ts* are the type definitions for the zod library:

```
// In node_modules/zod/index.d.ts
import * as z from "./external";
export * from "./external";
export {z};
export default z;
```

Additionally, every *.js* file in the *lib* folder has a corresponding *.d.ts* file that contains the type definitions for the JavaScript code.

Just like the DOM types, you can use Go to Definition to explore the types that ship with libraries. Understanding these types can help you use the library more effectively.

DefinitelyTyped

Not every library bundles *.d.ts* files alongside the JavaScript you download. This was a big issue in TypeScript's early days, when most open source packages weren't written in TypeScript.

The DefinitelyTyped GitHub repository (<https://github.com/DefinitelyTyped/DefinitelyTyped>) was built to house high-quality type definitions for numerous popular JavaScript libraries that didn't ship definitions of their own. It's now one of the largest open source repositories on GitHub.

By installing a package with `@types/*` and your library as a dev dependency, you can add type definitions that TypeScript will be able to use immediately.

For example, say you're using the `diff` library to check for the difference between two strings:

```
import Diff from "diff"; // red squiggly line under "diff"

const message1 = "Now playing: 'Run Run Run'";
const message2 = "Now playing: 'Bye Bye Bye'";

const differences = Diff.diffChars(message1, message2);
```

TypeScript reports an error underneath the `import` statement because it can't find type definitions, even though the library is installed more than 40 million times a week from NPM:

```
// hovering over "diff" shows:
Could not find a declaration file for module 'diff'. Try `npm
install --save-dev
@types/diff` if it exists or add a new declaration (.d.ts) f
ile containing
`declare module 'diff';`
```

Since you're using `pnpm` instead of `npm`, the installation command looks like this:

```
pnpm i -D @types/diff
```

Once the type definitions from DefinitelyTyped are installed, TypeScript will recognize the `diff` library and provide type checking and autocompletion for it:

```
// hovering over differences shows:  
const differences: Diff.Change[];
```

This is a great solution for libraries that haven't been updated in a while or for more commonly used libraries (like, say, React) that don't ship with type definitions.

The skipLibCheck Config Option

As you've seen, your project can contain hundreds of declaration files. By default, TypeScript considers these files part of your project. So, it checks them for type errors every single time.

This can result in extremely frustrating situations where a type error in a third-party library can prevent your project from compiling. To avoid this, TypeScript has a `skipLibCheck` setting. When set to `true`, TypeScript will skip checking declaration files for type errors:

```
{  
  "compilerOptions": {  
    "skipLibCheck": true  
  }  
}
```

This is a must-have in any TypeScript project because of the sheer number of declaration files that are included. Adding this setting speeds up compilation and prevents errors from code in your `node_modules`.

However, there is one enormous downside to the `skipLibCheck` option: It doesn't just skip declaration files in `node_modules`; it skips *all* declaration files.

This means that if you make a mistake authoring a declaration file, TypeScript won't catch it. This can lead to bugs that are difficult to track

down.

There's a trade-off here: On the one hand, `skipLibCheck` is a must-have because of the danger of incorrect third-party declaration files, but on the other, it makes it much harder to author your own declaration files.

Authoring Declaration Files

Now you know how to use declaration files and their downsides (thanks to `skipLibCheck`), so let's look at their use cases.

The most common use for declaration files is describing the global scope of your project. You've seen how you can use `declare const` in a script file to add a global variable. You can also use declaration merging, a feature you learned about earlier, to append to existing interfaces and namespaces.

As a reminder, declaration merging is when you define a type or interface with the same name as an existing type or interface. TypeScript will merge the two.

This means any interface declared in a declaration file is fair game for augmentation. For example, *lib.dom.d.ts* contains a `Document` interface. Let's imagine you want to add a `foo` property to it.

For example, you can create a *global.d.ts* file and declare a new `Document` interface:

```
// In global.d.ts

interface Document {
  foo: string;
}
```

This *global.d.ts* declaration file is being treated as a script, so the `Document` interface merges with the existing one.

Now, across the project, the `Document` interface will have a `foo` property:

```
// In app.ts
```

```
document.foo = "hello"; // No error!
```

Declaration merging can be extremely useful for describing JavaScript globals that TypeScript doesn't know about. You'll see more examples of these in the exercises for this chapter.

It's also possible to type non-JavaScript files. In some environments, such as Webpack, you can import files—including images—that will ultimately be incorporated into the bundle with a string identifier.

Consider this example in which several *.png* images are imported. TypeScript doesn't typically recognize PNG files as modules, so it reports an error underneath each `import` statement:

```
import pngUrl1 from "./example1.png"; // red squiggly line u
nder "./example1.png"
import pngUrl2 from "./example2.png"; // red squiggly line u
nder "./example2.png"
import pngUrl3 from "./example3.png"; // red squiggly line u
nder "./example3.png"
import pngUrl4 from "./example4.png"; // red squiggly line u
nder "./example4.png"

// hovering over "./example1.png" shows:
Cannot find module './example1.png' or its corresponding typ
e declarations.
```

TypeScript is trying to find either a module called *example1.png* or its type declaration. The `declare module` syntax can help you fix this error by allowing you to declare types for non-JavaScript files.

To add support for the *.png* imports, create a new file called *png.d.ts*. In the file, you'll start with `declare module`, but since you can't use relative module names, you'll use a wildcard `*` to match any **.png* file. In the declaration, you'll say that `png` is a string and export it as the default:

```
// In images.d.ts
declare module "*.png" {
  const png: string;
```

```
export default png;
}
```

With the *images.d.ts* file in place, TypeScript will recognize the imported *.png* files as strings without reporting any errors.

Should You Store Your Types in Declaration Files?

A common misconception among TypeScript developers is that declaration files are where you store your types.

You might be thinking that you'd create a *types.d.ts* file like so:

```
// types.d.ts
export type Example = string;
```

Then, you'd import the `Example` type into a TypeScript file:

```
// index.ts
import {Example} from "./types";

const myFunction = (example: Example) => {
  console.log(example);
};
```

This is a bad idea because the `skipLibCheck` configuration option will ignore these files, meaning they won't be type-checked.

This is a relatively natural thing to get wrong; after all, a “declaration file” sounds like where you put your type declarations. Instead, put your types in regular TypeScript files. As a general rule of thumb, you should try to use as few declaration files as possible to mitigate the risk of bugs.

Is Using Global Types a Good Idea?

Across your project, you'll end up with several commonly used types. For example, you might have a `User` type that's used in many different files.

One option is to put these into the global scope to avoid importing them everywhere. This can be done by using a *.d.ts* file as a script or using `declare global` in a *.ts* file.

However, we don't recommend you do this. Polluting the global scope with types can turn your project into a mess of implicit dependencies. It can be hard to know where a type is coming from and can make refactoring difficult.

As your project grows, you'll get naming conflicts between types. Two different parts of your system might define a `User` type, leading to confusion.

Instead, you should import types explicitly. This makes it clear where a type is coming from, makes your system more portable, and makes refactoring easier.

Exercise 13-1: Typing a JavaScript Module

Consider this *example.js* JavaScript file that exports `myFunc`:

```
// example.js
export const myFunc = () => {
  return "Hello World!";
};
```

The `myFunc` function is then imported into a TypeScript *index.ts* file:

```
// index.ts
import {myFunc} from "./example"; // red squiggly line under
./example

myFunc();
```

However, there is an error in the import statement because TypeScript expects a declaration file for this JavaScript module:

```
// hovering over ./example shows:
Could not find a declaration file for module './example'.
```

Your task is to create a declaration file for the *example.js* file.

See <https://totalts.link/essentials-13-1/>.

Solution

The solution is to create a declaration file alongside the JavaScript file with a matching name. In this case, the declaration file should be named *example.d.ts*. In the declaration file, we declare the `myFunc` function with its type signature:

```
// example.d.ts
export function myFunc(): string;

export {};
```

With *example.d.ts* in place, the import statement in *index.ts* will no longer show an error.

Exercise 13-2: Ambient Context

Consider a variable called `state` that is returned from a global `DEBUG.getState()` function:

```
const state = DEBUG.getState(); // red squiggly line under D
DEBUG

type test = Expect<Equal<typeof state, {id: string}>>;
```

Here, `DEBUG` acts like a global variable. In our hypothetical project, `DEBUG` is referenced only in this file and is introduced into the global scope by an external script that you don't have control over.

Currently, there is an error below `DEBUG` because TypeScript cannot resolve the type of `state` returned by `DEBUG.getState()`.

As shown in the test, you expect `state` to be an object with an `id` of type `string`, but TypeScript currently interprets it as `any`:

```
// hovering over state shows:
const state: any;
```

Your task is to specify that `DEBUG` is available in this module (and this module only) without needing to provide its implementation. This will help TypeScript understand the type of state and provide the expected type checking.

See <https://totalts.link/essentials-13-2/>.

Solution

The first step is to use `declare const` to simulate a global variable within the local scope of the module. You'll start by declaring `DEBUG` as an empty object:

```
declare const DEBUG: {};
```

Now that you've typed `DEBUG`, the error message has moved to appear beneath `getState()`:

```
const state = DEBUG.getState(); // red squiggly line under g  
etState()  
  
type test = Expect<Equal<typeof state, {id: string}>>;
```

Referencing the test, you can see the `DEBUG` object needs a `getState` property that returns an object with an `id` of type `string`. You can update the `DEBUG` object to reflect this:

```
declare const DEBUG: {  
  getState: () => {  
    id: string;  
  };  
};
```

With this change, the errors have been resolved!

Exercise 13-3: Modifying window

Let's imagine now that you want your `DEBUG` object to be accessible only through the `window` object:

```
// In index.ts

const state = window.DEBUG.getState(); // red squiggly line
    under DEBUG

type test = Expect>
```

You expect state to be an object with an id string property, but it is currently typed as any.

There's also an error on DEBUG that tells us TypeScript doesn't see the DEBUG type:

```
// hovering over DEBUG shows:
Property 'DEBUG' does not exist on type 'Window & typeof globalThis'.
```

Your task is to specify that DEBUG is available on the window object. This will help TypeScript understand the type of state and provide the expected type checking.

See <https://totalts.link/essentials-13-3/>.

Solution

The first thing you'll do is create a new *window.d.ts* declaration file in the *src* directory. You need this file to be treated as a script to access the global scope, so you will not include the `export` keyword.

In the file, you'll create a new interface named `Window` that extends the built-in `Window` interface in *lib.dom.d.ts*. This will allow you to add new properties to the window interface—in this case, the `DEBUG` property with the `getState` method:

```
// window.d.ts
interface Window {
  DEBUG: {
    getState: () => {
      id: string;
    };
  };
}
```

```
};  
}
```

With this change, the errors have been resolved.

Alternative Solution

An alternative solution would be to use `declare global` with the interface directly in the *index.ts* file:

```
// index.ts  
const state = window.DEBUG.getState();  
  
type test = Expect<Equal<typeof state, {id: string}>>;  
  
declare global {  
  interface Window {  
    DEBUG: {  
      getState: () => {  
        id: string;  
      };  
    };  
  }  
}
```

Either approach will work, but keeping the global types in a separate file can often make them easier to find.

Exercise 13-4: Modifying process.env

Node.js introduces a global entity called `process`, which includes several properties that are typed with `@types/node`.

The `env` property is an object encapsulating all the environment variables that have been incorporated into the current running process. This can come in handy for feature flagging or for pinpointing different APIs across various environments.

Here's an example of using an `envVariable`, along with a test that checks to see if it is a string:

```
const envVariable = process.env.MY_ENV_VAR;

type test = Expect<Equal<typeof envVariable, string>>; // red squiggly line under Equal
```

TypeScript isn't aware of the `MY_ENV_VAR` environment variable, so it can't be certain that it will be a string. Thus, the `Equal` test fails because `envVariable` is typed as `string | undefined` instead of just `string`.

Your task is to determine how to specify the `MY_ENV_VAR` environment variable as a string in the global scope. This will be slightly different from the solution for modifying `window` in the first exercise.

As a tip, in `@types/node` from `DefinitelyTyped`, the `ProcessEnv` interface is responsible for environment variables. It can be found in the NodeJS namespace. You might need to revisit previous chapters to refresh your memory on declaration merging of types and namespaces to solve this exercise.

See <https://totalts.link/essentials-13-4/>.

Solution

There are two options for modifying the global scope in TypeScript: using `declare global` or creating a `.d.ts` declaration file.

For this solution, you'll create a `process.d.ts` file in the `src` directory. It doesn't matter what you call it, but `process.d.ts` indicates that you're modifying the `process` object. Since you know that `ProcessEnv` is in the NodeJS namespace, you'll use `declare namespace` to add your own properties to the `ProcessEnv` interface.

In this case, you'll declare a namespace `NodeJS` that contains an interface `ProcessEnv`. In it will be the property `MY_ENV_VAR` of type `string`:

```
// src/process.d.ts

declare namespace NodeJS {
  interface ProcessEnv {
    MY_ENV_VAR: string;
  }
}
```

With this new file in place, you can see that `MY_ENV_VAR` is now recognized as a string in `index.ts`. The error is resolved, and you have autocompletion support for the variable.

Remember, just because the error is resolved doesn't mean that `MY_ENV_VAR` will actually be a string at runtime. This update is merely a contract you're setting up with TypeScript. You still need to make sure that this contract is respected in your runtime environment.

Summary

This chapter explored how TypeScript distinguishes between modules and scripts and how declaration files describe JavaScript code and add types to the global scope.

TypeScript determines whether a file is a module or a script based on import or export statements. Modules have local scope while scripts execute globally. Use `moduleDetection: "force"` to treat all files as modules and avoid global scope issues.

Declaration files (`.d.ts`) describe existing JavaScript without implementation code and can add global types. The `declare` keyword defines ambient values, while `declare global` and `declare module` help augment existing types or create new module definitions.

TypeScript ships with declaration files for JavaScript built-ins and DOM APIs. Libraries often include their own `.d.ts` files or rely on DefinitelyTyped's `@types/*` packages for community-maintained definitions.

Use `skipLibCheck: true` to avoid third-party declaration file errors, but this makes authoring your own declaration files riskier. Store application types in regular TypeScript files rather than declaration files, and prefer explicit imports over global types to keep your codebase maintainable.

14

CONFIGURING TYPESCRIPT



We've dipped into TypeScript's *tsconfig.json* configuration file a few times in this book, but now it's time to take a deeper look. You'll find a recommended base configuration that includes descriptions of the individual options and then find additional recommendations for different project types. We won't cover every available option in *tsconfig.json* since many of them are old and rarely used, but we'll cover the most important ones.

Recommended Configuration

To start, here's a recommended base *tsconfig.json* configuration with options appropriate for most applications you're building:

```
{
  "compilerOptions": {
    /* Base Options: */
    "skipLibCheck": true,
    "target": "es2022",
    "esModuleInterop": true,
```

```
"allowJs": true,  
"resolveJsonModule": true,  
"moduleDetection": "force",  
"isolatedModules": true,  
"strict": true,  
"noUncheckedIndexedAccess": true  
}
```

Here's what each setting does:

skipLibCheck Skips type checking of declaration files, which improves compilation speed. We covered this in [Chapter 13](#).

target Specifies the ECMAScript target version for the compiled JavaScript code. Targeting es2022 provides access to some relatively recent JavaScript features, but by the time you read this book, you might want to target a newer version.

esModuleInterop Enables better compatibility between CommonJS and ECMAScript modules.

allowJs Allows JavaScript files without declaration files to be imported into the TypeScript project.

resolveJsonModule Allows JSON files to be imported into your TypeScript project.

moduleDetection Has a force option that tells TypeScript to treat all `.ts` files as a module instead of a script. We covered this in [Chapter 13](#).

isolatedModules Ensures that each file can be independently transpiled without relying on information from other files. This ensures that bundlers like ESBUILD and swc (commonly used by frontend frameworks) can be run on your code.

strict Enables a set of strict type checking options that catch more errors and generally promote better code quality.

noUncheckedIndexedAccess Enforces stricter type checking for indexed access operations, catching potential runtime errors.

Once these base options are set, there are several more to add depending on the type of project you're working on.

Additional Configuration Options

After setting the base *tsconfig.json* settings, there are several questions to ask yourself to determine which additional options to include:

- Are you transpiling your code with TypeScript? If yes, set `module` to `NodeNext`.
- Are you building for a library? If you're building for a library, set `declaration` to `true`. If you're building for a library in a monorepo, set `composite` to `true` and `declarationMap` to `true`.
- Are you not transpiling with TypeScript? If you're using a different tool to transpile your code, such as ESbuild or Babel, set `module` to `Preserve`, and set `noEmit` to `true`.
- Does your code run in the DOM? If yes, set `lib` to `["dom", "dom.iterable", "es2022"]`. If not, set it to `["es2022"]`.

The Complete Base Configuration

Based on your answers to those questions, here's how the complete *tsconfig.json* file would look:

```
{
  "compilerOptions": {
    /* Base Options: */
    "skipLibCheck": true,
    "target": "es2022",
    "esModuleInterop": true,
    "allowJs": true,
    "resolveJsonModule": true,
    "moduleDetection": "force",
    "isolatedModules": true,

    /* Strictness */
    "strict": true,
    "noUncheckedIndexedAccess": true,

    /* If transpiling with tsc: */
    "module": "NodeNext",
    "outDir": "dist",
```

```
    "sourceMap": true,

    /* AND if you're building for a library: */
    "declaration": true,

    /* AND if you're building for a library in a monorepo:
    */
    "composite": true,
    "declarationMap": true,

    /* If NOT transpiling with tsc: */
    "module": "Preserve",
    "noEmit": true,

    /* If your code runs in the DOM: */
    "lib": ["es2022", "dom", "dom.iterable"],

    /* If your code doesn't run in the DOM: */
    "lib": ["es2022"]
  }
}
```

Now that you understand the lay of the land, let's take a look at each of these options in more detail.

Base Options

In this section, we'll cover some of the most important base configuration options: `target`, `esModuleInterop`, and `isolatedModules`.

target

The `target` option specifies the ECMAScript version that TypeScript should target when generating JavaScript code.

For example, setting `target` to ES5 will attempt to transform your code to be compatible with ECMAScript 5.

Language features like optional chaining and nullish coalescing, which were introduced later than ES5, are still available:

```
// Optional chaining
const search = input?.search;

// Nullish coalescing
const defaultedSearch = search ?? "Hello";
```

But when they are turned into JavaScript, they'll be transformed into code that works in ES5 environments:

```
// Optional chaining
var search = input === null || input === void 0 ? void 0 : input.search;

// Nullish coalescing
var defaultedSearch = search !== null && search !== void 0 ? search : "Hello";
```

While target can transpile newer syntaxes into older environments, it won't do the same with APIs that don't exist in the target environment.

For example, if you're targeting a version of JavaScript that doesn't support `.replaceAll` on strings, TypeScript won't polyfill it for you:

```
const str = "Hello, world!";

str.replaceAll("Hello,", "Goodbye, cruel");
```

This code will cause an error in your target environment because target won't transform it for you. If you need to support older environments, you'll need to find your own polyfills. You configure the environment your code executes in with `lib`, as we saw in [Chapter 13](#).

If you're not sure what to specify for target, keep it up to date with the version you have specified in `lib`.

esModuleInterop

`esModuleInterop` is an old flag, released in 2018. It helps with interoperability between CommonJS and ECMAScript modules. At the

time, TypeScript had deviated slightly from commonly used tools like Babel in how it handled wildcard imports and default exports. `esModuleInterop` brought TypeScript in line with these tools.

You can read the release notes (<https://totalts.link/esmoduleinterop>) for more details. Suffice to say, when you're building an application, `esModuleInterop` should always be turned on.

isolatedModules

`isolatedModules` prevents some TypeScript language features that single-file transpilers can't handle.

Sometimes you'll be using tools other than `tsc` to turn your TypeScript into JavaScript. Tools such as `esbuild`, `babel`, or `swc` can't handle all TypeScript features. `isolatedModules` disables these features, making it easier to use these tools.

Consider the following example of an `AlbumFormat` enum that has been created with `declare const`:

```
declare const enum AlbumFormat {  
  CD,  
  Vinyl,  
  Digital,  
}  
  
const largestPhysicalSize = AlbumFormat.Vinyl;  
// red squiggly line under AlbumFormat when isolatedModules  
is enabled
```

Recall that the `declare` keyword will place `const enum` in an ambient context, which means that it would be erased at runtime.

When `isolatedModules` is disabled, this code will compile without any errors.

However, when `isolatedModules` is enabled, the `AlbumFormat` enum will not be erased and TypeScript will raise an error:

```
// hovering over AlbumFormat.Vinyl shows:  
Cannot access ambient const enums when 'isolatedModules' is
```


enabled.

```
// index.js after transpilation
"use strict";
Object.defineProperty(exports, "__esModule", {value: true});
const largestPhysicalSize = AlbumFormat.Vinyl;
```

This is because only tsc has enough context to understand what value `AlbumFormat.Vinyl` should have. TypeScript checks your entire project at once and stores the values for `AlbumFormat` in memory.

When using a single-file transpiler like esbuild, it doesn't have this context, so it can't know what `AlbumFormat.Vinyl` should be. So, `isolatedModules` is a way to make sure you're not using TypeScript features that can be difficult to transpile.

`isolatedModules` is a sensible default because it makes your code more portable if you ever need to switch to a different transpiler. It disables so few patterns that it's worth always turning on.

Strictness Options

The `strict` option in `tsconfig.json` acts as shorthand for enabling several different type checking options all at once, including catching potential null or undefined issues and stronger checks for function parameters, among others.

Setting `strict` to `false` makes TypeScript behave in ways that are much less safe. As an example, `strict` includes a setting called `strictNullChecks`. Setting `strictNullChecks` to `false` will allow you to assign `null` to a variable that is supposed to be a string:

```
let name: string = null; // No error
```

With `strict` enabled, TypeScript will, of course, catch this error.

Another example of a strict setting is `noImplicitAny`. Setting `noImplicitAny` to `false` will allow you to leave function parameters unannotated:

```
function greet(name) {  
  console.log(`Hello, ${name}!`);  
  
}
```

This can lead to a flood of any types in your codebase, which, as you know, leads to a loss of type safety.

In fact, we've written this entire book on the premise that you have `strict` enabled in your codebase. It's the baseline for all modern TypeScript apps.

One argument you often hear for turning `strict` off is that it's a good on-ramp for beginners. You can get a project up and running more quickly without having to worry about all the strictness rules.

However, we don't think this is a good idea. A lot of prominent TypeScript libraries like `zod`, `trpc`, `@redux/toolkit`, and `xstate` won't behave as expected when `strict` is off. Most community resources, like StackOverflow and React TypeScript Cheatsheet, assume you have `strict` enabled.

Not only that, but a project that starts with `strict: false` is likely to stay that way. On a mature codebase, it can be time-consuming to turn on `strict` and fix all of the errors.

Consider `strict: false` to be like a fork of TypeScript: It means you can't work with many libraries, makes it harder to seek help, and leads to more runtime errors.

noUncheckedIndexedAccess

One strictness rule that isn't part of `strict` is `noUncheckedIndexedAccess`. When enabled, it helps catch potential runtime errors by detecting cases where accessing an array or object index might return `undefined`.

Consider this example of a `VinylSingle` interface with an array of `tracks`:

```
interface VinylSingle {  
  title: string;  
  artist: string;  
  tracks: string[];
```

```
}

const egoMirror: VinylSingle = {
  title: "Ego / Mirror",
  artist: "Burial / Four Tet / Thom Yorke",
  tracks: ["Ego", "Mirror"],
};
```

To access the B-side of `egoMirror`, we would index into its `tracks` like this:

```
const bSide = egoMirror.tracks[1];

console.log(bSide.toUpperCase()); // 'MIRROR'
```

Without `noUncheckedIndexedAccess` enabled in `tsconfig.json`, TypeScript assumes that indexing will always return a valid value, even if the index is out of bounds.

Trying to access a nonexistent fourth track would not raise an error in VS Code, but it does result in a runtime error:

```
const nonExistentTrack = egoMirror.tracks[3];
console.log(nonExistentTrack.toUpperCase()); // No error in
VS Code

// However, running the code results in a runtime error:
TypeError: Cannot read property 'toUpperCase' of undefined
```

By setting `noUncheckedIndexedAccess` to `true`, TypeScript will infer the type of every indexed access to be `T | undefined` instead of just `T`. In this case, every entry in `egoMirror.tracks` would be of type `string | undefined`:

```
const ego = egoMirror.tracks[0];
const mirror = egoMirror.tracks[1];
const nonExistentTrack = egoMirror.tracks[3];
```

```
// hovering over ego shows:  
const ego: string | undefined
```

However, because the types of each of the tracks are now `string | undefined`, you get errors when attempting to call `toUpperCase` even for the valid tracks:

```
console.log(ego.toUpperCase()); // red squiggly line under e  
go  
  
// hovering over ego shows:  
'ego' is possibly 'undefined'
```

This means you have to handle the possibility of undefined values when accessing array or object indices.

So, `noUncheckedIndexedAccess` makes TypeScript stricter, but at the cost of having to handle undefined values more carefully.

Usually, this is a good trade-off, as it helps catch potential runtime errors early in the development process. But we wouldn't blame you if you end up turning it off in some cases.

Other Strictness Options

In most cases, setting up *tsconfig.json* with `strict` and `noUncheckedIndexedAccess` will be sufficient. If you want to go further, there are several other strictness options you can enable:

`allowUnreachableCode` Errors when unreachable code is detected, like code after a `return` statement.

`exactOptionalPropertyTypes` Requires that optional properties are exactly the type they're declared as, instead of allowing `undefined`.

`noFallthroughCasesInSwitch` Ensures that any nonempty case block in a `switch` statement ends with a `break`, `return`, or `throw` statement.

noImplicitOverride Requires that `override` is used when overriding a method from a base class.

noImplicitReturns Ensures that every code path in a function returns a value.

noPropertyAccessFromIndexSignature Forces you to use `example['access']` when accessing a property on an object with an index signature.

noUnusedLocals Errors when a local variable is declared but never used.

noUnusedParameters Errors when a function parameter is declared but never used.

The Two Choices for module

The `module` setting in `tsconfig.json` specifies how TypeScript should treat your imports and exports. There are two main choices: `NodeNext` and `Preserve`. When you want to transpile your TypeScript code with the TypeScript compiler, choose `NodeNext`. When you're using an external bundler like Webpack or Parcel, choose `Preserve`.

NodeNext

If you are transpiling your TypeScript code using the TypeScript compiler, you should choose `module: "NodeNext"` in your `tsconfig.json` file:

```
{
  "compilerOptions": {
    "module": "NodeNext"
  }
}
```

`module: "NodeNext"` also implies `moduleResolution: "NodeNext"`, so we'll discuss their behavior together.

When using `NodeNext`, TypeScript emulates Node's module resolution behavior, which includes support for features like `package.json`'s exports

field. Code emitted using `module: NodeNext` will be able to be run in a Node.js environment without any additional processing.

One thing you'll notice when using `module: NodeNext` is that you'll need to use `.js` extensions when importing TypeScript files:

```
// Importing from album.ts
import {Album} from "../album.js";
```

This can feel strange at first, but it's necessary because TypeScript doesn't transform your imports. This is how the import will look when it's transpiled to JavaScript, and TypeScript prefers for the code you write to match the code you'll run.

NODE16

NodeNext is a shorthand for "the most up-to-date Node.js module behavior." If you prefer to pin your TypeScript to a specific Node version, you can use Node16 instead:

```
{
  "compilerOptions": {
    "module": "Node16"
  }
}
```

At the time of writing, Node.js 16 is now end-of-life, but each Node version after it copied its module resolution behavior. This may change in the future, so it's worth checking the TypeScript documentation for the most up-to-date information or sticking to NodeNext.

Preserve

If you're using a bundler like Webpack, Rollup, or Parcel to transpile your TypeScript code, you should choose `module: "Preserve"` in your *tsconfig.json* file:

```
{
  "compilerOptions": {
    "module": "Preserve"
  }
}
```

When using Preserve, TypeScript assumes that the bundler will handle module resolution. This means you don't have to include `.js` extensions when importing TypeScript files because the bundler will take care of resolving the filepaths and extensions for you:

```
// Importing from album.ts
import {Album} from "../album";
```

If you're using an external bundler or transpiler, you should use `module: "Preserve"` in your `tsconfig.json` file. This is also true if you're using a frontend framework like Next.js, Remix, Vite, or SvelteKit: It will handle the bundling for you.

noEmit

The `noEmit` option in `tsconfig.json` tells TypeScript not to emit any JavaScript files when transpiling your TypeScript code:

```
{
  "compilerOptions": {
    "noEmit": true
  }
}
```

This pairs well with `module: "Preserve"`; in both cases, you're telling TypeScript that an external tool will handle the transpilation for you.

TypeScript's default for this option is `false`, so if you're finding that running `tsc` emits JavaScript files when you don't want it to, set `noEmit` to `true`.

Source Maps

TypeScript can generate source maps that link your compiled JavaScript code back to your original TypeScript code. You can enable them by setting `sourceMap` to `true` in your *tsconfig.json* file:

```
{
  "compilerOptions": {
    "sourceMap": true
  }
}
```

When you run your code with Node, you can add the flag `--enable-source-maps`:

```
node --enable-source-maps dist/index.js
```

Now when an error occurs in your compiled JavaScript code, the stack trace will point to the original TypeScript file.

Transpiling Code for Library Use

A common way to use TypeScript is to build a library that others can use in their projects. When building a library, there are a few additional settings you should consider in your *tsconfig.json* file.

outDir

The `outDir` option in *tsconfig.json* specifies the directory where TypeScript should output the transpiled JavaScript files.

Using TypeScript to transpile your code, it's common to set `outDir` to a `dist` directory in the root of your project:

```
{
  "compilerOptions": {
    "outDir": "dist"
  }
}
```

This can be combined with `rootDir` to specify the root directory of your TypeScript files:

```
{
  "compilerOptions": {
    "outDir": "dist",
    "rootDir": "src"
  }
}
```

Now a file at `src/index.ts` will be transpiled to `dist/index.js`.

Creating Declaration Files

We've already discussed how `.d.ts` declaration files are used to provide type information for JavaScript code, but so far you've created them only manually.

By setting `"declaration": true` in your `tsconfig.json` file, TypeScript will automatically generate `.d.ts` files and save them alongside your compiled JavaScript files:

```
{
  "compilerOptions": {
    "declaration": true
  }
}
```

For example, consider this `album.ts` file:

```
// In album.ts

export interface Album {
  title: string;
  artist: string;
  year: number;
}

export function createAlbum(
  title: string,
```

```
    artist: string,  
    year: number,  
  ): Album {  
    return {title, artist, year};  
  }  
}
```

After running the TypeScript compiler with the declaration option enabled, it will generate an *album.js* file and an *album.d.ts* file in the project's dist directory.

Here's the declaration file code with the type information:

```
// album.d.ts  
  
export interface Album {  
  title: string;  
  artist: string;  
  year: number;  
}  
  
export declare function createAlbum(  
  title: string,  
  artist: string,  
  year: number,  
): Album;
```

Here's the *album.js* file transpiled from TypeScript:

```
// album.js  
"use strict";  
Object.defineProperty(exports, "__esModule", {value: true});  
exports.createAlbum = void 0;  
function createAlbum(title, artist, year) {  
  return {title, artist, year};  
}  
exports.createAlbum = createAlbum;
```

Now anyone who uses your library will have access to the type information in the *.d.ts* files.

Declaration Maps

One common use case for libraries is in monorepos. A *monorepo* is a collection of libraries, each with their own *package.json*, that can be shared across different applications. This means you'll often be developing the library and the application that uses it side by side.

For example, a monorepo might look like this:

```
monorepo
├── apps
│   ├── my-app
│   └── package.json
└── packages
    ├── my-library
    └── package.json
```

However, if you're working on code in *my-app* that imports from *my-library*, you'll be working with the compiled JavaScript code, not the TypeScript source code. This means when you COMMAND-click (or CTRL-click) an import, you'll be taken to the *.d.ts* file instead of the original TypeScript source.

This is where declaration maps come in. They provide a mapping between the generated *.d.ts* and the original *.ts* source files.

To create them, the *declarationMap* setting should be added to your *tsconfig.json* file as well as the *sourceMap* setting:

```
{
  "compilerOptions": {
    "declarationMap": true,
    "sourceMap": true
  }
}
```

With this option in place, the TypeScript compiler will generate *.d.ts.map* files alongside the *.d.ts* files. Now when you COMMAND-click an

import in `my-app`, you'll be taken to the original TypeScript source file in `my-library`.

This is less useful when your library is on npm unless you also ship your source files, but that's a little outside the scope of this book. In a monorepo, however, declaration maps are a great quality-of-life improvement.

jsx

TypeScript has built-in support for transpiling JSX syntax, which is a syntax extension for JavaScript that allows you to write HTML-like code in your JavaScript files. In TypeScript, these need to be written in files with a `.tsx` extension:

```
// Component.tsx
const Component = () => {
  return <div />
}
```

The `jsx` option tells TypeScript how to handle JSX syntax and has five possible values. The most common are `preserve`, `react`, and `react-jsx`. Here's what each of them does:

preserve Keeps JSX syntax as is.

react Transforms JSX into `React.createElement` calls. This is useful for React 16 and earlier.

react-jsx Transforms JSX into `_jsx` calls and automatically imports from `react/jsx-runtime`. This is useful for React 17 and later.

A Note on Node's TypeScript Support

Recently, Node has been moving closer and closer to supporting TypeScript as a first-class citizen.

In Node 23 and 24, you can now use `node` on the command line to run TypeScript files. This is currently in the experimental stage and is known

colloquially as `--experimental-strip-types`. In Node 23 or greater, you can run `node index.ts`. In versions 20 through 22, a flag is required:

```
node index.ts # Node 23+
node --experimental-strip-types index.ts # Node 20-22
```

`--experimental-strip-types` requires some additional configuration. Five *tsconfig.json* settings are required:

```
{
  "compilerOptions": {
    "module": "NodeNext",
    "noEmit": true,
    "rewriteRelativeImportExtensions": true,
    "verbatimModuleSyntax": true,
    "erasableSyntaxOnly": true
  }
}
```

Let's break down each one:

module Lets TypeScript know that we're using Node's module system. This was described in the previous sections.

noEmit Tells TypeScript not to emit any files. We don't need to emit files because we're running the TypeScript files directly with Node.

rewriteRelativeImportExtensions Allows us to import TypeScript files using the *.ts* extension:

```
import {example} from "./example.ts";
```

Since we're not creating JavaScript files, we need to point Node to the TypeScript files directly.

Next, we come to the issue of imports. Node's TypeScript support runs an extremely fast type stripper to strip the types in the file before running the code. One issue with this is that it doesn't know the difference between a type and a value, especially when it comes to imports.

verbatimModuleSyntax Forces you to import types using the import type syntax:

```
import type {MyExampleType} from "../example.ts";
```

Without `import type` here, Node will throw a runtime error.

erasableSyntaxOnly Finally, Node supports only *some* TypeScript features. The ones it doesn't support might feel familiar: enums, namespaces, and class parameter properties.

As we mentioned in [Chapter 9](#), each of these features requires some kind of transformation. For instance, the transpiled enum code looks very different from the original code:

```
// Original
enum MyEnum {
  A = "a",
  B = "b",
}

// Transpiled
var MyEnum;
(function (MyEnum) {
  MyEnum["A"] = "a";
  MyEnum["B"] = "b";
})(MyEnum || (MyEnum = {}));
```

However, features that *don't* require transformation are trivial to support. You simply strip the types and you're done:

```
// Original
const myFunction = (a: number, b: number) => a + b;

// Transpiled
const myFunction = (a, b) => a + b;
```

By supporting only type stripping (and not transformation), Node simplifies its mental model to just “JavaScript with types.” This reduces the maintenance burden on Node itself.

You can disable these features in TypeScript by turning on `erasableSyntaxOnly`. This will raise an error if any of these features are used.

Managing Multiple TypeScript Configurations

As projects grow in size and complexity, it’s common to have different environments or targets within the same project.

For example, your single repo might include both a client-side application and a server-side API, each with different requirements and configurations.

This means you might want different *tsconfig.json* settings for different parts of your project. In this section, you’ll learn how multiple *tsconfig.json* files can be composed together.

How TypeScript Finds tsconfig.json

In a project with multiple *tsconfig.json* files, your IDE will need to know which one to use for each file. It determines which *tsconfig.json* to use by looking for the closest one to the current *.ts* file in question.

For example, given this file structure,

```
project
├── client
│   └── tsconfig.json
├── server
│   └── tsconfig.json
└── tsconfig.json
```

a file in the `client` directory will use the *client/tsconfig.json* file, while a file in the `server` directory will use the *server/tsconfig.json* file. Anything in the `project` directory that isn’t in `client` or `server` will use the *tsconfig.json* file in the root of the project.

This means *client/tsconfig.json* can contain settings specific to the client-side application, such as adding the dom types:

```
{
  "compilerOptions": {
    // . . .Other options
    "lib": ["es2022", "dom", "dom.iterable"]
  }
}
```

But *server/tsconfig.json* can contain settings specific to the server-side application, such as removing the dom types:

```
{
  "compilerOptions": {
    // . . .Other options
    "lib": ["es2022"]
  }
}
```

Globals with Multiple tsconfig.json Files

A useful feature of having multiple *tsconfig.json* files is that globals are tied to a single configuration file.

For example, say a declaration file *server.d.ts* in the server directory has a global declaration for an `ONLY_ON_SERVER` variable. This variable will be available only in files that are part of the server configuration:

```
// In server/server.d.ts
declare const ONLY_ON_SERVER: string;
```

Trying to use `ONLY_ON_SERVER` in a file that's part of the client configuration will result in an error:

```
// In client/index.ts
console.log(ONLY_ON_SERVER); // red squiggly line under ONLY_ON_SERVER
```

This feature is useful when dealing with environment-specific variables or globals that come from testing tools like Jest or Cypress, and it avoids polluting the global scope.

Extending Configurations

When you have multiple *tsconfig.json* files, it's common to have shared settings between them:

```
// client/tsconfig.json
{
  "compilerOptions": {
    "target": "es2022",
    "module": "Preserve",
    "esModuleInterop": true,
    "noEmit": true,
    "strict": true,
    "skipLibCheck": true,
    "isolatedModules": true,
    "lib": [
      "es2022",
      "dom",
      "dom.iterable"
    ],
    "jsx": "react-jsx",
  }
}

// server/tsconfig.json
{
  "compilerOptions": {
    "target": "es2022",
    "module": "Preserve",
    "esModuleInterop": true,
    "noEmit": true,
    "strict": true,
    "skipLibCheck": true,
    "isolatedModules": true,
    "lib": [
      "es2022"
```

```
    ]  
  },  
}
```

Instead of repeating the same settings in both *client/tsconfig.json* and *server/tsconfig.json*, you can create a new *tsconfig.base.json* file that can be extended as a shared base.

The common settings can be moved to *tsconfig.base.json*:

```
// tsconfig.base.json  
{  
  "compilerOptions": {  
    "target": "es2022",  
    "module": "Preserve",  
    "esModuleInterop": true,  
    "noEmit": true,  
    "strict": true,  
    "skipLibCheck": true,  
    "isolatedModules": true  
  }  
}
```

Then, the *client/tsconfig.json* would extend the base configuration with the *extends* option that points to the *tsconfig.base.json* file:

```
// client/tsconfig.json  
{  
  "extends": "../tsconfig.base.json",  
  "compilerOptions": {  
    "lib": [  
      "es2022",  
      "dom",  
      "dom.Iterable"  
    ],  
    "jsx": "react-jsx"  
  }  
}
```

The *server/tsconfig.json* file would do the same:

```
// server/tsconfig.json
{
  "extends": "../tsconfig.base.json",
  "compilerOptions": {
    "lib": [
      "es2022"
    ]
  }
}
```

This approach is particularly useful for monorepos, where many different *tsconfig.json* files might need to reference the same base configuration.

Any changes to the base configuration will be automatically inherited by the client and server configurations. However, it's important to note that using *extends* will copy over only *compilerOptions* from the base configuration, not other settings like *include* or *exclude* (which are used to specify which files to include or exclude from compilation):

```
{
  "compilerOptions": {}, // Will be inherited by 'extends'
  "include": [], // Will NOT be inherited by 'extends'
  "exclude": [] // Will NOT be inherited by 'extends'
}
```

--project

Now you have a set of *tsconfig.json* files that are organized and share common settings. This works okay in the IDE, which automatically detects which *tsconfig.json* file to use based on the file's location.

But what if you want to run a command that checks the entire project at once?

To do this, you need to run *tsc* using the *--project* flag and point it to each *tsconfig.json* file:

```
tsc --project ./client/tsconfig.json
tsc -p ./server/tsconfig.json # -p works as an alias for --project
```

This can work for a small number of configurations, but it can quickly become unwieldy as the number of configurations grows.

Project References

To simplify this process, TypeScript has a feature called *project references*. This allows you to specify a list of projects that depend on each other, and TypeScript will build them in the correct order.

You can configure a single *tsconfig.json* file at the root of the project that references the client and server configurations:

```
// tsconfig.json
{
  "references": [
    {
      "path": "./client/tsconfig.json"
    },
    {
      "path": "./server/tsconfig.json"
    }
  ],
  "files": []
}
```

Note that there is also an empty `files` array in the configuration shown here. This is to prevent the root *tsconfig.json* from checking any files itself; it just references the other configurations.

Next, you need to add the `composite` option to the *tsconfig.base.json* file. This option tells TypeScript that client and server are child project configurations that need to be run with project references:

```
// tsconfig.base.json
{
  "compilerOptions": {
```

```
    // . . .other options
    "composite": true
  },
}
```

Now, from the root, you can run `tsc` with the `-b` flag to run each of the projects:

```
tsc -b
```

The `-b` flag tells TypeScript to run the project references. This will typecheck and build the `client` and `server` configurations in the correct order.

When you run this for the first time, some `.tsbuildinfo` files will be created in the `client` and `server` directories. These files are used by TypeScript to cache information about the project and speed up subsequent builds.

So, to sum up:

- Project references allow you to run several `tsconfig.json` files in a single `tsc` command.
- Each subconfiguration should have `composite: true` in its `tsconfig.json` file.
- The root `tsconfig.json` file should have a `references` array that points to each subconfiguration, and it should have `files: []` to prevent it from checking any files itself.
- Run `tsc -b` from the root to build all the configurations.
- Each `tsconfig.json` will have its own global scope, and globals will not be shared between configurations.
- `.tsbuildinfo` files will be created in each subconfiguration to speed up subsequent builds.

Project references can be used in all sorts of ways to manage complex TypeScript projects. They're especially useful when you want globals to affect only a certain part of your project, like types from the `dom` or test frameworks that add functions to the global scope.

Summary

This chapter provided comprehensive guidance on configuring TypeScript through *tsconfig.json*. We recommend starting with a base configuration that includes essential options like `strict`, `skipLibCheck`, `target: "es2022"`, and `noUncheckedIndexedAccess`.

The key decisions revolve around your project setup. If you're transpiling with TypeScript, use `module: "NodeNext"`. If you're using external bundlers like Webpack, choose `module: "Preserve"` with `noEmit: true`. For DOM environments, include `"dom"` in your `lib` array.

The `strict` option is non-negotiable: It enables crucial type safety features that prevent runtime errors. While `noUncheckedIndexedAccess` adds extra safety by treating array access as potentially undefined, it requires more careful handling of indexed operations.

For library development, enable `declaration: true` to generate *.d.ts* files. In monorepos, add `composite: true` and `declarationMap: true` for better development experience with source navigation.

Complex projects benefit from multiple *tsconfig.json* files that can extend a shared base configuration. Project references allow you to manage these configurations efficiently using `tsc -b`, ensuring proper build order and isolated global scopes.

Node's experimental TypeScript support offers a glimpse into the future with direct *.ts* file execution, though it requires specific configuration and supports only type stripping, not transformation features like enums or namespaces.

PART VI

ADVANCED APPLICATION DEVELOPMENT

15

DESIGNING YOUR TYPES



As you build out your TypeScript applications, you're going to notice something. The way you design your types will significantly change how easy your application is to maintain.

Your types are more than just a way to catch errors at compile time. They help reflect and communicate the business logic they represent.

You've used syntax like `interface extends` and type helpers like `Pick` and `Omit`. You understand the benefits and trade-offs of deriving types from other types. In this chapter, you'll look at some more advanced techniques for designing your types in TypeScript.

You will learn several more techniques for composing and transforming types. You'll work with generic types, which can turn your types into "type functions." Also introduced are template literal types for defining and enforcing specific string formats as well as mapped types for deriving the shape of one type from another.

Generic Types

Generic types let you turn a type into a "type function" that can receive arguments. You've seen generic types before, like `Pick` and `Omit`. These types take in a type and a key and return a new type based on that key:

```
type Example = Pick<{a: string; b: number}, "a">;
```

Now you're going to be creating your own generic types. These are most useful for reducing repetition in your code.

Consider these `StreamingPlaylist` and `StreamingAlbum` types, which share similar structures:

```
type StreamingPlaylist =  
  | {  
    status: "available";  
    content: {  
      id: number;  
      name: string;  
      tracks: string[];  
    };  
  }  
  | {  
    status: "unavailable";  
    reason: string;  
  };  
  
type StreamingAlbum =  
  | {  
    status: "available";  
    content: {  
      id: number;  
      title: string;  
      artist: string;  
      tracks: string[];  
    };  
  }  
  | {  
    status: "unavailable";  
    reason: string;  
  };
```

Both the `StreamingPlaylist` and `StreamingAlbum` types represent a streaming resource that is either available with specific content or

unavailable with a reason for its unavailability.

The primary difference lies in the structure of the content object: The `StreamingPlaylist` type has a `name` property, while the `StreamingAlbum` type has a `title` property and an `artist` property. Despite this difference, the overall structure of the types is the same.

To reduce repetition, you can create a generic type called `ResourceStatus` that can represent both `StreamingPlaylist` and `StreamingAlbum`.

To create a generic type, you use a *type parameter* that declares what type of argument the type must receive. To specify the parameter, use the angle bracket syntax, which will look familiar from working with the various type helpers seen earlier in the book:

```
type ResourceStatus<TContent> = unknown;
```

The `ResourceStatus` type will take in a type parameter of `TContent`, which will represent the shape of the content object that is specific to each resource. For now, set the resolved type to `unknown`.

Often, type parameters are named with single-letter names like `T`, `K`, or `V`, but you can name them anything you like.

Now that `ResourceStatus` has been declared as a generic type, you can pass it a *type argument*. Let's create an `Example` type and provide an object type as the type argument for `TContent`:

```
type StreamingPlaylist = ResourceStatus<{  
  id: string;  
  name: string;  
  tracks: string[];  


---


```

Just like with `Pick` and `Omit`, the type argument is passed in as an argument to the generic type.

But what type will `Example` be?

```
// hovering over Example shows:  
type Example = unknown;
```

The result of `ResourceStatus` is unknown. Why is this happening? You can get a clue by hovering over the `TContent` parameter in the `ResourceStatus` type:

```
type ResourceStatus<TContent> = unknown;  
  
// hovering over TContent shows:  
Type 'TContent' is declared but its value is never read.
```

The `TContent` parameter is not being used; it's just returning unknown, no matter what is passed in. So, the `Example` type is also unknown.

Let's use the parameter. Let's update the `ResourceStatus` type to match the structure of the `StreamingPlaylist` and `StreamingAlbum` types, with the bit you want to be dynamic replaced with the `TContent` type parameter:

```
type ResourceStatus<TContent> =  
  | {  
    status: "available";  
    content: TContent;  
  }  
  | {  
    status: "unavailable";  
    reason: string;  
  };
```

You can now redefine `StreamingPlaylist` and `StreamingAlbum` to use the parameter:

```
type StreamingPlaylist = ResourceStatus<{  
  id: number;  
  name: string;  
  tracks: string[];  

```

```
type StreamingAlbum = ResourceStatus<{
  id: number;
  title: string;
  artist: string;
  tracks: string[];
}>;
```

Now if you hover over `StreamingPlaylist`, you will see that it has the same structure as it did originally, but it's now defined with the `ResourceStatus` type without having to manually provide the additional properties:

```
// hovering over StreamingPlaylist shows:
type StreamingPlaylist =
  | {
    status: "unavailable";
    reason: string;
  }
  | {
    status: "available";
    content: {
      id: number;
      name: string;
      tracks: string[];
    };
  };
};
```

`ResourceStatus` is now a generic type. It's a kind of type function, which means it's useful in all the ways runtime functions are useful. You can use generic types to capture repeated patterns in your types and make your types more flexible and reusable.

Multiple Type Parameters

Generic types can accept multiple type parameters, allowing for even more flexibility.

You can expand the `ResourceStatus` type to include a second type parameter that represents metadata that accompanies the resource:

```
type ResourceStatus<TContent, TMetadata> =  
  | {  
    status: "available";  
    content: TContent;  
    metadata: TMetadata;  
  }  
  | {  
    status: "unavailable";  
    reason: string;  
  };
```

Now that the ResourceStatus type is set up to accept TMetadata, the StreamingPlaylist and StreamingAlbum types can include metadata specific to each resource:

```
type StreamingPlaylist = ResourceStatus<  
  {  
    id: number;  
    name: string;  
    tracks: string[];  
  },  
  {  
    creator: string;  
    artwork: string;  
    dateUpdated: Date;  
  }  
>;
```

```
type StreamingAlbum = ResourceStatus<  
  {  
    id: number;  
    title: string;  
    artist: string;  
    tracks: string[];  
  },  
  {  
    recordLabel: string;  
    upc: string;  
    yearOfRelease: number;  
  }  
>;
```

```
}  
>;
```

Like before, each type maintains the same structure defined in `ResourceStatus`, but with its own content and metadata.

You can use as many type parameters as you need in a generic type. But just like with functions, the more parameters you have, the more complex your types can become.

All Type Arguments Must Be Provided

What happens if you don't pass a type argument to a generic type? Let's try it with the `ResourceStatus` type:

```
type Example = ResourceStatus; // red squiggly line under ResourceStatus  
  
// hovering over ResourceStatus shows:  
Generic type 'ResourceStatus' requires 2 type argument(s).
```

TypeScript shows an error that tells you that `ResourceStatus` requires two type arguments. This is because by default, all generic types *require* their type arguments to be passed in, just like runtime functions.

Default Type Parameters

In some cases, you may want to provide default types for generic type parameters. Like with functions, you can use the `=` to assign a default value.

By setting `TMetadata`'s default value to an empty object, you can essentially make `TMetadata` optional:

```
type ResourceStatus<TContent, TMetadata = {}> =  
  | {  
    status: "available";  
    content: TContent;  
    metadata: TMetadata;  
  }  
  | {  
    status: "unavailable";
```

```
    reason: string;
};
```

Now you can create a `StreamingPlaylist` type without providing a `TMetadata` type argument:

```
type StreamingPlaylist = ResourceStatus<{
  id: number;
  name: string;
  tracks: string[];
}>;
```

If you hover over it, you'll see that it's typed as expected, with `metadata` being an empty object:

```
// hovering over StreamingPlaylist shows:
type StreamingPlaylist =
  | {
    status: "unavailable";
    reason: string;
  }
  | {
    status: "available";
    content: {
      id: number;
      name: string;
      tracks: string[];
    };
    metadata: {};
  };
};
```

Defaults can help make your generic types more flexible and easier to use.

Type Parameter Constraints

To set constraints on type parameters, you can use the `extends` keyword. You can force the `TMetadata` type parameter to be an object while still defaulting to an empty object:

```
type ResourceStatus<TContent, TMetadata extends object = {}>
= // . . .
```

There's also an opportunity to provide a constraint for the TContent type parameter.

Both of the StreamingPlaylist and StreamingAlbum types have an id property in their content objects. This would be a good candidate for a constraint.

You can create a HasId type that enforces the presence of an id property:

```
type HasId = {
  id: number;
};

type ResourceStatus<TContent extends HasId, TMetadata extends
  object = {}> =
  | {
    status: "available";
    content: TContent;
    metadata: TMetadata;
  }
  | {
    status: "unavailable";
    reason: string;
  };
};
```

With these changes in place, it is now required that the TContent type parameter must include an id property. The TMetadata type parameter is optional, but if it is provided, it must be an object.

When you try to create a type with ResourceStatus that doesn't have an id property, TypeScript will raise an error that tells you exactly what's wrong:

```
type StreamingPlaylist = ResourceStatus<
  {
    name: string;
```



```
    tracks: string[];
  }, // red squiggly line under the object
  {
    creator: string;
    artwork: string;
    dateUpdated: Date;
  }
>;

// hovering over the error shows:
Type '{name: string; tracks: string[];}' does not satisfy the
constraint 'HasId'.
Property 'id' is missing in type '{name: string; tracks: string[];}'
but required in type 'HasId'.
```

The error message tells you that you're missing the `id` property. Once the `id` property is added to the `TContent` type parameter, the error will go away.

Note that the constraints we're providing here are just descriptions for properties the object must contain. You can pass `name` and `tracks` into `TContent` as long as it has an `id` property.

In other words, these constraints are *open*, not *closed*. You won't get excess property warnings here. Any excess properties you pass in will be added to the type.

extends, extends, extends

By now, you've seen `extends` used in a few different contexts:

- In generic types, to set constraints on type parameters
- In classes, to extend another class
- In interfaces, to extend another interface

There is even another use for `extends`: conditional types, which you'll look at later in this chapter.

One of TypeScript's annoying habits is that it tends to reuse the same keywords in different contexts. So, it's important to understand that `extends` means different things in different places.

Template Literal Types in TypeScript

Similar to how template literals in JavaScript allow you to interpolate values into strings, template literal types in TypeScript can be used to interpolate other types into string types.

For example, let's create a `PngFile` type that represents a string that ends with `.png`:

```
type PngFile = `${string}.png`;
```

Now when you type a new variable as `PngFile`, it must end with `.png`:

```
let myImage: PngFile = "my-image.png"; // OK
```

When a string does not match the pattern defined in the `PngFile` type, TypeScript will raise an error:

```
let myImage: PngFile = "my-image.jpg"; // Type '"my-image.jpg"'  
is not assignable to type 'PngFile'.
```

Combining Template Literal Types with Union Types

Template literal types become even more powerful when combined with union types. By passing a union to a template literal type, you can generate a type that represents all possible combinations of the union.

For example, let's imagine you have a set of colors, each with possible shades from 100 to 900:

```
type ColorShade = 100 | 200 | 300 | 400 | 500 | 600 | 700 |  
800 | 900;  
type Color = "red" | "blue" | "green";
```

If you want a combination of all possible colors and shades, you can use a template literal type to generate a new type:

```
type ColorPalette = `${Color}-${ColorShade}`;
```

Now `ColorPalette` will represent all possible combinations of colors and shades:

```
let myColor: ColorPalette = "red-500"; // OK
let myColor2: ColorPalette = "blue-900"; // OK
```

That's 27 possible combinations: three colors times nine shades.

If you have any kind of string pattern you want to enforce in your application, from routes to URIs to hex codes, template literal types can help. But it's worth noting that these types can quickly get combinatorially explosive and create enormous unions. So, tread carefully when building complex string literal types.

Transforming String Types

TypeScript even has several built-in utility types for transforming string types. For example, `Uppercase` and `Lowercase` can be used to convert a string to uppercase or lowercase:

```
type UppercaseHello = Uppercase<"hello">; // "HELLO"
type LowercaseHELLO = Lowercase<"HELLO">; // "hello"
```

The `Capitalize` type can be used to capitalize the first letter of a string:

```
type CapitalizeMatt = Capitalize<"matt">; // "Matt"
```

The `Uncapitalize` type can be used to lowercase the first letter of a string:

```
type UncapitalizePHD = Uncapitalize<"PHD">; // "pHD"
```

These utility types are occasionally useful for transforming string types in your applications, and they prove how flexible TypeScript's type system can be.

Conditional Types

You can use conditional types in TypeScript to create if/else logic in your types. This is mostly useful in a library setting when working with really complex code, but this section gives a simple example in case you ever run into it.

Imagine you want to create a `ToArray` generic type that converts a type to an array type:

```
type ToArray<T> = T[];
```

This is fine, except when you pass in a type that's already an array. If you do, you'll get an array of arrays:

```
type Example = ToArray<string>; // string[]
type Example2 = ToArray<string[]>; // string[][]
```

You actually want `Example2` to end up as `string[]` too. So, you need to check if `T` is already an array, and if it is, return `T` instead of `T[]`.

You can use a conditional type to do that. This uses a ternary operator, similar to JavaScript:

```
type ToArray<T> = T extends any[] ? T : T[];
```

This will look pretty scary the first time you see it, but let's break it down:

```
type ToArray<T> = T extends any[] ? T : T[];
//                ^^^^^^^^^^^^^^^^^      ^   ^^^
//                Condition             true/false
```

The “condition” in a conditional type is the part before the `?`. In this case, it's `T extends any[]`:

```
type ToArray<T> = T extends any[] ? T : T[];
//                ^^^^^^^^^^^^^^^^^
```

```
// Condition
```

This checks if τ can be assigned to `any[]`. To make sense of this check, imagine it like a function. In this case, only arrays can be passed in:

```
const toArray = (t: any[]) => {  
  // Implementation  
};  
toArray([1, 2, 3]); // OK  
toArray("hello"); // Error
```

τ extends `any[]` checks if τ could be passed to a function expecting `any[]`. If you wanted to check if τ was a string, for example, you would use `T extends string`.

If the condition is true, it resolves to the “true” part, just like a normal ternary. If it’s false, it resolves to the “false” part.

For this example case, if τ is an array, it resolves to τ . If it’s not, it resolves to $\tau[]$. This means that our previous examples now work as expected:

```
type Example = ToArray<string>; // string[]  
type Example2 = ToArray<string[]>; // string[]
```

Conditional types turn TypeScript’s type system into a full programming language. They’re incredibly powerful, but they can also be incredibly complex. You’ll rarely need them in application code, but they can perform wonders in library code.

Mapped Types

Mapped types in TypeScript allow you to create a new object type based on an existing type by iterating over its keys and values. This can let you be extremely expressive when creating new object types.

Consider this `Album` interface:

```
interface Album {  
  name: string;  
  artist: string;  
  songs: string[];  
}
```

Imagine you want to create a new type that makes all the properties optional and nullable. If it were only optional, you could use `Partial`, but you want to end up with a type that looks like this:

```
type AlbumWithNullable = {  
  name?: string | null;  
  artist?: string | null;  
  songs?: string[] | null;  
};
```

Let's start by using a mapped type instead of repeating the properties:

```
type AlbumWithNullable = {  
  [K in keyof Album]: K;  
};
```

This will look similar to an index signature, but instead of `[k: string]`, you use `[K in keyof Album]`. This will iterate over each key in `Album` and create a property in the object with that key. Note that `K` is a name you've chosen; you can choose any name you like.

In this case, you're using `K` as the type of the property too. This is not what you want eventually, but it's a good start:

```
// hovering over AlbumWithNullable shows:  
type AlbumWithNullable = {  
  name: "name";  
  artist: "artist";  
  songs: "songs";  
};
```

Hovering over `AlbumWithNullable`, you can see that `K` represents the *currently iterated key*. This means you can use `K` to get the type of the original `Album` property using an indexed access type:

```
type AlbumWithNullable = {
  [K in keyof Album]: Album[K];
};

// hovering over AlbumWithNullable shows:
type AlbumWithNullable = {
  name: string;
  artist: string;
  songs: string[];
};
```

Wonderful! You've now re-created the object type of `Album`. Now you can add `| null` to each property:

```
type AlbumWithNullable = {
  [K in keyof Album]: Album[K] | null;
};

// hovering over AlbumWithNullable shows:
type AlbumWithNullable = {
  name: string | null;
  artist: string | null;
  songs: string[] | null;
};
```

You're almost there. You just need to make each property optional by adding a `?` after the key:

```
type AlbumWithNullable = {
  [K in keyof Album]?: Album[K] | null;
};

// hovering over AlbumWithNullable shows:
type AlbumWithNullable = {
```

```
name?: string | null | undefined;
artist?: string | null | undefined;
songs?: string[] | null | undefined;
};
```

Now you have a new type that is derived from the `Album` type, but with all properties optional and nullable.

In the spirit of designing your types properly, you should make this behavior reusable by wrapping it in a generic type, `Nullable<T>`:

```
type Nullable<T> = {
  [K in keyof T]?: T[K] | null;
};
type AlbumWithNullable = Nullable<Album>;
```

Mapped types are an extremely useful method for transforming object types and have many different uses in application code.

Key Remapping with as

In the previous example, you didn't need to change the key of the object you were iterating over. But what if you did?

Let's say you want to create a new type that has the same properties as `Album` but with the key names uppercased. You could try using `Uppercase` on `keyof Album`:

```
type AlbumWithUppercaseKeys = {
  [K in Uppercase<keyof Album>]: Album[K]; // red squiggly line under Album[K]
};

// hovering over Album[K] shows:
Type 'K' cannot be used to index type 'Album'.
```

But this doesn't work. You can't use `K` to index `Album` because `K` has already been transformed to its uppercase version. Instead, you need to find a way to keep `K` the same as before, while using the uppercase version of `K` for the key.

You can do this by using the `as` keyword to remap the key:

```
type AlbumWithUppercaseKeys = {  
  [K in keyof Album as Uppercase<K>]: Album[K];  
};  
  
// hovering over AlbumWithUppercaseKeys shows:  
type AlbumWithUppercaseKeys = {  
  NAME: string;  
  ARTIST: string;  
  SONGS: string[];  
};
```

Using `as` allows you to remap the key while keeping the original key accessible in the loop. This isn't like when you use `as` for a type assertion; it's a completely different use of the keyword.

Using Mapped Types with Union Types

Mapped types don't always have to use `keyof` to iterate over an object. They can also map over a union of potential property keys for the object.

For example, you can create an `Example` type that is a union of `a`, `b`, and `c`:

```
type Example = "a" | "b" | "c";
```

Then, you can create a `MappedExample` type that maps over `Example` and returns the same values:

```
type MappedExample = {  
  [E in Example]: E;  
};  
  
// hovering over MappedExample shows:  
type MappedExample = {  
  a: "a";  
  b: "b";
```

```
c: "c";  
};
```

Here, each member of the `Example` union becomes both a key and a value in the `MappedExample` type.

Exercise 15-1: Creating a DataShape Type Helper

Consider the types `UserDataShape` and `PostDataShape`:

```
type ErrorShape = {  
  error: {  
    message: string;  
  };  
};  
  
type UserDataShape =  
  | {  
    data: {  
      id: string;  
      name: string;  
      email: string;  
    };  
  }  
  | ErrorShape;  
  
type PostDataShape =  
  | {  
    data: {  
      id: string;  
      title: string;  
      body: string;  
    };  
  }  
  | ErrorShape;
```

Looking at these types, they share a consistent pattern. Both `UserDataShape` and `PostDataShape` possess a data object and an error shape, with the error shape being identical in both. The only difference

between the two is the data object, which holds different properties for each type.

Your task is to create a generic DataShape type to reduce duplication in the UserDataShape and PostDataShape types.

See <https://totalts.link/essentials-15-1/>.

Solution

Here's how a generic DataShape type would look:

```
type DataShape<TData> =  
  | {  
    data: TData;  
  }  
  | ErrorShape;
```

With this type defined, the UserDataShape and PostDataShape types can be updated to use it:

```
type UserDataShape = DataShape<{  
  id: string;  
  name: string;  
  email: string;  
  
type PostDataShape = DataShape<{  
  id: string;  
  title: string;  
  body: string;  


---


```

Exercise 15-2: Typing PromiseFunc

This PromiseFunc type represents a function that returns a promise:

```
type PromiseFunc = (input: any) => Promise<any>;
```

Here are two example tests that use the PromiseFunc type with different inputs that currently have errors:

```
type Example1 = PromiseFunc<string, string>; // red squiggly
line under PromiseFunc<>

type test1 = Expect<Equal<Example1, (input: string) => Promise<string>>>;

type Example2 = PromiseFunc<boolean, number>; // red squiggly
line under PromiseFunc<>

type test2 = Expect<Equal<Example2, (input: boolean) => Promise<number>>>;
```

The error messages inform us that the PromiseFunc type is not generic. You're also expecting the PromiseFunc type to take in two type arguments: the input type and the return type of the promise.

Your task is to update PromiseFunc so that both of the tests pass without errors.

See <https://totalts.link/essentials-15-2/>.

Solution

The first thing you need to do is make PromiseFunc generic by adding type parameters to it. In this case, since you want it to have two parameters, you call them TInput and TOutput and separate them with a comma:

```
type PromiseFunc<TInput, TOutput> = (input: any) => Promise<
any>;
```

Next, you need to replace the any types with the type parameters. The input will use the TInput type, and the Promise will use the TOutput type:

```
type PromiseFunc<TInput, TOutput> = (input: TInput) => Promise<
TOutput>;
```

With these changes in place, the errors go away, and the tests pass because `PromiseFunc` is now a generic type. Any type passed as `TInput` will serve as the input type, and any type passed as `TOutput` will act as the output type of the `Promise`.

Exercise 15-3: Working with the Result Type

Let's say you have a `Result` type that can be either a success or an error:

```
type Result<TResult, TError> =  
  | {  
    success: true;  
    data: TResult;  
  }  
  | {  
    success: false;  
    error: TError;  
  };
```

You also have the `createRandomNumber` function that returns a `Result` type:

```
const createRandomNumber = (): Result<number> => {  
  // red squiggly line under number  
  const num = Math.random();  
  
  if (num > 0.5) {  
    return {  
      success: true,  
      data: 123,  
    };  
  }  
  
  return {  
    success: false,  
    error: new Error("Something went wrong"),  
  };  
};
```

Because only a number is being sent as a type argument, you have an error message:

```
// hovering over Result shows:  
Generic type 'Result' requires 2 type argument(s)
```

You're specifying only the number because it can be a bit of a hassle to always specify both the success and error types whenever you use the `Result` type.

It would be easier if you could designate the `Error` type as the default type for `Result`'s `TError`, since that's what most errors will be typed as.

Your task is to adjust the `Result` type so that `TError` defaults to type `Error`.

See <https://totalts.link/essentials-15-3/>.

Solution

Similar to other times you set default values, the solution is to use an equal sign.

In this case, you'll add the `=` after the `TError` type parameter and then specify `Error` as the default type:

```
type Result<TResult, TError = Error> =  
  | {  
    success: true;  
    data: TResult;  
  }  
  | {  
    success: false;  
    error: TError;  
  };
```

`Result` types are a great way to ensure that errors are handled properly. For instance, `result` here must be checked for success before accessing the `data` property:

```
const result = createRandomNumber();

if (result.success) {
  console.log(result.data);
} else {
  console.error(result.error.message);
}
```

This pattern can be a great alternative to try...catch blocks in JavaScript.

Exercise 15-4: Constraining the Result Type

After updating the Result type to have a default type for TError, it would be a good idea to add a constraint on the shape of what's being passed in.

Here are some examples of using the Result type:

```
type BadExample = Result<
  {id: string},
  // @ts-expect-error Should be an object with a message pro
  string
>;

type GoodExample = Result<{id: string}, TypeError>;
type GoodExample2 = Result<{id: string}, {message: string; c
ode: number}>;
type GoodExample3 = Result<{id: string}, {message: string}>;
type GoodExample4 = Result<{id: string}>;
```

The GoodExamples should pass without errors, but the BadExample should raise an error because the TError type is not an object with a message property. Currently this isn't the case, as you can see by the error under the @ts-expect-error directive.

Your task is to add a constraint to the Result type that ensures that the BadExample test raises an error, while the GoodExamples pass without errors.

See <https://totalts.link/essentials-15-4/>.

Solution

You want to set a constraint on `TError` to ensure that it is an object with a message string property, while also retaining `Error` as the default type for `TError`.

To do this, you'll use the `extends` keyword for `TError` and specify the object with a message property as the constraint:

```
type Result<TResult, TError extends {message: string} = Error> =  
  | {  
    success: true;  
    data: TResult;  
  }  
  | {  
    success: false;  
    error: TError;  
  };
```

Now if you remove the `@ts-expect-error` directive from `BadExample`, you will get an error under `string`:

```
type BadExample = Result<  
  {id: string},  
  string // red squiggly line under string  
>;  
  
// hovering over string shows:  
Type 'string' does not satisfy the constraint '{message: string}'.
```

The behavior of constraining type parameters and adding defaults is similar to runtime parameters. However, unlike runtime arguments, you can add properties inline and still satisfy the constraint:

```
type GoodExample2 = Result<{id: string}, {message: string; code: number}>;
```

Exercise 15-5: A Stricter Omit Type

Earlier in the book, you worked with the `omit` type helper, which allows you to create a new type that omits specific properties from an existing type.

Interestingly, the `omit` helper also lets you try to omit keys that don't exist in the type you're trying to omit from.

In this example, you're trying to omit the property `b` from a type that has only the property `a`:

```
type Example = Omit<{a: string}, "b">;
```

Since `b` isn't a part of the type, you might anticipate TypeScript would show an error, but it doesn't.

Instead, you want to implement a `StrictOmit` type that accepts only keys that exist in the provided type. Otherwise, an error should be shown.

Here's the start of `StrictOmit`, which currently has an error under `K`:

```
type StrictOmit<T, K> = Omit<T, K>; // red squiggly line under the second K
```

```
// hovering over K shows:
```

```
Type 'K' does not satisfy the constraint 'string | number | symbol'.
```

Currently the `StrictOmit` type behaves the same as a regular `Omit`, as evidenced by this failing `@ts-expect-error` directive:

```
type ShouldFail = StrictOmit<  
  // @ts-expect-error // red squiggly line under @ts-expect-error  
  {a: string},
```

```
    "b"  
>;
```

Your task is to update `StrictOmit` so that it accepts only keys that exist in the provided type τ . If a nonvalid key κ is passed, TypeScript should return an error.

Here are some tests to show how `StrictOmit` should behave:

```
type tests = [  
  Expect<Equal<StrictOmit<{a: string; b: number}, "b">, {a:  
    string}>>,  
  Expect<Equal<StrictOmit<{a: string; b: number}, "b" |  
    "a">, {}>>,  
  Expect<  
    Equal<StrictOmit<{a: string; b: number}, never>, {a: str  
ing; b: number}>  
  >,  
>];
```

You'll need to remember `keyof` and how to constrain type parameters to complete this exercise.

See <https://totalts.link/essentials-15-5/>.

Solution

Here's the starting point of the `StrictOmit` type and the `ShouldFail` example that you need to fix:

```
type StrictOmit<T, K> = Omit<T, K>;  
  
type ShouldFail = StrictOmit<  
  {a: string},  
  // @ts-expect-error // red squiggly line under @ts-expect-  
error  
  "b"  
>;
```

Your goal is to update `StrictOmit` so that it accepts only keys that exist in the provided type `T`. If a nonvalid key `K` is passed, TypeScript should return an error.

Since the `ShouldFail` type has a key of `a`, you'll start by updating the `StrictOmit`'s `K` to extend `a`:

```
type StrictOmit<T, K extends "a"> = Omit<T, K>;
```

Removing the `@ts-expect-error` directive from `ShouldFail` will now show an error under `"b"`:

```
type ShouldFail = StrictOmit<
  {a: string},
  "b" // red squiggly line under "b"
>;

// hovering over "b" shows:
Type '"b"' does not satisfy the constraint '"a"'.
```

This shows you that the `ShouldFail` type is failing as expected.

However, you want to make this more dynamic by specifying that `K` will extend any key in the object `T` that is passed in. You can do this by changing the constraint from `"a"` to `keyof T`:

```
type StrictOmit<T, K extends keyof T> = Omit<T, K>;
```

Now in the `StrictOmit` type, `K` is bound to extend the keys of `T`. This imposes a limitation on the type parameter `K` that it must belong to the keys of `T`.

With this change, all the tests pass as expected with any keys that are passed in:

```
type tests = [
  Expect<Equal<StrictOmit<{a: string; b: number}, "b">, {a:
string}>>,
  Expect<Equal<StrictOmit<{a: string; b: number}, "b" |
```

```
"a">, {}>>,
  Expect<
    Equal<StrictOmit<{a: string; b: number}, never>, {a: string; b: number}>
  >,
];
```

Exercise 15-6: Route Matching

Here you have a route typed as `AbsoluteRoute`:

```
type AbsoluteRoute = string;

const goToRoute = (route: AbsoluteRoute) => {
  // . . .
};
```

You're expecting that the `AbsoluteRoute` will represent any string that has a forward slash at the start of it. For example, you'd expect the following strings to be valid routes:

```
goToRoute("/home");
goToRoute("/about");
goToRoute("/contact");
```

However, if you attempt to pass a string that doesn't start with a forward slash, no error is currently generated:

```
goToRoute(
  // @ts-expect-error // red squiggly line under @ts-expect-error
  "somewhere",
);
```

Because `AbsoluteRoute` is currently typed as `string`, TypeScript fails to flag this as an error. Your task is to refine the `AbsoluteRoute` type to accurately expect that strings begin with a forward slash.

See <https://totalts.link/essentials-15-6/>.

Solution

To specify that `AbsoluteRoute` is a string that begins with a forward slash, you'll use a template literal type.

Here's how you could create a type representing a string that begins with a forward slash, followed by either "home", "about", or "contact":

```
type AbsoluteRoute = `/${"home" | "about" | "contact"}`;
```

With this setup, your tests would pass, but you'd be limited to only those three routes. Instead, you want to allow for any string that begins with a forward slash.

To do this, you can just use the `string` type in the template literal:

```
type AbsoluteRoute = `/${string}`;
```

With this change, the `somewhere` string will cause an error since it does not begin with a forward slash:

```
goToRoute(  
  // @ts-expect-error  
  "somewhere",  
);
```

Exercise 15-7: Sandwich Permutations

In this exercise, you want to build a `Sandwich` type.

This `Sandwich` is expected to encompass all possible combinations of three types of bread ("rye", "brown", "white") and three types of filling ("cheese", "ham", "salami"):

```
type BreadType = "rye" | "brown" | "white";  
  
type Filling = "cheese" | "ham" | "salami";  
type Sandwich = unknown;
```

As shown in the test, there are several possible combinations of bread and filling:

```
type tests = [  
  Expect<  
    Equal<  
      Sandwich,  
      | "rye sandwich with cheese"  
      | "rye sandwich with ham"  
      | "rye sandwich with salami"  
      | "brown sandwich with cheese"  
      | "brown sandwich with ham"  
      | "brown sandwich with salami"  
      | "white sandwich with cheese"  
      | "white sandwich with ham"  
      | "white sandwich with salami"  
    >  
  >,  
];
```

Your task is to use a template literal type to define the Sandwich type. This can be done in just one line of code!

See <https://totalts.link/essentials-15-7/>.

Solution

Following the pattern of the tests, you can see that the desired results are named:

```
bread "sandwich with" filling
```

That means you should pass the BreadType and Filling unions to the Sandwich template literal with the string "sandwich with" in between:

```
type BreadType = "rye" | "brown" | "white";  
type Filling = "cheese" | "ham" | "salami";  
type Sandwich = `${BreadType} sandwich with ${Filling}`;
```

TypeScript generates all the possible combinations, resulting in the type Sandwich being defined as follows:

```
| "rye sandwich with cheese"
| "rye sandwich with ham"
| "rye sandwich with salami"
| "brown sandwich with cheese"
| "brown sandwich with ham"
| "brown sandwich with salami"
| "white sandwich with cheese"
| "white sandwich with ham"
| "white sandwich with salami"
```

Exercise 15-8: Attribute Getters

Here you have an Attributes interface that contains a firstName, lastName, and age:

```
interface Attributes {
  firstName: string;
  lastName: string;
  age: number;
}
```

Following that, you have AttributeGetters, which is currently typed as unknown:

```
type AttributeGetters = unknown;
```

As shown in the tests, you expect AttributeGetters to be an object composed of functions. When invoked, each of these functions should return a value matching the key from the Attributes interface:

```
type tests = [
  Expect< // red squiggly line under Equal<>
    Equal<
      AttributeGetters,
```

```
    {
      firstName: () => string;
      lastName: () => string;
      age: () => number;
    }
  >
  >,
];
```

Your task is to define the `AttributeGetters` type so that it matches the expected output. To do this, you'll need to iterate over each key in `Attributes` and produce a function as a value that then returns the value of that key.

See <https://totalts.link/essentials-15-8/>.

Solution

The challenge is to derive the shape of `AttributeGetters` based on the `Attributes` interface:

```
interface Attributes {
  firstName: string;
  lastName: string;
  age: number;
}
```

To do this, you'll use a mapped type. You'll start by using `[K in keyof Attributes]` to iterate over each key in `Attributes`. Then, you'll create a new property for each key, which will be a function that returns the type of the corresponding property in `Attributes`:

```
type AttributeGetters = {
  [K in keyof Attributes]: () => Attributes[K];
};
```

The `Attributes[K]` part is the key to solving this challenge. It allows you to index into the `Attributes` object and return the actual values of each key.

With this approach, you get the expected output, and all tests pass as expected:

```
type tests = [  
  Expect<  
    Equal<  
      AttributeGetters,  
      {  
        firstName: () => string;  
        lastName: () => string;  
        age: () => number;  
      }  
    >  
  >,  
];
```

Exercise 15-9: Renaming Keys in a Mapped Type

After creating the `AttributeGetters` type in the previous exercise, it would be nice to rename the keys to be more descriptive.

Here's a test case for `AttributeGetters` that currently has an error:

```
type tests = [  
  Expect<  
    Equal<  
      AttributeGetters,  
      {  
        getFirstName: () => string;  
        getLastName: () => string;  
        getAge: () => number;  
      }  
    >  
  >,  
];
```

Your challenge is to adjust the `AttributeGetters` type to remap the keys as specified. You'll need to use the `as` keyword, template literals, and TypeScript's built-in `Capitalize<string>` type helper.

See <https://totalts.link/essentials-15-9/>.

Solution

Your goal is to create a new mapped type, `AttributeGetters`, that changes each key in `Attributes` into a new key that begins with “get” and capitalizes the original key. For example, `firstName` would become `getFirstName`.

Before looking at the solution, let’s look at an incorrect approach:

Attempt 1

It might be tempting to think that you should transform `keyof Attributes` before it even gets to the mapped type.

To do this, you’d be creating a `NewAttributeKeys` type and setting it to a template literal with `keyof Attributes` in it added to get:

```
type NewAttributeKeys = `get${keyof Attributes}`;
```

However, hovering over `NewAttributeKeys` shows that it’s not quite right:

```
// hovering over NewAttributeKeys shows:  
type NewAttributeKeys = "getfirstName" | "getlastName" | "getage";
```

Adding the global `Capitalize` helper results in the keys being formatted correctly:

```
type NewAttributeKeys = `get${Capitalize<keyof Attributes>}`;
```

Since you have formatted keys, you can now use `NewAttributeKeys` in the map type:

```
type AttributeGetters = {  
  [K in NewAttributeKeys]: () => Attributes[K]; // red squiggly line under Attributes[K]
```

```
};
```

```
// hovering over Attributes[K] shows:  
Type 'K' cannot be used to index type 'Attributes'.
```

However, there is a problem. You can't use `K` to index `Attributes` because each `k` has changed and no longer exists on `Attributes`.

You need to maintain access to the original key in the mapped type.

Attempt 2

To maintain access to the original key, you can use the `as` keyword.

Instead of using the `NewAttributeKeys` type you tried before, you can update the map type to use `keyof Attributes` as followed by the transformation you want to make:

```
type AttributeGetters = {  
  [K in keyof Attributes as `get${Capitalize<K>}`]: () => At  
tributes[K];  
};
```

You now iterate over each key `K` in `Attributes` and use it in a template literal type that prefixes “get” and capitalizes the original key. Then, the value paired with each new key is a function that returns the type of the original key in `Attributes`.

Now when you hover over the `AttributeGetters` type, you see that it's correct and the tests pass as expected:

```
// hovering over AttributeGetters shows:  
type AttributeGetters = {  
  getFirstName: () => string;  
  getLastName: () => string;  
  getAge: () => number;  
};
```

Summary

This chapter covered advanced TypeScript type design techniques that make your code more maintainable and expressive.

Generic types turn types into reusable “type functions” by using angle brackets, such as `<TContent>`. They support multiple parameters, defaults with `=`, and constraints with `extends` to ensure type safety while reducing repetition.

Template literal types enforce string patterns using backticks and interpolation. Types like ``${string}.png`` ensure that strings match specific formats. Combined with unions, they generate all possible combinations for type-safe APIs.

Conditional types add `if-else` logic using ternary operators: `T extends any[] ? T : T[]`. While powerful, they’re mainly useful in library code rather than application development.

Mapped types iterate over object keys with `[K in keyof T]` to transform properties. The `as` keyword enables key remapping while maintaining access to original types through indexed access.

16

BUILDING POWERFUL SHARED UTILITIES



It's commonly thought that there are two levels of TypeScript complexity.

On one end, you have library development. Here you take advantage of many of TypeScript's most arcane and powerful features. You'll need conditional types, mapped types, generics, and much more to create a library that's flexible enough to be used in a variety of scenarios.

On the other end, you have application development. Here, you're mostly concerned with making sure your code is type-safe. You want to make sure your types reflect what's happening in your application. Any complex types are housed in libraries you use. You'll need to know your way around TypeScript, but you won't need to use its advanced features much.

This is the rule of thumb most of the TypeScript community follows. "It's too complex for application code. You'll only need it in libraries." But there's a third level that's often overlooked: the */utils* folder.

If your application gets big enough, you'll start capturing common patterns in a set of reusable functions. These functions, like `groupBy`, `debounce`, and `retry`, might be used hundreds of times across a large application. They're like mini-libraries within the scope of your application.

Understanding how to build these types of functions can save your team a lot of time. Capturing common patterns means your code becomes easier to maintain and faster to build.

In this chapter, we'll cover how to build these functions. We'll start with generic functions and then head to type predicates, assertion functions, and function overloads.

Generic Functions

You've seen that in TypeScript, functions can receive not just values as arguments but types too. Here, you're passing a *value* and a *type* to new Set():

```
const set = new Set<number>([1, 2, 3]);
//           ^^^^^^^^^      ^^^^^^^^^^^
//           Type           value
```

Here, number is the type passed into the angle brackets, and [1, 2, 3] is the value in parentheses. This works because new Set() is a generic function.

Any function that can't receive types is a regular function, like JSON.parse:

```
const obj = JSON.parse<{hello: string}>>('{"hello": "world"}');
// red squiggly line under {hello: string}
// Expected 0 type arguments, but got 1.
```

TypeScript is telling you that JSON.parse doesn't accept type arguments because it's not generic. But what makes a function generic?

A function is generic if it declares a type parameter. Here's a generic function that takes a type parameter T:

```
function identity<T>(arg: T): T {
  //           ^^^ Type parameter
  return arg;
}
```

This example uses the function keyword, but you can also use arrow function syntax:

```
const identity = <T>(arg: T): T => arg;
```

You can even declare a generic function as a type:

```
type Identity = <T>(arg: T) => void;  
const identity: Identity = (arg) => arg;
```

Now you can pass a type argument to identity:

```
const answer = identity<number>(42);
```

This answer ends up as the number 42.

Generic Function Type Alias vs. Generic Type

It's important not to confuse the syntax for a generic type with the syntax for a type alias for a generic function. They look similar to the untrained eye. Here's the difference:

```
// Type alias for a generic function  
type Identity = <T>(arg: T) => void;  
//           ^^^  
// Type parameter belongs to the function.  
  
// Generic type  
type Identity<T> = (arg: T) => void;  
//           ^^^  
// Type parameter belongs to the type.
```

As the first example shows, with a type alias the `<T>` type parameter belongs to the function because it is attached to the parentheses. For a generic type, the `<T>` type parameter is attached to the type's name. It's all about the position of the type parameter.

Missing or Conflicting Type Arguments

When you looked at generic types, you saw that TypeScript *requires* you to pass in all type arguments when you use a generic type:

```
type StringArray = Array<string>;

type AnyArray = Array; // red squiggly line under Array

// hovering over Array shows:
Generic type 'Array<T>' requires 1 type argument(s).
```

This code has an error because `Array` is a generic type, and `AnyArray` doesn't pass it a type argument.

However, this is not true of generic functions. If you don't pass a type argument to a generic function, TypeScript won't complain:

```
function identity<T>(arg: T): T {
    return arg;
}

const result = identity(42); // No error!
```

This example demonstrates what makes generic functions some of the best tools in TypeScript. If you don't pass a type argument, TypeScript will attempt to *infer* it from the function's runtime arguments.

The `identity` function simply takes in an argument and returns it. We've referenced the type parameter in the runtime parameter: `arg: T`. This means that if you don't pass in a type argument, `T` will be inferred from the type of `arg`. In the previous example, `result` will be typed as `42`, but every time the function is called, it can potentially return a different type:

```
const result1 = identity("hello"); // result1: 'hello'
const result2 = identity({hello: "world"}); // result2: {hello: 'world'}
const result3 = identity([1, 2, 3]); // result3: number[]
```

This ability means your functions can understand what types they're working with and alter their suggestions and errors accordingly. It's TypeScript at its most powerful and flexible.

Let's go back to specifying type arguments instead of inferring them. What happens if in your type argument you pass conflicts with the runtime argument?

Let's try it with our identity function:

```
const result = identity<string>(42); // red squiggly line under 42
```

```
// hovering over 42 shows:  
Argument of type '42' is not assignable to parameter of type 'string'.
```

Here, TypeScript is telling us that 42 is not a string. This is because you've explicitly told TypeScript that `T` should be a string, which conflicts with the runtime argument.

Passing type arguments is an instruction to TypeScript override inference. If you pass in a type argument, TypeScript will use it as the source of truth. If you don't, TypeScript will use the type of the runtime argument as the source of truth.

There Is No Such Thing as a “Generic”

A quick note on terminology here. TypeScript “generics” has a reputation for being difficult to understand. A large part of that is probably based on how people use the word *generic*.

A lot of people think of a “generic” as a noun that is part of TypeScript. If you ask someone where the “generic” is in this piece of code, they will probably point to the `<T>`:

```
const identity = <T>(arg: T) => arg;
```

In the following example, someone might describe the code as “passing a generic to Set”:

```
const set = new Set<number>([1, 2, 3]);
```

This terminology gets very confusing, so to keep things clear, split them into different terms:

Type parameter The `<T>` in `identity<T>`

Type argument The number passed to `Set<number>`

Generic class/function/type A class, function, or type that declares a type parameter

When you break generics down into these terms, it becomes much easier to understand.

The Problem Generic Functions Solve

Let's put what you've learned into practice.

Consider this function called `getFirstElement` that takes an array and returns the first element:

```
const getFirstElement = (arr: any[]) => {  
  return arr[0];  
};
```

This function is dangerous. Because it accepts an array of type `any`, the value returned by `getFirstElement` is also `any`:

```
const first = getFirstElement([1, 2, 3]);  
  
// hovering over first shows:  
const first: any;
```

As you've seen, `any` can cause havoc in your code. Anyone who uses this function will be unwittingly opting out of TypeScript's type safety. So, how can you fix this?

TypeScript needs to understand the type of the array being passed in and use it to type what's returned. You need to make `getFirstElement` generic.

To do this, add a type parameter `TMember` before the function's parameter list, and then use `TMember[]` as the type for the array:

```
const getFirstElement = <TMember>(arr: TMember[]) => {  
  return arr[0];  
};
```

Just like generic types, it's common to prefix your type parameters with `T` to differentiate them from normal types.

Now when you call `getFirstElement`, TypeScript will infer the type of `TMember` based on the argument you pass in:

```
const firstNumber = getFirstElement([1, 2, 3]);  
const firstString = getFirstElement(["a", "b", "c"]);  
  
// hovering over firstNumber shows:  
const firstNumber: number | undefined;  
  
// hovering over firstString shows:  
const firstString: string | undefined;
```

In this case, `firstNumber` is typed as `number | undefined`, and `firstString` is typed as `string | undefined`. The `getFirstElement` function is now type-safe, and the type of the array passed in is the type of the thing you get back.

Debugging the Inferred Type of Generic Functions

When you're working with generic functions, it can be hard to know what type TypeScript has inferred. However, with a carefully placed hover, you can find out.

When you call the `getFirstElement` function, you can hover over the function name to see what TypeScript has inferred:

```
const first = getFirstElement([1, 2, 3]);  
  
// hovering over getFirstElement shows:  
const getFirstElement: <number>(arr: number[]) => number
```

Notice that within the angle brackets, TypeScript has inferred that `TMember` is `number` because an array of numbers was passed in.

This can be useful when you have more complex functions with multiple type parameters to debug. If you get stuck, try creating temporary function calls in the same file to see what TypeScript has inferred.

Type Parameter Defaults

Just like generic types, you can set default values for type parameters in generic functions. This can be useful when runtime arguments to the function are optional:

```
const createSet = <T = string>(arr?: T[]) => {  
  return new Set(arr);  
};
```

Here you set the default type of `T` to `string`. This means that if you don't pass in a type argument, TypeScript will assume `T` is `string`:

```
const defaultSet = createSet(); // Set<string>
```

The default doesn't impose a constraint on the type of `T`. This means you can still pass in any type you want:

```
const numberSet = createSet<number>([1, 2, 3]); // Set<number>
```

If you don't specify a default and TypeScript can't infer the type from the runtime arguments, it will default to `unknown`:

```
const createSet = <T>(arr?: T[]) => {  
  return new Set(arr);  
};  
  
const unknownSet = createSet(); // Set<unknown>
```

Here, you've removed the default type of τ , and TypeScript has defaulted to unknown.

Constraining Type Parameters

You can also add constraints to type parameters in generic functions. This can be useful when you want to ensure that a type has certain properties.

Let's imagine a `removeId` function that takes an object and removes the `id` property:

```
const removeId = <TObj>(obj: TObj) => {  
  const {id, . . .rest} = obj; // red squiggly line under id  
  return rest;  
};  
  
// hovering over id shows:  
Property 'id' does not exist on type 'unknown'.
```

The `TObj` type parameter, when used without a constraint, is treated as unknown. This means that TypeScript doesn't know if `id` exists on `obj`.

To fix this, you can add a constraint to `TObj` that ensures that it has an `id` property:

```
const removeId = <TObj extends {id: unknown}>(obj: TObj) =>  
{  
  const {id, . . .rest} = obj;  
  return rest;  
};
```

Now when you use `removeId`, TypeScript will error if you don't pass in an object with an `id` property:

```
const result = removeId({name: "Alice"}); // red squiggly li  
ne under name  
  
// hovering over name shows:  
Object literal may only specify known properties, and 'name'  
does not exist in type '{id: unknown;}'.
```

But if you pass in an object with an `id` property, TypeScript will know that `id` has been removed:

```
const result = removeId({id: 1, name: "Alice"});

// hovering over result shows:
const result: Omit<
  {
    id: number;
    name: string;
  },
  "id"
>;
```

Note how clever TypeScript is being here. Even though a return type wasn't specified for `removeId`, TypeScript has inferred that `result` is an object with all the properties of the input object except `id`.

Type Predicates

You were introduced to type predicates in [Chapter 5](#). Recall that type predicates are used to capture reusable logic that narrows the type of a variable.

For example, say you want to ensure that a variable is an `Album` before accessing its properties or passing it to a function that requires an `Album`.

You can write an `isAlbum` function that takes in an input and checks for all the required properties:

```
function isAlbum(input: unknown) {
  return (
    typeof input === "object" &&
    input !== null &&
    "id" in input &&
    "title" in input &&
    "artist" in input &&
    "year" in input
  );
}
```

This function expects that all properties will be present, but if you hover over `isAlbum`, you can see a rather ugly type signature:

```
// hovering over isAlbum shows:
function isAlbum(
  input: unknown,
): input is object &
  Record<"id", unknown> &
  Record<"title", unknown> &
  Record<"artist", unknown> &
  Record<"year", unknown>;
```

This is technically correct: a big intersection between an object and a bunch of Records. But it's not very helpful.

When you try to use `isAlbum` to narrow the type of a value, TypeScript will infer many of the values as `unknown`:

```
const run = (maybeAlbum: unknown) => {
  if (isAlbum(maybeAlbum)) {
    maybeAlbum.name.toUpperCase(); // red squiggly line unde
r name
  }
};

// hovering over name shows:
Object is of type 'unknown'.
```

To attempt to fix this, you might try to add even more checks to `isAlbum` to ensure that it also is checking the types of all the properties:

```
function isAlbum(input: unknown) {
  return (
    typeof input === "object" &&
    input !== null &&
    "id" in input &&
    "title" in input &&
    "artist" in input &&
    "year" in input &&
  )
}
```

```
    typeof input.id === "number" &&
    typeof input.title === "string" &&
    typeof input.artist === "string" &&
    typeof input.year === "number"
  );
}
```

But doing so actually breaks the narrowing! Once the checks become too complex, TypeScript defaults to inferring boolean as the return type. This means you're back to square one with your narrowing:

```
const run = (maybeAlbum: unknown) => {
  if (isAlbum(maybeAlbum)) {
    maybeAlbum.name.toUpperCase(); // red squiggly line under
    r maybeAlbum
  }
};

// hovering over maybeAlbum shows:
'maybeAlbum' is of type 'unknown'.
```

You can fix this by adding a type predicate to the function. In this case, input is Album:

```
function isAlbum(input: unknown): input is Album {
  return (
    typeof input === "object" &&
    input !== null &&
    "id" in input &&
    "title" in input &&
    "artist" in input &&
    "year" in input
  );
}
```

Now when you use the `isAlbum` function, TypeScript will know that the type of the value has been narrowed to `Album`:

```
const run = (maybeAlbum: unknown) => {  
  if (isAlbum(maybeAlbum)) {  
    maybeAlbum.name.toUpperCase(); // No error!  
  }  
};
```

In this code, TypeScript understands that if it reaches the call to `toUpperCase`, there must be an album name. This pattern is much more readable for complex type guards.

However, it's important to realize that authoring your own type predicates can be a little dangerous! If the type predicate doesn't accurately reflect the type being checked, TypeScript won't catch that discrepancy:

```
function isAlbum(input): input is Album {  
  return typeof input === "object";  
}
```

In this example, any object passed to `isAlbum` will be considered an `Album`, even if it doesn't have the required properties. This is a common pitfall when working with type predicates; it's important to consider them about as unsafe as `as` and `!`.

Assertion Functions

Assertion functions look similar to type predicates, but they're used slightly differently. Instead of returning a boolean to indicate whether a value is of a certain type, assertion functions throw an error if the value isn't of the expected type.

Here's how you could rework the `isAlbum` type predicate to be an `assertIsAlbum` assertion function:

```
function assertIsAlbum(input: unknown): asserts input is Album {  
  if (  
    typeof input === "object" &&  
    input !== null &&  
    "id" in input &&
```

```
    "title" in input &&  
    "artist" in input &&  
    "year" in input  
  ) {  
    throw new Error("Not an Album!");  
  }  
}
```

The `assertIsAlbum` function takes in an input of type `unknown` and asserts that it is an `Album` using the `asserts input is Album` syntax.

This means the narrowing is more aggressive. Instead of checking within an `if` statement, the function call itself is enough to assert that the input is an `Album`:

```
function getAlbumTitle(item: unknown) {  
  // 'item' is unknown here  
  
  assertIsAlbum(item);  
  
  // After the assertion, 'item' is narrowed to 'Album'.  
  console.log(item.title);  
}
```

Assertion functions can be useful when you want to ensure that a value is of a certain type before proceeding with further operations.

But just like type predicates, assertion functions can be misused. If the assertion function doesn't accurately reflect the type being checked, it can lead to runtime errors.

For example, if the `assertIsAlbum` function doesn't check for all the required properties of an `Album`, it can lead to unexpected behavior:

```
function assertIsAlbum(input: unknown): asserts input is Album {  
  if (typeof input === "object") {  
    throw new Error("Not an Album!");  
  }  
}
```

```
let item = null;

assertIsAlbum(item);

// 'item' is now narrowed to 'Album', even though it's
// null at runtime.
item.title;
```

In this case, the `assertIsAlbum` function doesn't check for the required properties of an `Album`; it just checks if `typeof input` is `"object"`. This means we've left ourselves open to a stray `null`. The famous JavaScript quirk where `typeof null === 'object'` will cause a runtime error when we try to access the `title` property.

Function Overloads

Function overloads provide a way to define multiple function signatures for a single function implementation. In other words, you can define different ways to call a function, each with its own set of parameters and return types. It's an interesting technique for creating a flexible API that can handle different use cases while maintaining type safety.

To demonstrate how function overloads work, you'll create a `searchMusic` function that allows for different ways to perform a search based on the provided arguments.

To define function overloads, the same function definition is written multiple times with different parameter and return types. Each definition is called an *overload signature* and is separated by semicolons. You'll also need to use the `function` keyword each time.

For the `searchMusic` example, you want to allow users to search by providing an artist, genre, and year. But for legacy reasons, you want users to be able to pass them as a single object or as separate arguments.

Here's how you could define these function overload signatures. The first signature takes in three separate arguments, while the second signature takes in a single object with the properties:

```
function searchMusic(artist: string, genre: string, year: number): void;
```

```
function searchMusic(criteria: {  
    // red squiggly line under searchMusic  
    artist: string;  
    genre: string;  
    year: number;  
}): void;  
  
// hovering over searchMusic shows:  
Function implementation is missing or not immediately following the declaration.
```

The overloads should allow for artist, genre, and year to be passed in as either individual parameters or a single object, but you're getting an error. The issue is that you've declared some ways this function should be declared, but you haven't provided the implementation yet.

The Implementation Signature

The implementation signature is the function declaration that contains the actual logic for the function. It is written below the overload signatures and must be compatible with all the defined overloads.

In this case, the implementation signature will take in a parameter called `artistOrCriteria` that can be either a string or an object with the specified properties. In the function, you'll check the type of `artistOrCriteria` and perform the appropriate search logic based on the provided arguments:

```
function searchMusic(artist: string, genre: string, year: number): void;  
function searchMusic(criteria: {  
    artist: string;  
    genre: string;  
    year: number;  
}): void;  
function searchMusic(  
    artistOrCriteria: string | {artist: string; genre: string;  
    year: number},  
    genre?: string,  
    year?: number,
```

```
) : void {
  if (typeof artistOrCriteria === "string") {
    // Search with separate arguments
    search(artistOrCriteria, genre, year);
  } else {
    // Search with object
    search(
      artistOrCriteria.artist,
      artistOrCriteria.genre,
      artistOrCriteria.year,
    );
  }
}
```

In this code, the implementation signature combines the overloads into a single function definition where `artistOrCriteria` will be either a string for the artist or an object with the criteria, and the genre and year become optional parameters.

With the implementation signature in place, you can call the `searchMusic` function with the different arguments defined in the overloads:

```
searchMusic("King Gizzard and the Lizard Wizard", "Psychedelic Rock", 2021);
searchMusic({
  artist: "Tame Impala",
  genre: "Psychedelic Rock",
  year: 2015,
});
```

Both of these searches would work just fine. However, TypeScript will warn you if you attempt to pass in an argument that doesn't match any of the defined overloads. In this case, you mix the criteria object with the genre parameter:

```
searchMusic(
  // red squiggly line under searchMusic
  {
```

```
        artist: "Tame Impala",
        genre: "Psychedelic Rock",
        year: 2015,
    },
    "Psychedelic Rock",
);

// hovering over searchMusic shows:
No overload expects 2 arguments, but overloads do exist that
expect either 1 or 3 arguments.
```

This error shows that you're trying to call `searchMusic` with two arguments, but the overloads expect only one or three arguments.

Function Overloads vs. Unions

Function overloads can be useful when you have multiple call signatures spread over different sets of arguments. In the previous example, you can call the function with either one argument or three.

When you have the same number of arguments but different types, you should use a union type instead of function overloads. For example, if you want to allow the user to search by either artist name or criteria object, you could use a union type:

```
function searchMusic(
    query: string | {artist: string; genre: string; year: number},
): void {
    if (typeof query === "string") {
        // Search by artist
        searchByArtist(query);
    } else {
        // Search by all
        search(query.artist, query.genre, query.year);
    }
}
```

This uses far fewer lines of code than defining two overloads and an implementation.

Exercise 16-1: Making a Function Generic

Here you have a function `createStringMap`. The purpose of this function is to generate a `Map` with keys as strings and values of the type passed in as arguments:

```
const createStringMap = () => {  
  return new Map();  
};
```

As it currently stands, you get back a `Map<any, any>`. However, the goal is to make this function generic so that you can pass in a type argument to define the type of the values in the `Map`.

For example, if you pass in `number` as the type argument, the function should return a `Map` with values of type `number`:

```
const numberMap = createStringMap<number>(); // red squiggly  
line under number  
  
numberMap.set("foo", 123);  
numberMap.set(  
  "bar",  
  // @ts-expect-error  
  true,  
);
```

Likewise, if you pass in an object type, the function should return a `Map` with values of that type:

```
const objMap = createStringMap<{a: number}>(); // red squiggly  
line under {a: number}  
  
objMap.set("foo", {a: 123});  
  
objMap.set(  
  "bar",  
  // @ts-expect-error // red squiggly line under @ts-expect-error
```

```
    {b: 123},  
  );
```

The function should also default to unknown if no type is provided:

```
const unknownMap = createStringMap();  
  
type test = Expect<Equal<typeof unknownMap, Map<string, unknown>>>;  
// red squiggly line under Equal<>
```

Your task is to transform `createStringMap` into a generic function capable of accepting a type argument to describe the values of `Map`. Make sure it functions as expected for the provided test cases.

See <https://totalts.link/essentials-16-1/>.

Solution

The first thing you can do to make this function generic is to add a type parameter `T`:

```
const createStringMap = <T>() => {  
  return new Map();  
};
```

With this change, the `createStringMap` function can now handle a type argument `T`. The error has disappeared from the `numberMap` variable, but the function is still returning a `Map<any, any>`:

```
const numberMap = createStringMap<number>();  
  
// hovering over createStringMap shows:  
const createStringMap: <number>() => Map<any, any>;
```

You need to specify the types for the map entries. Since you know that the keys will always be strings, you'll set the first type argument of `Map` to `string`. For the values, you'll use the type parameter `T`:

```
const createStringMap = <T>() => {  
  return new Map<string, T>();  
};
```

Now the function can correctly type the map's values. If you don't pass in a type argument, the function will default to unknown:

```
const unknownMap = createStringMap();  
  
// hovering over unknownMap shows:  
const unknownMap: Map<string, unknown>;
```

Through these steps, you've successfully transformed `createStringMap` from a regular function into a generic function capable of receiving type arguments.

Exercise 16-2: Default Type Arguments

After making the `createStringMap` function generic in Exercise 16-1, calling it without a type argument defaults to values being typed as `unknown`:

```
const stringMap = createStringMap();  
  
// hovering over stringMap shows:  
const stringMap: Map<string, unknown>;
```

Your goal is to add a default type argument to the `createStringMap` function so that it defaults to `string` if no type argument is provided. Note that you will still be able to override the default type by providing a type argument when calling the function.

See <https://totalts.link/essentials-16-2/>.

Solution

The syntax for setting default types for generic functions is the same as for generic types:

```
const createStringMap = <T = string>() => {  
  return new Map<string, T>();  
};
```

By using the `T = string` syntax, you tell the function that if no type argument is supplied, it should default to `string`.

Now when you call `createStringMap()` without a type argument, you end up with a `Map` where both keys and values are `string`:

```
const stringMap = createStringMap();  
  
// hovering over stringMap shows:  
const stringMap: Map<string, string>;
```

If you attempt to assign a number as a value, TypeScript gives you an error because it expects a `string`:

```
stringMap.set(  
  "bar",  
  123, // red squiggly line under 123  
);
```

However, you can still override the default type by providing a type argument when calling the function:

```
const numberMap = createStringMap<number>();  
numberMap.set("foo", 123);
```

In this code, `numberMap` will result in a `Map` with `string` keys and `number` values, and TypeScript will give an error if you try assigning a non-number value:

```
numberMap.set(  
  "bar",  
  // @ts-expect-error
```

```
    true,  
  );  
}
```

Exercise 16-3: Inference in Generic Functions

Consider this `uniqueArray` function:

```
const uniqueArray = (arr: any[]) => {  
  return Array.from(new Set(arr));  
};
```

The function accepts an array as an argument, then converts it to a `Set`, and then returns it as a new array. This is a common pattern for when you want to have unique values in your array.

While this function operates effectively at runtime, it lacks type safety. It transforms any array passed in into `any[]`:

```
it("returns an array of unique values", () => {  
  const result = uniqueArray([1, 1, 2, 3, 4, 4, 5]);  
  
  type test = Expect<Equal<typeof result, number[]>>; // red  
  squiggly line under Equal<>  
  
  expect(result).toEqual([1, 2, 3, 4, 5]);  
});  
  
it("should work on strings", () => {  
  const result = uniqueArray(["a", "b", "b", "c", "c",  
    "c"]);  
  
  type test = Expect<Equal<typeof result, string[]>>; // red  
  squiggly line under Equal<>  
  
  expect(result).toEqual(["a", "b", "c"]);  
});
```

Your task is to boost the type safety of the `uniqueArray` function by making it generic.

Note that in the tests, we do not explicitly provide type arguments when invoking the function. TypeScript should be able to infer the type from the argument.

Adjust the function and insert the necessary type annotations to ensure that the result type in both tests is inferred as `number[]` and `string[]`, respectively.

See <https://totalts.link/essentials-16-3/>.

Solution

The first step is to add a type parameter onto `uniqueArray`. This turns `uniqueArray` into a generic function that can receive type arguments:

```
const uniqueArray = <T>(arr: any[]) => {  
  return Array.from(new Set(arr));  
};
```

Now when you hover over a call to `uniqueArray`, you can see that it is inferring the type as `unknown`:

```
const result = uniqueArray([1, 1, 2, 3, 4, 4, 5]);  
  
// hovering over uniqueArray shows:  
const uniqueArray: <unknown>(arr: any[]) => any[];
```

This is because no type arguments have been passed to it. If there's no type argument and no default, it defaults to `unknown`.

You want the type argument to be inferred as a number because you know that the thing you're getting back is an array of numbers.

So, what you'll do is add a return type of `T[]` to the function:

```
const uniqueArray = <T>(arr: any[]): T[] => {  
  . . .
```

Now the result of `uniqueArray` is inferred as an unknown array:

```
const result = uniqueArray([1, 1, 2, 3, 4, 4, 5]);

// hovering over uniqueArray shows:
const uniqueArray: (arr: any[]) => unknown[];
```

Again, the reason for this is that you haven't passed any type arguments to it. If there's no type argument and no default, it defaults to `unknown`.

If you add a `<number>` type argument to the call, the `result` will now be inferred as a number array:

```
const result = uniqueArray<number>([1, 1, 2, 3, 4, 4, 5]);

// hovering over result shows:
const result: number[];
```

However, at this point there's no relationship between the things you're passing in and the thing you're getting out. Adding a type argument to the call returns an array of that type, but the `arr` parameter in the function itself is still typed as `any[]`.

What you need to do is tell TypeScript that the type of the `arr` parameter is the same type as what is passed in.

To do this, replace `arr: any[]` with `arr: T[]`:

```
const uniqueArray = <T>(arr: T[]): T[] => {
    . . .
}
```

The function's return type is an array of `T`, where `T` represents the type of elements in the array supplied to the function.

Thus, TypeScript can infer the return type as `number[]` for an input array of numbers or `string[]` for an input array of strings, even without explicit return type annotations. As you can see, the tests pass successfully:

```
// Number test
const result = uniqueArray([1, 1, 2, 3, 4, 4, 5]);
```

```
type test = Expect<Equal<typeof result, number[]>>;

// String test
const result = uniqueArray(["a", "b", "b", "c", "c", "c"]);

type test = Expect<Equal<typeof result, string[]>>;
```

If you explicitly pass a type argument, TypeScript will use it. If you don't, TypeScript attempts to infer it from the runtime arguments.

Exercise 16-4: Type Parameter Constraints

Consider this function `addCodeToError`, which accepts a type parameter `TError` and returns an object with a `code` property:

```
const UNKNOWN_CODE = 8000;

const addCodeToError = <TError>(error: TError) => {
  return {
    . . .error,
    code: error.code ?? UNKNOWN_CODE, // red squiggly line under code
  };
};

// hovering over code shows:
Property 'code' does not exist on type 'TError'.
```

If the incoming error doesn't include a code, the function assigns a default `UNKNOWN_CODE`. Currently there is an error under the `code` property.

Currently, there are no constraints on `TError`, which can be of any type. This leads to errors in the tests:

```
it("Should accept a standard error", () => {
  const errorWithCode = addCodeToError(new Error("Oh dear!"));

  type test1 = Expect<Equal<typeof errorWithCode, Error & {code: number}>>;
```

```

// red squiggly line under Equal<>

console.log(errorWithCode.message);

type test2 = Expect<Equal<typeof errorWithCode.message, string>>;
});

it("Should accept a custom error", () => {
  const customErrorWithCode = addCodeToError({
    message: "Oh no!",
    code: 123,
    filepath: "/",
  });

  type test3 = Expect<
    Equal<
      typeof customErrorWithCode,
      {
        message: string;
        code: number;
        filepath: string;
      } & {
        code: number;
      }
    >
  >; // red squiggly line under Equal<>

  type test4 = Expect<Equal<typeof customErrorWithCode.message, string>>;
});

```

Your task is to update the `addCodeToError` type signature to enforce the required constraints so that `TError` is required to have a `message` property and can optionally have a `code` property.

See <https://totalts.link/essentials-16-4/>.

Solution

The syntax to add constraints is the same as what you saw for generic types.

You need to use the `extends` keyword to add constraints to the generic type parameter `TError`. The object passed in is required to have a `message` property of type `string` and can optionally have a `code` of type `number`:

```
const UNKNOWN_CODE = 8000;

const addCodeToError = <TError extends {message: string; code?: number}>(  
  error: TError,  
) => {  
  return {  
    ...error,  
    code: error.code ?? UNKNOWN_CODE,  
  };  
};
```

This change ensures that `addCodeToError` must be called with an object that includes a `message` string property. TypeScript also knows that `code` could be either a number or undefined. If `code` is absent, it will default to `UNKNOWN_CODE`.

These constraints make the tests pass, including the case where you pass in an extra `filepath` property. This is because using `extends` in generics does not restrict you to passing in only the properties defined in the constraint.

Exercise 16-5: Combining Generic Types and Functions

Here you have `safeFunction`, which accepts a function `func` typed as `PromiseFunc` that returns a function itself. However, if `func` encounters an error, it is caught and returned instead:

```
type PromiseFunc = () => Promise<any>;

const safeFunction = (func: PromiseFunc) => async () => {  
  try {  
    const result = await func();  
    return result;  
  } catch (e) {
```



```
    if (e instanceof Error) {
      return e;
    }
    throw e;
  }
};
```

In short, the thing that you get back from `safeFunction` should be either the thing that's returned from `func` or an `Error`.

However, there are some issues with the current type definitions.

The `PromiseFunc` type is currently set to always return `Promise<any>`. This means the function returned by `safeFunction` is supposed to return either the result of `func` or an `Error`, but at the moment, it's just returning `Promise<any>`.

There are several tests that are failing due to these issues:

```
it("should return an error if the function throws", async ()
=> {
  const func = safeFunction(async () => {
    if (Math.random() > 0.5) {
      throw new Error("Something went wrong");
    }
    return 123;
  });

  type test1 = Expect<Equal<typeof func, () => Promise<Error
| number>>>;
  // red squiggly line under Equal<>
  const result = await func();

  type test2 = Expect<Equal<typeof result, Error | number>>;
  // red squiggly line under Equal<>
});

it("should return the result if the function succeeds", async
c () => {
  const func = safeFunction(() => {
    return Promise.resolve(`Hello!`);
  });
```

```

    type test1 =    type test1 = Expect<Equal<typeof func, () =
> Promise<string | Error>>>;
    // red squiggly line under Equal<>

    const result = await func();

    type test2 = Expect<Equal<typeof result, string | Error>>;
    // red squiggly line under Equal<>

    expect(result).toEqual("Hello!");
  });

```

Your task is to update `safeFunction` to have a generic type parameter and update `PromiseFunc` so that it doesn't return `Promise<Any>`. This will require you to combine generic types and functions to ensure that the tests pass successfully.

See <https://totalts.link/essentials-16-5/>.

Solution

Here's the starting point of our `safeFunction`:

```

type PromiseFunc = () => Promise<any>;

const safeFunction = (func: PromiseFunc) => async () => {
  try {
    const result = await func();
    return result;
  } catch (e) {
    if (e instanceof Error) {
      return e;
    }
    throw e;
  }
};

```

The first thing you'll do is update the `PromiseFunc` type to be a generic type. You'll call the type parameter `TResult` to represent the type of the

value returned by the promise, and you'll pass it to the return type of the function:

```
type PromiseFunc<TResult> = () => Promise<TResult>;
```

With this update, you now need to update the `PromiseFunc` in the `safeFunction` to include the type argument:

```
const safeFunction =  
  <TResult>(func: PromiseFunc<TResult>) =>  
  async () => {  
    . . .
```

With these changes in place, when you hover over the `safeFunction` call in the first test, you can see that the type argument is inferred as `number` as expected:

```
it("should return an error if the function throws", async ()  
=> {  
  const func = safeFunction(async () => {  
    if (Math.random() > 0.5) {  
      throw new Error("Something went wrong");  
    }  
    return 123;  
  });  
  . . .
```

```
// hovering over safeFunction shows:  
const safeFunction: <number>(func: PromiseFunc<number>) =>  
Promise<() => Promise<number | Error>>
```

The other tests pass as well.

Whatever you pass into `safeFunction` will be inferred as the type argument for `PromiseFunc`. This is because the type argument is being inferred *in* the generic function.

This combination of generic functions and generic types can make your generic functions a lot easier to read.

Exercise 16-6: Multiple Type Arguments in a Generic Function

After making the `safeFunction` generic in Exercise 16-5, it's been updated to allow for passing arguments:

```
// In safeFunction
async (. . .args: any[]) => {
  try {
    const result = await func(. . .args);
    return result;
  } catch (e) {
    if (e instanceof Error) {
      return e;
    }
    throw e;
  }
};
```

Now that the function being passed into `safeFunction` can receive arguments, the function you get back should *also* contain those arguments and require you to pass them in.

However, as shown in the tests, this isn't working:

```
it("should return the result if the function succeeds", async
c () => {
  const func = safeFunction((name: string) => {
    return Promise.resolve(`hello ${name}`);
  });

  type test1 = Expect<
    Equal<typeof func, (name: string) => Promise<Error | string>>
  >; // red squiggly line under Equal<>
```

For example, in this test the `name` isn't being inferred as a parameter of the function returned by `safeFunction`. Instead, it's actually saying that you

can pass in as many arguments as you want to into the function, which isn't correct:

```
// hovering over func shows:  
const func: (. . .args: any[]) => Promise<string | Error>;
```

Your task is to add a second type parameter to `PromiseFunc` and `safeFunction` to infer the argument types accurately.

As shown in the tests, in some cases no parameters are necessary, and in others a single parameter is needed:

```
it("should return an error if the function throws", async ()  
=> {  
  const func = safeFunction(async () => {  
    if (Math.random() > 0.5) {  
      throw new Error("Something went wrong");  
    }  
    return 123;  
  });  
  
  type test1 = Expect<Equal<typeof func, () => Promise<Error  
| number>>>;  
  // red squiggly line under Equal<>  
  
  const result = await func();  
  
  type test2 = Expect<Equal<typeof result, Error | number>>;  
});  
  
it("should return the result if the function succeeds", asyn  
c () => {  
  const func = safeFunction((name: string) => {  
    return Promise.resolve(`hello ${name}`);  
  });  
  
  type test1 = Expect<  
    Equal<typeof func, (name: string) => Promise<Error | str  
ing>>  
  >; // red squiggly line under Equal<>
```

```
const result = await func("world");

type test2 = Expect<Equal<typeof result, string | Error>>;

expect(result).toEqual("hello world");
});
```

Update the types of the function and the generic type, and make these tests pass successfully.

See <https://totalts.link/essentials-16-6/>.

Solution

Here's how PromiseFunc is currently defined:

```
type PromiseFunc<TResult> = (. . .args: any[]) => Promise<TResult>;
```

The first thing to do is figure out the types of the arguments being passed in. Currently they're set to one value, but they need to be different based on the type of function being passed in.

Instead of having args be of type `any[]`, you want to spread in all of the args and capture the entire array.

To do this, you'll update the type to be `TAargs`. Since args needs to be an array, you'll say that `TAargs` extends `any[]`. Note that this doesn't mean that `TAargs` will be typed as `any` but rather that it will accept any kind of array:

```
type PromiseFunc<TAargs extends any[], TResult> = (
  . . .args: TAargs
) => Promise<TResult>;
```

You might have tried this with `unknown[]`, but `any[]` is the only thing that works in this scenario.

Now you need to update the `safeFunction` so that it has the same arguments as `PromiseFunc`. To do this, you'll add `TAargs` to its type

parameters.

Note that you also need to update the args for the async function to be of type TArgs:

```
const safeFunction =  
  <TArgs extends any[], TResult>(func: PromiseFunc<TArgs, TResult>) =>  
  async (. . .args: TArgs) => {  
    try {  
      const result = await func(. . .args);  
      return result;  
    } catch (e) {  
      . . .  
    }  
  }  
}
```

This change is necessary to make sure the function returned by `safeFunction` has the same typed arguments as the original function.

With these changes, all the tests pass as expected.

Exercise 16-7: Assertion Functions

This exercise starts with an interface `User`, which has properties `id` and `name`. Then, you have an interface `AdminUser`, which extends `User`, inheriting all its properties and adding a `roles` string array property:

```
interface User {  
  id: string;  
  name: string;  
}  
  
interface AdminUser extends User {  
  roles: string[];  
}
```

The function `assertIsAdminUser` accepts either a `User` or an `AdminUser` object as an argument. If the `roles` property isn't present in the argument, the function throws an error:

```
function assertIsAdminUser(user: User | AdminUser) {  
  if (!("roles" in user)) {  
    throw new Error("User is not an admin");  
  }  
}
```

This function's purpose is to verify you are able to access properties that are specific to the AdminUser, such as roles.

In the `handleRequest` function, you call `assertIsAdminUser` and expect the type of `user` to be narrowed down to `AdminUser`.

But as shown in this test case, it doesn't work as expected:

```
const handleRequest = (user: User | AdminUser) => {  
  type test1 = Expect<Equal<typeof user, User | AdminUser>>;  
  
  assertIsAdminUser(user);  
  
  type test2 = Expect<Equal<typeof user, AdminUser>>; // red  
  squiggly line under Equal<>  
  
  user.roles; // red squiggly line under roles  
};
```

The `user` type is `User | AdminUser` before `assertIsAdminUser` is called, but it doesn't get narrowed down to just `AdminUser` after the function is called. This means you can't access the `roles` property:

```
// hovering over roles shows:  
Property 'roles' does not exist on type 'User | AdminUser'.
```

Your task is to update the `assertIsAdminUser` function with the proper type assertion so that the `user` is identified as an `AdminUser` after the function is called.

See <https://totalts.link/essentials-16-7/>.

Solution

The solution is to add a type annotation onto the return type of `assertIsAdminUser`.

If it was a type predicate, you would say `user is AdminUser`:

```
function assertIsAdminUser(user: User): user is AdminUser {  
  // red squiggly line under user is AdminUser  
  if (!("roles" in user)) {  
    throw new Error("User is not an admin");  
  }  
}
```

However, this leads to an error under `user is AdminUser`:

```
// hovering over user is AdminUser shows:  
A function whose declared type is neither 'undefined', 'void',  
nor 'any' must return a value.
```

You get this error because `assertIsAdminUser` is returning `void`, which is different from a type predicate that requires you to return a `boolean`.

Instead, you need to add the `asserts` keyword to the return type:

```
function assertIsAdminUser(user: User | AdminUser): asserts  
  user is AdminUser {  
  if (!("roles" in user)) {  
    throw new Error("User is not an admin");  
  }  
}
```

By adding the `asserts` keyword, just by the fact that `assertIsAdminUser` is called you can assert that the user is an `AdminUser`. You don't need to put it in an `if` statement or anywhere else.

With the `asserts` change in place, the `user` type is narrowed down to `AdminUser` after `assertIsAdminUser` is called, and the test passes as expected:

```
const handleRequest = (user: User | AdminUser) => {  
    type test1 = Expect<Equal<typeof user, User | AdminUser>>;  
  
    assertIsAdminUser(user);  
  
    type test2 = Expect<Equal<typeof user, AdminUser>>;  
  
    user.roles;  
};  
  
// hovering over roles shows:  
user: AdminUser;
```

Summary

This chapter covered utility folder development, the overlooked middle ground between simple application code and complex library development. Utility functions like `groupBy` and `debounce` are reused throughout large applications and benefit from TypeScript's advanced features.

Generic functions accept both values and types as arguments using type parameters like `<T>`. Unlike generic types, they can infer types from runtime arguments when no explicit type is provided, making functions like `getFirstElement` type-safe instead of returning `any`.

Type parameter constraints with `extends` ensure that generic types have required properties. You can set defaults for type parameters, with TypeScript falling back to `unknown` when it can't infer a type.

Type predicates use `input is Type` syntax to create reusable narrowing logic, while assertion functions use `asserts input is Type` and throw errors instead of returning Booleans. Both capture common validation patterns but require careful implementation.

Function overloads provide multiple call signatures for different parameter combinations, though union types are often simpler when parameter counts match.

INDEX

A

- abstract classes, [171–172](#)
- abstract keyword, [171–172](#)
- abstract methods, [172](#)
- `addClickListener` function, [71–72](#)
- `allowJs` setting, [308](#)
- `allowUnreachableCode` option, [314](#)
- ambient context, [291](#), [302–303](#)
- angle brackets (`<>`), [48](#), [56](#), [230](#), [333](#)
- annotating
 - empty objects, [269–270](#)
 - empty parameters, [39–40](#)
 - with `Equal` type, [38](#)
 - function parameters, [34](#), [36–37](#)
 - types
 - any, [41–42](#)
 - basic, [34](#), [40–41](#)
 - excess property checking, [271](#)
 - type inference, [35–37](#)
- values, [225–229](#)
- variables, [35–36](#)
 - vs. `as const`, [151](#)
 - vs. `as` and `satisfies`, [242–245](#)
 - vs. values, [225–229](#)
- any type, [37–38](#)
 - annotations with, [41–42](#)
 - assigning to `never`, [280](#)
 - evolving, [249–250](#)
 - vs. `unknown`, [91](#)
- application development, [359](#)
- arguments
 - adding to constructors, [161](#)
 - in generic types, [336](#)
 - types, [333](#)
- array methods, [295](#)
- arrays, [49](#)

- creating, [48–49](#)
- mutation of, [148–149](#)
- named tuples, [50–55](#)
- of objects
 - creating, [52–54](#)
 - inferring, [146–148](#), [153–156](#)
 - read-only and mutable, [144–145](#)
 - tuples, [50](#)
- Array type, [48](#), [51–52](#)
- arrow functions
 - to define class methods, [165–166](#)
 - implementing class methods, [174–175](#)
 - and this keyword, [265](#)
- as any tool, [234](#)
- as const assertion, [150–151](#)
 - arrays, [210–212](#)
 - creating deeply read-only objects, [246–247](#)
 - deep immutability with, [150–153](#)
 - inferring literal values in arrays, [153–156](#)
 - for JavaScript-style enums, [203–205](#)
 - vs. `Object.freeze`, [151–153](#)
 - vs. variable annotation, [151](#)
- as keyword
 - for assertions, [229–231](#)
 - providing additional information, [236–238](#)
 - remapping keys, [343–344](#), [356–357](#)
 - vs. `satisfies` and variable annotations, [242–245](#)
- assertion functions, [368–369](#), [384–386](#)
- assertions
 - as keyword, [229–231](#)
 - forcing type of values, [229–232](#)
 - never type, [280](#)
 - non-null, [231–232](#)
 - providing additional information, [236–238](#)
 - solving issues with, [238–240](#)
- `asserts` keyword, [368–369](#), [385–386](#)
- assignability of types, [90](#), [93](#), [257](#)
- asynchronous (async) functions
 - `Awaited` utility type, [213](#)
 - expecting certain properties, [135–136](#)
 - and generic types, [379–381](#)
 - typing, [64–65](#), [73–75](#)
 - unwrapping promises, [217–218](#)
- attribute getters, [354–355](#)
- augmenting modules, [294–295](#)
- autocomplete, [9–11](#)
- automated testing, [5](#)
- `Awaited` utility type, [213](#)

B

babel tool, [311](#)
basic types, [34](#), [38–39](#), [40–41](#)
bigint type, [34](#)
book repository, [xxvi](#)
boolean type, [34](#)
browsers, [24–25](#)
Bun, [28](#)
bundlers, [314–316](#)

C

C#, [xxiv](#)
capitalize type, [339](#)
classes
 creating, [159–162](#), [173–174](#)
 extending, [167–168](#), [177–178](#)
 generic, [363](#)
 getters, [175–176](#)
 inheritance
 abstract classes, [171–172](#)
 abstract methods, [172](#)
 implements keyword, [169–171](#)
 overriding, [168–169](#)
 protected properties, [168](#)
 methods, [165–166](#), [174–175](#)
 parameter properties, [181–182](#)
 properties, [162–165](#)
 property initializers, [162–163](#)
 setters, [177](#)
 in types and values, [259](#)
 using as types, [161–162](#)
class keyword, [159](#)
CLI (command line interface), [5](#), [25](#), [27–28](#)
closed object types, [253–254](#)
colon (:), [33](#), [61](#), [62](#)
comment directives, [232–234](#)
comments, [16–17](#)
CommonJS, [310](#)
composite option, [308](#), [326](#)
conditional types, [340–341](#)
configuring TypeScript
 base options, [310–312](#)
 jsx option, [319–320](#)
 module options, [314–316](#)
 multiple configurations
 extending configurations, [323–325](#)

- finding *tsconfig.json* files, [321–323](#)
- project, [325](#)
- project references, [325–326](#)
- Node support, [320–321](#)
- noEmit, [316](#)
- recommended configuration, [307–309](#)
- source maps, [316](#)
- strictness options, [312–314](#)
- transpiling code for library use, [316–319](#)
- console.log function, [64](#)
- constants, [21](#)
- const enums, [187](#)
- const keyword, [140–141](#)
- constructors, [160–161](#), [167–168](#)
- curly brackets ({})
 - empty object annotation, [269–270](#)
 - empty object types, [256–258](#)
 - object literal types, [43–44](#)

D

- declaration setting, [308](#), [317](#)
- declaration files, [288–291](#). *See also* [tsconfig.json files](#)
 - authoring, [299–302](#)
 - creating, [317–318](#)
 - declare keyword, [291–295](#)
 - from TypeScript and third parties, [295–299](#)
- declaration maps, [318–319](#)
- declarationMap setting, [308](#), [319](#)
- declaration merging, [116–117](#), [189–190](#), [299–300](#)
- declarations, [14–16](#)
- declare keyword, [291](#)
 - const enum, [311](#)
 - global, [292–293](#)
 - global scope, [301](#)
 - global variables, [291–292](#)
 - module, [293–295](#)
- decoupling vs. deriving, [221–223](#)
- deeply read-only objects, [246–247](#)
- deeply Required type, [131](#)
- default parameters, [61–62](#), [66–67](#)
- DefinitelyTyped GitHub repository, [298–299](#)
- deriving types
 - creating values from types, [200–201](#)
 - vs. decoupling, [221–223](#)
 - from functions, [212–214](#)
 - indexed access types, [201–205](#)
 - from other types, [198–200](#)

- transforming, [218–220](#)
- `diff` library, [298](#)
- discriminants, [98](#)
- discriminated tuples, [103–105](#)
- discriminated unions, [98–100](#)
 - bag of optionals problem, [97–98](#)
 - destructuring, [100–102](#)
 - handling defaults with, [105–107](#)
 - narrowing with switch statement, [102–103](#)
- `dist` directory, [317](#)
- `DistributiveOmit` type, [134](#)
- `DistributivePick` type, [134](#)
- documentation, [16–17](#)
- `document.getElementById` function, [57](#)
- `document` global, [296–297](#)
- `document.querySelector` function, [57–58](#)
- `dom.iterable` function, [297](#)
- DOM types, [296–297](#)
- `.d.ts` files, [288–291](#)
- dynamic keys
 - combining with known keys, [122–123](#)
 - default properties with, [125–126](#)
 - index signatures, [121–122](#)
 - using, [124](#)
 - `PropertyKey` type, [123–124](#), [128](#)
 - `Record` type, [122](#)
 - support for, [127–128](#)

E

- ECMAScript, [191–192](#), [310](#)
- empty object type (`{}`), [256–258](#), [269–270](#)
- empty parameters, [39–40](#)
- `enum` keyword, [182](#)
- enums
 - as `const` for JavaScript-style, [203–205](#)
 - numeric, [182–187](#)
 - string, [183–187](#)
 - transpiling, [184–185](#)
 - in types and values, [260](#)
 - whether to use, [187](#)
 - workings of, [184–187](#)
- environment variables, [305](#)
- `env` property, [305](#)
- `Equal` type annotation, [38](#)
- `--erasableSyntaxOnly` flag, [192](#), [321](#)
- Error class, [94](#)
- errors

- checking, [11–14](#)
- multiline, [13–14](#)
- narrowing with instanceof, [93–94](#)
- runtime, [11–12](#)
- suppressing, [232–236](#)
- TError, [347–348](#)
- @ts-expect-error directive, [xxvii](#), [232–233](#)
- in TypeScript CLI, [27–28](#)
- es5 option, [296](#)
- es2022 option, [296](#)
- ESbuild, [309](#)
- esbuild tool, [311](#)
- esModuleInterop flag, [308](#), [310–311](#)
- exactOptionalPropertyTypes option, [314](#)
- @example tag, [16](#)
- excess property warnings
 - comparing functions, [252–253](#)
 - fresh and stale objects, [254–255](#)
 - in functions, [272–274](#)
 - in objects, [270–272](#)
 - open vs. closed objects, [253–254](#)
 - on variables, [251–252](#)
- Exclude utility type, [218–219](#), [220](#)
- exercises in book, [xxvi–xxviii](#)
- Expect annotation, [38](#)
- experimental-strip-types flag, [320](#)
- export statement, [285–286](#)
- extends keyword
 - with abstract classes, [172](#)
 - constraining generic type parameters, [379](#)
 - constraining Result types, [349](#)
 - creating new interfaces, [114](#)
 - extending classes, [167–168](#)
 - setting parameter constraints, [337](#)
 - uses of, [338](#)
- Extract utility type, [220](#)

F

- feedback loop, [5](#)
- fetch function, [73–74](#)
- files: [], [325–326](#)
- Flow language, [253–254](#)
- frameworks, [28](#)
- fresh objects, [254–255](#)
- function keyword, [360](#), [370](#)
- function overloads, [369–372](#)
- functions. *See also* [asynchronous functions](#)

- arrow type, [165–166](#)
- assignability, [265–267](#)
- deriving types from, [212–214](#)
- excess property checks when comparing, [252–253](#)
- excess property warnings in, [272–274](#)
- extracting, [21](#)
- generic, [360–361](#), [363](#)
 - combining with generic types, [379–381](#)
 - constraining type parameters, [365–366](#)
 - debugging inferred type of, [364](#)
 - inference in, [375–377](#)
 - making, [372–374](#)
 - problems solved by, [363–364](#)
 - type aliases vs. generic types, [361–363](#)
 - type arguments in, [382–384](#)
 - type parameter defaults, [364–365](#)
- parameters
 - annotations, [34](#), [36–37](#)
 - basic types with, [38–39](#)
 - comparing, [276–278](#)
 - default, [61–62](#)
 - implementation, [265–267](#)
 - optional, [61](#)
 - rest, [62–63](#), [67–68](#)
 - restricting, [81–82](#)
- passing
 - read-only arrays to, [144–145](#)
 - read-only objects to, [145–146](#)
 - types to, [56–60](#)
- returning void, [71–72](#)
- return types, [62](#)
- this keyword in, [263–264](#)
- types, [63–64](#), [68–70](#)
- unions of
 - intersecting parameters, [267–269](#)
 - with object parameters, [278–281](#)
- void type, [64](#)

G

- generic classes, [363](#)
- generic functions, [360–361](#), [363](#)
 - combining with generic types, [379–381](#)
 - constraining type parameters, [365–366](#)
 - debugging inferred type of, [364](#)
 - inference in, [375–377](#)
 - making, [372–374](#)
 - problems solved by, [363–364](#)

- type aliases vs. generic types, [361–363](#)
- type arguments in, [382–384](#)
- type parameter defaults, [364–365](#)
- “generic” term, [362–363](#)
- generic types, [332–334](#), [363](#)
 - arguments, [336](#)
 - combining with generic functions, [379–381](#)
 - creating, [344–345](#)
 - default type parameters, [336–337](#)
 - vs. generic function type aliases, [361–363](#)
 - multiple type parameters, [334–335](#)
 - passing arguments to, [361](#)
 - type parameter constraints, [337–338](#)
- get keyword, [176](#)
- getters, [175–176](#)
- GitHub repository, [xxvii](#)
- global identifiers with multiple *tsconfig.json* files, [322–323](#)
- global scope
 - adding types to, [301](#)
 - declaration files, [290](#), [299](#)
 - scripts, [286](#)
 - using declare keyword, [292–293](#), [301](#)
- global types, [301–306](#)
- global variables, [291–292](#)
- Go to Definition, [17–18](#)
- Go to References, [17–18](#)

H

- Hejlsberg, Anders, [xxiv](#)
- hot module replacement (HMR), [28](#)
- hovering, [14–17](#)
- HTMLAudioElement API, [10](#)
- Hungarian notation, [170](#)

I

- IDE (integrated development environment)
 - autocomplete, [9–11](#)
 - automatic imports, [18](#)
 - Go to Definition and Go to References, [17–18](#)
 - introspecting variables and declarations, [14–16](#)
 - JSDoc comments, [16–17](#)
 - navigation in, [17–18](#)
 - quick fixes, [19](#)
 - rename symbol, [18](#)
 - restarting VS Code server, [19](#)
 - tsconfig.json* files, [322](#)
 - for TypeScript, [5](#)

- TypeScript error checking, [11–14](#)
 - working in JavaScript, [19–21](#)
- implementations, [290–291](#)
- implementation signature, [370–371](#)
- implements keyword, [169–171](#)
- imports, [18](#)
- import statement, [285–286](#)
- indexed access types, [201–205](#)
 - accessing specific values, [207–208](#)
 - chaining multiple, [202](#)
 - unions with, [208–209](#)
- indexing, [121](#)
- index signatures, [121–122](#), [124](#)
- inference
 - with array of objects, [146–148](#), [153–156](#)
 - debugging inferred type of generic functions, [364](#)
 - in generic functions, [375–377](#)
 - of literal values in arrays, [153–156](#)
 - and mutability
 - array mutation, [148–149](#)
 - array of objects, [146–148](#)
 - const keyword, [140–141](#)
 - let keyword, [140](#)
 - of object properties, [141–142](#)
 - readonly object properties, [143–146](#)
 - readonly tuples, [149–150](#)
 - of types, [35–37](#)
- inheritance
 - abstract classes, [171–172](#)
 - abstract methods, [172](#)
 - extending classes, [167–168](#)
 - implements keyword, [169–171](#)
 - overriding, [168–169](#)
 - protected properties, [168](#)
- initializers of class properties, [162–163](#)
- in operator, [88–90](#), [96](#)
- instanceof operator, [93–94](#)
- integrated development environment. *See* [IDE](#)
- interface extends syntax, [115–116](#)
- interfaces
 - in declaration files, [289](#)
 - expecting certain properties, [135–136](#)
 - extending, [114–117](#), [119–120](#)
 - implements keyword, [169–171](#)
 - merging within namespaces, [189–191](#)
 - vs. types, [116–117](#)
- intersection types, [112–113](#)
 - creating, [117–118](#)

- vs. `interface` extends syntax, [115–116](#)
- introspecting variables and declarations, [14–16](#)
- `I` prefix, [170](#)
- `is` keyword, [367–369](#)
- `isolatedModules` setting, [308](#), [311–312](#)

J

JavaScript

- choosing version with `lib` setting, [296](#)
- declaration files describing, [288–290](#)
- files, [316](#)
- integrated development environment, [19–21](#)
- leading to TypeScript, [xxiii–xxiv](#)
- Plain Old JavaScript Object, [203](#)
- template literals in, [338](#)
- vs. TypeScript, [3–4](#)
- TypeScript changing, [26–27](#)
- TypeScript declaration files for, [295–296](#)
- typing module, [302](#)
- workflow, [4](#)
- working in, [19–21](#)

JavaScript-style enums, [203–205](#)

JSDoc comments, [16–17](#)

`JSON.parse` function, [59–60](#), [360](#)

`jsx` option, [319–320](#)

JSX syntax, [319–320](#)

K

`keyof` operator, [198–199](#)

- deriving types from values, [206–207](#)
- extracting union of all values, [209–210](#)
- getting an object's values with, [203](#)
- iterating over objects, [275](#)
- reducing key repetition, [205–206](#)
- and `typeof`, [200](#)

keys

- dynamic
 - combining with known keys, [122–123](#)
 - default properties with, [125–126](#)
 - index signatures, [121–122](#), [124](#)
 - `PropertyKey` type, [123–124](#), [128](#)
 - `Record` type, [122](#)
 - support for, [127–128](#)
- iterating over, [255–256](#), [274–276](#)
- omitting, [349–351](#)
- reducing repetition of, [205–206](#)
- remapping with `as`, [343–344](#), [356–357](#)

renaming in mapped types, [355–357](#)

L

language server, [4](#)

large numbers, [41](#)

let keyword, [140](#)

lib.dom.d.ts file, [296–297](#)

lib.es5.d.ts file, [296](#)

lib.es2021.string.d.ts file, [296](#)

libraries

building, [308](#)

development, [359](#)

transpiling code for library use, [316–319](#)

types shipping with, [297–298](#)

lib setting, [296](#), [297](#)

linters, [28](#)

literal types, [79–82](#)

inferring values in arrays, [153–156](#)

objects, [43–44](#)

templates

in JavaScript, [338](#)

in TypeScript, [338–340](#), [352–353](#)

local scope, [286](#)

Long Term Support (LTS), [6](#)

Lowercase type, [339](#)

M

Map feature, [58–59](#)

.map method, [267](#), [295](#)

mapped types, [341–343](#)

key remapping with as, [343–344](#)

renaming keys in, [355–357](#)

and union types, [344](#)

members of unions, [78](#)

merging namespaces, [189–191](#)

methods, [165–166](#)

abstract keyword, [171–172](#)

implementing, [174–175](#)

Microsoft, [xxiii–xxiv](#)

NodeNext, [308](#), [314–315](#)

Preserve, [309](#), [315–316](#)

settings, [314–316](#)

moduleDetection configuration, [288](#), [308](#)

modules

augmentation vs. overriding, [294–295](#)

and scripts, [285–288](#)

sharing types across, [46–48](#)

- typing JavaScript types, [302](#)
- module setting, [308](#), [320](#)
- monorepos, [318–319](#), [325](#)
- multiline errors, [13–14](#)
- mutability
 - with `as const`, [150–153](#)
 - inferring literal values in arrays, [153–156](#)
 - and inference
 - array mutation, [148–149](#)
 - array of objects, [146–148](#)
 - `const` keyword, [140–141](#)
 - `let` keyword, [140](#)
 - of object properties, [141–142](#)
 - readonly object properties, [143–146](#)
 - readonly tuples, [149–150](#)

N

- named tuples, [50–55](#)
- name global variable, [287](#)
- namespaces, [188–191](#)
- naming types and values the same, [261–263](#)
- narrowing
 - discriminated unions with switch statement, [102–103](#)
 - never type, [92–93](#)
 - process, [84–90](#)
 - unions wider than members, [83–84](#)
 - unknown type, [95–97](#)
 - values with `satisfies`, [228–229](#)
 - wide and narrow types, [82–83](#)
- never type, [92–93](#), [113](#), [257](#), [280](#)
- new keyword, [56](#), [160](#)
- Next.js, [316](#)
- NODE16 version, [315](#)
- node `index.ts` command, [320](#)
- Node.js
 - installing, [6](#)
 - process, [305–306](#)
 - runtime environment, [4](#), [5](#)
 - support for TypeScript, [320–321](#)
 - type checking, [28](#)
- NodeJS namespace, [305–306](#)
- `node_modules` folder, [6](#), [297](#), [299](#)
- `node_modules/typescript/lib`, [296](#)
- NodeNext setting, [308](#), [314–315](#)
- `noEmit` option, [28](#), [309](#), [316](#), [320](#)
- `noFallthroughCasesInSwitch` option, [314](#)
- `noImplicitAny` setting, [312](#)

- `noImplicitOverride` option, [169](#), [314](#)
- `noImplicitReturns` option, [314](#)
- `NonNullable` utility type, [219–220](#)
- non-null assertion, [231–232](#)
- non-runtime errors, [12](#)
- `noPropertyAccessFromIndexSignature` option, [314](#)
- `noUncheckedIndexedAccess` option, [308](#), [312–314](#)
- `noUnusedLocals` option, [314](#)
- `noUnusedParameters` option, [314](#)
- npm package manager, [6](#), [297](#)
- null type
 - empty object type, [257–258](#), [269–270](#)
 - handling, [80–81](#)
 - runtime errors, [369](#)
- `Number.MAX_SAFE_INTEGER` value, [41](#)
- numbers, large, [41](#)
- number type, [34](#)
- numeric enums, [182–183](#)
 - behavior of, [185–187](#)
 - transpiling, [184–185](#)

O

- `Object.freeze` method, [151–153](#)
- `Object.keys()` method, [275](#)
- object literal types, [43–44](#)
- objects
 - arrays of, [49](#)
 - closed vs. open types, [253–254](#)
 - dynamic object keys
 - combining with known keys, [122–123](#)
 - index signatures, [121–122](#)
 - `PropertyKey` type, [123–124](#), [128](#)
 - `Record` type, [122](#)
 - excess properties
 - detecting, [270–272](#)
 - warnings, [251–255](#)
 - extending
 - interfaces, [114–117](#)
 - intersection types, [112–113](#)
 - fresh and stale, [254–255](#)
 - getting values with `keyof` operator, [203](#)
 - inference of properties, [141–142](#)
 - inference with array of, [146–148](#), [153–156](#)
 - iterating over keys of, [255–256](#), [274–276](#)
 - optional properties, [43–45](#)
 - restricting keys with `Record` keyword, [126–127](#)
 - utility types

- Omit, [131–132](#)
 - Partial, [129](#)
 - Pick, [131](#)
 - Required, [129–131](#)
 - union types, [132–134](#)
- Omit type, [131–132](#)
 - action of, [220](#)
 - DistributiveOmit type, [134](#)
 - strictness of, [349](#)
 - StrictOmit type, [349–351](#)
 - union types with, [132–134](#)
- online resources, [xxvi](#)
- open object types, [253–254](#)
- operators
 - ?, [43](#), [45](#), [75](#), [232](#)
 - !, [231](#), [240](#)
 - &, [112](#), [118](#)
 - |, [78–79](#)
 - =, [61](#), [67](#), [336](#), [347](#)
 - in, [88–90](#), [96](#)
 - instanceof, [93–94](#)
 - keyof, [198–199](#)
 - deriving types from values, [206–207](#)
 - extracting union of all values, [209–210](#)
 - getting an object’s values with, [203](#)
 - iterating over objects, [275](#)
 - reducing key repetition, [205–206](#)
 - and typeof, [200](#)
- satisfies
 - annotating values with, [227–228](#)
 - vs. as and variable annotations, [242–245](#)
 - creating deeply read-only objects, [246–247](#)
 - enforcing valid configuration, [240–242](#)
 - excess property checking, [272](#), [274](#)
 - narrowing values with, [228–229](#)
- ternary, [340](#)
- typeof
 - extracting types from values, [199–200](#), [206–207](#)
 - extracting union of all values, [209–210](#)
 - for narrowing, [84](#), [86–87](#)
- optional class properties, [163](#)
- optional modifier (?), [55](#), [61](#)
- optional parameters, [61](#)
 - function parameters, [65–66](#)
- opts argument, [161](#)
- outDir option, [317](#)
- overload signature, [370](#)
- override keyword, [168–169](#)

overriding modules, [294–295](#)

P

package.json, [6](#), [315](#)

parameters. *See also* [type parameters](#)

- annotations on, [34](#), [36–37](#)

- comparing, [276–278](#)

- default, [61–62](#), [66–67](#)

- in function callbacks, [265–267](#)

- in generic types, [334–338](#)

- optional, [61](#)

- rest, [62–63](#)

- type, [333](#)

- in unions of functions, [267–269](#), [278–281](#)

Parameters utility type, [212](#), [214–216](#)

@param tag, [16](#)

Parcel, [314–315](#)

Partial type, [129](#)

Pick type, [131](#)

- action of, [220](#)

- union types with, [132–134](#)

Plain Old JavaScript Object (POJO), [203](#)

PNG files, [300](#)

pnpm package manager, [6–7](#), [298](#)

polyfills, [310](#)

Preserve module, [309](#), [315–316](#), [320](#)

private class properties, [163–165](#)

private keyword, [163–165](#)

process entity, [305–306](#)

--project flag, [325](#)

project references, [325–326](#)

projects, [25](#)

projects folder, [7](#)

Promise type, [65](#), [217–218](#), [346](#)

properties

- in classes, [162–165](#)

- excess property warnings

 - comparing functions, [252–253](#)

 - fresh and stale objects, [254–255](#)

 - in objects, [270–272](#)

 - open vs. closed objects, [253–254](#)

 - on variables, [251–252](#)

- expecting certain, [135–136](#)

- of objects

 - inferring, [141–142](#)

 - readonly, [143–146](#)

- optional, [43–45](#)

- protected, [168](#)
- PropertyKey type, [123–124](#), [128](#)
- property types, [44–45](#)
- protected keyword, [168](#)
- public class properties, [163–165](#)
- public keyword, [163–165](#)

Q

- Quick Fixes menu, [19](#), [20](#)

R

- react-jsx value, [320](#)
- React TypeScript Cheatsheet, [312](#)
- react value, [320](#)
- ReadonlyArray type helper, [144](#)
- readonly keyword
 - class properties, [163](#)
 - object properties, [143–146](#)
 - tuples, [149–150](#)
- Record type, [122](#), [126–127](#)
- @redux/toolkit library, [312](#)
- Remix, [316](#)
- rename symbol feature, [18](#)
- .replaceAll method, [296](#)
- Required type, [129–131](#)
- resolveJsonModule setting, [308](#)
- response.json() function, [73–74](#)
- Response type, [73–74](#)
- rest parameters, [62–63](#), [67–68](#)
- Result types, [346–349](#)
- return types, [62](#)
- ReturnType utility type, [213](#), [216–217](#)
- return value–based typing, [216–217](#)
- rewriteRelativeImportExtensions setting, [320](#)
- Rollup, [315](#)
- root directory, [317](#)
- rootDir option, [317](#)
- Rosenwasser, Daniel, [192](#)
- route matching, [351–352](#)
- runtime errors, [11–12](#), [369](#)
- runtime tests, [xxvii](#)

S

- satisfies operator
 - annotating values with, [227–228](#)
 - vs. as and variable annotations, [242–245](#)

- creating deeply read-only objects, [246–247](#)
- enforcing valid configuration, [240–242](#)
- excess property checking, [272](#), [274](#)
- narrowing values with, [228–229](#)
- Script# (ScriptSharp), [xxiii–xxiv](#)
- scripts, [285–288](#)
- Set data structure, [56](#)
- set keyword, [177](#)
- sets, [56–57](#)
- setters, [177](#)
- setup for TypeScript development, [3–7](#)
- shared utilities
 - assertion functions, [368–369](#)
 - function overloads, [369–372](#)
 - generic functions, [360–361](#)
 - problem solved by generic functions, [363–364](#)
 - generic function type alias vs. generic types
 - constraining type parameters, [365–366](#)
 - debugging inferred type of generic functions, [364](#)
 - “generic” term, [362–363](#)
 - missing or conflicting type arguments, [361–362](#)
 - type parameter defaults, [364–365](#)
 - type predicates, [366–368](#)
- single-file transpilers, [311](#)
- single source of truth, [214–216](#)
- skipLibCheck option, [299](#), [308](#)
- Sorhus, Sindre, [131](#)
- source maps, [316](#)
- square brackets ([]), [48](#)
- src directory, [6](#)
- StackOverflow, [312](#)
- stale objects, [254–255](#)
- strictness options, [312–314](#)
- strictNullChecks setting, [312](#)
- StrictOmit type, [349–351](#)
- strict option, [308](#), [312](#)
- string enums, [183–187](#)
- string types, [34](#), [80–81](#), [339–340](#)
- subtypes, [83](#)
- super() method, [167](#)
- supertypes, [83](#)
- SvelteKit, [316](#)
- swc tool, [311](#)
- switch statement, [102–103](#)
- symbol type, [34](#)

T

- target option, [308](#), [310](#)
- TC39 committee, [192](#)
- template literal types
 - combining with union types, [339](#)
 - in JavaScript, [338](#)
 - in TypeScript, [338–340](#)
 - permutations, [352–353](#)
- ternary operator, [340](#)
- TError parameter, [347–348](#)
- testing, automated, [5](#)
- tests, runtime, [xxvii](#)
- this keyword
 - accessing classes, [161](#)
 - and arrow functions, [265](#)
 - in functions, [263–264](#)
 - in types and values, [260–261](#)
- throw keyword, [87–88](#)
- tools for TypeScript development, [5–7](#)
- @totaltypescript/helpers library, [38](#)
- τ prefix, [170](#), [363](#)
- transforming derived types, [218–220](#)
- transpiling TypeScript, [25–26](#)
 - enums, [184–185](#)
 - for library use, [316–319](#)
 - module settings, [308](#)
 - namespaces, [189](#)
 - noEmit option, [316](#)
- trpc library, [312](#)
- try. . .catch statement, [93–94](#)
- tsconfig.json* files. *See also* [declaration files](#)
 - base options, [310–312](#)
 - generating, [25](#)
 - jsx option, [319–320](#)
 - lib setting, [296](#)
 - moduleDetection, [288](#)
 - module options, [314–316](#)
 - multiple configurations
 - extending configurations, [323–325](#)
 - finding *tsconfig.json* files, [321–323](#)
 - project, [325](#)
 - project references, [325–326](#)
 - Node's TypeScript support, [320–321](#)
 - noEmit, [28](#), [316](#)
 - noImplicitOverride, [169](#)
 - recommended configuration, [307–309](#)
 - source maps, [316](#)
 - strictness options, [312–314](#)
 - transpiling code for library use, [316–319](#)

- tsc tool, [4](#), [25–26](#), [311](#), [326](#)
- @ts-expect-error directive, [xxvii](#), [232–233](#)
- .ts files, [4](#), [46–47](#), [285](#)
- @ts-ignore directive, [164](#), [233](#)
- @ts-nocheck directive, [233–234](#)
- .tsx files, [4](#)
- tuples, [50](#)
 - discriminated, [103–105](#)
 - extracting types from, [202](#)
 - in functions, [54–55](#)
 - named, [50–55](#)
 - and readonly, [149–150](#)
- type aliases, [46](#), [361–363](#)
- type annotations
 - any type, [37–38](#)
 - basic types, [34](#), [40–41](#)
 - excess property checking, [271](#)
 - function parameter annotations, [34](#)
 - type inference, [35–37](#)
 - variable annotations, [35–36](#)
- type arguments
 - in generic functions, [363](#), [382–384](#)
 - missing or conflicting, [361–362](#)
 - passing
 - to functions, [57](#)
 - to generic functions, [362](#)
 - to generic types, [333](#)
 - setting defaults, [374–375](#)
- type-fest library (Sorhus), [131](#)
- type inference, [35–37](#)
- type keyword, [46](#), [47–48](#)
- type="module" attribute, [287](#)
- typeof operator
 - extracting types from values, [199–200](#), [206–207](#)
 - extracting union of all values, [209–210](#)
 - for narrowing, [84](#), [86–87](#)
- type-only namespaces, [191](#)
- type parameters
 - constraining
 - in functions, [377–379](#)
 - generic functions, [365–366](#)
 - generic types, [337–338](#)
 - creating generic types, [333](#), [346](#)
 - defaults for generic types, [336–337](#)
 - in generic functions, [363](#)
 - generic types accepting multiple, [334–335](#)
 - setting defaults, [364–365](#)
- type predicates, [366–368](#)

types

- arguments, [333](#)
- assignability of, [90](#), [93](#)
- based on return value, [216–217](#)
- basic, [34](#), [38–39](#), [40–41](#)
- conditional, [340–341](#)
- creating values from, [200–201](#)
- in declaration files, [289](#)
- deriving
 - creating values from types, [200–201](#)
 - vs. decoupling, [221–223](#)
 - from functions, [212–214](#)
 - indexed access types, [201–205](#)
 - from other types, [198–200](#)
 - transforming derived types, [218–220](#)
 - from values, [206–207](#)
- DistributiveOmit, [134](#)
- DistributivePick, [134](#)
- functions, [63–64](#)
- generic, [332–334](#)
 - arguments, [336](#)
 - default type parameters, [336–337](#)
 - multiple type parameters, [334–335](#)
 - type parameter constraints, [337–338](#)
- indexed access, [201–205](#)
- vs. interfaces, [116–117](#)
- intersection, [112–118](#)
- from libraries, [297–298](#)
- literal, [79–82](#)
- mapped, [341–343](#)
 - key remapping with as, [343–344](#)
 - renaming keys in, [355–357](#)
 - and union types, [344](#)
- naming the same as values, [261–263](#)
- narrow, [82–83](#)
- never, [92–93](#), [113](#), [257](#), [280](#)
- open vs. closed objects, [253–254](#)
- passing to functions, [56](#), [57–60](#)
- passing to sets, [56–57](#)
- sharing across modules, [46–48](#)
- storing in declaration files, [301](#)
- template literals in TypeScript, [338–340](#), [352–353](#)
- throw narrowing, [87–88](#)
- union, [77–78](#)
- unknown, [90–91](#)
- using classes as, [161–162](#)
- and value world, [258–261](#)
- wide and narrow, [82–83](#)

TypeScript

- birth of, [xxiii–xxiv](#)
- choosing modules and scripts, [287–288](#)
- configuring
 - base options, [310–312](#)
 - jsx option, [319–320](#)
 - module options, [314–316](#)
 - multiple configurations, [321–326](#)
 - Node support for, [320–321](#)
 - noEmit, [316](#)
 - recommended configuration, [307–309](#)
 - source maps, [316](#)
 - strictness options, [312–314](#)
 - transpiling code for library use, [316–319](#)
- declaration files, [295–296](#)
- in development pipeline
 - browser support, [24–25](#)
 - changing JavaScript, [26–27](#)
 - errors in TypeScript CLI, [27–28](#)
 - frameworks, [28](#)
 - linter functionality, [28](#)
 - transpiling, [25–26](#)
 - version control, [27](#)
 - watch mode, [27](#)
- vs. ECMAScript, [191–192](#)
- error checking, [11–14](#)
- future of, [192](#)
- installing, [7](#)
- vs. JavaScript, [3–4](#)
- performance, [116](#)
- project workflow, [5](#)
- setup, [3–7](#)
- tools for development, [5–7](#)
- workflow, [4–5](#)

TypeScript command line interface (CLI), [5](#), [25](#), [27–28](#)

TypeScript Official Handbook, [xxvi](#)

typescript package, [7](#)

@types/node package, [305](#)

@types/* package, [298](#)

U

Uncapitalize type, [340](#)

undefined type

- and empty object type, [257–258](#), [269–270](#)

- vs. void, [72–73](#)

unions

- combining with other unions, [79–82](#)

- creating from as const array, [210–212](#)
- discriminated, [98–100](#)
 - bag of optionals problem, [97–98](#)
 - destructuring, [100–102](#)
 - handling defaults with, [105–107](#)
 - narrowing with switch statement, [102–103](#)
- extracting all values from object, [209–210](#)
- vs. function overloads, [372](#)
- of functions
 - intersecting parameters, [267–269](#)
 - with object parameters, [278–281](#)
- with indexed access types, [208–209](#)
- members of, [78](#)
- passing to indexed access types, [202–203](#)
- types of, [77–78](#)
- wider than members, [83–84](#)
- union types, [77–78](#)
 - combining template literal types with, [339](#)
 - with Omit and Pick, [132–134](#)
 - using mapped types with, [344](#)
- unknown type, [90–91](#)
 - narrowing to single value, [95–97](#)
 - type assignability chart for, [257](#)
 - type parameter defaults, [365](#)
- Uppercase type, [339](#)
- utility types
 - expecting certain properties, [135–136](#)
 - Omit, [131–132](#)
 - Partial, [129](#)
 - Pick, [131](#)
 - Required, [129–131](#)
 - union types, [132–134](#)
 - updating products, [136–137](#)
- /utils folder, [359](#)

V

values

- accessing specific, [207–208](#)
- annotating, [225–229](#)
- creating from types, [200–201](#)
- deriving types from, [206–207](#)
- forcing type of, [229–232](#)
- naming the same as types, [261–263](#)
- narrowing with satisfies, [228–229](#)
- and type world, [258–261](#)

variables

- annotating, [35–36](#)

- vs. annotating values, [225–229](#)
- with as and satisfies, [242–245](#)
- vs. as const, [151](#)
- environment, [305](#)
- excess property checks on, [251–252](#)
- global, [291–292](#)
- inlining, [20](#)
- introspecting, [14–16](#)
- verbatimModuleSyntax function, [321](#)
- version control, [27](#)
- Visual Studio Code (VS Code), [5](#)
 - server, [19](#)
- Vite, [28](#), [316](#)
- Vitest, [xxvii–xxviii](#)
- void type, [64](#), [71–73](#)

W

- warning locations, [12–13](#)
- watch flag, [27](#)
- watch mode, [27](#)
- Webpack, [300](#), [314–315](#)
- wide types, [82–83](#)
- window object, [304–305](#)

X

- xstate library, [312](#)

Z

- zod library, [97](#), [297–298](#), [312](#)